

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2014

Huấn luyện viên: Hu Weidong

China Computer Federation, 2014

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2014

I0I China candidate team papers / National training team 2014 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2014. Phần mục lục và đầu sách có thể đọc trực tiếp, còn thân bài cần đối chiếu kỹ với bố cục PDF. Bản dịch bao gồm phần đầu sách, mục lục và toàn bộ mười lăm bài của tập.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2014*. Huấn luyện viên ghi trên bìa là Hu Weidong. Đơn vị xuất bản là China Computer Federation.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Báo cáo ra đề <i>MSS</i>	Wang Ziyu, Trường Trung học số 1 Shengli, Đông Doanh, Sơn Đông
9	Báo cáo ra đề <i>Matrix</i>	Yu Xingjiang, Trường Trường Quận, Trường Sa, Hồ Nam
19	Đa giác biến đổi	Dong Honghua, Trường Trung học số 1 Thiệu Hưng, Chiết Giang
35	Bước đầu nghiên cứu thuật toán liên quan tới nhóm hoán vị	Cen Ruoxu, Trường Trung học Trần Hải, Chiết Giang
45	Bàn về phụ thuộc tuyến tính	Kuang Zhengfei, Trường Yali, Trường Sa, Hồ Nam
57	Một số nghiên cứu về cây khung tích nhỏ nhất ba chiều	Zhang Hengjie, Trường Trung học số 1 Thiệu Hưng, Chiết Giang
65	Bàn về bài toán xâu con palindrome	Xu Yi, Trường Trung học cao cấp Thường Châu, Giang Tô
77	Bàn về ứng dụng phương pháp duy trì mảng nhiều chiều trong bài cấu trúc dữ liệu	Liang Zeyu, Trường Trung học số 1 Hợp Phì, An Huy
91	Ứng dụng cây đoạn trong một lớp bài chia để trị	Xu Yinzhan, Trường Học Quân Hàng Châu, Chiết Giang
105	Thuật toán căn bậc hai: không chỉ là chia khối	Wang Yuetong, Trường Ngoại ngữ Nam Kinh, Giang Tô
115	Bàn về các vấn đề liên quan tới cây động và một số mở rộng đơn giản	Huang Zhi'ao, Trường Yali, Trường Sa, Hồ Nam
137	Ứng dụng thuật toán ngẫu nhiên trong thi tin học	Hu Zecong, Trường THPT trực thuộc Đại học Sư phạm Hồ Nam
155	Hiện thực chương trình tính tế: bàn về tối ưu hằng số trong OI	He Qi, Trường Trung học Nam Khai, Thiên Tân
169	Trở về gốc rễ: phép toán bit và ứng dụng	Shen Yang, Trường Trung học Trần Hải, Chiết Giang
195	Một số phương pháp tìm nghiệm tốt thứ k	Yu Dingli, Trường Trung học số 1 Thiệu Hưng, Chiết Giang

3 Báo cáo ra đề *MSS*

Wang Ziyu, Trường Trung học số 1 Shengli, Đông Doanh

3.1 Mô tả bài toán

3.1.1 Phát biểu

Trên mặt phẳng có N điểm. Mỗi điểm có một trọng số; ban đầu mỗi điểm thuộc một tập khác nhau. Gọi tập hiện tại thứ i là S_i . Cần duy trì một cấu trúc dữ liệu hỗ trợ bốn loại thao tác sau.

- Merge $x\ y$: chuyển mọi điểm trong tập S_x sang tập S_y .
- Split $i\ d\ v$, với $d \in \{0, 1\}$: tạo hai tập mới S_{c+1} và S_{c+2} , trong đó c là số tập hiện có, kể cả các tập rỗng sinh ra trước đó. Sau đó, trong tập S_i , các điểm có tọa độ chiều thứ d không vượt quá v được chuyển vào S_{c+1} , còn các điểm có tọa độ lớn hơn v được chuyển vào S_{c+2} .
- Query i : hỏi giá trị lớn nhất, nhỏ nhất và tổng trọng số của các điểm trong tập S_i .
- Add $i\ d$: cộng d vào trọng số của mọi điểm trong tập S_i .

Để thuận tiện, với mỗi $d \in \{0, 1\}$, mọi điểm trong dữ liệu vào có tọa độ chiều thứ d đôi một khác nhau. Mọi tập được nhắc tới trong các thao tác đều không rỗng.

3.1.2 Dữ liệu vào

Dòng đầu chứa một số nguyên, biểu thị số điểm. Tiếp theo là N dòng, mỗi dòng gồm ba số nguyên lần lượt là hai tọa độ và trọng số của một điểm. Sau đó là một số nguyên Q , biểu thị số thao tác. Tiếp theo là Q dòng, mỗi dòng mô tả một thao tác theo định dạng ở trên.

3.1.3 Dữ liệu ra

Với mỗi thao tác Query, in ra ba số nguyên, lần lượt là trọng số lớn nhất, trọng số nhỏ nhất và tổng trọng số trong tập được hỏi.

3.1.4 Ví dụ

Dữ liệu vào

```
5
4 7 3
18 16 2
14 13 1
10 2 10
3 8 5
11
Merge 2 3
Merge 4 5
Split 2 1 13
Query 4
Query 6
Add 6 2
Query 6
Merge 6 4
Split 6 0 10
```

Query 9
Split 8 0 3

Dữ liệu ra

10 5 15
1 1 1
3 3 3
3 3 3

3.1.5 Quy mô dữ liệu và ràng buộc

Dữ liệu được chia thành năm nhóm:

- nhóm A chiếm 10%: $N, Q \leq 500$;
- nhóm B chiếm 20%: dữ liệu không có thao tác tách;
- nhóm C chiếm 10%: sau khi thao tác tách đầu tiên xuất hiện thì không còn thao tác gộp;
- nhóm D chiếm 20%: các điểm thỏa một quan hệ đặc biệt khiến bài toán suy biến về một chiều;
- nhóm E không có đặc trưng bổ sung.

Ở các nhóm B đến E, $N \leq 50000$, $Q \leq 100000$. Giới hạn thời gian là 3 giây và giới hạn bộ nhớ là 512 MB. Dòng ràng buộc của nhóm D trong bản nguồn không rõ hoàn toàn; phần lời giải phía sau cho thấy nhóm này là trường hợp các điểm có thứ tự một chiều tương thích, nên có thể chuyển mọi thao tác tách về cùng một trục.

3.2 Phân tích bài toán

3.2.1 Các thuật toán đơn giản

Mô phỏng. Với nhóm A, có thể mô phỏng trực tiếp các tập điểm. Độ phức tạp thời gian là $O(NQ)$.

Gộp theo heuristic. Với nhóm B, vì không có thao tác tách, có thể dùng gộp theo kích thước: mỗi tập được duy trì bằng một cây cân bằng. Khi gộp hai tập, ta chèn toàn bộ điểm của tập nhỏ hơn vào cây cân bằng của tập lớn hơn. Mỗi lần một điểm bị chèn lại, kích thước tập chứa nó ít nhất tăng gấp đôi; do đó mỗi điểm bị chèn lại nhiều nhất $O(\log N)$ lần. Độ phức tạp là $O(N \log^2 N + Q)$.

Tách theo heuristic. Với nhóm C, trước hết dùng gộp theo heuristic để xử lý toàn bộ thao tác gộp trước khi thao tác tách đầu tiên xuất hiện. Sau đó, nếu mỗi thao tác tách có thể thực hiện trong thời gian

$$O(T \cdot |\text{tập nhỏ hơn sau khi tách}|),$$

trong đó T không phụ thuộc vào kích thước tập nhỏ hơn, thì phân tích ngược lại giống hệt gộp theo heuristic.

Nếu mỗi lần chỉ tách theo một tọa độ, ta chỉ cần lập cây cân bằng theo thứ tự tọa độ đó. Quay lại bài này, với mỗi tập ta lập hai cây cân bằng, lần lượt sắp theo hoành độ và tung độ. Khi tách, dùng cây theo chiều được hỏi để lấy tập nhỏ hơn; trong cây còn lại, xóa các điểm tương ứng rồi dựng lại. Khi đó $T = O(\log N)$, nên độ phức tạp là $O(N \log^2 N + Q)$.

3.2.2 Thuật toán một chiều

Trong nhóm D, vì các điểm có thứ tự tương thích theo một chiều, mọi phép tách theo tung độ có thể chuyển thành phép tách theo hoành độ. Do đó có thể xem bài toán là duy trì các tập số trên một đường thẳng.

Dựa trên cây cân bằng. Ta tiếp tục thử dùng cây cân bằng để duy trì giá trị trong từng tập. Cây cân bằng hỗ trợ tách trong $O(\log N)$ và gộp trong $O(\log N)$ khi hai tập có khoảng giá trị không giao nhau; vấn đề còn lại là gộp khi các khoảng giá trị có thể xen kẽ nhau.

Thuật toán gộp có thể viết như sau.

```
merge(x, y):
  if x = nil: return y
  if y = nil: return x
  if x.min > y.min: swap(x, y)
  return merge(merge(splitL(x, y.min), y), splitR(x, y.min))
```

Ở đây $\text{merge}(a, b)$ trả về hợp của hai cây cân bằng a, b khi phạm vi giá trị của chúng không giao nhau; $\text{splitL}(a, v)$ trả về cây gồm các phần tử của a có giá trị không vượt quá v ; $\text{splitR}(a, v)$ trả về cây gồm các phần tử của a có giá trị lớn hơn v .

Ta xét độ phức tạp của thuật toán. Cần chứng minh rằng tổng chi phí của Q_1 thao tác tách và Q_2 thao tác gộp là

$$O((N + Q_1) \log^2 N).$$

Định nghĩa

$$W(X, Y) = \sum_{a \in X, b \in Y} [|\text{rank}(X \cup Y, a) - \text{rank}(X \cup Y, b)| = 1],$$

trong đó $\text{rank}(S, x)$ là thứ hạng theo giá trị của x trong S . Nói không hoàn toàn hình thức, $W(X, Y)$ là số đoạn, trừ đi 1, thu được sau khi gộp X và Y rồi nén các phần tử vốn thuộc cùng một tập thành một đoạn. Một thao tác gộp có chi phí $O(W(X, Y) \log N)$.

Gọi U là hợp của tất cả các tập. Với một phần tử x thuộc tập S , đặt

$$\begin{aligned} g_0(x) &= \text{rank}(U, x) - \text{rank}(U, \text{prev}(S, x)), \\ g_1(x) &= \text{rank}(U, \text{succ}(S, x)) - \text{rank}(U, x), \end{aligned}$$

trong đó $\text{prev}(S, x)$ và $\text{succ}(S, x)$ lần lượt là phần tử trước và sau của x trong tập S ; nếu không tồn tại thì dùng $\min(U)$ hoặc $\max(U)$.

Sau thao tác thứ i , định nghĩa thể năng của cấu trúc dữ liệu là

$$\Phi_i = c \log N \sum_{x \in U} (\log g_0(x) + \log g_1(x)),$$

với c là hằng số sẽ chọn sau. Ban đầu $\Phi_0 = cN \log^2 N$.

Một thao tác tách có chi phí thực $O(\log N)$. Vì chỉ có một giá trị g_0 và một giá trị g_1 tăng, phần tăng thể năng không vượt quá $2c \log^2 N$.

Xét thao tác gộp X và Y . Đặt

$$\begin{aligned} P_X &= \{x \in X \mid \text{prev}(X, x) \neq \text{prev}(X \cup Y, x)\} \cup \{\min(X)\}, \\ S_X &= \{x \in X \mid \text{succ}(X, x) \neq \text{succ}(X \cup Y, x)\} \cup \{\max(X)\}. \end{aligned}$$

Sau khi gộp X, Y và nén các điểm vốn thuộc cùng một tập thành đoạn, P_X chứa mọi đầu trái của các đoạn đến từ X , còn S_X chứa mọi đầu phải. Định nghĩa tương tự P_Y, S_Y . Khi đó

$$|P_X| + |P_Y| = W(X, Y) + 1.$$

Với các cặp biên kề nhau sau khi gộp, trong hai đại lượng g_0 và g_1 sẽ có một đại lượng giảm ít nhất còn một nửa giá trị cũ. Vì có $W(X, Y) - 1$ cặp như vậy, phần biến thiên thế năng do gộp không vượt quá

$$-c(W(X, Y) - 1) \log N.$$

Chọn c đủ lớn, tổng chi phí mọi thao tác được chặn bởi

$$\begin{aligned} \Phi_0 + \sum_{i=1}^{Q_1} O(\log N + c \log^2 N) \\ + \sum_{i=1}^{Q_2} \left(-c(W(X_i, Y_i) - 1) \log N + O(W(X_i, Y_i) \log N) \right) \\ = O((Q_1 + N) \log^2 N). \end{aligned}$$

Dựa trên cây đoạn. Một cách khác là dùng cây đoạn động để duy trì từng tập. Cây đoạn động hỗ trợ gộp; sau khi rời rạc hóa tọa độ, thao tác tách tốn $O(\log N)$. Cần chứng minh tổng chi phí của Q_1 thao tác tách và Q_2 thao tác gộp là

$$O((N + Q_1) \log N).$$

Gọi $\text{size}(T)$ là số nút của cây đoạn động biểu diễn tập T . Chi phí gộp hai tập X, Y là

$$O(\text{size}(X) + \text{size}(Y) - \text{size}(X \cup Y)).$$

Định nghĩa thế năng sau thao tác thứ i :

$$\Phi_i = c \sum_{S \in \mathcal{S}_i} \text{size}(S),$$

trong đó $\mathcal{S}_i$ là họ các tập hiện tại. Một thao tác tách có chi phí thực $O(\log N)$ và chỉ làm tăng thế năng thêm $O(c \log N)$. Một thao tác gộp X, Y làm thế năng giảm

$$c(\text{size}(X) + \text{size}(Y) - \text{size}(X \cup Y)).$$

Thế năng ban đầu là $cN \log N$. Chọn c đủ lớn, tổng chi phí không vượt quá

$$\begin{aligned} \Phi_0 + \sum_{i=1}^{Q_1} (O(\log N) + c \log N) \\ + \sum_{i=1}^{Q_2} \left(O(\text{size}(X_i) + \text{size}(Y_i) - \text{size}(X_i \cup Y_i)) \right. \\ \left. - c(\text{size}(X_i) + \text{size}(Y_i) - \text{size}(X_i \cup Y_i)) \right) \\ = O((N + Q) \log N). \end{aligned}$$

Từ đó nhóm D được giải với độ phức tạp $O((N + Q) \log N)$.

Thuật toán thay thế. Đây là một bài toán kinh điển. Tài liệu tham khảo [3] đưa ra thuật toán trung bình $O(\log N)$ dựa trên *biased skip list*, nhưng cả phân tích lẫn hiện thực đều phức tạp hơn; độc giả quan tâm có thể tự tham khảo.

3.2.3 Thuật toán tổng quát

Xét phạm vi áp dụng của thuật toán một chiều. Định nghĩa quan hệ thứ tự riêng phần trên tập điểm:

$$(x_1, y_1) < (x_2, y_2) \quad \text{khi và chỉ khi} \quad x_1 < x_2, y_1 < y_2.$$

Nếu hợp của mọi tập hiện tại U , theo quan hệ này, là một chuỗi, thì thuật toán một chiều áp dụng được: do y tăng đơn điệu theo x , mọi phép chia theo tọa độ y đều có thể chuyển thành phép chia theo tọa độ x . Tương tự, khi U là một phản chuỗi, tức y giảm đơn điệu theo x , thuật toán một chiều cũng áp dụng được.

Trong trường hợp tổng quát, chia U thành một số tập con $U_1, \dots, U_K$ sao cho theo thứ tự riêng phần ở trên, mỗi U_i là một chuỗi hoặc phản chuỗi. Với mỗi U_i , xét giao của nó với từng tập hiện tại:

$$\{S \cap U_i \mid S \in \mathcal{S}\}.$$

Khi đó các thao tác gộp, tách và hỏi đều có thể xử lý độc lập trên từng U_i , và mỗi U_i được duy trì bằng thuật toán một chiều.

Độ phức tạp của cách này là $O((N + Q)K \log N)$. Còn cần chứng minh $K = O(\sqrt{N})$.

Bổ đề 1. Trong một tập có thứ tự riêng phần P kích thước N , hoặc tồn tại một chuỗi kích thước $\sqrt{N}$, hoặc tồn tại một phản chuỗi kích thước $\sqrt{N}$.

Chứng minh. Giả sử P không chứa chuỗi độ dài L . Theo định lý Dilworth, P có thể được chia thành không quá $L - 1$ phản chuỗi. Vì vậy tồn tại một phản chuỗi có kích thước ít nhất $N/(L - 1)$. Chọn $L = \lceil \sqrt{N} \rceil$ là đủ.

Bổ đề 2. Một tập có thứ tự riêng phần U kích thước N có thể được chia thành $O(\sqrt{N})$ chuỗi hoặc phản chuỗi.

Chứng minh. Chia đệ quy: mỗi lần xóa khỏi tập còn lại một chuỗi dài nhất hoặc một phản chuỗi dài nhất. Nếu tập còn lại có kích thước n , theo bổ đề 1 có thể xóa ít nhất $\sqrt{n}$ phần tử. Số phần trong phép chia thỏa

$$T(N) = T(N - \sqrt{N}) + 1 < T(N/2) + O(\sqrt{N}) = O(\sqrt{N}).$$

Như vậy bài toán được giải với độ phức tạp

$$O((N + Q)\sqrt{N} \log N).$$

3.3 Kết luận

Kiến thức dùng trong bài chỉ gồm cấu trúc dữ liệu cơ bản và phân tích khấu hao, đều thuộc phạm vi của thí sinh NOI. Tuy nhiên, việc tìm ra thuật toán tổng quát đòi hỏi năng lực phân tích khá tốt.

Đối với 40 điểm cuối, tuy còn tồn tại những cách giải khác, tác giả cho rằng cách chia tập có thứ tự riêng phần được trình bày ở đây thú vị và giàu tính gợi mở hơn. Khi đối tượng trong bài toán thỏa một quan hệ thứ tự riêng phần khác, chẳng hạn quan hệ bao hàm giữa các đoạn, những phương pháp tương tự đáng được cân nhắc.

Tài liệu tham khảo

1. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa.
2. Richard A. Brualdi, *Introductory Combinatorics*, 5th ed., Prentice Hall.
3. John Iacono, Ozgur Ozkan, “Mergeable Dictionaries”.

4 Báo cáo ra đề *Matrix*

Yu Xingjiang, Trường Trường Quận, Trường Sa

4.1 Đề bài

4.1.1 Mô tả

Cho một ma trận số thực không âm A kích thước $N \times N$. Cần tìm một ma trận số thực B kích thước $N \times N$ sao cho

$$\sum_{k=1}^N B_{i,k} + \sum_{k=1}^N B_{k,j} \geq A_{i,j}$$

với mọi i, j . Nói cách khác, với mỗi ô (i, j) , tổng các phần tử trên hàng i của B cộng với tổng các phần tử trên cột j của B phải không nhỏ hơn $A_{i,j}$. Trong điều kiện đó, hãy tối thiểu hóa tổng mọi phần tử của B .

4.1.2 Dữ liệu vào

Có hai nhiệm vụ, chi tiết xem ở phần phạm vi dữ liệu. Với mỗi nhiệm vụ, dòng đầu chứa một số nguyên N . Tiếp theo là N dòng, mỗi dòng chứa N số thực không âm, mô tả ma trận A đã cho.

4.1.3 Dữ liệu ra

In hai dòng. Mỗi dòng in một số thực không âm giữ lại 4 chữ số sau dấu thập phân, lần lượt là đáp án của hai nhiệm vụ.

4.1.4 Ví dụ

Dữ liệu vào

```
3
1 7 3
3 2 1
5 4 1
2
1 1
1 1
```

Dữ liệu ra

```
6.5000
1.0000
```


4.1.5 Nhiệm vụ con và quy ước

- Task0: mọi phần tử trong A bằng nhau.
- Task1: trong cả nhiệm vụ một và nhiệm vụ hai, $N \leq 50$.
- Task2: các phần tử của A là ngẫu nhiên.
- Task3: trong nhiệm vụ hai, $N \leq 50$.
- Task4: trong nhiệm vụ một, $N \leq 300$; trong nhiệm vụ hai, $N \leq 80$.

Ở nhiệm vụ một, ma trận B cần dựng không có ràng buộc nào khác. Ở nhiệm vụ hai, mọi phần tử của B phải không âm. Mọi số thực trong dữ liệu vào giữ lại 3 chữ số sau dấu thập phân và có trị tuyệt đối không vượt quá 1000.

Giới hạn thời gian là 3 giây, giới hạn bộ nhớ là 512 MB.

4.2 Ý tưởng giải

4.2.1 Task0

Dễ thấy khi mọi phần tử của A bằng nhau, mọi phần tử trong một nghiệm tối ưu của B cũng có thể chọn bằng nhau. Do đó có thể tính trực tiếp đáp án bằng tay.

4.2.2 Lập mô hình quy hoạch tuyến tính

Theo đề bài, mô hình quy hoạch tuyến tính trực tiếp là

$$\begin{cases} \sum_{i=1}^N B_{i,1} + \sum_{j=1}^N B_{1,j} \geq A_{1,1}, \\ \sum_{i=1}^N B_{i,2} + \sum_{j=1}^N B_{1,j} \geq A_{1,2}, \\ \dots \\ \sum_{i=1}^N B_{i,1} + \sum_{j=1}^N B_{2,j} \geq A_{2,1}, \\ \sum_{i=1}^N B_{i,2} + \sum_{j=1}^N B_{2,j} \geq A_{2,2}, \\ \dots \\ \sum_{i=1}^N B_{i,N} + \sum_{j=1}^N B_{N,j} \geq A_{N,N}, \end{cases} \quad \min z = \sum_{i=1}^N \sum_{j=1}^N B_{i,j}.$$

Cách lập này có $O(N^2)$ biến, cần tối ưu thêm. Đề bài thực ra đã gợi ý rằng có thể dùng tổng hàng và tổng cột làm biến, nên ta thử hướng đó. Gọi R_i là tổng trọng số của hàng i , C_j là tổng trọng số của cột j . Khi đó có một mô hình mới:

$$\begin{cases} R_1 + C_1 \geq A_{1,1}, \\ R_1 + C_2 \geq A_{1,2}, \\ \dots \\ R_2 + C_1 \geq A_{2,1}, \\ R_2 + C_2 \geq A_{2,2}, \\ \dots \\ R_N + C_N \geq A_{N,N}, \\ \sum_{i=1}^N R_i - \sum_{j=1}^N C_j = 0. \end{cases}$$

Trong bài gốc, hàm mục tiêu của mô hình này được viết là

$$\min z = \sum_{i=1}^N R_i + \sum_{j=1}^N C_j.$$

Vì luôn có $\sum R_i = \sum C_j$, việc tối thiểu hóa biểu thức trên tương đương với tối thiểu hóa tổng phần tử của B , chỉ khác một hệ số hằng.

So với mô hình ban đầu, số biến giảm xuống còn $O(N)$. Tuy nhiên, còn một câu hỏi: có chắc mọi nghiệm của mô hình tổng hàng/tổng cột đều tương ứng với một ma trận B hay không?

Với nhiệm vụ một, do các phần tử của B có thể nhận giá trị tùy ý, nghiệm tương ứng luôn tồn tại.

Với nhiệm vụ hai, xét các ràng buộc hiện tại:

$$\sum_{i=1}^N R_i = \sum_{j=1}^N C_j, \quad R_i \geq 0, \quad C_j \geq 0.$$

Lấy tùy ý một R_i , rồi rút một phần giá trị từ các C_j để đưa vào hàng i . Do $\sum_j C_j \geq R_i$, luôn tồn tại cách rút sao cho sau đó mọi C_j vẫn không âm. Sau khi xử lý xong R_i , bài toán còn lại có dạng ràng buộc không đổi. Vì vậy chắc chắn tồn tại một ma trận không âm B tương ứng với nghiệm của mô hình trên.

4.2.3 Task1

Khi cả hai nhiệm vụ đều có $N \leq 50$, có thể dùng đơn hình hoặc thuật toán tổng quát tương tự để giải trực tiếp quy hoạch tuyến tính.

Với nhiệm vụ hai, mô hình đúng là dạng chuẩn của quy hoạch tuyến tính. Với nhiệm vụ một, cần tách mỗi R_i thành $R_i^a - R_i^b$, trong đó $R_i^a, R_i^b \geq 0$; với C_j làm tương tự. Trên dữ liệu ngẫu nhiên, đơn hình chạy rất tốt, thậm chí có thể nhanh hơn lời giải chính. Nếu hiện thực tốt, ví dụ thêm cắt tĩa bằng danh sách liên kết, cách này cũng có thể qua được dữ liệu của Task3. Độ phức tạp thời gian: không xác định.

4.2.4 Nguyên lý đối ngẫu

Quy hoạch tuyến tính gốc dường như không dễ xử lý tiếp, nên ta thử chuyển sang bài toán đối ngẫu.

Nhiệm vụ một. Có thể dự đoán rằng trị tuyệt đối của các phần tử trong ma trận đáp án không quá lớn. Nếu cộng cho mỗi phần tử một hằng số rất lớn, rồi dùng kết luận của Task0, nhiệm vụ một sẽ được chuyển thành nhiệm vụ hai. Dĩ nhiên làm trực tiếp như vậy chắc chắn sẽ quá thời gian, nên ta dùng thẳng nguyên lý đối ngẫu.

Bài toán đối ngẫu thu được có dạng

$$\begin{cases} Y_{1,1} + Y_{1,2} + \dots + Y_{1,N} + P = 1, \\ Y_{2,1} + Y_{2,2} + \dots + Y_{2,N} + P = 1, \\ \dots \\ Y_{1,1} + Y_{2,1} + \dots + Y_{N,1} - P = 1, \\ Y_{1,2} + Y_{2,2} + \dots + Y_{N,2} - P = 1, \\ \dots \\ Y_{1,N} + Y_{2,N} + \dots + Y_{N,N} - P = 1, \\ Y_{i,j} \geq 0, \end{cases} \quad \max z = 0 \cdot P + \sum_{i=1}^N \sum_{j=1}^N A_{i,j} Y_{i,j}.$$

Nếu không có biến P , đây là mô hình luồng chi phí lớn nhất rất quen thuộc trên đồ thị hai phía. Chú ý rằng mọi ràng buộc đều lấy dấu bằng. Điều này có nghĩa là trong mạng chi phí đó, tổng dung lượng các cạnh rời khỏi S bằng tổng dung lượng các cạnh đi vào T ; từ đó suy ra ngay $P = 0$. Phần còn lại chỉ là một bài toán luồng chi phí đơn giản.

Nhiệm vụ hai, cách 1. Vẫn đối ngẫu theo cách trên, ta nhận được

$$\begin{cases} Y_{1,1} + Y_{1,2} + \dots + Y_{1,N} + P \leq 1, \\ Y_{2,1} + Y_{2,2} + \dots + Y_{2,N} + P \leq 1, \\ \dots \\ Y_{1,1} + Y_{2,1} + \dots + Y_{N,1} - P \leq 1, \\ Y_{1,2} + Y_{2,2} + \dots + Y_{N,2} - P \leq 1, \\ \dots \\ Y_{1,N} + Y_{2,N} + \dots + Y_{N,N} - P \leq 1, \\ Y_{i,j} \geq 0, \end{cases} \quad \max z = 0 \cdot P + \sum_{i=1}^N \sum_{j=1}^N A_{i,j} Y_{i,j}.$$

Vì $Y_{i,j} \geq 0$, có thể thấy $P \in [-1, 1]$. Ý nghĩa của mô hình này thực chất là luồng chi phí lớn nhất trên đồ thị hai phía đầy đủ, chỉ khác ở chỗ ta có thể tăng giới hạn lượng ra của mọi đỉnh ở một phía thêm P , đồng thời giảm giới hạn lượng vào của mọi đỉnh phía còn lại đi P .

Trực giác cho thấy, khi P nằm trong các đoạn $[-1, 0]$ và $[0, 1]$, đáp án là một hàm đơn đỉnh. Tác giả ghi rằng việc chứng minh vẫn còn khó, nên trước hết thử lập bảng giá trị; kết quả cho thấy đúng là hàm đơn đỉnh. Gọi hàm đó là $f(P)$. Nó có một vài tính chất sau:

- Chỉ khi $f'(P) = 0$ mới có nhiều giá trị P cho cùng một giá trị $f(P)$. Vì vậy có thể tam phân hàm này.
- Với hàm đơn đỉnh trên $[-1, 1]$, chỉ cần thực hiện một lần tam phân.

Cách này giải được Task4.

Nhiệm vụ hai, cách 2. Đây là ý tưởng ban đầu của tác giả. Tuy độ phức tạp cao hơn, tư tưởng của nó khá khéo.

Nhận xét rằng đáp án của bài toán gốc có tính đơn điệu rõ ràng theo giá trị cần kiểm tra. Vì thế có thể nhị phân trực tiếp đáp án; giả sử đáp án đang thử là K . Khi đó ràng buộc ban đầu

$$\sum_{i=1}^N R_i - \sum_{j=1}^N C_j = 0$$

được thay bằng

$$\sum_{i=1}^N R_i = K, \quad \sum_{j=1}^N C_j = K.$$

Đối ngẫu tiếp, ta được

$$\begin{cases} Y_{1,1} + Y_{1,2} + \dots + Y_{1,N} + A' \leq 1, \\ Y_{2,1} + Y_{2,2} + \dots + Y_{2,N} + A' \leq 1, \\ \dots \\ Y_{1,1} + Y_{2,1} + \dots + Y_{N,1} + B' \leq 1, \\ Y_{1,2} + Y_{2,2} + \dots + Y_{N,2} + B' \leq 1, \\ \dots \\ Y_{1,N} + Y_{2,N} + \dots + Y_{N,N} + B' \leq 1, \\ Y_{i,j} \geq 0, \end{cases}$$

$$\max z = KA' + KB' + \sum_{i=1}^N \sum_{j=1}^N A_{i,j} Y_{i,j}.$$

Mô hình luồng của quy hoạch tuyến tính này cũng rất rõ: có thể dùng chi phí K để nối giới hạn lưu lượng của mọi đỉnh ở một phía thêm K . Theo tính chất của đối ngẫu, khi và chỉ khi mô hình này có nghiệm cực đại, bài toán gốc không có nghiệm, tức là giá trị K đang nhị phân nhỏ hơn đáp án đúng.

Rõ ràng khi mô hình này đạt nghiệm cực đại thì phải có $A' \rightarrow -\infty$ và $B' \rightarrow -\infty$. Nhưng ta không thể trực tiếp tìm A' và B' trên khoảng $(-\infty, 1]$. Có thể dự đoán rằng khi A' và B' đủ nhỏ, với một A' bất kỳ, cứ giảm A' đi ΔA thì lượng B' cần giảm, ΔB , sẽ tiến tới một hằng số.

Giả sử lúc này đã biết A' và B' . Nếu $\Delta A \geq \Delta B$, đặt $\Delta A = 1$, rồi tìm ΔB tương đối với ΔA . Rõ ràng ΔB nằm trong đoạn $[0, 1]$ và có thể tam phân: nếu ΔB quá nhỏ thì ΔA còn dư; nếu ΔB quá lớn thì ΔA không đủ bù chi phí $-\Delta B \cdot K$. Trường hợp $\Delta A \leq \Delta B$ làm tương tự bằng cách đặt $\Delta B = 1$.

Còn cần xác định A' và B' ban đầu. Vì A'/B' cuối cùng sẽ tiến tới $\Delta A/\Delta B$, chỉ cần thay $\Delta A = 1$ bằng $\Delta A = P_0$, với P_0 đủ lớn. Khi K càng gần đáp án, độ chính xác cần thiết của tỉ số A'/B' càng cao; nếu muốn nhận được chính xác đáp án K , P_0 phải rất lớn. Tuy nhiên đáp án chỉ cần 4 chữ số sau dấu thập phân, nên không cần chính xác đến vậy; P_0 có thể chọn tương đối nhỏ. Qua thử nghiệm, trong tình huống thông thường, lấy $P_0 = N^2$ là đủ để đạt nghiệm tối ưu.

So với thuật toán thứ nhất, cách này có thêm một lớp nhị phân, nên độ phức tạp tăng thêm một thừa số log. Đáng tiếc là nó chỉ vừa đủ qua dữ liệu $N \leq 50$, tức Task3.

4.2.5 Task2

Trong nhiệm vụ hai, với phần lớn dữ liệu ngẫu nhiên, $P = 0$ đã là nghiệm tối ưu. Chỉ có rất ít bộ dữ liệu làm P dao động trong khoảng $[-10^{-4}, 10^{-4}]$, về cơ bản có thể bỏ qua. Do đó có thể xuất đáp án của nhiệm vụ một cho câu hỏi thứ hai.

4.2.6 Khác

Nếu ma trận có kích thước $N \times M$ thì đương nhiên cũng có thể làm tương tự. Khi đó với nhiệm vụ một, P không còn bằng 0, mà bằng

$$P = \frac{M - N}{N + M}.$$

Bài này là một đồ thị hai phía đầy đủ. Nếu dùng SPFA để chạy luồng chi phí thì hiệu quả khá thấp; cần dùng zkw hoặc nguyên thủy-đối ngẫu.

Đơn hình trong nhiều trường hợp cũng có hiệu suất không tệ, gần với zkw luồng chi phí. Nhưng nếu người ra dữ liệu tận dụng đủ tri thức của mình thì vẫn có thể làm nó bị kẹt.

Đồ thị hai phía của nhiệm vụ hai có tính chất đặc biệt nhất định. Nếu có thể bỏ được bước tam phân, hoặc dùng cách khác để tối ưu mỗi lần chạy luồng chi phí, tác giả rất hoan nghênh trao đổi thêm.

4.3 Tổng kết

Bài này có nhiều phiên bản dẫn xuất. Tuy nhiên các phiên bản khác hoặc có đề bài quá phức tạp, hoặc tác giả chưa tìm được thuật toán đủ tốt. Nếu có ý tưởng tốt hơn cho lớp bài này, tác giả rất mong được liên hệ.

Cảm hứng của bài đến từ một bài trước đây của tác giả tên là *Chessboard*. Mô hình ban đầu là đặt quân xe trên bàn cờ, yêu cầu mỗi ô chỉ được đặt nhiều nhất một quân, và ô (i, j) cần được ít nhất $W(i, j)$ quân xe khống chế. Mô hình này tuy mô tả đơn giản, nhưng dường như là NP-khó: nó yêu cầu nghiệm nguyên của một quy hoạch tuyến tính, trong khi nghiệm tối ưu của quy hoạch tuyến tính này hầu như đều là số thực. Sau nhiều lần làm yếu ràng buộc, tác giả cuối cùng thu được bài toán hiện tại.

Bài toán kiểm tra kiến thức cơ bản của thí sinh về quy hoạch tuyến tính và mô hình hóa luồng mạng, đồng thời dùng nguyên lý đối ngẫu. Tính chất của thuật toán thứ nhất rất đẹp, dù phần chứng minh vẫn còn để ngỏ trong bài gốc. Với thuật toán thứ hai, ta lồng nhị phân và tam phân, rồi dùng luồng chi phí để phán định có nghiệm hay không. Tác giả cho rằng đây là một tư tưởng khá hay: chuyển bài toán tối ưu hóa thành bài toán phán định tính khả thi, rồi hiện thực nó trên mạng luồng. Tiếc rằng với bài này, đó không phải thuật toán tốt nhất.

Hy vọng bài toán giúp độc giả có thêm gợi mở và hiểu sâu hơn về quy hoạch tuyến tính cũng như các bài toán luồng mạng.

4.4 Lời cảm ơn

Cảm ơn CCF đã cung cấp nền tảng học tập và trao đổi. Cảm ơn thầy Xiang Qizhong đã hướng dẫn tác giả. Cảm ơn các bạn học đã giúp đỡ tác giả.

5 Đa giác biến đổi

Dong Honghua, Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Bài viết tổng kết các phép biến đổi thường dùng với đa giác, cho thấy tính đa dạng của cách nhìn một đa giác, đồng thời minh họa tác dụng của từng phép biến đổi qua ví dụ. Những biến đổi này cung cấp hướng suy nghĩ cho các bài toán liên quan tới đa giác. Bài viết cũng đi sâu vào một dạng biến đổi để nghiên cứu bài toán thống kê thông tin điểm bên trong đa giác, từ đó thu được một thuật toán có độ phức tạp khá tốt.

5.1 Dẫn nhập

Những năm gần đây, đa giác xuất hiện thường xuyên trong các kỳ chọn đội tuyển cấp tỉnh, các cuộc thi OI, cũng như ACM World Finals. Hình thức xuất hiện của chúng phức tạp và thay đổi đa dạng.

Tác giả tổng kết một số cách xử lý như sau. Phần 2 giới thiệu vài kiến thức cơ bản sẽ dùng về sau. Phần 3 liệt kê nhiều phép biến đổi của đa giác, kèm ví dụ để người đọc dễ hiểu và mở rộng. Phần 4 xuất phát từ GIS, tức hệ thống thông tin địa lý, để dẫn vào bài toán thống kê thông tin điểm bên trong đa giác; bài toán được thảo luận từ trường hợp đặc biệt tới tổng quát, từ dễ tới khó, và cuối cùng cho một thuật toán tổng quát có độ phức tạp tốt.

5.2 Cơ sở

5.2.1 Định nghĩa đa giác

Đa giác. Một hình phẳng khép kín tạo bởi ít nhất ba đoạn thẳng nối đuôi nhau theo thứ tự.

Đa giác đơn. Một đa giác mà các cạnh không kề nhau không giao nhau.

Đa giác lồi. Một đa giác đơn mà mọi góc trong đều không vượt quá 180° .

5.2.2 Tích có hướng

Với hai vector hai chiều (x_1, y_1) , (x_2, y_2) , tích có hướng của chúng là

$$x_1y_2 - x_2y_1.$$

Giá trị này bằng diện tích có hướng của hình bình hành do hai vector tạo thành.

5.2.3 Phương pháp tia

Phương pháp tia dùng để phán định một điểm có nằm trong đa giác hay không. Từ điểm hiện tại, kẻ một tia theo một hướng nào đó rồi đếm số giao điểm giữa đa giác và tia này. Nếu số giao điểm là lẻ thì điểm nằm trong đa giác; nếu là chẵn thì điểm nằm ngoài.

Để tiện tính toán, có thể chọn tia hướng lên trên và hơi lệch sang phải, nhằm tránh trường hợp giao điểm rơi đúng vào đỉnh đa giác. Phương pháp này cũng áp dụng được cho đa giác tự cắt.

Ví dụ 1: Bean. Nguồn: [bzoj 1294] SCOI2009 Bean.

Đại ý đề bài. Cho $D \leq 9$ điểm có trọng số trên một lưới $n \times m$. Hãy tìm một đường đi xuất phát từ một điểm rồi quay lại điểm đó sao cho tổng trọng số các điểm bị bao quanh trừ đi số bước là lớn nhất.

Phân tích. Đường đi tạo thành một đa giác có thể tự cắt. Vì vậy việc phán định mỗi điểm có trọng số có bị bao quanh hay không trở thành bài toán điểm nằm trong đa giác.

Dùng phương pháp tia: với mỗi điểm có trọng số, kẻ tia hướng lên trên; khi đi qua một cạnh giao với tia đó thì trạng thái của điểm được lật. Dùng nhị phân để ghi trạng thái của từng điểm có trọng số, khi đó tổng giá trị điểm có thể suy ra từ trạng thái cuối.

Do đó chỉ cần liệt kê điểm xuất phát, tính đường ngắn nhất để đạt tới từng trạng thái nhị phân rồi quay lại điểm xuất phát. Vì trọng số cạnh đều bằng 1, có thể dùng bfs.

5.3 Các phép biến đổi

5.3.1 Kết hợp với gốc tọa độ

Cạnh là thành phần chính của đa giác. Nếu kết hợp mỗi cạnh với gốc tọa độ, ta thu được một số tam giác.

Theo cách phán định của phương pháp tia, có thể chứng minh diện tích đa giác bằng tổng diện tích có dấu của các tam giác này. Diện tích mỗi tam giác có thể tính trực tiếp bằng tích có hướng. Cần chú ý thứ tự lấy tích có hướng khác nhau sẽ cho dấu diện tích khác nhau; điều này vừa đúng lúc phản ánh chiều đi quanh của đa giác.

Ví dụ 2: Pollution Solution. Nguồn: *World Finals 2013 Problem J.*

Đại ý đề bài. Tính diện tích phần giao giữa một đa giác và một hình tròn.

Phân tích. Diện tích đa giác có thể tách thành một số diện tích có hướng của tam giác, nên bài toán rút gọn thành tính diện tích phần giao giữa tam giác và hình tròn. Nếu tách theo tâm đường tròn, ta bảo đảm một đỉnh của tam giác nằm ở tâm. Sau đó chia bốn trường hợp theo quan hệ vị trí giữa tam giác và đường tròn rồi tính riêng từng trường hợp.

5.3.2 Tách theo trục tọa độ

Tương tự, có thể kết hợp cạnh với trục tọa độ: trừ các cạnh thẳng đứng, mỗi cạnh được chiếu xuống một nửa trục, và cạnh cùng hình chiếu của nó tạo thành một hình thang vuông hoặc hình chữ nhật.

5.3.3 Nửa mặt phẳng

Một đa giác lồi có thể xem là giao của một số nửa mặt phẳng.

Ví dụ 3: Giao đa giác lồi. Nguồn: *[bzoj 2618] CQOI2006.*

Đại ý đề bài. Tính diện tích giao của n đa giác lồi.

Phân tích. Giao của các đa giác lồi chính là giao của toàn bộ các nửa mặt phẳng tương ứng.

Ở đây giới thiệu một thuật toán giao nửa mặt phẳng dùng phương pháp tăng dần, độ phức tạp bậc hai nhưng dễ viết. Với mỗi nửa mặt phẳng mới, tìm các điểm của giao hiện tại nằm bên trong và bên ngoài nửa mặt phẳng đó; xóa các điểm bên ngoài, đồng thời thêm giao điểm giữa nửa mặt phẳng mới và các đoạn nối một điểm trong với một điểm ngoài. Như vậy ta thu được giao mới.

5.3.4 Cây

Mọi đường chéo của đa giác lồi đều nằm bên trong nó, nên một đường chéo có thể chia đa giác lồi thành hai đa giác lồi. Làm đệ quy sẽ thu được một phép tam giác hóa. Các tam giác được tách ra tạo thành một cây.

Đa giác đơn cũng có thể biến thành cây. Từ mọi đỉnh, kéo dài theo phương ngang sang trái và phải cho tới giao điểm đầu tiên; kéo dài thành đường thẳng cũng được. Cách này chia đa giác thành nhiều hình thang, và các hình thang đó tạo thành một cây.

Ví dụ 4: Journey. Nguồn: *[bzoj 2657] ZJOI2012 journey.*

Đại ý đề bài. Cho phép tam giác hóa của một đa giác lồi. Hãy tìm một đường chéo sao cho số tam giác mà nó đi qua là lớn nhất.

Phân tích. Phép tam giác hóa tạo thành cấu trúc cây, nên thử biến đường chéo thành một đường đi trên cây.

Kết luận là: các đường chéo của đa giác lồi có số tam giác đi qua khác 0 tương ứng một-một với mọi đường đi trên cây.

Khi $n = 3$, kết luận hiển nhiên đúng. Giả sử kết luận đúng với $n = k$, ta chứng minh với $n = k + 1$. Xóa một đỉnh của đa giác lồi $k + 1$ cạnh thì thu được đa giác lồi k cạnh; trên cây chỉ cần xét trường hợp thêm một nút lá x , có cha là y . Theo giả thiết quy nạp, các đường chéo cũ tương ứng với các đường đi trong cây cũ. Khi thêm đỉnh bị xóa trở lại, vì y là cha của x , mọi đoạn nối từ đỉnh mới trong vùng x tới các đỉnh khác đều phải đi qua cạnh nối giữa x và y . Vì vậy các đoạn đó tương ứng với mọi đường đi trên cây xuất phát từ x . Kết luận đúng với $n = k + 1$.

Như vậy, tìm đường chéo đi qua nhiều tam giác nhất trở thành tìm đường dài nhất trên cây; có thể giải bằng quy hoạch động trên cây hoặc hai lần bfs.

Ví dụ 5: *Toil for Oil.* Nguồn: *World Finals 2002 Problem F.*

Đại ý đề bài. Cho một đa giác có lỗ trên biên. Hãy tính diện tích phần bên trong đa giác có thể chứa dầu.

Phân tích. Dùng các đường thẳng nằm ngang qua mọi đỉnh để cắt đa giác thành nhiều hình thang. Trong mỗi hình thang, hoặc toàn bộ có dầu, hoặc hoàn toàn không có dầu.

Quét từ dưới lên trên để sinh từng vùng. Có thể dùng đường trung tuyến của dải hiện tại để cắt: nếu nó giao với cạnh đa giác thì thêm biên trên và biên dưới; sắp xếp hai dãy biên này sẽ thu được quan hệ tương ứng. Nếu phía dưới thành phần liên thông hiện tại đã có lỗ, phần phía trên không thể có dầu, nên đánh dấu thành phần liên thông đó là không khả thi. Phần phía trên không ảnh hưởng tới dải hiện tại, nên trực tiếp cộng diện tích khả thi trong dải hiện tại vào đáp án.

Dùng DSU để duy trì tính liên thông của cây, đồng thời dùng các lỗ trên biên dưới trước khi thống kê dải hiện tại và trên biên trên sau khi thống kê để cập nhật trạng thái của thành phần liên thông.

5.4 Loạt *Architect*: thống kê thông tin điểm trong đa giác

5.4.1 Ứng dụng trong GIS

Trong GIS, hệ thống thông tin địa lý, ta thường cần thống kê các giá trị đặc trưng nằm trong một vùng. Vùng thường có thể biểu diễn bằng đa giác, nên các bài toán này trở thành bài toán thống kê thông tin điểm bên trong đa giác.

Trong phần lớn hệ thống GIS, cách làm là dùng phương pháp tia để lần lượt phán định từng điểm có nằm trong đa giác hay không. Thuật toán này có hiệu suất thấp và chưa tận dụng tốt các tính chất của bản đồ.

5.4.2 *Architect*

Đại ý đề bài. Nguồn: *POJ Challenge Round 5, lời mời của Trường Trung học số 1 Thiệu Hưng.*

Hỗ trợ thêm điểm động và hỏi số điểm nằm trong một đa giác đơn mà mọi góc trong đều là bội của 45° . Các điểm nằm ở tâm ô lưới, còn đa giác nằm trên biên ô lưới.

Vết cặn. Dùng phương pháp tia để lần lượt phán định từng điểm có nằm bên trong hay không.

Tách đa giác. Vì đa giác khá phức tạp, ta xét cách tách. Nếu đem phương pháp dùng gốc tọa độ và từng cạnh để tạo tam giác, vốn dùng khi tính diện tích đa giác, áp dụng vào bài này, ta sẽ sinh ra một số cạnh vừa không song song với trục tọa độ, vừa không tạo góc 45° ; điều này phá hỏng điều kiện của đề.

Vì vậy xét một cách tách khác. Trừ cạnh thẳng đứng, mỗi cạnh được chiếu xuống trục x ; cạnh đó và hình chiếu tạo thành hình thang vuông 45° hoặc hình chữ nhật. Số điểm trong toàn bộ các hình thang vuông 45° và hình chữ nhật, cộng trừ theo hướng duyệt, sẽ cho số điểm trong đa giác ban đầu: nếu duyệt theo chiều kim đồng hồ thì cạnh hướng sang phải lấy dấu $+$, cạnh hướng sang trái lấy dấu $-$. Cần chú ý rằng điểm nằm trên biên đa giác cũng được tính, nên hình thang vuông 45° mang dấu $-$ phải dịch xuống một ô.

Chuyển hóa bài toán. Sau khi tách đa giác, bài toán trở thành đếm số điểm trong các hình thang vuông 45° và hình chữ nhật.

Trước hết bỏ qua thao tác thêm điểm động; giả sử mọi điểm đã được thêm xong rồi mới hỏi. Với truy vấn số điểm trong một hình chữ nhật có một cạnh nằm trên trục x , có thể chuyển nó thành: phần đỏ cộng phần xanh, trừ phần xanh. Cả hai phần đều là truy vấn số điểm nằm phía dưới bên trái một điểm truy vấn.

Sau đó dùng thuật toán quét: sắp xếp thao tác thêm điểm và truy vấn theo tọa độ x , dùng tọa độ y làm khóa phụ; khi quét tới một truy vấn, đưa mọi điểm ở bên trái điểm truy vấn vào cấu trúc dữ liệu, chẳng hạn cây Fenwick, rồi theo tọa độ y để hỏi tổng đoạn hoặc tổng tiền tố trong cấu trúc dữ liệu.

Với hình thang vuông 45° , cũng dùng cách tương tự; lúc này khóa y được thay bằng $x + y$.

Chia để trị cdq. Cuối cùng, do đề bài cho thêm điểm và hỏi xen kẽ, còn đóng góp của điểm cho mỗi truy vấn là độc lập, có thể chia để trị theo thời gian, tức cdq. Tất cả sự kiện ban đầu được sắp theo thời gian; chọn trung điểm, chia dãy thành hai nửa, đệ quy trái và phải, rồi tính đóng góp của các điểm bên trái cho các truy vấn bên phải. Vì thời gian của bên trái đều nhỏ hơn bên phải, phần này trở lại trường hợp thêm điểm trước rồi mới hỏi, dùng phương pháp vừa nêu là được.

Độ phức tạp thời gian.

$$O(n \log^2 n).$$

5.4.3 Architekt1

Đại ý đề bài. Nguồn: lần hỏi thứ nhất của kỳ hồ tuyển đội tuyển 2014.

Ban đầu có n điểm trên mặt phẳng, mỗi điểm có trọng số. Có Q thao tác; mỗi thao tác là sửa trọng số điểm hoặc hỏi tổng trọng số các điểm nằm trong một đa giác đơn. Bảo đảm các cạnh của mọi đa giác truy vấn tạo thành một đồ thị phẳng liên thông.

Không có sửa đổi. Trước hết xét trường hợp không có sửa đổi. Tổng trọng số điểm có thể cộng trừ. Tách đa giác theo trục tọa độ: mỗi cạnh và hình chiếu của nó lên trục x tạo thành một hình thang. Ta cần tính tổng trọng số trong hình thang và cộng trừ theo hướng cạnh khi thống kê.

Chuyển sang cách nhìn “điểm đóng góp cho hình thang”. Dùng đường quét để tối ưu: dùng cây cân bằng duy trì các đoạn thẳng còn giao với đường quét; khi gặp một điểm ứng viên, cộng một giá trị cho mọi đoạn nằm phía trên nó. Cuối cùng thống kê tổng trọng số trên mọi đoạn và cộng trừ theo hướng.

Độ phức tạp thời gian.

$$O(S \log S),$$

trong đó S là tổng quy mô cạnh cần xử lý.

Chia để trị cdq. Nếu dùng thuật toán trên mà cần tăng hoặc giảm trọng số điểm thì làm thế nào?

Phần cộng trừ vẫn có thể tách được, nên có thể lồng cdq: biến tình huống sửa và hỏi xen kẽ thành sửa trước rồi hỏi sau. Cụ thể, với dãy hiện tại, chọn trung điểm, tính ảnh hưởng của các sửa đổi bên trái lên các truy vấn bên phải; phần này chính là trường hợp sửa trước hỏi sau, quay về bài toán trên. Sau đó đệ quy xử lý riêng nửa trái và nửa phải.

Độ phức tạp thời gian.

$$O(S \log S \log Q).$$

5.4.4 Archtext2

Đại ý đề bài. Nguồn: kỳ hồ tuyển đội tuyển 2014.

Trên cơ sở bài trước, ngoài tổng trọng số điểm còn phải hỏi trọng số điểm lớn nhất.

Chuyển hóa bài toán. Vì đã cho đồ thị phẳng, ta thử tìm cấu trúc trên đồ thị đối ngẫu. Trong đồ thị đối ngẫu, mỗi đa giác truy vấn bao quanh một thành phần liên thông, và thành phần đó cũng là liên thông trên đồ thị đối ngẫu. Mỗi cạnh của đa giác chính là một cạnh trên đồ thị đối ngẫu. Vì vậy bài toán chuyển thành: xóa một số cạnh rồi hỏi thông tin trong một thành phần liên thông. Khi đa giác suy biến thành đoạn thẳng thì bên trong không có miền nào; trường hợp này có thể đặc biệt xử lý trực tiếp.

Tiếp theo dẫn vào bài toán đồ thị động.

Bài toán đồ thị động. Mô tả ngắn gọn: nhiều lần hỏi thông tin của một thành phần liên thông sau khi xóa một số cạnh.

Dùng cdq. Tại mọi thời điểm, bảo đảm mọi cạnh không xuất hiện trong bất kỳ truy vấn nào của đoạn hiện tại đều đã được gộp. Đồng thời ghi số lần mỗi cạnh xuất hiện trong các truy vấn của đoạn.

Khi đệ quy sang nửa trái, thêm vào đồ thị các cạnh xuất hiện ở truy vấn nửa phải nhưng không xuất hiện ở truy vấn nửa trái, rồi dùng DSU để duy trì thông tin trong thành phần liên thông. Sau khi đệ quy trở về, cần khôi phục DSU về trạng thái trước khi đệ quy, tức xóa các cạnh vừa thêm do nửa trái.

Vì những cạnh bị xóa là các cạnh được thêm sau cùng, chỉ cần thao tác trên ngăn xếp thêm cạnh: hoàn tác thao tác gán cha trong DSU và khôi phục thông tin được duy trì. Khi đệ quy sang nửa phải cũng làm tương tự và khi quay lại cũng khôi phục.

Khi đoạn chỉ còn một truy vấn, trực tiếp lấy thông tin của thành phần liên thông chứa nó.

Tương ứng thông tin. Khó khăn còn lại là đưa thông tin điểm trong đồ thị phẳng ban đầu lên các đỉnh của đồ thị đối ngẫu, tức các miền của đồ thị phẳng.

Một truy vấn là phần bên trong của một đa giác. Cần tìm ít nhất một miền nằm bên trong, chẳng hạn miền ở bên phải một cạnh khi duyệt đa giác theo chiều kim đồng hồ, rồi hỏi thông tin của thành phần liên thông chứa miền đó.

Không có sửa đổi. Khi tính max, một điểm có thể bị tính lặp nhiều lần. Nếu không có sửa đổi, cứ lấy trọng số lớn nhất của mọi điểm thuộc một miền làm trọng số của miền đó, rồi áp dụng thuật toán đồ thị động để hỏi trọng số điểm lớn nhất trong một thành phần liên thông.

Độ phức tạp thời gian.

$$O(S \log S \log n).$$

Số điểm. Xét tiếp trường hợp đặc biệt mọi trọng số điểm đều bằng 1, tức cần hỏi số điểm bên trong. Có thể dùng công thức Euler trên đồ thị phẳng liên thông:

$$\text{số điểm} = \text{số cạnh} - \text{số miền} - 1.$$

Số miền và số cạnh trong một thành phần liên thông có thể tính tương đối dễ, từ đó suy ra số điểm.

Độ phức tạp thời gian.

$$O(S \log S \log n).$$

Trường hợp tổng quát. Với trường hợp tổng quát, ta cần ánh xạ mỗi điểm ban đầu vào một miền nào đó.

Tiếp tục phân tích truy vấn, có thể thấy lân cận của mọi điểm nằm bên trong đa giác cũng nằm trong thành phần liên thông tương ứng trên đồ thị đối ngẫu. Vì vậy, với điểm bên trong, chọn tùy ý một miền làm miền tương ứng là được.

Điểm trên biên không thỏa tính chất này, nhưng các điểm đó có thể lấy trực tiếp từ thông tin của truy vấn. Ngoài ra cần cưỡng chế rằng điểm trên biên không được thêm vào miền khi xử lý truy vấn; trong cdq, phải ghi thêm thông tin về số lần một điểm xuất hiện. Cụ thể, khi đệ quy nửa trái, cần đưa các điểm xuất hiện ở nửa phải nhưng không xuất hiện ở nửa trái vào miền tương ứng của chúng.

Độ phức tạp thời gian.

$$O(S \log S \log n).$$

Thêm sửa đổi. Khi có thao tác sửa đổi, do ta đã có thông tin về số thao tác mà một điểm liên quan tới, chỉ cần tính cả sửa đổi vào đó. Khi đệ quy tới đoạn chỉ còn thao tác sửa đổi thì thực hiện sửa đổi.

Độ phức tạp thời gian.

$$O(S \log S \log n).$$

5.4.5 ArchiEXT

Đại ý đề bài. Trên cơ sở bài trước, không cho trước đồ thị phẳng liên thông tạo bởi toàn bộ đa giác truy vấn.

Tự lực xây dựng. Không cho đồ thị phẳng nhưng vẫn muốn dùng thuật toán trên, nên chỉ còn cách tự xây dựng đồ thị phẳng. Dùng đường quét để tìm mọi giao điểm giữa các cạnh của các đa giác; các giao điểm này cùng với các cạnh đa giác tạo thành đồ thị phẳng.

Bước này có độ phức tạp

$$O(\text{số giao điểm} \cdot \log S).$$

Không liên thông. Cần chú ý đồ thị phẳng thu được bằng cách này không nhất thiết liên thông. Vì vậy phải làm cho nó liên thông. Áp dụng phương pháp tác giả trình bày trong buổi trao đổi học viên WC2014, có thể chuyển nó thành đồ thị phẳng liên thông. Sau khi có đồ thị phẳng liên thông, bài toán trở thành bài trước.

Tối ưu. Điểm nghẽn của độ phức tạp là bước tìm toàn bộ giao điểm, tức $O(\text{số giao điểm})$. Tìm hết mọi giao điểm vừa không hợp lý lắm, vừa quá lãng phí.

Xét chia khối: cứ mỗi T truy vấn tạo thành một khối. Trong khối này cần dựng đồ thị phẳng; những điểm còn lại dùng định vị điểm bằng đường quét để đưa vào một miền. Khi đó dựng đồ thị phẳng tốn

$$O\left(\frac{S}{T}T^2 \log S\right),$$

định vị các điểm còn lại tốn

$$O\left(\frac{S}{T}S \log S\right),$$

và giải T truy vấn tốn

$$O\left(\frac{S}{T}T \log^2 T\right).$$

Tổng hợp lại, chọn $T = \sqrt{S}$ là tối ưu, cho độ phức tạp cuối cùng

$$O(S^{1.5} \log S).$$

Vẫn còn một vấn đề: nếu một đa giác truy vấn có quá nhiều cạnh, làm đây không thể chia theo $\sqrt{S}$ thì sao? Chú ý số đa giác như vậy không nhiều, và bên trong chỉ có một miền, nên có thể dùng trực tiếp định vị điểm để phán định, với tổng độ phức tạp $O((nQ + S) \log S)$. Lấy riêng mọi truy vấn có số cạnh lớn hơn $\sqrt{S}$, số lượng không vượt quá $\sqrt{S}$, rồi giải từng truy vấn độc lập; như vậy vẫn bảo đảm độ phức tạp $O(S^{1.5} \log S)$.

5.4.6 Tiểu kết

Khi các thuật toán thường dùng gặp khó ở truy vấn max, tác giả chọn một hướng khác: ghép nhiều đa giác thành một đồ thị phẳng rồi chuyển bài toán thành bài toán đồ thị.

Trong bài toán này còn nhiều lần dùng chia để trị theo thời gian, tức cdq, để biến phức tạp thành đơn giản. Ở cuối, bài viết dùng thêm tư tưởng chia khối và phân loại trường hợp để giảm độ phức tạp xấu nhất, từ đó thu được một thuật toán khá tốt.

5.5 Tổng kết

Mặc dù số cạnh của đa giác không cố định và hình dạng phức tạp, gây nhiều khó khăn khi giải bài, nhưng thông qua những cách tách khéo léo, đa giác có thể biến thành một số phần nhỏ hơn, tạo điều kiện để xử lý từng phần.

Sau khi phân tích đặc điểm của đa giác trong đề, áp dụng các phương pháp tách thường dùng thường đem lại hiệu quả tốt.

5.6 Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Cảm ơn các thầy Chen Heli, Shao Hongxiang, You Guanghui và Dong Yehua của Trường Trung học số 1 Thiệu Hưng đã quan tâm và hướng dẫn tác giả trong nhiều năm. Cảm ơn huấn luyện viên đội tuyển quốc gia Hu Weidong và Yu Linyun đã hướng dẫn. Cảm ơn các anh chị Xu Jie, Zhong Yiqin, Fei Jiezhong, Liang Jiawen của Đại học Thanh Hoa và Jiang Youzhi của Đại học Giao thông Thượng Hải đã giúp đỡ tác giả. Cảm ơn các bạn He Qizheng, Zheng Zhongyi, Xu Zhengjie của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở. Cảm ơn các bạn Yu Yili, Zhang Hengjie, Wang Jianhao, Guo Yu và nhiều bạn học khác của Trường Trung học số 1 Thiệu Hưng đã giúp đỡ và gợi mở. Cảm ơn tất cả thầy cô và bạn bè từng giúp đỡ tác giả. Cảm ơn cha mẹ và gia đình đã ủng hộ, quan tâm và chăm sóc chu đáo.

Tài liệu tham khảo

1. SUN, “Bàn về một lớp thuật toán chia để trị”, bài giảng WC2013.
2. RRR, “Bàn về vài cách giải phi kinh điển cho bài cấu trúc dữ liệu”, luận văn đội tuyển quốc gia 2013.
3. HA, “Ứng dụng đường quét trong hình học tính toán”, trao đổi học viên WC2014.
4. 246, “Báo cáo ra đề *Architext*”, bài tập đội tuyển quốc gia 2014.

6 Bước đầu nghiên cứu thuật toán liên quan tới nhóm hoán vị

Cen Ruoxu, Trường Trung học Trần Hải, Chiết Giang

Tóm tắt

Nhóm hoán vị là một mô hình thường gặp trong các kỳ thi tin học. Bài viết này giới thiệu những kiến thức cơ bản về nhóm hoán vị, rồi xuất phát từ bài toán tổng quát là xác định một phần tử có thuộc nhóm hoán vị hay không để nghiên cứu sơ bộ thuật toán Schreier–Sims.

6.1 Kiến thức chuẩn bị

6.1.1 Nhóm

Giả sử G là một tập không rỗng, còn $\circ$ là một phép toán hai ngôi được định nghĩa trên G . Nếu thỏa mãn các điều kiện sau:

- tính đóng: $\forall a, b \in G, a \circ b \in G$;
- tính kết hợp: $\forall a, b, c \in G, (a \circ b) \circ c = a \circ (b \circ c)$;
- phần tử đơn vị: $\exists e \in G, \forall a \in G, e \circ a = a \circ e = a$;
- phần tử nghịch đảo: $\forall a \in G, \exists a^{-1} \in G, aa^{-1} = a^{-1}a = e$,

thì $(G, \circ)$ được gọi là một nhóm; viết gọn là G là nhóm. Từ đây trở đi, ta viết $a \circ b$ thành ab . Số phần tử trong một nhóm được gọi là bậc của nhóm; theo đó có nhóm hữu hạn và nhóm vô hạn. Nhóm là sự trừu tượng hóa của nhiều hệ đại số. Chẳng hạn, toàn bộ các số nguyên đối với phép cộng, hoặc các số nguyên trong $[1, n]$ nguyên tố cùng nhau với n đối với phép nhân modulo n , đều tạo thành nhóm. Cần chú ý rằng nhóm không nhất thiết thỏa mãn tính giao hoán.

Nếu $(G, \circ)$ là nhóm, H là tập con của G , và $(H, \circ)$ cũng là nhóm, thì H được gọi là nhóm con của G , ký hiệu $H \leq G$. Mọi nhóm G đều có hai nhóm con tầm thường là G và $\{e\}$; các nhóm con khác được gọi là nhóm con thực sự.

Với một tập con M của G , giao của tất cả các nhóm con chứa M cũng là một nhóm con; nhóm này được gọi là nhóm con sinh bởi M , ký hiệu $\langle M \rangle$. Có thể hiểu nhóm con sinh là nhóm thu được bằng cách lấy các phần tử trong M và thực hiện phép toán tùy ý nhiều lần. Khi đó M được gọi là một tập sinh của $\langle M \rangle$.

Ví dụ, xét nhóm đơn giản $G = \{0, 1, 2, 3, 4, 5\}$, phép toán là cộng modulo 6, phần tử đơn vị là 0. Khi đó $H_1 = \{0, 2, 4\}$ và $H_2 = \{0, 3\}$ là các nhóm con, trong đó H_1 cũng là nhóm con sinh bởi $\{2\}$.

6.1.2 Lớp kề

Nếu H là nhóm con của G , với mọi $a \in G$, gọi

$$aH = \{ah \mid h \in H\}$$

là một lớp kề trái của H , và gọi

$$Ha = \{ha \mid h \in H\}$$

là một lớp kề phải của H .

Trong G , số lớp kề trái của H bằng số lớp kề phải của H . Thật vậy, ánh xạ $Ha \mapsto a^{-1}H$ là một song ánh. Dưới đây ta chủ yếu xét lớp kề phải.

Trong ví dụ trên,

$$H_1 + 0 = H_1 + 2 = H_1 + 4 = \{0, 2, 4\},$$

$$H_1 + 1 = H_1 + 3 = H_1 + 5 = \{1, 3, 5\}.$$

Với H_2 , ta có $H_2 + 0 = H_2 + 3 = \{0, 3\}$, $H_2 + 1 = H_2 + 4 = \{1, 4\}$, và $H_2 + 2 = H_2 + 5 = \{2, 5\}$. Từ ví dụ này có thể quan sát được định lý sau.

Định lý 1. Nếu H là nhóm con của G , thì với mọi hai lớp kề phải Ha, Hb của H , hoặc $Ha = Hb$, hoặc $Ha \cap Hb = \emptyset$.

Chứng minh. Nếu tồn tại $x \in Ha \cap Hb$, thì tồn tại $h_1, h_2 \in H$ sao cho $x = h_1a = h_2b$. Do đó

$$\forall ha \in Ha, \quad ha = hh_1^{-1}h_1a = hh_1^{-1}h_2b = h'b \in Hb,$$

với $h' = hh_1^{-1}h_2 \in H$. Suy ra $Ha \subseteq Hb$; tương tự $Hb \subseteq Ha$. Vậy $Ha = Hb$. Định lý được chứng minh.

Định lý này cho thấy nhóm G có thể được chia thành hợp của các lớp kề phải đôi một rời nhau, từ đó cho ta một cách phân tích và giản hóa cấu trúc của nhóm.

Ký hiệu $|G : H|$ là số lớp kề phải phân biệt của nhóm con H trong G . Dùng 1 để chỉ nhóm con chỉ chứa phần tử đơn vị, thì $|G : 1|$ chính là bậc của G . Ta lập tức có định lý sau.

Định lý 2 (Lagrange). Nếu H là nhóm con của nhóm hữu hạn G , thì

$$|G : 1| = |G : H| |H : 1|.$$

6.1.3 Hoán vị

Một song ánh từ một tập hữu hạn Ω đến chính Ω được gọi là một hoán vị của Ω . Không mất tính tổng quát, coi $\Omega = \{1, 2, \dots, n\}$. Khi đó một hoán vị có thể biểu diễn dưới dạng

$$\begin{pmatrix} 1 & 2 & \cdots & n \\ a_1 & a_2 & \cdots & a_n \end{pmatrix},$$

trong đó $(a_1, a_2, \dots, a_n)$ là một hoán vị của dãy $1, 2, \dots, n$. Phép toán giữa các hoán vị là hợp thành:

$$\begin{pmatrix} 1 & 2 & \cdots & n \\ a_1 & a_2 & \cdots & a_n \end{pmatrix} \begin{pmatrix} 1 & 2 & \cdots & n \\ b_1 & b_2 & \cdots & b_n \end{pmatrix} = \begin{pmatrix} a_1 & a_2 & \cdots & a_n \\ b_1 & b_2 & \cdots & b_n \end{pmatrix}.$$

Trên n phần tử có $n!$ hoán vị. Để kiểm tra rằng $n!$ hoán vị này tạo thành một nhóm đối với phép hợp thành hoán vị, gọi là nhóm đối xứng bậc n , ký hiệu S_n . Nhóm hoán vị là nhóm con của S_n ; các phần tử của nó là các hoán vị.

Ảnh của phần tử β qua hoán vị g được ký hiệu là β^g . Tập các ảnh của β dưới mọi hoán vị trong nhóm hoán vị G được gọi là quỹ đạo của β , ký hiệu β^G . Trong nhóm hoán vị G , các hoán vị không làm thay đổi tập phần tử $A = \{a_1, \dots, a_m\}$ tạo thành nhóm con ổn định của A , ký hiệu G_A hoặc $G_{a_1, \dots, a_m}$.

6.1.4 Ví dụ 1: *Shift it!*

Tóm tắt đề bài. Có một lưới $n \times n$. Trên các ô lần lượt viết các số từ 1 đến n^2 , mỗi số xuất hiện đúng một lần. Mỗi thao tác có thể dịch vòng một hàng sang trái hoặc sang phải một ô, hoặc dịch vòng một cột lên trên hoặc xuống dưới một ô. Hãy xác định có thể dùng các thao tác đó để biến lưới ban đầu thành lưới đích hay không; nếu có thì hãy tìm một phương án bất kỳ không quá 10000 bước.

Quy mô dữ liệu. $n \leq 6$.

Phân tích. Tuy quy mô dữ liệu có vẻ rất nhỏ, số lượng dãy thao tác lại rất lớn nên khó tìm kiếm trực tiếp. Nhìn theo quan điểm nhóm hoán vị, ta thấy mỗi loại thao tác có thể biểu diễn thành một hoán vị trên n^2 số. Tập hợp tất cả thao tác có thể thực hiện chính là nhóm con G của S_{n^2} sinh bởi những hoán vị đó. Câu hỏi trở thành: hoán vị h biến lưới ban đầu thành lưới đích có thuộc G hay không.

Khi n là số chẵn, ta có thể dựng một phương án hoán đổi hai số kề nhau bất kỳ. Chẳng hạn, để hoán đổi 1 và 2, lần lượt thực hiện một chuỗi dịch vòng trên hàng đầu và cột đầu: lên, trái, xuống, phải, lên. Hiệu quả của chuỗi thao tác này là hoán đổi 1 và 2, đồng thời các số còn lại trong cột đầu bị dịch vòng lên một ô. Lặp lại $n - 1$ lần thì cuối cùng chỉ còn tác dụng hoán đổi 1 và 2.

Vì có thể hoán đổi hai số kề nhau bất kỳ, hiển nhiên mọi lưới đích đều đạt được: ta chỉ cần lần lượt đưa từng số về vị trí cần đến bằng các phép hoán đổi. Kết hợp thêm vài chiến lược tham lam đơn giản có thể giảm số bước xuống trong giới hạn 10000.

Điểm thú vị là khi n lẻ thì không phải lúc nào cũng có lời giải. Ta biết một hoán vị có thể được tách thành tích của một số chu trình. Chu trình $(a_1, a_2, \dots, a_m)$ biểu thị a_1 chuyển thành a_2 , a_2 chuyển thành a_3 , ..., và a_m chuyển thành a_1 . Chu trình độ dài 2 được gọi là một phép đổi chỗ; mọi chu trình lại có thể tách thành tích của các phép đổi chỗ. Cách biểu diễn một hoán vị thành tích các phép đổi chỗ không duy nhất, nhưng tính chẵn lẻ là bất biến, từ đó chia hoán vị thành hoán vị lẻ và hoán vị chẵn. Khi n lẻ, mọi thao tác đều là hoán vị chẵn, nên không thể sinh ra hoán vị lẻ.

6.2 Thuật toán Schreier–Sims

6.2.1 Bài toán tổng quát về tính giải được của trò chơi hoán vị

Trong ví dụ 1, tuy ta thu được một kết luận đẹp, lời giải đã dùng một cấu trúc khó nghĩ ra và các tính chất rất đặc thù. Dưới đây ta xét bài toán tổng quát hơn: thực hiện thao tác trên n số, mỗi thao tác là một hoán vị, tập thao tác là S ; hỏi có thể biến một trạng thái thành một trạng thái khác hay không. Nói cách khác, cho $S \subseteq S_n$ và $h \in S_n$, cần xác định $h \in \langle S \rangle$ hay không. Dưới đây giả sử tập phần tử là $\Omega = \{1, 2, \dots, n\}$, và $G = \langle S \rangle \subseteq S_n$.

6.2.2 Cơ sở và tập sinh mạnh

Cơ sở của G là một dãy phần tử của Ω ,

$$B = (\beta_1, \dots, \beta_m),$$

thỏa $G_B = 1$, tức là trong G , hoán vị giữ nguyên mọi phần tử của B chỉ có thể là phần tử đơn vị. B định nghĩa một chuỗi nhóm con

$$G = G^{[1]} \geq G^{[2]} \geq \dots \geq G^{[m]} \geq G^{[m+1]} = 1. \quad (1)$$

Trong đó

$$G^{[i]} = G_{\beta_1, \dots, \beta_{i-1}},$$

tức là nhóm con gồm các hoán vị giữ nguyên $\{\beta_1, \dots, \beta_{i-1}\}$. Nếu với mọi $1 \leq i \leq m$ đều có $G^{[i+1]} \neq G^{[i]}$, thì cơ sở này được gọi là không dư thừa. Các cơ sở không dư thừa khác nhau có thể có độ dài khác nhau.

Một tập sinh T của nhóm G được gọi là tập sinh mạnh của G đối với cơ sở B , nếu

$$\forall 1 \leq i \leq m+1, \quad \langle T \cap G^{[i]} \rangle = G^{[i]}. \quad (2)$$

Nói cách khác, tập sinh mạnh phải chứa các tập sinh của tất cả nhóm con trong (1).

Ví dụ, xét $G = S_4$, dãy $B = (1, 2, 3)$ là một cơ sở không dư thừa của G . Ở đây $G^{[1]}, G^{[2]}, G^{[3]}$ lần lượt là tập tất cả hoán vị trên $\{1, 2, 3, 4\}$, trên $\{2, 3, 4\}$, và trên $\{3, 4\}$; $G^{[4]} = 1$, thỏa (1). Hai tập

$$T_1 = \{(1, 2, 3, 4), (3, 4)\}$$

và

$$T_2 = \{(1, 2, 3, 4), (2, 3, 4), (3, 4)\}$$

đều là tập sinh của G , nhưng T_1 không phải là tập sinh mạnh đối với B , vì $\langle T_1 \cap G^{[2]} \rangle \neq G^{[2]}$. Còn T_2 là một tập sinh mạnh đối với B .

6.2.3 Quá trình phân rã

Giả sử đã tìm được cơ sở B và tập sinh mạnh T . Ta sẽ dùng chúng để xác định h có thuộc G hay không.

Dùng các lớp kề của $G^{[i+1]}$ để chia $G^{[i]}$. Trong $G^{[i]}$, β_i có thể biến thành các phần tử thuộc quỹ đạo $\beta_i^{G^{[i]}}$, còn $G^{[i+1]}$ là nhóm ổn định của β_i . Do đó mỗi phần tử trong $\beta_i^{G^{[i]}}$ tương ứng với một lớp kề của $G^{[i+1]}$. Với mỗi lớp kề, ta chọn một phần tử đại diện r để biểu diễn lớp kề $G^{[i+1]}r$. Tập các phần tử đại diện này ký hiệu là R_i .

Có thể tìm R_i bằng BFS: lấy phần tử trong quỹ đạo làm đỉnh, lấy các hoán vị trong $T \cap G^{[i]}$ làm cạnh, và bắt đầu BFS từ β_i . Nếu đi được đến γ , điều đó chứng tỏ $\gamma \in \beta_i^{G^{[i]}}$. Nhân lần lượt các hoán vị trên đường đi từ β_i đến γ , ta thu được phần tử đại diện của lớp kề biến β_i thành γ .

Tiếp tục ví dụ trên, $G = S_4$, $B = (1, 2, 3)$, và

$$T = \{(1, 2, 3, 4), (2, 3, 4), (3, 4)\}.$$

Ta có $G^{[3]} = \{e, (3, 4)\}$. Nhóm này chia $G^{[2]}$ thành ba lớp kề:

$$G^{[3]}e = \{e, (3, 4)\},$$

$$G^{[3]}(2, 3) = \{(2, 3), (2, 3, 4)\},$$

$$G^{[3]}(2, 4) = \{(2, 4), (2, 3, 4)\}.$$

Do đó $R_2 = \{e, (2, 3), (2, 4)\}$. Các hoán vị trong ba lớp kề này lần lượt biến $\beta_2 = 2$ thành 2, 3, 4.

Bây giờ, mọi $g \in G$ có thể biểu diễn duy nhất dưới dạng

$$g = r_m r_{m-1} \dots r_1, \quad r_i \in R_i.$$

Quá trình phân rã rất đơn giản: với $g \in G$, trước hết tìm phần tử đại diện lớp kề $r_1 \in R_1$ thỏa $\beta_1^g = \beta_1^{r_1}$; sau đó đặt $g_2 = gr_1^{-1} \in G^{[2]}$. Tiếp tục tìm $r_2 \in R_2$ thỏa $\beta_2^{g_2} = \beta_2^{r_2}$, đặt $g_3 = g_2r_2^{-1}$, rồi lặp lại tương tự.

Quá trình phân rã có thể xác định h có thuộc G hay không. Ta thử phân rã h bằng phương pháp trên; nếu phân rã thành công thì hiển nhiên $h \in G$, ngược lại $h \notin G$. Có hai kiểu thất bại: hoặc trong quá trình phân rã, hoán vị

$$h_i = hr_1^{-1}r_2^{-1} \cdots r_{i-1}^{-1}$$

đưa β_i ra ngoài $\beta_i^{G^{[i]}}$, khiến ta không thể tiếp tục; hoán vị h_i này được gọi là hoán vị dư. Hoặc ở cuối quá trình, $h_{m+1} \neq e$; khi đó h_{m+1} cũng được gọi là một hoán vị dư.

6.2.4 Bổ đề Schreier

Nếu R là tập các phần tử đại diện lớp kề của H trong G , thì với mọi $g \in G$, $Hg \cap R$ chỉ có đúng một phần tử; ký hiệu phần tử đó là $\bar{g}$.

Định lý 3. Giả sử $H \leq G = \langle S \rangle$, R là tập các phần tử đại diện lớp kề của H trong G , và $e \in R$. Khi đó tập

$$T = \{rs\bar{r}s^{-1} \mid r \in R, s \in S\}$$

là một tập sinh của H .

Chứng minh. Theo định nghĩa, mọi phần tử trong T đều thuộc H , nên $\langle T \rangle \subseteq H$.

Lấy tùy ý $h \in H \leq G$. Viết $h = s_1s_2 \cdots s_k$, trong đó $s_i \in S$. Ta định nghĩa một dãy phần tử $h_0, h_1, \dots, h_k$ của G sao cho

$$h_j = t_1t_2 \cdots t_jr_{j+1}s_{j+1}s_{j+2} \cdots s_k. \quad (3)$$

Trong đó $t_i \in T$, $r_{j+1} \in R$, và $h_j = h$. Ban đầu đặt $h_0 = es_1s_2 \cdots s_k = h$. Nếu h_j đã được xác định, đặt

$$t_{j+1} = r_{j+1}s_{j+1}\overline{r_{j+1}s_{j+1}}^{-1}, \quad r_{j+2} = \overline{r_{j+1}s_{j+1}}.$$

Rõ ràng $h_{j+1} = h_j = h$, đúng theo dạng yêu cầu của (3). Ta có

$$h = h_k = t_1t_2 \cdots t_kr_{k+1}.$$

Do $h \in H$ và $t_1t_2 \cdots t_k \in \langle T \rangle \leq H$, suy ra $r_{k+1} \in H \cap R = 1$. Vì vậy $h \in \langle T \rangle$.

Tóm lại, $H \subseteq \langle T \rangle$, nên $\langle T \rangle = H$.

6.2.5 Quy trình thuật toán

Ta liên tục điều chỉnh và duy trì dãy phần tử

$$B = (\beta_1, \beta_2, \dots, \beta_m)$$

cùng dãy tập sinh $T_1, T_2, \dots, T_{m+1}$, bảo đảm T_i là tập ổn định của $\{\beta_1, \dots, \beta_{i-1}\}$, đồng thời $\langle T_i \rangle \geq \langle T_{i+1} \rangle$ với $1 \leq i \leq m$, $\langle T_1 \rangle = G$, và $T_{m+1} = 1$. Ta còn duy trì tập đại diện lớp kề R_i của $\langle T_i \rangle_{\beta_i}$ trong $\langle T_i \rangle$. Dùng một con trỏ cur , bảo đảm rằng khi $i > cur$ thì

$$\langle T_i \rangle_{\beta_i} = \langle T_{i+1} \rangle. \quad (4)$$

Khi đó $(\beta_{cur+1}, \dots, \beta_m)$ là một cơ sở của $\langle T_{cur} \rangle$, và

$$\bigcup_{cur+1 \leq j \leq m} T_j$$

là tập sinh mạnh của $\langle T_{cur} \rangle$.

1. Ban đầu đặt $m = 1$, chọn β_1 là một phần tử bất kỳ bị một hoán vị trong S làm thay đổi, đặt $T_1 = S$, $cur = 1$, rồi dùng BFS để tìm R_1 .
2. Mỗi lần xét xem (4) có đúng tại $i = cur$ hay không. Theo điều đã bảo đảm, $\langle T_{cur} \rangle_{\beta_{cur}} \supseteq \langle T_{cur+1} \rangle$, nên chỉ cần kiểm tra chiều ngược lại. Dùng định lý 3 để tìm tập sinh T' của $\langle T_{cur} \rangle_{\beta_{cur}}$, rồi dùng quá trình phân rã để xác định mọi phần tử của T' có thuộc $\langle T_{cur+1} \rangle$ hay không.
Nếu (4) đúng, giảm cur đi 1. Nếu không đúng, có một phần tử $h \in T'$ không thuộc $\langle T_{cur+1} \rangle$. Đưa hoán vị dư thu được khi phân rã h vào T_{cur+1} . Nếu $cur = m$, chọn một phần tử bất kỳ bị hoán vị dư này làm thay đổi làm β_{m+1} , rồi chạy lại BFS để cập nhật R_{cur+1} . Lúc này (4) tại $i = cur + 1$ chưa chắc đúng, nên tăng cur thêm 1.
3. Lặp lại quá trình trên đến khi $cur = 0$. Khi đó ta thu được cơ sở B của G và tập sinh mạnh

$$T = \bigcup_{1 \leq j \leq m} T_j.$$

6.2.6 Phân tích độ phức tạp

Kích thước của cơ sở nhiều nhất là n . Mỗi T_i nhiều nhất thay đổi n lần, vì sau mỗi lần thay đổi, quỹ đạo của β_i sẽ tăng thêm một phần tử. Vì vậy bước 2 được thực hiện $O(n^2)$ lần. Mỗi lần, kích thước tập sinh mạnh của $\langle T_{cur} \rangle_{\beta_{cur}}$ là

$$O(|R_{cur}| |T_{cur}|) = O(n^2),$$

và phân rã từng phần tử trong đó cần $O(n^2)$. Từ đó thu được cận trên thời gian $O(n^6)$.

Chú ý rằng nếu đã biết một phần tử nào đó của tập sinh mạnh của $\langle T_{cur} \rangle_{\beta_{cur}}$ thuộc $\langle T_{cur+1} \rangle$, về sau không cần phân rã nó lại nữa. Do đó, với mỗi $1 \leq cur \leq m$, chỉ cần thực hiện

$$O(|R_{cur}| |T_{cur}|) = O(n^2)$$

lần phân rã. Tổng độ phức tạp thời gian là $O(n^5)$. Nếu kích thước nhóm là $|G|$, độ phức tạp thời gian cũng có thể viết là $O(n^2 \log^3 |G|)$.

Trong thực tế, hằng số của thuật toán rất nhỏ, thường không cần đến nhiều thao tác như vậy. Chương trình của tác giả có thể chạy dữ liệu $n = 100$ trong 2 giây.

Độ phức tạp bộ nhớ là $O(n^3)$.

6.2.7 Ví dụ 2: Group

Tóm tắt đề bài. Cho một tập S gồm các hoán vị độ dài N , $|S| = M$. Hãy tìm bậc nhỏ nhất của một tập G thỏa các điều kiện:

$$S \subset G,$$

và

$$\forall \sigma, \tau \in G, \quad \sigma \tau^{-1} \in G.$$

Quy mô dữ liệu.

$$N \leq 50, \quad M \leq 100.$$

Phân tích. Trong G , tính kết hợp và phần tử nghịch đảo hiển nhiên đúng theo định nghĩa, tính đóng cũng đúng, và phần tử đơn vị $e = \sigma \sigma^{-1} \in G$. Do đó G là một nhóm. Nhóm nhỏ nhất chứa S chính là nhóm do S sinh ra. Vì vậy $G = \langle S \rangle$, và thứ cần tìm là $|\langle S \rangle|$.

Dùng thuật toán Schreier–Sims để tìm cơ sở B của $\langle S \rangle$. Ta biết B định nghĩa một chuỗi nhóm con

$$G = G^{[1]} \geq G^{[2]} \geq \dots \geq G^{[m]} \geq G^{[m+1]} = 1.$$

Theo định lý Lagrange,

$$|G| = \prod_{i=1}^m |G^{[i]} : G^{[i+1]}|.$$

Số lớp kề của $G^{[i+1]}$ trong $G^{[i]}$ chính là $|R_i|$. Thuật toán Schreier–Sims đã tìm được R_i , nên chỉ cần tính trực tiếp $\prod_{i=1}^m |R_i|$. Cần dùng số nguyên độ chính xác cao.

6.3 Tổng kết

Thuật toán Schreier–Sims là một thuật toán cơ bản trong nhóm tính toán. Còn có những thuật toán tương tự tốt hơn và các tối ưu cho thuật toán này, nhưng do trình độ của tác giả có hạn, đồng thời xét đến độ khó hiện thực trong OI, bài viết không giới thiệu thêm. Thuật toán này thực ra biểu diễn rõ ràng cấu trúc của nhóm hoán vị, nên có thể thuận tiện hoàn thành các nhiệm vụ như xác định tính thành viên và tính bậc. Tuy nhiên, đây là thuật toán tổng quát, không tận dụng tính chất đặc thù của một nhóm hoán vị cụ thể, nên độ phức tạp thời gian khá lớn. Khi đối mặt với bài cụ thể, ta vẫn cần khai thác tính chất riêng của đề rồi mới thiết kế thuật toán.

6.4 Lời cảm ơn

Cảm ơn bạn Du Yuhao đã giúp đỡ tác giả trong quá trình viết bài.

Cảm ơn thầy Li Jian đã hướng dẫn.

Tài liệu tham khảo

1. Akos Seress, *Permutation Group Algorithms*, Cambridge University Press.
2. Donald E. Knuth, “Efficient Representation of Perm Groups”, *Combinatorica* 11 (1991).
3. Nhóm biên soạn Khoa Toán ứng dụng, Đại học Đồng Tế, *Toán rời rạc*, Nhà xuất bản Đại học Đồng Tế.

7 Bàn về phụ thuộc tuyến tính

Kuang Zhengfei, Trường Yali, Trường Sa

Tóm tắt

Trong đời OI, rồi ở đại học hoặc những bậc học cao hơn, rất nhiều học sinh sẽ ít nhiều tiếp xúc với đại số tuyến tính. Phụ thuộc tuyến tính là một thành phần quan trọng của đại số tuyến tính. Bản thân nó không phức tạp, nhưng dấu vết của nó xuất hiện trong rất nhiều mảng kiến thức quan trọng. Bài viết này giới thiệu phụ thuộc tuyến tính, thảo luận các mở rộng của nó trong đại số tuyến tính, và một số ứng dụng kinh điển trong thi tin học.

7.1 Kiến thức chuẩn bị

Trước khi đi vào chủ đề chính, ta cần giới thiệu một ít kiến thức đại số tuyến tính.

7.1.1 Định thức của ma trận

Định thức là một hàm trong đại số tuyến tính, ánh xạ một ma trận vuông cấp n A tới một đại lượng vô hướng. Nó được định nghĩa bằng công thức toán học; bạn đọc có thể tra cứu tài liệu liên quan, ở đây không nhắc lại. Trong bài viết này, ta dùng $\det(A)$ hoặc $|A|$ để biểu thị định thức của ma trận A .

7.1.2 Ma trận chuyển vị

Trong đại số tuyến tính, chuyển vị của một ma trận $n \times m$ A là một ma trận khác, ký hiệu A^T . Mỗi hàng của A tương ứng với một cột của A^T . Ví dụ, với ma trận 3×2

$$\begin{bmatrix} a & b \\ c & d \\ e & f \end{bmatrix},$$

chuyển vị của nó là

$$\begin{bmatrix} a & c & e \\ b & d & f \end{bmatrix}.$$

7.1.3 Ma trận nghịch đảo

Với một ma trận vuông cấp n A , nếu tồn tại một ma trận vuông cấp n khác B sao cho

$$AB = BA = I_n,$$

trong đó I_n là ma trận đơn vị cấp n , thì gọi A là khả nghịch, còn B là ma trận nghịch đảo của A .

Bổ đề 1. Một ma trận vuông A khả nghịch khi và chỉ khi $\det(A) \neq 0$ (quy tắc Cramer).

7.1.4 Không gian tuyến tính

Định nghĩa đầy đủ của không gian tuyến tính khá dài; ở đây ta chỉ giải thích không gian tuyến tính tạo bởi các vector k chiều, tức các bộ có thứ tự gồm k phần tử.

Giả sử V là một tập gồm các vector k chiều, còn $\mathbb{F}$ là một trường, thường là trường số. Nếu

$$\forall a, b \in V, \quad a + b \in V,$$

và

$$\forall a \in V, k \in \mathbb{F}, \quad ka \in V,$$

thì gọi V là một không gian tuyến tính trên trường $\mathbb{F}$. Cần đặc biệt chú ý rằng bất kể phần tử thật sự trong không gian tuyến tính là gì, ta đều gọi chúng là vector.

7.1.5 Tổ hợp tuyến tính

Giả sử V là một không gian tuyến tính trên trường $\mathbb{F}$, và

$$S = \{v_1, v_2, \dots, v_n\}$$

là một tập con của V . Với một phần tử $v \in V$, nếu tồn tại một bộ hệ số $\{a_1, a_2, \dots, a_n\} \in \mathbb{F}$ sao cho

$$v = a_1 v_1 + a_2 v_2 + \dots + a_n v_n,$$

thì gọi v là một tổ hợp tuyến tính của S .

7.2 Phụ thuộc tuyến tính

7.2.1 Phụ thuộc tuyến tính là gì

Gọi V là một không gian tuyến tính trên trường $\mathbb{F}$. Với một tập con

$$S = \{v_1, v_2, \dots, v_n\}$$

của V , nếu tồn tại một bộ hệ số không đồng thời bằng 0 $a_1, a_2, \dots, a_n$ trong $\mathbb{F}$ sao cho

$$a_1 v_1 + a_2 v_2 + \dots + a_n v_n = 0,$$

thì gọi S là phụ thuộc tuyến tính. Ở đây 0 là vector không. Ngược lại, nếu không tìm được bộ hệ số như vậy, thì gọi nhóm vector này là độc lập tuyến tính.

Nói cách khác, nếu vector không là một tổ hợp tuyến tính của S , thì gọi S phụ thuộc tuyến tính; nếu không thì gọi S độc lập tuyến tính. Một nhóm vector phụ thuộc tuyến tính được gọi là một nhóm phụ thuộc tuyến tính; trường hợp ngược lại tương tự.

Ví dụ, các vector $[1, 2, 0]$, $[0, -1, 1]$, $[1, 1, 1]$ phụ thuộc tuyến tính, vì vector cuối là tổng của hai vector đầu. Còn các vector $[1, 0, 0]$, $[0, 1, 0]$, $[0, 0, 1]$ độc lập tuyến tính.

Có một điểm cần nhắc tới: nếu trong công thức trên, ta thay toàn bộ phép toán bằng phép xor, thì cũng có thể nói rằng giữa chúng tồn tại một quan hệ tương tự. Trong OI, mối liên hệ giữa xor và phụ thuộc tuyến tính còn xuất hiện tương đối nhiều hơn.

7.2.2 Các định lý cơ bản về phụ thuộc tuyến tính

Phụ thuộc tuyến tính có nhiều định lý cơ bản.

Định lý 1. Giữa $k + 1$ vector k chiều nhất định có phụ thuộc tuyến tính.

Định lý 2. Mọi siêu tập của một nhóm phụ thuộc tuyến tính đều phụ thuộc tuyến tính.

Định lý 3. Mọi tập con của một nhóm độc lập tuyến tính đều độc lập tuyến tính.

Định lý 4. Nếu trong một nhóm vector S có một vector không, hoặc có hai vector bằng nhau, thì nhóm đó phụ thuộc tuyến tính.

Định lý 5. Mọi nhóm phụ thuộc tuyến tính đều có thể biểu diễn bằng tổ hợp tuyến tính của một nhóm độc lập tuyến tính.

Bản thân phụ thuộc tuyến tính không có quá nhiều điều đáng kể riêng lẻ, nhưng tần suất nó xuất hiện trong đại số tuyến tính lại rất cao. Đó là hướng mà ta sẽ thảo luận tiếp theo.

7.3 Cơ sở của không gian tuyến tính

Định nghĩa không gian tuyến tính đã được nhắc tới ở trên. Rõ ràng các không gian Euclid $\mathbb{R}^n$ với số chiều khác nhau đều là không gian tuyến tính trên trường số thực $\mathbb{R}$. Trong không gian Euclid hai chiều $\mathbb{R}^2$, mọi vector đều có thể được biểu diễn duy nhất bằng tổ hợp tuyến tính của hai vector độc lập tuyến tính $[0, 1]$, $[1, 0]$. Nếu mở rộng hiện tượng này, ta có thể nhận được tính chất tương tự cho mọi không gian tuyến tính. Vì vậy, ta gọi những vector tương tự $[0, 1]$, $[1, 0]$ là cơ sở của không gian tuyến tính. Định nghĩa chuẩn hơn là:

Giả sử V là một không gian tuyến tính, B là một tập con của V . Nếu mọi vector trong V đều có thể biểu diễn duy nhất thành tổ hợp tuyến tính của các vector trong B , thì gọi B là một cơ sở của V .

Từ trường hợp $\mathbb{R}^2$, không khó để nghĩ rằng có thể mọi cơ sở của không gian tuyến tính đều độc lập tuyến tính. Thực tế đúng là như vậy, vì nếu một cơ sở phụ thuộc tuyến tính thì sẽ có vô số cách tổ hợp tuyến tính để biểu diễn vector trong không gian. Vì thế, ta có thể mô tả cơ sở của không gian tuyến tính bằng một cách khác:

Cơ sở của một không gian tuyến tính là một tập con cực đại thỏa độc lập tuyến tính.

Hiển nhiên, một không gian tuyến tính có thể có nhiều cơ sở, nhưng số phần tử của chúng, dưới đây gọi là lực lượng cơ sở, đều bằng nhau; số đó được gọi là số chiều của không gian tuyến tính. Ngược lại, một nhóm độc lập tuyến tính chỉ có thể trở thành cơ sở của một không gian tuyến tính. Do đó, ta có thể dùng cơ sở để biểu diễn không gian tuyến tính. Tương tự, ta cũng có thể dùng một nhóm vector S , không nhất thiết độc lập tuyến tính, để xây dựng một không gian tuyến tính; không gian được xây dựng này gọi là $\text{span}(S)$.

Ngoài ra, cơ sở của một số không gian tuyến tính có kích thước vô hạn; ở đây chỉ nhắc qua và không trình bày kỹ. Bạn đọc có hứng thú có thể tìm đọc thêm tài liệu khác.

7.4 Hạng của ma trận

Ma trận có thể được xem như vector tạo bởi nhiều vector, lại có nhiều tính chất đẹp, nên giữ vị trí rất quan trọng trong đại số tuyến tính.

7.4.1 Hạng của ma trận là gì

Chính vì ma trận có thể được xem như một nhóm vector hàng hoặc vector cột, việc nghiên cứu các vector ấy có phụ thuộc tuyến tính hay không cũng là một vấn đề đáng xét. Hạng của ma trận xuất hiện từ đó.

Một ma trận A có hai loại hạng: hạng hàng và hạng cột. Hạng hàng là số lượng lớn nhất các hàng độc lập tuyến tính của ma trận A ; hạng cột là số lượng lớn nhất các cột thỏa điều kiện tương tự. Hiển nhiên, hạng hàng của A bằng hạng cột của A^T . Nhưng điều đẹp hơn là:

Định lý 6. Hạng hàng của một ma trận bằng hạng cột của nó.

Có rất nhiều cách chứng minh; ở đây chỉ đưa ra một cách. Tuy nhiên, trước khi chứng minh, ta còn cần thảo luận một vài thứ khác.

7.4.2 Quan hệ giữa hạng và cơ sở

Ma trận có hạng, còn không gian tuyến tính có cơ sở. Trước đó ta đã nhắc rằng một nhóm vector có thể xây dựng ra một không gian tuyến tính, còn ma trận vừa khéo chính là một nhóm vector như vậy. Không khó nhận ra rằng tập tương ứng với hạng hàng của ma trận chính là cơ sở của không gian tuyến tính tạo bởi các vector hàng của nó, dưới đây gọi là không gian hàng; hạng cột thì tương ứng với cơ sở của không gian cột.

7.4.3 Hạng hàng bằng hạng cột

Với thảo luận ở phần trước, ta có thể chứng minh định lý trên.

Xét một ma trận $n \times m$ A . Giả sử hạng hàng của nó là r , tức số chiều của không gian hàng V của A là r . Trong V , chọn tùy ý một nhóm vector

$$S = \{v_1, v_2, \dots, v_r\},$$

rồi dùng các vector này làm hàng để tạo thành một ma trận $r \times m$ R . Hiển nhiên, mỗi vector hàng của A đều có thể biểu diễn bằng một tổ hợp tuyến tính của S . Nói bằng phép nhân ma trận, nhất định tồn tại một ma trận $n \times r$ C sao cho

$$A = CR.$$

Vì $A = CR$, mỗi vector cột của A đều có thể biểu diễn bằng tổ hợp tuyến tính của các vector cột trong C . Rõ ràng không gian cột của A được chứa hoàn toàn trong không gian cột của C , nên hạng cột của A không lớn hơn hạng cột của C . Lại vì C chỉ có r cột, nên hạng cột của C không lớn hơn r , tức hạng hàng của A . Do đó hạng cột của A không lớn hơn hạng hàng của A . Theo tính đối xứng, hạng hàng của A cũng không lớn hơn hạng cột của A . Vì vậy hạng hàng của A bằng hạng cột của A . Chứng minh xong.

Vì hạng hàng bằng hạng cột, ta thống nhất hai khái niệm này và gọi chung là hạng của ma trận, ký hiệu $R(A)$. Hiển nhiên, với một ma trận $n \times m$ A ,

$$R(A) \leq \min(n, m).$$

Khi $R(A) = \min(n, m)$, ta gọi A là đầy hạng; ngược lại gọi là thiếu hạng.

7.4.4 Hạng và định thức

Khi định thức của một ma trận vuông cấp n A khác 0, không khó chứng minh n vector hàng của A độc lập tuyến tính. Nói cách khác, ma trận này đầy hạng. Vì thế, nếu dùng định thức để hiểu hạng, ta có một định nghĩa mới:

Khi trong một ma trận A tồn tại một định thức con cấp r , tức chọn tùy ý r vector hàng để lập ma trận rồi lấy định thức, khác 0, còn mọi định thức con cấp $r + 1$ đều bằng 0, ta quy định hạng của A là $R(A) = r$.

Đến đây, quan hệ giữa cơ sở, hạng và định thức đều đã được phân tích. Bây giờ chỉ còn một vấn đề: tính hạng như thế nào.

7.4.5 Cách tính hạng

Nếu cần tính hạng, phương pháp phổ biến nhất là khử Gauss. Tin rằng tất cả học sinh từng học OI đều đã viết khử Gauss, và xem đây là cách thông dụng để giải một hệ phương trình tuyến tính. Thực ra, sau khi khử Gauss một ma trận, hay nói cách khác một nhóm vector hàng, A , thứ thu được chính là một cơ sở trong $\text{span}(A)$. Nói cách khác, sau khi khử Gauss một lần, số phần tử khác 0 còn lại chính là $R(A)$.

Thông thường, độ phức tạp thời gian của khử Gauss là $O(n^3)$. Nhưng nếu thay tổ hợp tuyến tính giữa các vector bằng xor giữa các phần tử, và gọi P là cỡ lớn nhất của phần tử, thì số chiều của không gian tuyến tính tương ứng với một nhóm phần tử nhiều nhất là $O(\log P)$; khi đó độ phức tạp của khử Gauss trở thành $O(n \log P)$.

7.4.6 Tổng xor lớn nhất

Nhắc đến khử Gauss thì không thể không nhắc đến bài toán tổng xor lớn nhất. Đây là một bài toán tương đối kinh điển trong OI: cho một nhóm số tự nhiên, hãy tìm tập con có xor lớn nhất. Rõ ràng, sau khi khử Gauss theo xor nhóm số này, ta sẽ thu được một tam giác ngược, với các phần tử giảm dần từ trên xuống dưới. Khi đó, ta duyệt từng phần tử từ trên xuống dưới, lấy giá trị lớn hơn giữa đáp án hiện tại và xor của nó với phần tử đang xét làm đáp án mới. Không khó chứng minh đáp án sinh ra theo cách này nhất định là tối ưu.

7.5 Liên hệ ngoài đại số tuyến tính

Các chương trước đều nói về một vài mở rộng của phụ thuộc tuyến tính trong đại số tuyến tính, thiên về toán học hơn. Thực ra phụ thuộc tuyến tính cũng có liên hệ với rất nhiều thứ khác.

7.5.1 Phụ thuộc tuyến tính và matroid

Nhiều bạn từng học tham lam đều biết matroid, tức nguồn gốc của cách chứng minh thuật toán tham lam. Dưới đây, ta dùng cặp $M = \{S, L\}$ để biểu diễn một matroid. Giả sử A là một nhóm vector hàng, có thể chứng minh:

Định lý 7. Lấy tập nền S là nhóm vector A , và lấy L là họ mọi tập con độc lập tuyến tính của A ; khi đó (S, L) cấu thành một matroid.

Matroid có bốn tính chất. Nếu cả bốn tính chất đều được thỏa, định lý trên được chứng minh. Trong đó hai tính chất là hiển nhiên, chỉ cần chứng minh tính di truyền và tính trao đổi.

Tính di truyền. Suy ra từ Định lý 3.

Tính trao đổi. Dùng phản chứng. Giả sử $X, Y \in L$ và $|X| > |Y|$. Nếu với mọi $v \in X - Y$ đều có $Y \cup \{v\} \notin L$, thì mọi vector trong X đều có thể biểu diễn bằng tổ hợp tuyến tính của các vector trong Y . Nhưng X độc lập tuyến tính, không thể được biểu diễn hoàn toàn bằng một tập có số phần tử nhỏ hơn; điều này mâu thuẫn với $|X| > |Y|$. Vì vậy tính trao đổi nhất định được thỏa. Chứng minh xong.

Với tính chất này, dùng tham lam là có thể giải bài toán nhóm độc lập tuyến tính có trọng số lớn nhất. Ta xem tiếp bài sau.

7.5.2 CQOI2013 Problem 1 Nim

Tóm tắt đề bài. Hiện có hai người A, B chơi một trò chơi. Luật chơi như sau.

Trong trò chơi có N đồng diêm. Ở lượt đầu tiên, A lấy ra một số đồng diêm, nhưng không được lấy hết; sau đó B cũng thực hiện thao tác tương tự. Tiếp theo, nếu xor số que diêm trong các đồng còn lại bằng 0, thì B thắng; ngược lại A thắng.

Hỏi A cần lấy ít nhất bao nhiêu que diêm để có thể thắng.

Quy mô dữ liệu.

$$N \leq 100, \quad \text{mỗi số que diêm} \leq 10^9.$$

Phân tích. Nếu sau khi A lấy một số đồng diêm, các phần tử còn lại độc lập tuyến tính, thì A thắng.

Vì vậy bài toán trở thành: cho một số tự nhiên, cần chọn một tập con độc lập tuyến tính sao cho tổng mọi số trong tập con là lớn nhất. Theo thảo luận trước đó, dùng tham lam là có thể giải được.

7.5.3 Phụ thuộc tuyến tính và đồ thị

Nếu biểu diễn một đồ thị vô hướng bằng ma trận kề điểm-cạnh, thì một nhóm độc lập tuyến tính trong ma trận, tức nhóm vector, tương ứng với một rừng khung trong đồ thị.

Đi sâu hơn một chút, nếu ta có thể tính được một nhóm vector có bao nhiêu nhóm độc lập tuyến tính, thì có thể giải bài toán đếm số rừng khung trong một đồ thị. Không may là bài sau là một bài toán #P, tương tự vai trò của NP trong các bài toán tối ưu, nên bài trước chắc chắn cũng là bài toán #P. Nếu bạn đọc có hứng thú với hướng này, có thể thử tự khám phá.

7.5.4 Phụ thuộc tuyến tính và hình học tính toán

Trong $\mathbb{R}^2$, điều kiện cần và đủ để hai vector đồng tuyến là giữa chúng có phụ thuộc tuyến tính. Trong $\mathbb{R}^3$, điều tương ứng là quan hệ đồng phẳng, và cứ thế suy rộng. Nếu quan sát kỹ, ta có thể thấy tính chất này tương đương với việc dùng tích có hướng, tức lấy định thức, để phán đoán.

7.6 Bài ví dụ

Tục ngữ nói thực hành mới sinh hiểu biết. Để thể hiện tốt hơn vai trò của đại số tuyến tính trong OI, dưới đây liệt kê vài bài ví dụ về phụ thuộc tuyến tính.

7.6.1 XXor, bài tự ra

Tóm tắt đề bài. Định nghĩa một hàm $f(S)$, ánh xạ một tập số tự nhiên tới một số. Nếu trong S có tổng cộng k tập con sao cho xor của mọi số trong tập con ấy bằng 0, thì $f(S) = k^2$. Cho một tập L gồm n số tự nhiên, hãy tính

$$\sum_{T \subset L} f(T).$$

Quy mô dữ liệu.

$$n \leq 24, \quad \text{mọi phần tử trong } S \text{ đều } \leq 10^{18}.$$

Phân tích. Không khó nhận ra $f(S)$ thực chất tương ứng với

$$2^{2(|S| - \dim \text{span}(S))}.$$

Chỉ cần biết số chiều của mọi không gian tuyến tính là bài toán được giải.

Trước đó đã nhắc rằng, với một nhóm phần tử, độ phức tạp của khử Gauss theo xor là $O(n \log P)$, trong đó P là cỡ lớn nhất của phần tử. Bài này có thể khử Gauss theo xor trực tiếp trên mọi tập, nhưng tổng độ phức tạp vẫn quá chậm. Ta liên tưởng tới việc dùng hàm `lowbit` để tính số bit 1 trong mỗi xâu nhị phân độ dài n trong thời gian $O(2^n)$; bài này cũng có thể dùng cách tương tự.

Với mỗi tập S , ta dùng một danh sách liên kết để ghi lại dãy sinh bởi cơ sở của $\text{span}(S)$ sau khi sắp theo kích thước. Ta duyệt S theo số phần tử từ lớn đến nhỏ. Nếu $S = \emptyset$, lực lượng cơ sở của nó là 0. Ngược lại, chọn tùy ý một phần tử $x \in S$, dùng cơ sở của $\text{span}(S - \{x\})$ từ lớn đến nhỏ để lần lượt khử x , tức nếu có thể làm x nhỏ hơn thì xor nó. Nếu cuối cùng $x = 0$, thì

$$\text{span}(S) = \text{span}(S - \{x\});$$

ngược lại, cơ sở của $\text{span}(S)$ bằng cơ sở của $\text{span}(S - \{x\})$ hợp với x sinh ra cuối cùng, rồi lưu bằng danh sách liên kết.

Đến đây bài toán được giải. Đặt P là số lớn nhất trong L ; độ phức tạp thời gian và bộ nhớ đều là $O(2^n \log P)$.

7.6.2 Codeforces Round #228 Div.1 D

Tóm tắt đề bài. Gọi S là một tập trên miền số tự nhiên $\mathbb{Z}_+$. Nếu với mọi $a, b \in S$ đều có

$$a \oplus b \in S,$$

thì gọi S là một tập hoàn hảo.

Cho một số n , hỏi có bao nhiêu tập hoàn hảo sao cho số lớn nhất trong tập không vượt quá n .

Quy mô dữ liệu.

$$n \leq 10^9.$$

Phân tích. Để đơn giản hóa đề bài, ta xem mọi số trong bài này là số 32 bit. Khi đó một số thực chất tương ứng với một vector 32 chiều.

Không khó nhận ra một tập hoàn hảo thực chất tương ứng với một không gian tuyến tính; ta có thể dùng cơ sở để biểu diễn nó. Nói cách khác, nếu với mỗi không gian tuyến tính thỏa điều kiện, ta đều có thể tìm ra một cơ sở tương ứng duy nhất, thì bài toán được giải.

Dựa trên ý tưởng khử Gauss, ta có thể biểu diễn cơ sở tương ứng với một tập hoàn hảo bằng một tam giác ngược, hoặc hình thang. Lại vì phần tử lớn nhất trong tập không được vượt quá n , ta có thể dùng quy hoạch động chữ số, xử lý từng bit từ cao xuống thấp, lần lượt thêm các vector cơ sở.

Trước hết bỏ qua ràng buộc không lớn hơn n . Khi đang xử lý bit thứ i , và đã thêm j vector cơ sở:

1. Nếu thêm một vector cơ sở mới 2^i , thì các bit thứ i của những vector trước đó có thể được tùy ý không chế, nên có thể đặt tất cả bằng 0. Khi đó có 1 cách chọn.
2. Nếu không thêm vector mới, có thể chứng minh rằng dù bit thứ i của các vector trước đó lấy giá trị thế nào, không gian tuyến tính tương ứng đều không bị đếm lặp. Khi đó có 2^j cách chọn.

Dùng DP là có thể giải được bài toán, nhưng lúc này còn phải xét ràng buộc không lớn hơn n .

Đặt $g_{i,j}$ là số không gian tuyến tính có hậu tố i bit của giá trị lớn nhất bằng hậu tố i bit của n , sau khi đã xử lý i bit thấp và sinh ra j vector cơ sở. Tương tự, đặt $f_{i,j}$ là số không gian tuyến tính có hậu tố i bit của giá trị lớn nhất nhỏ hơn hậu tố i bit của n . Có thể thấy quá trình chuyển của $f_{i,j}$ hoàn toàn giống quá trình trên; ta chỉ cần xét cách chuyển $g_{i,j}$. Khi xử lý bit thứ i và đang có j vector cơ sở:

1. Nếu bit thứ i của n bằng 1:

- Nếu thêm vector cơ sở mới 2^i , thì hậu tố của giá trị lớn nhất trong không gian tuyến tính vẫn bằng hậu tố của n .
- Nếu không thêm, có 2^{j-1} cách gán để hậu tố vẫn bằng nhau, và cũng có 2^{j-1} cách để hậu tố nhỏ hơn n .

2. Nếu không, ta không thể thêm lại một vector, và chỉ có 2^{j-1} cách gán để hậu tố vẫn bằng nhau. Chú ý trường hợp đặc biệt $j = 0$ cũng có 1 cách.

Trên đây là toàn bộ quá trình chuyển. Đáp án của bài này là

$$\sum_{i=0}^{32} g_{0,i} + f_{0,i}.$$

Độ phức tạp thời gian là $O(\log^2 n)$, độ phức tạp bộ nhớ là $O(\log^2 n)$.

7.6.3 HEOI2013 Canxi sắt kẽm selen vitamin

Tóm tắt đề bài. Cho n vector n chiều, gọi là loại A , rồi cho tiếp n vector n chiều, gọi là loại B . Bảo đảm các vector loại A độc lập tuyến tính. Cần chọn cho mỗi vector loại A một vector dự phòng từ các vector loại B , sao cho sau khi vector ấy bị thay bằng vector dự phòng, n vector mới vẫn độc lập tuyến tính. Hai vector loại A bất kỳ không được chọn cùng một vector dự phòng. Hãy tìm một phương án như vậy.

Quy mô dữ liệu.

$$n \leq 300, \quad \text{mọi số xuất hiện đều là số nguyên trong } [0, 10000].$$

Phân tích. Gọi tập các vector loại A là S_A , và tập các vector loại B là S_B .

Bài toán thực chất là hỏi giữa mỗi cặp vector A, B có thể thay thế được hay không; phần còn lại chỉ cần dùng ghép cặp đồ thị hai phía. Hiển nhiên $\text{span}(S_A) = \mathbb{R}^n$, mọi vector loại B đều có thể được biểu diễn duy nhất bằng một tổ hợp tuyến tính của S_A . Trên thực tế có thể chứng minh:

Định lý 8. Điều kiện cần và đủ để một vector loại B x có thể thay một vector loại A y là: trong tổ hợp tuyến tính biểu diễn x bằng các phần tử của S_A , hệ số của y khác 0.

Chứng minh. Điều này là hiển nhiên. Nếu hệ số của y khác 0, thì y có thể được biểu diễn bằng x và các phần tử khác của S_A . Nói cách khác, sau khi thay thế vẫn có $\text{span}(S_A) = \mathbb{R}^n$, và S_A độc lập tuyến tính. Ngược lại, nếu hệ số bằng 0, các phần tử khác trong S_A có thể trực tiếp tổ hợp ra x , nên các vector này nhất định phụ thuộc tuyến tính. Chứng minh xong.

Bây giờ bài toán trở thành cách tính các hệ số trong tổ hợp tuyến tính tương ứng với x . Có thể giải phần này bằng một phương pháp tương tự khử Gauss.

Đến đây bài toán được giải. Độ phức tạp thời gian của thuật toán là $O(N^3)$, độ phức tạp bộ nhớ là $O(N^2)$. Chú ý rằng đây không phải độ phức tạp của lời giải gốc của bài.

7.7 Tổng kết

Tuy phụ thuộc tuyến tính không xuất hiện quá thường xuyên trong OI, vẫn có rất nhiều bài hay liên quan tới nó. Khác với số học đang phổ biến hiện nay, các bài về phụ thuộc tuyến tính thường kết hợp với một số thứ ngoài toán học. Bởi vì bản thân nó không phức tạp, nhưng yêu cầu thí sinh có năng lực hiểu và phân tích tương đối mạnh thì mới vận dụng đến nơi đến chốn. Phụ thuộc tuyến tính là một chủ đề đẹp; hy vọng trong tương lai OI sẽ có thêm nhiều bài liên quan tới nó, để nó được ứng dụng nhiều hơn.

7.8 Lời cảm ơn

- Cảm ơn cha mẹ và nhà trường đã bồi dưỡng tác giả.
- Cảm ơn CCF đã trao cho tác giả cơ hội quý báu này.
- Cảm ơn thầy Qu Yunhua đã hỗ trợ và hướng dẫn tác giả.
- Cảm ơn các bạn Liu Yanyi, Huang Zhi'ao và những bạn khác đã giúp đỡ tích cực.

Tài liệu tham khảo

1. Wikipedia, <http://www.wikipedia.org>.
2. Khoa Toán, Đại học Đồng Tế, *Đại số tuyến tính*, Nhà xuất bản Đại học Đồng Tế.
3. XFS, *Bước đầu nghiên cứu về matroid*, Tập luận văn đội tuyển quốc gia năm 2007.
4. Heidi Gebauer, Yoshio Okamoto, “Fast Exponential-Time Algorithms for the Forest Counting and the Tutte Polynomial Computation in Graph Classes”.

8 Một số nghiên cứu về cây khung tích nhỏ nhất ba chiều

Zhang Hengjie, Trường Trung học số 1 Thiệu Hưng, Chiết Giang

Tóm tắt

Dựa trên mô hình tích nhỏ nhất hai chiều, bài viết này dùng nguyên lý tìm bao lồi của thuật toán gói quà 3D để nghiên cứu một vài tình huống khi mở rộng mô hình tích nhỏ nhất lên ba chiều.

8.1 Mô hình cây khung tích nhỏ nhất ba chiều

Cho n điểm và m cạnh. Mỗi cạnh có ba thuộc tính a, b, c . Cần tìm một tập cạnh sao cho n điểm liên thông đôi một qua các cạnh được chọn. Mục tiêu là cực tiểu hóa tích của tổng thuộc tính a , tổng thuộc tính b và tổng thuộc tính c trên tập cạnh được chọn.

Chú ý rằng ở đây a, b, c đều phải không âm.

8.2 Trường hợp một chiều

Đây chính là trường hợp thông thường.

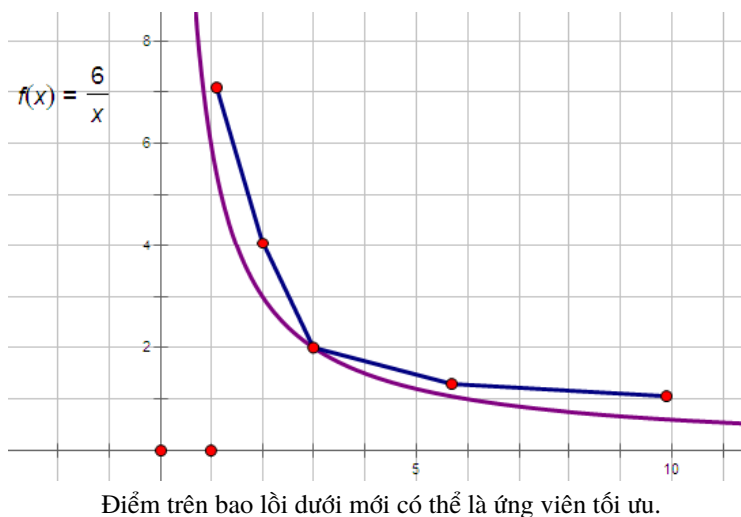
8.3 Trường hợp hai chiều

Với một phương án khả thi, lấy tổng thuộc tính a và tổng thuộc tính b trong phương án đó làm tọa độ trên mặt phẳng Descartes hai chiều. Khi đó mỗi phương án tương ứng với một điểm.

Có một tính chất:

Trong tất cả các điểm, chỉ những điểm nằm trên bao lồi dưới mới có thể trở thành đáp án.

Hình dưới minh họa các điểm khả thi, bao lồi dưới và một đường $f(x) = 6/x$ đi qua một điểm trên cạnh của bao lồi.



Điểm trên bao lồi dưới mới có thể là ứng viên tối ưu.

Chứng minh. Ta chứng minh rằng điểm không nằm trên bao lồi dưới nhất định không phải đáp án; chỉ cần chứng minh các điểm trên biên của bao lồi dưới nhưng không phải đỉnh không tốt hơn đáp án. Xét hai đỉnh kề nhau trên bao lồi. Giả sử có một điểm nào đó trên đoạn nối hai đỉnh này cho giá trị tốt hơn cả hai đỉnh. Khi đó ta vẽ một đường hàm nghịch tỉ lệ đi qua điểm ấy, thì hai đỉnh sẽ đều nằm phía trên đường hàm này. Nhưng không thể tồn tại một đường hàm nghịch tỉ lệ sao cho có hai điểm ở phía trên nó mà đoạn nối hai điểm ấy lại cắt đường hàm. Vì vậy, mọi điểm trên đoạn nối hai đỉnh kề nhau của bao lồi đều kém hơn đỉnh tốt hơn trong hai đỉnh đó. Chứng minh xong.

Cách hiện thực. Định nghĩa $\text{work}(A, B)$ là thao tác tìm điểm có hình chiếu gần gốc tọa độ nhất trên một vector vuông góc với đường thẳng AB , hướng ngược với gốc tọa độ. Giả sử $\text{work}(A, B)$ tìm được điểm C . Khi đó gọi đệ quy $\text{work}(A, C)$ và $\text{work}(C, B)$ sẽ tìm được mọi điểm của bao lồi dưới nằm giữa A và B . Nếu A và B đều là hai điểm ở vô cực và mọi điểm đều nằm trong nửa mặt phẳng hướng về gốc tọa độ so với đường thẳng nối hai điểm này, thì $\text{work}(A, B)$ có thể tìm được mọi điểm trên bao lồi dưới.

Làm thế nào để tìm điểm có hình chiếu gần nhất trên một vector cho trước? Gọi vector này là u , và gọi $d(v)$ là độ dài hình chiếu của vector v lên u . Ta có thể chứng minh

$$\sum_i d(v_i) = d\left(\sum_i v_i\right).$$

Vì vậy, chỉ cần đặt trọng số của cạnh thứ i thành độ dài hình chiếu của vector (a_i, b_i) lên u , rồi chạy thuật toán cây khung nhỏ nhất thông thường là tìm được phương án.

8.4 Trường hợp ba chiều

Tương tự hai chiều, trong tất cả các điểm, chỉ những điểm nằm trên bao lồi dưới mới có thể trở thành đáp án.

8.4.1 Heuristic ngẫu nhiên

Khi dữ liệu ngẫu nhiên, vì các điểm cực trị trên bao lồi có thể rất nhô ra, nên phương pháp chọn ngẫu nhiên một vector để tìm giá trị tối ưu, hoặc mô phỏng tối luyện, có tỉ lệ trúng rất cao. Trong tình huống dữ liệu ngẫu nhiên, đây là một thuật toán rất mạnh.

8.4.2 Vector ba chiều

Trước hết ta giới thiệu các phép toán của vector ba chiều và ý nghĩa hình học của chúng.

Cộng vector A và vector B . Tương tự hai chiều: nối đầu của B vào cuối của A , vector từ đầu của A tới cuối của B là vector cần tìm. Viết bằng công thức:

$$(A_x + B_x, A_y + B_y, A_z + B_z).$$

Trừ vector B khỏi vector A . Tương đương với cộng vector ngược hướng của B . Viết bằng công thức:

$$(A_x - B_x, A_y - B_y, A_z - B_z).$$

Tích vô hướng của vector A và vector B . Kết quả là một số, biểu thị tích giữa độ dài hình chiếu của A lên B và độ dài của B . Nó cũng có thể được biểu diễn bằng tích độ dài của A , độ dài của B và cos của góc kẹp giữa chúng. Viết bằng công thức:

$$A_x B_x + A_y B_y + A_z B_z.$$

Tích có hướng của vector A và vector B . Kết quả là một vector vuông góc với cả A lẫn B , và độ dài của nó bằng diện tích hình bình hành tạo bởi A và B . Hướng được xác định bằng quy tắc bàn tay phải. Viết bằng công thức:

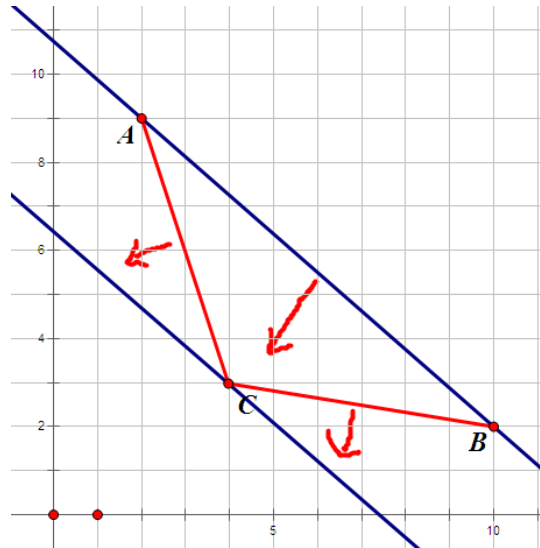
$$(A_y B_z - A_z B_y, A_z B_x - A_x B_z, A_x B_y - A_y B_x).$$

Vector pháp tuyến của một mặt phẳng. Đó là một vector vuông góc với mặt phẳng.

8.4.3 Một cách làm “hiển nhiên”

Tương tự cách làm hai chiều, định nghĩa $\text{work}(A, B, C)$ là thao tác tìm điểm có hình chiếu nhỏ nhất trên vector pháp tuyến của mặt phẳng ABC . Giả sử $\text{work}(A, B, C)$ tìm được điểm D , rồi gọi đệ quy $\text{work}(A, B, D)$, $\text{work}(A, D, C)$ và $\text{work}(D, B, C)$.

Đáng tiếc là cách này sai. Trong không gian hai chiều, giao của nửa mặt phẳng phía gốc tọa độ so với đường thẳng AC và nửa mặt phẳng phía gốc tọa độ so với đường thẳng BC sẽ không còn trở thành điểm trên bao lồi nữa. Nhưng trong trường hợp ba chiều ta không bảo đảm được điều này. Chẳng hạn, hình dưới vẽ một điểm E nằm khuất ở bên trái cạnh AD ; khi đó $\text{work}(A, B, D)$ và $\text{work}(A, C, D)$ có thể cùng tìm tới E .



Điểm E có thể bị hai lời gọi đệ quy cùng tìm lại.

8.4.4 Một cách làm đúng

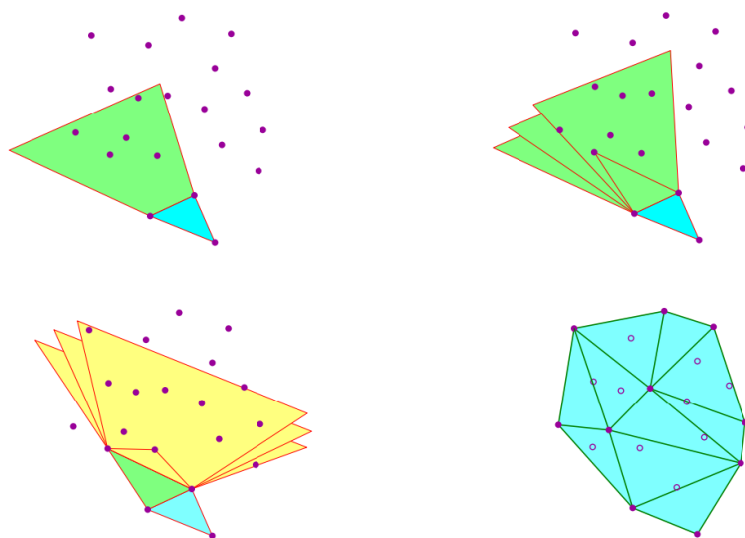
Xét một bài toán khác:

Trong không gian ba chiều có n điểm; hãy tìm bao lồi của chúng.

Thực ra bản chất sai lầm của cách làm phía trên là chính nó không thể tìm được một bao lồi hoàn chỉnh trong không gian ba chiều. Vì vậy tác giả chọn thuật toán gói quà 3D làm nền tảng cho thuật toán này.

Cách làm gói quà thông thường cho bao lồi 3D là: ban đầu chọn một cạnh chắc chắn nằm trên bao lồi, rồi tìm một điểm sao cho phía bên kia của mặt phẳng tạo bởi cạnh này và điểm ấy không có điểm nào. Như vậy ta tìm được một mặt của bao lồi; sau đó đệ quy trên hai cạnh còn lại của mặt này và làm việc tương tự, cho tới khi không còn cạnh mới được thêm vào.

Các hình dưới minh họa quá trình mở dần các mặt tam giác của bao lồi quanh một cạnh ban đầu, cho tới khi toàn bộ các mặt biên được tìm thấy.

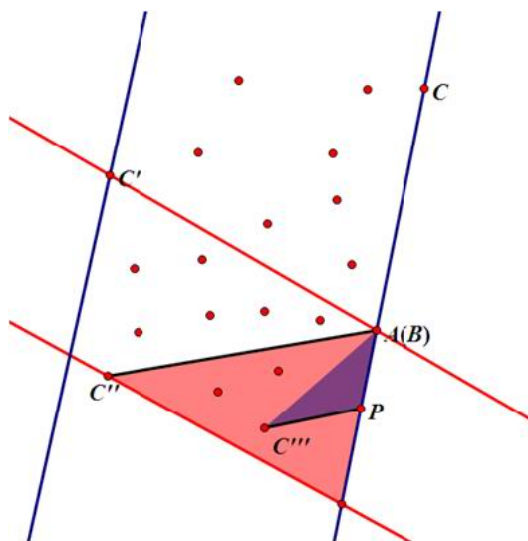


Một chuỗi bước của thuật toán gói quà trong không gian ba chiều.

Tương tự thuật toán gói quà, giả sử ta đã tìm được một mặt ABC trên bao lồi, và cần tìm điểm D sao cho một phía của mặt phẳng ABD không có điểm nào. Tìm trực tiếp có thể khá khó; dưới đây là một phương pháp tác giả tìm được. Trước hết tìm điểm C' xa mặt ABC nhất, rồi tìm điểm C'' xa mặt ABC' nhất. Cứ lặp như vậy thì có thể tìm được D .

Chứng minh độ phức tạp. Để tiện xét, chiếu mọi điểm lên mặt phẳng có vector pháp tuyến là vector $\overrightarrow{AB}$, rồi xét diện tích S của vùng mà D có thể xuất hiện.

Hình dưới minh họa vùng ứng viên này: sau khi đã tìm được C'' , gọi S'' là diện tích tam giác màu đỏ; sau khi tìm thêm C''' , vùng mới S''' là tam giác màu tím. Vì đường thẳng AC'' song song với đường thẳng PC''' , nên S''' giảm ít nhất một nửa so với S'' .



Diện tích vùng ứng viên giảm theo từng bước tìm điểm xa nhất.

Trong giả thiết mọi điểm đều là điểm lưới, khi cuối cùng $S < 1$ thì điểm D nhất định đã được tìm thấy. Vì vậy độ phức tạp không vượt quá $\log_2 \text{area}$.

Do đó, chỉ khi mọi điểm đều có các thuộc tính a, b, c là số nguyên, độ phức tạp của thuật toán mới là

$$O(h \log_2 \text{area}),$$

trong đó h là số điểm trên bao lồi.

8.5 Mở rộng

Thực ra không chỉ cây khung mới có thể lồng vào mô hình tích nhỏ nhất; các thuật toán kinh điển khác cũng có thể làm như vậy. Dưới đây là ví dụ mô hình cắt nhỏ nhất kết hợp thành công với tích nhỏ nhất.

Cho đồ thị vô hướng có n điểm và m cạnh, mỗi cạnh có ba thuộc tính a, b, c . Cần tìm một tập cạnh sao cho sau khi xóa tập cạnh này, s không thể tới t . Mục tiêu là cực tiểu hóa tích của tổng thuộc tính a , tổng thuộc tính b và tổng thuộc tính c trên tập cạnh đó.

Ban đầu, cách làm giống với cây khung: duy trì các mặt trên bao lồi, và khi cần tìm điểm có độ dài hình chiếu ngắn nhất thì đặt lại trọng số cạnh thành một trọng số khác rồi chạy trực tiếp luồng cực đại. Nhưng khi cần lấy ra phương án, dùng luồng cực đại để lấy cắt nhỏ nhất trông hơi kỳ lạ.

Ở đây đưa ra một cách tìm phương án. Chia tập điểm thành hai phần: tập các điểm có thể đi tới từ s trong mạng dư, ký hiệu là A , và tập các điểm không thể đi tới, ký hiệu là B . Các cạnh nối giữa hai tập này chính là một phương án hợp lệ. Chứng minh ngắn gọn như sau. Đặt S là tổng dung lượng của các

cạnh này. Khi chạy thuật toán luồng cực đại, một đường tăng luồng sẽ liên tục đi qua lại giữa hai tập; dù thế nào, số lần đi từ A vào B nhất định bằng số lần đi từ B vào A cộng 1, nên S sẽ giảm đúng bằng lượng luồng của lần tăng này. Vì cuối cùng $S = 0$, S ban đầu bằng giá trị luồng cực đại. Lại vì luồng cực đại bằng cắt nhỏ nhất, ta chứng minh được phương án này là một nghiệm khả thi tối ưu.

Có thể còn có những mô hình khác lồng được với mô hình tích nhỏ nhất; chỉ cần mô hình ấy cho phép tìm ra phương án là đủ.

Tài liệu tham khảo

1. Tang Wenbin, bài giảng Trại mùa đông năm 2012.

8.6 Lời cảm ơn

Cảm ơn các thầy cô hướng dẫn Chen Heli, You Guangjiao, Dong Yehua và Shao Hongxiang đã giúp đỡ tác giả trong một thời gian dài.

Cảm ơn Yu Yili và Dong Honghua đã giúp đỡ tác giả.

9 Bàn về bài toán xâu con palindrome

Xu Yi, Trường Trung học cao cấp Thường Châu, Giang Tô

Tóm tắt

Bài viết trước hết điểm lại ngắn gọn cách dùng mảng hậu tố để tính bán kính palindrome, sau đó giới thiệu chi tiết thuật toán Manacher. Trên nền tảng này, tác giả phân tích một số dạng bài xâu con palindrome qua các ví dụ, rồi tổng kết cách suy nghĩ chung khi giải loại bài này.

9.1 Dẫn nhập

Những năm gần đây, khi các thuật toán và cấu trúc dữ liệu xử lý xâu ngày càng phổ biến trong nước, các bài toán liên quan tới xâu cũng xuất hiện ngày càng nhiều.

Bài toán xâu con palindrome là một trong những chủ đề khá được quan tâm trong nhóm bài xâu. Vì xâu con palindrome có nhiều tính chất đẹp, các bài thuộc loại này thường có độ linh hoạt cao. Bài viết này phân tích và tổng kết một phần chủ đề ấy, với hy vọng gợi mở thêm cho người đọc.

9.2 Khái niệm cơ bản

- Một tập hữu hạn khác rỗng các ký tự được gọi là bảng chữ cái.
- Một dãy hữu hạn các ký tự trong bảng chữ cái được gọi là xâu.
- Số ký tự trong xâu được gọi là độ dài của xâu.
- Một đoạn liên tiếp trong xâu được gọi là xâu con.
- Một xâu con không đổi sau khi đảo ngược được gọi là xâu con palindrome.
- Xâu con palindrome dài nhất trong một xâu được gọi là xâu con palindrome dài nhất.
- Một nửa độ dài lớn nhất của xâu con palindrome có tâm tại vị trí i được gọi là bán kính palindrome tại vị trí i .

9.3 Các thuật toán thường dùng

Khi xử lý bài toán xử con palindrome, thông thường ta cần tính bán kính palindrome tại mọi vị trí của xử.

Để tránh bàn riêng lẫn lộn và biên, từ đó giảm độ phức tạp của mã nguồn, ta chèn cùng một ký tự đặc biệt vào hai bên mỗi ký tự của xử, chèn thêm một ký tự đặc biệt khác trước ký tự đầu tiên, và một ký tự đặc biệt khác nữa sau ký tự cuối cùng. Chẳng hạn, `abbabcba` được đổi thành `#a#b#b#a#b#c#b#a#@`.

Như vậy ta thu được một xử mới $S[1 \dots n]$. Để tính mảng bán kính palindrome $R[1 \dots n]$ của xử này, thường có hai cách sau.

S	#	a	#	b	#	b	#	a	#	b	#	c	#	b	#	a	#
R	1	2	1	2	5	2	1	4	1	2	1	6	1	2	1	2	1

9.3.1 Mảng hậu tố

Cách dựng mảng hậu tố đã rất quen thuộc nên không nhắc lại ở đây. Cách dùng mảng hậu tố để tính bán kính palindrome tại từng vị trí cũng đã được nhiều tài liệu trước đây giới thiệu; ta chỉ điểm lại ý tưởng.

Rõ ràng $R[i]$ bằng LCP của $S[i \dots n]$ và xử $S[1 \dots i]$ sau khi đảo ngược. Vì vậy, ta viết xử gốc bị đảo ngược vào sau xử gốc, ngăn cách bằng một ký tự đặc biệt. Bài toán trở thành tìm LCP của hai hậu tố trong xử mới, nên có thể xử lý thuận tiện bằng mảng hậu tố.

- Nếu dựng mảng hậu tố bằng thuật toán nhân đôi và tiền xử lý RMQ trong $O(n \log n)$, tổng độ phức tạp thời gian là $O(n \log n)$.
- Nếu dựng mảng hậu tố bằng DC3 và tiền xử lý RMQ trong $O(n)$, tổng độ phức tạp thời gian là $O(n)$.

9.3.2 Thuật toán Manacher

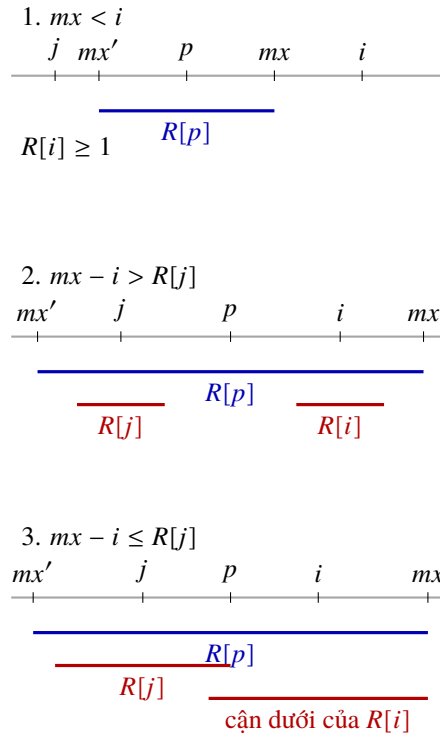
Muốn giải trơn tru các bài toán xử con palindrome, thường cần tính $R[1 \dots n]$ trong $O(n)$. Cách hiện thực dựa trên mảng hậu tố khá dài, nên ta nên tận dụng tính chất riêng của bài toán và dùng thành thạo thuật toán Manacher ngắn gọn dưới đây.

Quy trình thuật toán. Trong thuật toán Manacher, ta thêm hai biến phụ mx và p , lần lượt biểu thị biên phải xa nhất đã được một palindrome hiện có phủ tới và vị trí tâm tương ứng.

Khi tính $R[i]$, trước hết đặt cho nó một cận dưới. Gọi $j = 2p - i$ là điểm đối xứng của i qua p . Có ba trường hợp:

- Nếu $mx < i$, chỉ biết được $R[i] \geq 1$.
- Nếu $mx - i > R[j]$, palindrome tâm j nằm trọn trong palindrome tâm p . Do i và j đối xứng qua p , palindrome tâm i cũng nằm trọn trong palindrome tâm p , nên $R[i] = R[j]$.
- Nếu $mx - i \leq R[j]$, palindrome tâm j không nhất thiết nằm trọn trong palindrome tâm p . Tuy vậy, do đối xứng, palindrome tâm i chắc chắn mở rộng sang phải ít nhất tới mx , nên $R[i] \geq mx - i$.

Ba sơ đồ dưới minh họa lần lượt các trường hợp $mx < i$, $mx - i > R[j]$ và $mx - i \leq R[j]$: đường phía trên biểu diễn xử, đường giữa biểu diễn $R[p]$, đường phía dưới bên trái biểu diễn $R[j]$, còn đường phía dưới bên phải biểu diễn cận dưới của $R[i]$.



Với phần vượt quá cận dưới, ta chỉ cần tiếp tục so khớp từng ký tự. Mỗi lần so khớp thành công đều làm mx dịch sang phải, mà mx chỉ có thể dịch sang phải nhiều nhất n lần. Vì vậy thuật toán Manacher có độ phức tạp thời gian $O(n)$.

Giải mã. Giả mã tương ứng khởi tạo $p = 0, mx = 0$, duyệt i từ trái sang phải, đặt

$$R[i] = \begin{cases} \min(R[2p - i], mx - i), & mx > i, \\ 1, & mx \leq i, \end{cases}$$

rồi mở rộng bằng cách so sánh $S[i + R[i]]$ và $S[i - R[i]]$. Nếu $i + R[i] > mx$, thuật toán cập nhật $p = i$ và $mx = i + R[i]$. Mảng R thu được chính là kết quả cần tìm.

9.4 Phân tích ví dụ

9.4.1 Ví dụ một: Luyện tập đội cổ vũ

Tóm tắt đề bài. Cho một xâu độ dài n ($1 \leq n \leq 10^6$). Hãy tính tích độ dài của k ($1 \leq k \leq 10^{12}$) xâu con palindrome độ dài lẻ dài nhất, lấy dư 19930726, hoặc xác định rằng số xâu con palindrome độ dài lẻ không đủ k . Nguồn bài: đợt thi chọn đội tuyển quốc gia năm 2011.

Phân tích. Trước hết dùng Manacher để tính bán kính palindrome tại mọi vị trí.

Rõ ràng độ dài xâu con palindrome chỉ có không quá n loại, nên ta có thể thống kê số lượng $cnt[1 \dots n]$ của từng độ dài. Giả sử các độ dài palindrome có thể xuất hiện quanh tâm i tạo thành đoạn $[l_i, r_i]$. Khi đó cần cộng 1 cho toàn bộ $cnt[l_i \dots r_i]$. Đây là bài toán hiệu phân kinh điển: mỗi lần chỉ cần cộng 1 tại $cnt[l_i]$, trừ 1 tại $cnt[r_i + 1]$, rồi lấy tổng tiền tố một lần.

Sau đó xử lý các độ dài palindrome từ lớn xuống nhỏ, chỉ xét độ dài lẻ. Mỗi lần dùng lũy thừa nhanh để nhân độ dài tương ứng vào đáp án, cho tới khi tổng số xâu con palindrome độ dài lẻ đã đạt k . Độ phức tạp thời gian là $O(n \log n)$.

9.4.2 Ví dụ hai: Palisection

Tóm tắt đề bài. Cho một xâu độ dài n ($1 \leq n \leq 2 \cdot 10^6$). Hãy tính số cặp xâu con palindrome giao nhau. Nguồn bài: Codeforces Beta Round #17.

Phân tích. Trước hết dùng Manacher để tính bán kính palindrome tại mọi vị trí, đồng thời tính tổng số xâu con palindrome, từ đó suy ra tổng số cặp xâu con palindrome.

Dùng tư tưởng chuyển sang phần bù, ta có thể đổi bài toán thành tính số cặp xâu con palindrome không giao nhau. Lấy tổng số cặp trừ đi số cặp không giao nhau sẽ được số cặp giao nhau.

Gọi $S_1[i]$ là số xâu con palindrome kết thúc tại vị trí i , và $S_2[i]$ là tổng số xâu con palindrome kết thúc tại một vị trí j ($1 \leq j \leq i$). Rõ ràng $S_2[i] = S_2[i-1] + S_1[i]$. Khi đang xét các xâu con palindrome có tâm tại i , số cặp không giao nhau với các palindrome bên trái là

$$\sum_{j=i-R[i]}^{i-1} S_2[j].$$

Để tính nhanh giá trị này, lấy thêm tổng tiền tố $S_3[i] = S_3[i-1] + S_2[i]$. Khi đó biểu thức trên bằng

$$S_3[i-1] - S_3[i-R[i]-1].$$

Vì vậy mỗi tâm được xử lý trong $O(1)$. Độ phức tạp thời gian là $O(n)$.

9.4.3 Tiểu kết một

Với các bài đếm đơn giản liên quan tới xâu con palindrome như ví dụ một và ví dụ hai, cách làm thường là dùng Manacher để tính bán kính palindrome tại mọi vị trí, rồi dùng tổng tiền tố hoặc những kỹ thuật quen thuộc tương tự để giải phần cốt lõi trong $O(n)$.

9.4.4 Ví dụ ba: Xâu song palindrome dài nhất

Tóm tắt đề bài. Cho một xâu độ dài n ($2 \leq n \leq 10^5$). Hãy tìm độ dài lớn nhất của một xâu con song palindrome T . Ở đây song palindrome nghĩa là T có thể được tách thành hai phần liên tiếp không rỗng X, Y , và cả X lẫn Y đều là palindrome. Nguồn bài: trại huấn luyện Thanh Hoa năm 2012 của đội tuyển quốc gia.

Phân tích. Trước hết dùng Manacher để tính bán kính palindrome tại mọi vị trí.

Xét việc liệt kê điểm chia i . Ta cần palindrome dài nhất có biên phải ở i và palindrome dài nhất có biên trái ở i , tức là cần tìm ở hai phía trái và phải của i vị trí xa i nhất mà bán kính palindrome vẫn phủ được tới i .

Lấy việc tìm vị trí tốt nhất bên trái làm ví dụ. Ta quét từ trái sang phải và duy trì biên phải hiện đang xử lý. Khi quét tới i , ta cập nhật vị trí tốt nhất bên trái cho mọi vị trí trong đoạn từ sau biên phải hiện tại tới $i + R[i]$, rồi cập nhật biên phải thành $i + R[i]$. Tính đúng đắn là hiển nhiên: nếu hai vị trí bên trái đều phủ được cùng một điểm, chọn vị trí trái hơn luôn tốt hơn. Phía phải làm tương tự.

Sau khi có vị trí tốt nhất bên trái và bên phải, ta biết độ dài song palindrome dài nhất có điểm chia i ; lấy giá trị lớn nhất là đáp án. Độ phức tạp thời gian là $O(n)$.

9.4.5 Ví dụ bốn: Palindrome gấp đôi

Tóm tắt đề bài. Cho một chuỗi độ dài n ($1 \leq n \leq 5 \cdot 10^5$). Hãy tìm độ dài lớn nhất của một chuỗi con palindrome gấp đôi T . Ở đây palindrome gấp đôi nghĩa là T là palindrome có độ dài chia hết cho 4, đồng thời nửa trước và nửa sau của T cũng đều là palindrome. Nguồn bài: Shanghai Team Selection Contest 2011.

Phân tích. Trước hết dùng Manacher để tính bán kính palindrome tại mọi vị trí.

Xét việc liệt kê tâm i từ trái sang phải. Ta cần tìm tâm con nhỏ nhất j ($j < i$) sao cho bán kính palindrome tại j phủ được tới i , đồng thời hai lần khoảng cách từ i tới j không vượt quá $R[i]$.

Ta dễ dàng suy ra biên trái của các j thỏa điều kiện khoảng cách là

$$k = i - \left\lfloor \frac{R[i]}{2} \right\rfloor.$$

Vì vậy cần tìm bên phải vị trí k một vị trí j gần k nhất mà bán kính palindrome của nó phủ được tới i .

Duy trì một bảng các vị trí còn có thể tối ưu. Trong bảng này, tìm vị trí chưa bị xóa gần nhất bên phải k . Nếu bán kính palindrome tại vị trí ấy không phủ được tới i , xóa nó khỏi bảng, vì nó cũng không thể được các tâm ở bên phải sử dụng nữa. Lặp lại cho tới khi tìm được vị trí phủ được tới i , hoặc đã tới i .

Bảng này chỉ cần hỗ trợ xóa và tìm vị trí chưa xóa gần nhất bên phải một vị trí cho trước, nên có thể duy trì dễ dàng bằng DSU. Lấy giá trị lớn nhất trên mọi tâm là đáp án. Độ phức tạp thời gian là $O(n\alpha(n))$.

9.4.6 Ví dụ năm: Tricky and Clever Password

Tóm tắt đề bài. Một palindrome T có độ dài l lẻ có thể được mã hóa thành

$$A + T[1 \dots x] + B + T[x + 1 \dots l - x] + C + T[l - x + 1 \dots l],$$

trong đó $+$ là phép nối chuỗi, x là số nguyên không âm bất kỳ thỏa $2x < l$, còn A, B, C là các chuỗi bất kỳ, có thể rỗng. Cho một văn bản S độ dài n ($1 \leq n \leq 10^5$). Hãy tìm độ dài lớn nhất có thể của T . Nguồn bài: Codeforces Beta Round #30.

Phân tích. Trước hết dùng Manacher để tính bán kính palindrome tại mọi vị trí.

Nhận xét rằng $T[x + 1 \dots l - x]$ cũng là một palindrome độ dài lẻ, và tâm của nó chính là tâm của T . Xét việc liệt kê tâm i . Đặt

$$T[x + 1 \dots l - x] = S[i - R[i] + 1 \dots i + R[i] - 1]$$

luôn là tối ưu; điều này có thể chứng minh dễ dàng bằng cách xét B, C có rỗng hay không.

Tiếp theo cần tối đa hóa x , tức là cần tìm độ dài khớp lớn nhất của chuỗi $S[i + R[i] \dots n]$ sau khi đảo ngược trong đoạn $S[1 \dots i - R[i]]$. Gọi chuỗi đảo của S là S' . Ta dùng KMP tiền xử lý $S_1[i]$, trong đó $S_1[i]$ là k lớn nhất thỏa

$$S[i - k + 1 \dots i] = S'[1 \dots k],$$

thì lấy cực đại tiền tố $S_2[i] = \max\{S_2[i - 1], S_1[i]\}$. Khi đó độ dài lớn nhất có thể của T với tâm i là

$$2(R[i] + \min\{S_2[i - R[i]], n - i - R[i] + 1\}) - 1.$$

Lấy lớn nhất trên mọi tâm là đáp án. Độ phức tạp thời gian là $O(n)$.

9.4.7 Tiểu kết hai

Với các bài tối ưu một chuỗi liên quan tới palindrome như ví dụ ba, bốn và năm, cách làm thường là dùng Manacher để tính bán kính palindrome tại mọi vị trí, rồi liệt kê tâm hoặc điểm chia trong $O(n)$. Sau khi phân tích ràng buộc của đề, ta kết hợp thêm cấu trúc dữ liệu hoặc thuật toán phụ để xử lý nhanh mỗi lần liệt kê.

9.4.8 Ví dụ sáu: Palindromic Substring

Tóm tắt đề bài. Cho một chuỗi chữ thường độ dài $n (1 \leq n \leq 10^5)$. Có $m (1 \leq m \leq 20)$ truy vấn; ở truy vấn thứ i , mỗi chữ cái nhận một trọng số trong $[0, 25]$. Hãy tìm trọng số của chuỗi con palindrome có trọng số nhỏ thứ k_i , bảo đảm tồn tại. Trọng số của một chuỗi con palindrome được định nghĩa bằng cách lấy nửa đầu của chuỗi con ấy, thay mỗi ký tự bằng trọng số tương ứng, xem dãy này như một số cơ số 26, rồi lấy dư 777777777. Nguồn bài: Asia Changchun Regional Contest 2012.

Định lý 1. Một chuỗi chỉ có $O(n)$ chuỗi con palindrome khác nhau về bản chất.

Chứng minh. Trong thuật toán Manacher, chỉ khi mx dịch sang phải mới sinh ra chuỗi con palindrome mới; nếu không, nhất định đã tồn tại một chuỗi con palindrome đối xứng tương ứng. Mà mx chỉ dịch sang phải không quá n lần, nên một chuỗi chỉ có $O(n)$ chuỗi con palindrome khác nhau về bản chất. Chứng minh xong.

Phân tích. Theo định lý trên, ta có thể dùng Manacher trong $O(n)$ để lấy ra vị trí của tất cả chuỗi con palindrome khác nhau về bản chất.

Với mỗi truy vấn, dùng cách tính tương tự hash chuỗi để nhận được trọng số của từng loại chuỗi con palindrome. Để tìm phần tử nhỏ thứ k , còn cần biết số lần xuất hiện của từng loại chuỗi con palindrome trong chuỗi; việc này có thể giải bằng cách dựng mảng hậu tố rồi tìm kiếm nhị phân.

Sau đó sắp các loại chuỗi con palindrome theo trọng số tăng dần, cộng dồn số lần xuất hiện cho tới khi không nhỏ hơn k ; trọng số tương ứng chính là đáp án. Độ phức tạp thời gian là $O(mn \log n)$.

9.4.9 Ví dụ bảy: Palindromic Equivalence

Tóm tắt đề bài. Cho một chuỗi chữ thường S độ dài $n (1 \leq n \leq 10^6)$. Hãy tính số chuỗi T có cùng tính chất palindrome với S , nghĩa là $T[i \dots j]$ là palindrome khi và chỉ khi $S[i \dots j]$ là palindrome. Nguồn bài: 11th POI Training Camp.

Phân tích. Ta nhận thấy tính chất palindrome của S và T có thể được biểu diễn bằng một số quan hệ bằng nhau và khác nhau. Với palindrome tâm i , rõ ràng có

$$T[i - k] = T[i + k] \quad (1 \leq k < R[i]),$$

và thêm quan hệ

$$T[i - R[i]] \neq T[i + R[i]].$$

Để đếm, chắc chắn cần thu được các quan hệ bằng nhau và khác nhau này.

Định lý 2. Tính chất palindrome của một chuỗi có thể được biểu diễn bằng $O(n)$ quan hệ bằng nhau và khác nhau.

Chứng minh. Trong thuật toán Manacher, chỉ khi mx dịch sang phải mới sinh ra quan hệ bằng nhau mới; nếu không, nhất định đã tồn tại quan hệ bằng nhau đối xứng. Mà mx dịch sang phải không quá n lần, nên số quan hệ bằng nhau là $O(n)$.

Vì mỗi tâm chỉ sinh ra một quan hệ khác nhau, số quan hệ khác nhau cũng là $O(n)$. Do đó tính chất palindrome của một xâu có thể được biểu diễn bằng $O(n)$ quan hệ bằng nhau và khác nhau. Chứng minh xong.

Theo đó, ta có thể dùng Manacher trong $O(n)$ để gộp các vị trí bằng nhau, chỉ giữ vị trí xuất hiện sớm nhất, rồi nối cạnh giữa các vị trí khác nhau. Việc này tương đương tạo ra một xâu mới; mỗi cạnh biểu diễn một quan hệ khác nhau.

Bài toán còn lại trở thành: cho một đồ thị vô hướng, hãy tô màu các đỉnh bằng 26 màu sao cho hai đầu của mỗi cạnh có màu khác nhau, và đếm số cách tô. Tuy nhiên với đồ thị vô hướng tổng quát, bài toán này không có thuật toán hiệu quả, nên cần tiếp tục khai thác tính chất đặc biệt của đồ thị này.

Định lý 3. Sau khi gộp các vị trí bằng nhau trong một xâu T , chỉ giữ vị trí xuất hiện sớm nhất, ta thu được xâu mới T' . Nếu tồn tại các quan hệ khác nhau

$$T'[i] \neq T'[j] \quad (j < i), \quad T'[i] \neq T'[k] \quad (k < i),$$

thì nhất định cũng tồn tại quan hệ khác nhau $T'[j] \neq T'[k]$.

Chứng minh. Vẫn suy nghĩ theo quy trình của Manacher. Khi tính $R[i]$, gọi $j = 2p - i$ là điểm đối xứng của i qua p , rồi xét các trường hợp:

- Nếu $mx < i$, mx dịch sang phải. Điều này tương đương thêm ký tự $T[mx]$ vào cuối T' , khiến nó trở thành ký tự đang hoạt động hiện tại, và sinh quan hệ khác nhau ban đầu $T[i - R[i]] \neq T[mx]$.
- Nếu $mx - i > R[j]$, quan hệ khác nhau thu được $T[i - R[i]] \neq T[i + R[i]]$ là dư thừa, vì nhất định tồn tại một quan hệ khác nhau đối xứng; do đó không cần xét.
- Nếu $mx - i \leq R[j]$, khi mx dịch sang phải thì giống trường hợp $mx < i$. Nếu mx không dịch sang phải, ta làm cho ký tự đang hoạt động hiện tại có thêm một quan hệ khác nhau $T[i - R[i]] \neq T[mx]$.

Giả sử ký tự đang hoạt động hiện tại là $T[i]$, và tồn tại các quan hệ khác nhau $T[i] \neq T[j]$ với $j < i$ và $T[i] \neq T[k]$ với $k < i$. Không mất tính tổng quát, giả sử $k < j$. Khi đó $T[k + 1 \dots i - 1]$ và $T[j + 1 \dots i - 1]$ đều là xâu con palindrome. Từ tính đối xứng, suy ra

$$T[j] = T[k + i - j],$$

và $T[k + 1 \dots k + i - j - 1]$ là một xâu con palindrome.

Vì vậy trong thuật toán Manacher nhất định sẽ có bước so sánh $T[k]$ với $T[k + i - j]$. Lại do $T[j] = T[k + i - j]$, nên $T[j]$ và $T[k]$ hoặc nhất định bằng nhau, hoặc nhất định khác nhau. Nếu chúng nhất định bằng nhau, thì $T[k]$ và $T[j]$ là cùng một vị trí trong T' . Suy ra nếu tồn tại hai quan hệ khác nhau từ $T'[i]$ tới các vị trí $T'[j]$ và $T'[k]$ ở bên trái, thì nhất định cũng tồn tại quan hệ khác nhau $T'[j] \neq T'[k]$. Chứng minh xong.

Sau khi có tính chất này, số chữ cái có thể đặt tại $T'[i]$ chính là 26 trừ đi số quan hệ khác nhau $T'[i] \neq T'[j]$ với $j < i$. Vì thế việc tính toán trở nên trực tiếp. Độ phức tạp thời gian là $O(n)$.

9.4.10 Tiểu kết ba

Với các bài liên quan tới xâu con palindrome nhưng khó tấn công trực tiếp như ví dụ sáu và bảy, ta thường cần phân tích điểm nghẽn của bài toán, rồi dùng chính quy trình Manacher để chứng minh cấp độ số

lượng của một đối tượng nào đó $O(n)$, đồng thời thu được một số tính chất hữu ích để tiếp tục đơn giản hóa bài toán.

9.5 Tổng kết

Với bài toán xâu con palindrome, trước hết ta phải nắm chắc các thuật toán cơ bản để tính bán kính palindrome, đặc biệt là hiểu sâu thuật toán Manacher. Điều này là thiết yếu khi phân tích nhiều tính chất của xâu con palindrome.

Trên nền tảng ấy, cần nắm bắt điều kiện của đề và phân tích cẩn thận, biết tận dụng tính chất của xâu con palindrome để đơn giản hóa bài toán, đồng thời kết hợp được nhiều thuật toán và cấu trúc dữ liệu khác nhau.

9.6 Lời cảm ơn

- Cảm ơn CCF đã cung cấp nền tảng học tập và giao lưu.
- Cảm ơn nhà trường đã hỗ trợ và thầy Cao Wen đã bồi dưỡng tác giả.
- Cảm ơn nhiều bạn học đã giúp tác giả trong quá trình hoàn thành luận văn.
- Cuối cùng, cảm ơn cha mẹ đã nuôi dưỡng tác giả.

Tài liệu tham khảo

1. Maxime Crochemore, Wojciech Rytter Mokhtar, “Text Algorithms”, Oxford University Press.
2. Luo Suiqian, *Mảng hậu tố: công cụ mạnh để xử lý xâu*, Tập luận văn đội tuyển quốc gia năm 2009.

10 Bàn về ứng dụng phương pháp duy trì mảng nhiều chiều trong bài cấu trúc dữ liệu

Liang Zeyu, Trường Trung học số 1 Hợp Phì

Tóm tắt

Trong vài năm gần đây, mảng cấu trúc dữ liệu trong thi tin học phát triển rất nhanh, xuất hiện nhiều phương pháp và tư tưởng mới mẻ, hữu dụng để giải bài cấu trúc dữ liệu, như tư tưởng chia khối, cấu trúc dữ liệu lồng nhau, cấu trúc dữ liệu bền vững và có thể hoàn tác, chia để trị theo thời gian, nhị phân toàn cục, phương pháp đánh nhãn, v.v. Tuy nhiên các phương pháp ấy đôi khi cũng có nhược điểm: lượng mã lớn, dễ viết sai, hằng số lớn. Bài viết này nêu một cách giải bài cấu trúc dữ liệu dựa trên việc duy trì mảng nhiều chiều, giới thiệu nguyên lý cơ bản, ứng dụng và so sánh của phương pháp này với các phương pháp khác, đồng thời đưa ra một số ví dụ.

10.1 Dẫn nhập

10.1.1 Bài toán phần tử nhỏ thứ K trên đoạn có sửa đổi

Cho một dãy số nguyên dương $A[1 \dots n]$. Cần hỗ trợ hai loại thao tác:

- sửa $A[i]$ thành x ;
- hỏi phần tử nhỏ thứ K trong $A[l \dots r]$.

Đây là một bài toán kinh điển. Các lời giải thường gặp gồm cây đoạn theo giá trị lồng cây cân bằng, cây Fenwick theo giá trị lồng cây cân bằng, cây cân bằng theo trọng lượng lồng cây đoạn, mảng khối theo giá trị lồng cây cân bằng, v.v. Điểm chung của chúng là phải dùng cấu trúc dữ liệu lồng nhau, mà lượng mã và hằng số của cấu trúc lồng nhau thường đều rất đáng kể, hiệu quả thực dụng không tốt.

Ta nhìn các cấu trúc lồng nhau ấy từ một góc khác. Nếu trong nút i của cấu trúc dữ liệu ngoài tồn tại nút j của cấu trúc dữ liệu trong, thì có thể mô tả phần tử này bằng một cặp (i, j) . Vì mọi cấu trúc dữ liệu đều có thể lưu bằng mảng, cả i và j đều có thể xem là chỉ số nguyên. Như vậy toàn bộ cấu trúc lồng nhau có thể được biểu diễn bằng các cặp trong một mảng hai chiều, và thao tác trên cấu trúc lồng nhau trở thành thao tác trên mảng hai chiều ấy.

Ta còn có thể làm trực tiếp hơn. Lập mảng hai chiều theo giá trị $A'[1 \dots n][1 \dots W]$, trong đó W là miền giá trị của dãy, thỏa

$$A'[i][j] = \begin{cases} 1, & A[i] = j, \\ 0, & A[i] \neq j. \end{cases}$$

Gọi A' là mảng hai chiều theo giá trị của A . Khi hỏi, nếu nhị phân một giá trị W_r , thì

$$\sum_{i=l}^r \sum_{j=1}^{W_r} A'[i][j]$$

chính là số phần tử trong $A[l \dots r]$ có giá trị thuộc $[1, W_r]$. Khi sửa đổi, chỉ cần đặt $A'[i][A[i]]$ về 0, đặt $A'[i][x]$ thành 1.

Kích thước của A' là $O(nW)$, không thể lưu toàn bộ. Nhưng các giá trị trong A' chỉ là 0 và 1, khi thao tác cũng không cần biết từng ô, mà chỉ cần một số tổng đoạn của A' . Vì bài toán này là kiểu “sửa điểm, hỏi đoạn”, ta dùng cây Fenwick hai chiều để lưu các tổng bộ phận của A' . Đặt

$$FT(A')[i][j] = \sum_{i'=i-\text{lowbit}(i)+1}^i \sum_{j'=j-\text{lowbit}(j)+1}^j A'[i'][j'].$$

Mảng $FT(A')$ cũng có kích thước $O(nW)$, nên cũng không thể lưu hết. Tuy nhiên ta có tính chất sau.

Tính chất 1.1. Nếu độ dài dãy là n , trước đó đã thực hiện Q thao tác sửa đổi, thì số phần tử khác 0 trong $FT(A')$ là

$$O((n + Q) \log n \log W).$$

Chứng minh. Ban đầu A' và $FT(A')$ đều toàn 0. Khi lần lượt đưa n phần tử ban đầu vào, mỗi phần tử tương đương với việc sửa một ô trong A' , kéo theo $O(\log n \log W)$ ô trong $FT(A')$ bị sửa. Mỗi thao tác sửa đổi chỉ thay đổi $O(1)$ ô của A' , nên cũng chỉ làm thay đổi $O(\log n \log W)$ ô trong $FT(A')$. Suy ra tổng số ô khác 0 cần ghi lại không vượt quá $O((n + Q) \log n \log W)$.

Vì vậy chỉ cần ghi lại các ô khác 0 của $FT(A')$. Khi truy cập một ô, ta tìm trong tập ô đã ghi: nếu có thì dùng giá trị ấy, nếu không thì xem là 0. Khi sửa, nếu một ô chuyển từ 0 sang khác 0 thì ghi thêm, nếu chuyển từ khác 0 về 0 thì xóa khỏi tập ghi. Một cách tốt là dùng bảng băm để lưu các ô này, nhờ đó thời gian tìm kiếm kỳ vọng là $O(1)$.

Với thao tác sửa, vì nhiều nhất chỉ liên quan tới $O(\log n \log W)$ ô của $FT(A')$, độ phức tạp là $O(\log n \log W)$. Với thao tác hỏi, nếu nhị phân thô W_r rồi tính tổng đoạn thì độ phức tạp sẽ lên tới $O(\log n \log^2 W)$. Nhưng bản chất cây Fenwick là thao tác bit: trong quá trình dựng dần đáp án theo bit, giả sử giá trị đã có là W'_r , bước hiện tại là bit thứ s từ phải sang trái, thì $\text{lowbit}(W'_r + 2^{s-1}) = 2^{s-1}$. Do đó có thể cộng trực tiếp các ô Fenwick tương ứng với cột $W'_r + 2^{s-1}$, mỗi bước chỉ tốn $O(\log n)$. Vậy một truy vấn có độ phức tạp $O(\log n \log W)$, và toàn bộ phương pháp đạt $O((n + Q) \log n \log W)$ thời gian, đồng thời tránh được cấu trúc lồng nhau phức tạp.

10.1.2 Duy trì mảng nhiều chiều

Từ lời giải bài phần tử nhỏ thứ K trên đoạn có sửa đổi, có thể thấy phương pháp duy trì mảng nhiều chiều thường giúp bỏ cấu trúc dữ liệu lồng nhau, thay bằng các cấu trúc đơn giản như cây Fenwick, nhờ đó giảm đáng kể lượng mã và hằng số.

Với nhiều bài cấu trúc dữ liệu, ta có thể lập một mảng nhiều chiều, thường là hai chiều, đôi khi ba chiều hoặc nhiều hơn, rồi dùng tổng đoạn hoặc thông tin khác của mảng đó để thu được đại lượng bài toán cần. Bản thân mảng thường quá lớn để lưu toàn bộ, nên chỉ lưu một phần các ô; các ô còn lại được suy ra trực tiếp khi cần dựa trên đặc điểm của chúng, chẳng hạn đều bằng 0, đều bằng một giá trị, giống ô khác, hoặc có quy luật rõ ràng.

Để duy trì mảng nhiều chiều, thường dùng các cấu trúc cơ bản sau cho từng chiều, giả sử quy mô chiều đó là m .

Cây Fenwick hoặc cây đoạn. Với mảng D chiều, một hình hộp truy vấn được tách thành $O(\log^D m)$ khối, và một phần tử liên quan tới $O(\log^D m)$ khối.

Mảng khối phân tầng. Với mảng một chiều, chọn số nguyên dương K . Lập K tầng khối; tầng thứ i từ trên xuống ($1 \leq i \leq K$) chia chiều này thành $n^{i/K}$ khối, mỗi khối có kích thước $n^{(K-i)/K}$. Nếu bảo đảm điểm chia của tầng trên cũng là điểm chia của tầng dưới, thì một đoạn bất kỳ của dãy có thể tách thành $O(\sqrt[K]{n})$ khối ở mỗi tầng, tổng cộng $O(K\sqrt[K]{n})$ khối. Mỗi phần tử liên quan tới K khối.

Trường hợp nhiều chiều phức tạp hơn. Với mảng hai chiều, nếu chỉ lập K tầng khối vuông, tầng thứ i có khối kích thước $n^{(K-i)/K} \times n^{(K-i)/K}$, thì một hình chữ nhật không thể luôn tách thành chỉ $O(\sqrt[K]{n^2})$ khối trên mỗi tầng. Cách đúng là với mọi cặp (i, j) thỏa $0 \leq i, j < K$, lập một tầng có kích thước khối $n^{i/K} \times n^{j/K}$. Khi đó một hình chữ nhật bất kỳ tách thành $O(\sqrt[K]{n^2})$ khối trên mỗi tầng; tổng số tầng là K^2 . Lúc này K không còn là số tầng, mà là số mũ quyết định cỡ khối.

Tính chất 1.2. Với mảng D chiều, nếu lập mảng khối phân tầng có số mũ K , thì tổng cộng cần K^D tầng; mỗi hình hộp D chiều tách thành

$$O(K^D \sqrt[K]{N})$$

khối, và mỗi phần tử liên quan tới $O(K^D)$ khối.

Trong ứng dụng thực tế, với số chiều khác nhau, cách chọn K cũng khác nhau. Thường mảng một chiều lấy $K = 3$ hoặc 4, mảng hai chiều lấy $K = 5$ hoặc 6 cho hiệu quả tốt.

10.1.3 Ví dụ một: middle

Tóm tắt đề bài. Cho một dãy $A[0 \dots n-1]$. Đặt $\text{midv}[i][j]$ là trung vị của $A[i \dots j]$, tức phần tử nhỏ thứ $\lfloor (j-i)/2 \rfloor + 1$ trong đoạn đó. Mỗi truy vấn cho bốn số a, b, c, d thỏa $0 \leq a < b < c < d < n$, yêu cầu tính

$$\max_{a \leq l \leq b, c \leq r \leq d} \text{midv}[l][r].$$

Truy vấn bị bắt buộc trực tuyến. Tác giả đề: Chen Lijie.

Lời giải. Trung vị về bản chất cũng là phần tử nhỏ thứ K , nên ta có thể mượn cách làm của bài phần tử nhỏ thứ K trên đoạn có sửa đổi. Nhị phân một giá trị W_r , rồi lập mảng $C[W_r][0 \dots n-1]$: nếu $A[i] \geq W_r$,

thì $C[W_r][i] = 1$, ngược lại $C[W_r][i] = -1$. Khi đó

$$\begin{aligned} & \max_{a \leq l \leq b, c \leq r \leq d} \text{midv}[l][r] \geq W_r \\ \Leftrightarrow & \max_{a \leq l \leq b, c \leq r \leq d} \sum_{i=l}^r C[W_r][i] \geq 0 \\ \Leftrightarrow & \sum_{i=b+1}^{c-1} C[W_r][i] + \max_{a \leq l \leq b} \sum_{i=l}^b C[W_r][i] + \max_{c \leq r \leq d} \sum_{i=c}^r C[W_r][i] \geq 0. \end{aligned}$$

Như vậy bài toán chuyển thành tìm tổng đoạn, tiền tố lớn nhất và hậu tố lớn nhất của một dãy, có thể duy trì bằng cây đoạn.

Mảng C là mảng hai chiều kích thước $O(nW)$, không thể lưu toàn bộ. Với một đoạn $[l \dots r]$, hai dãy $C[x][l \dots r]$ và $C[x+1][l \dots r]$ khác nhau ở ít nhất một phần tử khi và chỉ khi tồn tại $i \in [l, r]$ sao cho $A[i] = x$. Do đó trên một đoạn của cây đoạn chỉ có nhiều nhất $r - l + 1$ giá trị x mà thông tin tiền tố hoặc hậu tố lớn nhất thật sự cần được ghi lại; các giá trị còn lại có thể suy ra từ giá trị đã ghi có x nhỏ nhất không nhỏ hơn nó. Theo tính chất cây đoạn, tổng cộng cần ghi $O(n \log n)$ giá trị. Dùng bảng băm lưu các giá trị ấy, bài toán được giải trong $O(n \log n)$ thời gian và bộ nhớ.

10.1.4 Duy trì mảng nhiều chiều cho dãy động

Một số bài yêu cầu duy trì một dãy động, có thao tác chèn và xóa. Phương pháp đánh nhãn động có thể xử lý loại bài này với thời gian chèn/khấu hao $O(\log n)$. Trước đây, để lưu phân bố giá trị trong một đoạn, ta thường cần cây đoạn theo nhãn lồng cây đoạn theo giá trị. Với công cụ mảng nhiều chiều, có một cách đơn giản hơn: lập mảng hai chiều với chiều thứ nhất là nhãn, chiều thứ hai là giá trị. Khi phần tử có nhãn i trong dãy hiện tại mang giá trị j , đặt ô $[i][j]$ bằng 1, còn lại bằng 0. Chiều nhãn vẫn dùng cây đoạn, chiều giá trị dùng cây Fenwick. Mỗi lần phân phối lại nhãn, xóa toàn bộ thông tin nhãn cũ khỏi mảng hai chiều rồi thêm thông tin nhãn mới. Tương tự, chỉ cần dùng bảng băm ghi các ô khác 0 trong cấu trúc “cây đoạn lồng cây Fenwick” của mảng hai chiều này.

Cũng có thể dùng mảng khối phân tầng để duy trì mảng nhiều chiều của dãy động, nhưng độ phức tạp thường cao hơn và hiệu quả không tốt.

10.1.5 Xử lý trên cây

Tiếp theo mở rộng phương pháp từ dãy sang cây. Một ý tưởng tự nhiên là dùng thứ tự DFS để chuyển bài toán trên cây thành bài toán trên dãy.

Định nghĩa 1.1 (thứ tự DFS). Duyệt DFS toàn bộ cây, mỗi đỉnh được ghi một lần khi vào ngăn xếp. Dãy thu được gọi là thứ tự DFS của cây. Đặt $\text{in}[i]$ là vị trí xuất hiện của đỉnh i trong dãy.

Định nghĩa 1.2 (thứ tự DFS Euler). Duyệt DFS toàn bộ cây, mỗi đỉnh được ghi một lần khi vào ngăn xếp và một lần nữa khi ra khỏi ngăn xếp. Dãy thu được gọi là thứ tự DFS Euler, còn gọi là thứ tự ngoặc. Đặt $\text{in}[i]$ là vị trí bản ghi vào của đỉnh i , còn $\text{out}[i]$ là vị trí bản ghi ra của đỉnh i .

Thông tin trên cây chủ yếu gồm hai loại: đường đi và cây con. Với cây con, dùng thứ tự DFS vì mỗi cây con là một đoạn liên tục trong thứ tự ấy. Với đường đi, cần dùng thứ tự DFS Euler. Hai tính chất quan trọng là:

Tính chất 1.3. Với hai đỉnh u, v không có quan hệ tổ tiên–hậu duệ, nếu $\text{out}[u] \leq \text{in}[v]$, thì trong đoạn $[\text{out}[u] \dots \text{in}[v]]$ của thứ tự DFS Euler, các đỉnh xuất hiện đúng một lần cùng với $\text{LCA}(u, v)$ chính là toàn bộ đỉnh trên đường đi $u \rightarrow v$.

Tính chất 1.4. Với hai đỉnh u, v , nếu u là tổ tiên của v , thì trong đoạn $[in[u] \dots in[v]]$ của thứ tự DFS Euler, các đỉnh xuất hiện đúng một lần chính là toàn bộ đỉnh trên đường đi $u \rightarrow v$.

Theo hai tính chất này, để xử lý thông tin của một đường đi trên cây, chỉ cần tìm trong một đoạn của thứ tự DFS Euler các đỉnh xuất hiện đúng một lần; LCA có thể xử lý riêng. Nếu phép toán là khả nghịch, có thể dùng cách “phép ngược triệt tiêu”: lần xuất hiện thứ hai của một đỉnh triệt tiêu hiệu ứng của lần xuất hiện thứ nhất. Một cách thường dùng dựa trên tính chất 1.4 là tách đường đi $u \rightarrow v$ thành

$$[1, in[u]] + [1, in[v]] - [1, in[LCA(u, v)]] - [1, in[LCA(u, v).pr]],$$

trong đó $x.pr$ là cha của đỉnh x .

10.2 Ứng dụng

10.2.1 Một lớp bài mà hiệu ứng phụ thuộc vào số lần xuất hiện

Có một lớp bài yêu cầu tính tổng hiệu ứng của một đoạn, một đường đi hoặc một cây con, nhưng hiệu ứng của một phần tử lại phụ thuộc vào số lần giá trị của chính phần tử ấy xuất hiện trong đoạn, đường đi hoặc cây con đó. Khi ấy các cấu trúc dữ liệu thông thường như cây đoạn hay cây cân bằng thường khó áp dụng; các phương pháp chia khối cũng không thật thuận lợi, còn duy trì mảng nhiều chiều lại phát huy hiệu quả.

Trước hết nêu một định nghĩa sẽ dùng về sau.

Định nghĩa 2.1 (dãy vị trí xuất hiện trước thứ k). Với dãy $A[1 \dots n]$, định nghĩa

$$L_k(i) = \max \left\{ 0, x \mid k = \sum_{\substack{j=x \\ A[j]=A[i]}}^{i-1} 1 \right\}.$$

Nói cách khác, $L_k(i)$ là vị trí lần xuất hiện thứ k trước i của một phần tử bằng $A[i]$; nếu không tồn tại thì bằng 0.

10.2.2 Ví dụ hai: homework

Tóm tắt đề bài. Cho dãy $A[1 \dots n]$ và nhiều truy vấn. Mỗi truy vấn cho l, r, a, b , với $1 \leq l \leq r \leq n$ và $0 \leq a < b$, yêu cầu trả lời:

- trong $A[l \dots r]$ có bao nhiêu phần tử có giá trị thuộc $[a \dots b]$;
- trong $[a \dots b]$ có bao nhiêu số xuất hiện ít nhất một lần trong $A[l \dots r]$.

Ràng buộc $1 \leq n \leq 10^5$, thời gian 10 giây. Nguồn: AHOI2013 Round 2 Day 2.

Lời giải. Trước hết rời rạc hóa A , để miền giá trị trở thành $[0 \dots n-1]$. Với câu hỏi thứ nhất, làm trực tiếp như bài phần tử nhỏ thứ K có sửa đổi: lập mảng hai chiều theo giá trị A' của A , rồi lấy tổng

$$\sum_{i=l}^r \sum_{j=a}^b A'[i][j],$$

duy trì bằng cây Fenwick hai chiều.

Với câu hỏi thứ hai, kết quả chính là kết quả câu hỏi thứ nhất trừ đi số phần tử trong $[l \dots r]$ mà vị trí xuất hiện trước đó gần nhất cũng nằm trong $[l \dots r]$. Tính dãy $L_1[]$ của A , rồi lập mảng hai chiều có chiều thứ nhất là chỉ số của A , chiều thứ hai là giá trị L_1 . Truy vấn tương ứng trở thành lấy tổng đoạn. Mỗi câu hỏi trong một thao tác đều có độ phức tạp $O(\log^2 n)$ khi dùng cây Fenwick.

10.2.3 Ví dụ ba: Candy Park

Tóm tắt đề bài. Cho một cây có n đỉnh, mỗi đỉnh i có trọng số $V[i]$. Ngoài ra cho một dãy $W[1 \dots n]$. Có hai loại thao tác:

- sửa trọng số của một đỉnh;
- hỏi tổng hiệu ứng của mọi đỉnh trên đường đi $u \rightarrow v$. Với một đỉnh i trên đường đi, nếu trọng số của nó là lần xuất hiện thứ k của giá trị ấy trên đường đi, thì hiệu ứng của nó là $V[i]W[k]$.

Ràng buộc $1 \leq n, Q \leq 10^5$, thời gian 30 giây. Nguồn: WC2013.

Lời giải. Đây là bài toán trên cây, nên trước hết lấy thứ tự DFS Euler. Một đường đi có thể chuyển thành tập các vị trí xuất hiện đúng một lần trong một đoạn; LCA xử lý riêng.

Lập mảng $C[1 \dots 2n][1 \dots 2n]$. Với $i \leq j$, $C[i][j]$ biểu thị ảnh hưởng gia lượng của sự tồn tại tại vị trí i trong thứ tự DFS Euler lên vị trí j . Khi hỏi một đoạn $[l \dots r]$, chỉ cần lấy tổng đoạn $C[l \dots r][l \dots r]$ hoặc dạng tương đương $C[1 \dots r][l \dots 2n]$. Đặt $V'[i]$ là trọng số của đỉnh ứng với vị trí i trong thứ tự DFS Euler. Với mảng C , một ô chỉ có ý nghĩa khi $i \leq j$ và $V'[i] = V'[j]$.

Với các trọng số xuất hiện hơn $n^{1/3}$ lần trong cây gốc, ta lưu riêng và xử lý đặc biệt khi truy vấn; số trọng số như vậy chỉ là $O(n^{2/3})$. Với các trọng số còn lại, tính giá trị của mọi ô tương ứng trong C . Cụ thể, giả sử $W[0] = 0$:

- Nếu vị trí i và j ứng với cùng một đỉnh, thì $C[i][j] = -2V'[i]W[1]$, vì hiệu ứng của hai lần xuất hiện đầu đều bị triệt tiêu.
- Nếu trong đoạn $[i + 1 \dots j]$ có một vị trí ứng với cùng đỉnh với j , thì $C[i][j] = 0$, vì hiệu ứng tại j đã bị triệt tiêu hoàn toàn; sự xuất hiện tại i không làm thay đổi nó.
- Nếu trong đoạn $[i \dots j]$ không có vị trí nào ứng với cùng đỉnh với j , giả sử trong đoạn ấy có K đỉnh có trọng số $V'[i]$ và đều xuất hiện đúng một lần. Khi i xuất hiện một lần,

$$C[i][j] = V'[i](W[K] - W[K - 1]),$$

tức cộng ảnh hưởng của đỉnh tại i . Khi i xuất hiện hai lần,

$$C[i][j] = V'[i](W[K - 1] - W[K]),$$

tức triệt tiêu ảnh hưởng của đỉnh tại i .

Khi sửa đổi, nếu trọng số liên quan xuất hiện không quá $n^{1/3}$ lần, thì tính lại thô toàn bộ ô ứng với trọng số ấy trong C , tổng cộng $O(n^{2/3})$ ô; nếu số lần xuất hiện vượt $n^{1/3}$, chỉ cần xử lý riêng. Nếu dùng cây Fenwick hai chiều để duy trì C , một thao tác tốn $O(n^{2/3} \log^2 n)$. Nếu dùng mảng khối phân tầng với số mũ 3, độ phức tạp một thao tác là $O(9n^{2/3})$; trong bài này cách chia khối tốt hơn.

10.2.4 Dùng mảng nhiều chiều để chuyển phép toán không khả nghịch thành khả nghịch

Thao tác trong bài cấu trúc dữ liệu có thể chia thành hai loại. Một số thao tác có phép ngược, như cộng một giá trị, xor, đảo ngược, lấy tổng đoạn; gọi là khả nghịch. Một số thao tác không có phép ngược, hoặc phép ngược rất khó hiện thực, như gán giá trị, lấy min/max đoạn, lấy phần tử nhỏ hoặc lớn thứ K ; gọi là không khả nghịch.

Thông thường, phép toán khả nghịch dễ xử lý hơn phép toán không khả nghịch. Vì vậy khi bài có phép toán không khả nghịch, có thể tìm cách chuyển nó thành khả nghịch. Mảng nhiều chiều thường giúp làm điều này. Ví dụ bài phần tử nhỏ thứ K có sửa đổi chuyển “nhỏ thứ K ” thành “tổng đoạn” bằng

cách thêm một chiều giá trị. Với thao tác gán, có thể xem giá trị được gán là một chiều mới, đồng thời ghi lại thời điểm mới nhất mà mỗi phần tử được gán từng giá trị; khi cần dùng giá trị của phần tử, lấy thời điểm lớn nhất đã ghi, rồi chuyển về tổng đoạn hoặc trực tiếp dùng giá trị lớn nhất.

10.2.5 Ví dụ bốn: Tree

Tóm tắt đề bài. Cho một cây có n đỉnh, mỗi đỉnh có một trọng số nguyên không âm. Có q truy vấn (u, v, z) , mỗi truy vấn yêu cầu tìm giá trị lớn nhất của $w \text{ xor } z$ trên mọi trọng số w của các đỉnh thuộc đường đi $u \rightarrow v$. Ràng buộc $1 \leq n, q \leq 10^5$, $0 \leq z, w < 2^{16}$, thời gian 5 giây. Bài gốc cho phép offline, nhưng ở đây xét bản bắt buộc trực tuyến. Nguồn: HDU 4757.

Lời giải. Bài vẫn là đường đi trên cây, nên lấy thứ tự DFS Euler và lập mảng hai chiều theo giá trị C : khi trọng số đỉnh i là j , đặt

$$C[\text{in}[i]][j] = 1, \quad C[\text{out}[i]][j] = -1,$$

các ô khác bằng 0. Theo tính chất 1.3, đường đi $u \rightarrow v$ có thể tách thành

$$[1, \text{in}[u]] + [1, \text{in}[v]] - [1, \text{in}[\text{LCA}(u, v)]] - [1, \text{in}[\text{LCA}(u, v). \text{pr}]].$$

Để tìm giá trị lớn nhất của $w \text{ xor } z$, duyệt bit từ cao xuống thấp. Ở mỗi bit, trước hết thử chọn những trọng số có bit khác với bit tương ứng của z ; nếu trên đường đi không có trọng số trong miền ấy thì chọn miền có bit giống z . Số đỉnh có trọng số trong một miền được tính trực tiếp bằng tổng đoạn của mảng C , duy trì bằng cây Fenwick hai chiều. Kết hợp với tính chất nhị phân tự nhiên của cây Fenwick, bài được giải trong

$$O((n + q) \log n \log W), \quad W = 2^{16}.$$

Ở đây phép “lấy giá trị lớn nhất” là không khả nghịch, nhưng thêm chiều giá trị đã biến nó thành phép tổng đoạn khả nghịch.

10.2.6 Ví dụ năm: Just for fun EXT

Tóm tắt đề bài. Cho n đỉnh ban đầu chưa có cạnh, mỗi đỉnh có một trọng số nguyên không âm. Có bốn loại thao tác:

- thêm cạnh giữa u và v ; nếu u, v đã liên thông thì bỏ qua;
- sửa trọng số của một đỉnh;
- hỏi đỉnh có trọng số lớn thứ k trên đường đi u tới v ; nếu u, v chưa liên thông thì bỏ qua;
- hỏi tích các trọng số không vượt quá z trên đường đi u tới v , lấy dư theo 28256292; nếu u, v chưa liên thông thì bỏ qua.

Ràng buộc $1 \leq n \leq 2 \cdot 10^5$, $1 \leq q \leq 4 \cdot 10^5$. Bài bắt buộc trực tuyến. Nguồn: HDU 4757, tác giả: Chen Lijie.

Lời giải. Trước hết xét trường hợp đơn giản: cây đã cho sẵn từ đầu, hoặc cho phép offline. Khi ấy làm tương tự ví dụ trước: lấy thứ tự DFS Euler và lập các mảng hai chiều theo giá trị. Cần năm mảng. Một mảng có giá trị tại vị trí tương ứng là 1 và -1 , cây Fenwick duy trì tổng đoạn để trả lời truy vấn trọng số lớn thứ k . Ba mảng tiếp theo lưu số mũ của các thừa số 2, 3 và 784897 trong trọng số cùng với số đối của chúng; cây Fenwick duy trì tổng đoạn. Mảng cuối lưu phần còn lại của trọng số sau khi bỏ hết các thừa

số 2, 3, 784897, cùng nghịch đảo modulo 28256292 của nó; cây Fenwick duy trì tích đoạn. Bốn mảng sau phục vụ truy vấn tích trọng số. Khi sửa trọng số, cập nhật các mảng tương ứng. Độ phức tạp là

$$O((n + q) \log n \log W), \quad W = 28256292,$$

với $28256292 = 2^2 \cdot 3^2 \cdot 784897$.

Quay lại bài gốc. Điểm khó nhất là thao tác thêm cạnh: cần gộp hai cây, tức gộp thứ tự DFS Euler và các mảng hai chiều theo giá trị tương ứng. Có thể dùng gộp theo kích thước: mỗi lần tháo toàn bộ cây nhỏ hơn, chọn đỉnh đầu mút tương ứng làm gốc để dựng lại cây có gốc, lấy thứ tự DFS Euler rồi chèn vào vị trí phù hợp trong thứ tự DFS Euler của cây lớn hơn và cập nhật mảng hai chiều theo giá trị.

Để hiện thực, một cách là xem thứ tự Euler như dãy động và dùng phương pháp đánh nhãn động. Cách khác là dùng mảng khối để duy trì các mảng hai chiều theo giá trị; dấu hiệu của mỗi khối không phải là vị trí trong thứ tự DFS Euler, mà là số hiệu đỉnh ứng với phần tử trái nhất của khối. Khi gộp, trước hết chia khối cho các mảng hai chiều của thứ tự Euler của cây nhỏ đã dựng lại, rồi chèn trực tiếp các khối ấy vào khối tương ứng của cây lớn. Việc duy nhất cần xử lý là khối cuối của cây nhỏ có thể phải gộp với khối sau vị trí chèn của cây lớn, nếu tổng kích thước không vượt quá $\sqrt{n}$. Cách dùng nhãn động cho thao tác thêm cạnh có độ phức tạp khấu hao $O(\log^2 n \log W)$, còn cách chia khối đạt $O((\sqrt{n} + \sqrt{W}) \log n)$.

10.2.7 Dùng mảng nhiều chiều để hiện thực bền vững

Tư tưởng bền vững có vai trò quan trọng trong bài cấu trúc dữ liệu. Về bản chất, bền vững là ghi lại trạng thái của cấu trúc dữ liệu ở mọi thời điểm, tức các phiên bản, rồi nén phần dùng chung giữa các phiên bản để truy cập được lịch sử. Khi hiện thực cấu trúc dữ liệu bền vững, thường biến “sửa” thành “chép”: chép phần tử sau sửa, không ghi đè phần tử cũ.

Mọi cấu trúc dữ liệu đều có thể lưu bằng mảng: dùng chỉ số mảng biểu diễn phần tử hoặc nút, còn các trường tham chiếu chỉ số biểu diễn quan hệ giữa các phần tử. Theo nghĩa này, lưu bằng địa chỉ và lưu bằng mảng là tương tự, chỉ khác là địa chỉ trong bộ nhớ được xem như chỉ số. Nếu lấy chỉ số làm chiều thứ nhất, lấy trục thời gian làm chiều thứ hai, ta được một mảng hai chiều. Khi nút chỉ số i bị sửa ở thời điểm j , đặt ô $[i][j]$ thành giá trị sau sửa, bao gồm cả các trường tham chiếu tới chỉ số nút khác. Khi cần giá trị của một nút ở một phiên bản lịch sử, chỉ cần tìm trong mảng hai chiều này lần sửa cuối cùng của nút đó không muộn hơn phiên bản cần hỏi.

Nếu mỗi lần chỉ truy cập một phần tử riêng lẻ, có thể không cần lưu bằng mảng hai chiều, mà dùng set của C++ STL: lập một mảng set, mỗi set lưu lịch sử sửa của một chỉ số và sắp theo thời gian.

10.2.8 Ví dụ sáu: Version Controlled IDE

Tóm tắt đề bài. Duy trì một xâu, có ba loại thao tác:

- chèn một xâu độ dài không quá 100 vào một vị trí;
- xóa một xâu con;
- hỏi ở thời điểm tm , tức trước thao tác chèn/xóa thứ tm , xâu con gồm các ký tự từ vị trí l tới r .

Ràng buộc $1 \leq q \leq 5 \cdot 10^5$, tổng độ dài các xâu được chèn không quá 10^6 , tổng độ dài các xâu được xuất trong truy vấn không quá $2 \cdot 10^5$. Nguồn: UVA 12538.

Lời giải. Với bài duy trì dãy động như xâu, thường dùng cây splay. Một xâu con có thể được biến thành một cây con hoàn chỉnh bằng thao tác splay, nên chèn và xóa về bản chất là chèn hoặc xóa một cây con.

Khó khăn là truy vấn phiên bản lịch sử. Vì splay cũng có thể lưu bằng mảng, chỉ cần dùng mảng lưu cây splay, chỉ số biểu diễn nút, rồi theo phương pháp ở trên dùng mảng hai chiều, hoặc một mảng các set, để lưu lịch sử sửa của từng nút; khi truy vấn thì gọi từ đó.

Còn một vấn đề khác: độ phức tạp của splay dựa trên phân tích khấu hao. Có thể tồn tại một phiên bản mà độ sâu của splay đạt cấp $O(n)$; nếu truy vấn đều hỏi phiên bản ấy thì không bảo đảm thời gian. Vì vậy sau mỗi lần chèn/xóa, nếu độ sâu hiện tại của splay quá lớn, cần chủ động thực hiện một số thao tác splay để điều chỉnh, chẳng hạn liên tục splay nút có độ sâu lớn nhất lên gốc cho tới khi độ sâu lớn nhất về cấp $O(\log n)$. Các sửa đổi nút phát sinh từ những thao tác splay này cũng phải được ghi lại.

Trong bài này, ta chỉ truy cập các phần tử đơn lẻ và dùng một mảng set để hiện thực bền vững, nên chưa thật sự phát huy sức mạnh của mảng nhiều chiều. Thực ra, bằng phương pháp “chỉ số + trục thời gian”, ta có thể kết hợp với các phép như tổng đoạn để làm các ứng dụng phức tạp hơn, chẳng hạn hỏi thông tin liên quan tới các phiên bản từ l tới r , hoặc đếm số phiên bản thỏa một điều kiện nào đó.

10.3 Tổng kết

Bản chất của mọi cấu trúc dữ liệu đều là mảng. Dùng mảng nhiều chiều thường có thể đơn giản hóa các cấu trúc phức tạp trong bài, biểu diễn trực quan hơn các quan hệ định lượng, từ đó khiến bài toán dễ giải hơn. So với phương pháp truyền thống trong bài cấu trúc dữ liệu, cách duy trì mảng nhiều chiều có ưu thế rất rõ về độ phức tạp. Việc tìm ra phương pháp này cũng chính là quá trình không thỏa mãn với phương pháp sẵn có, cải tiến phương pháp sẵn có cho tới khi có đổi mới. Thực ra, bản chất của mọi phương pháp mới đều là quá trình “phương pháp đã có + trí tuệ con người”.

Ta cần phát huy đầy đủ năng lực của trí tuệ con người trong việc khám phá, kiểm thử và cải tiến lời giải.

Tài liệu tham khảo

1. Chen Lijie, *Nghiên cứu cấu trúc dữ liệu bền vững*, 2012.
2. Xu Haoran, *Bàn luận về cấu trúc dữ liệu*, 2012.
3. List Order Maintenance & Monotonic List Labeling Density Maintenance.
4. http://en.wikipedia.org/wiki/Euler_tour_technique

11 Ứng dụng cây đoạn trong một lớp bài chia để trị

Xu Yinzhan, Trường Học Quân Hàng Châu, Chiết Giang

Tóm tắt

Cây đoạn và chia để trị theo thời gian đều là những thuật toán thường dùng trong thi tin học. Bài viết này nêu tư tưởng dùng cây đoạn để giải các bài chia để trị, đưa ra một phương pháp chuyển bài sửa đoạn thành bài chia để trị, mở rộng tư tưởng ấy sang một số bài bền vững đơn giản, rồi giới thiệu khái niệm và cách dựng cây ảo. Các ví dụ trong bài tương đối cơ bản, nhằm gợi mở cách dùng phương pháp này trong các bài khác.

11.1 Cây đoạn và chia để trị theo thời gian

11.1.1 Cây đoạn

Cây đoạn là một cấu trúc dữ liệu rất thông dụng. Gốc biểu diễn toàn bộ đoạn, mỗi nút không phải lá có một con trái và một con phải. Nếu nút đang xét biểu diễn đoạn $[l, r]$, thì con trái biểu diễn $[l, \lfloor (l+r)/2 \rfloor]$, còn con phải biểu diễn $[\lfloor (l+r)/2 \rfloor + 1, r]$.

Thời gian tìm các nút tương ứng với một đoạn trên cây đoạn là $O(\log n)$, trong đó n là độ dài toàn đoạn.

Chứng minh ngắn gọn như sau. Giả sử đoạn cần chen là $[a, b]$, nút i biểu diễn đoạn $[l_i, r_i]$, và

$$\text{mid}_i = \left\lfloor \frac{l_i + r_i}{2} \right\rfloor.$$

Nếu tại một lần thăm nút i ta phải đi xuống cả hai con, thì $a \leq \text{mid}_i$ và $\text{mid}_i < b$. Trong cây con trái của i , nếu lại có một nút j khiến ta phải đi xuống cả hai con, thì $a \leq \text{mid}_j$ và $\text{mid}_j < b$; do $\text{mid}_j \leq \text{mid}_i$, các nút ở nhánh phải được che phủ trọn bởi $[a, b]$. Trường hợp cây con phải tương tự. Vì vậy số lần phân nhánh ở mỗi tầng là hằng số, nên tổng số nút được thăm là $O(\log n)$.

11.1.2 Chia để trị theo thời gian

Chia để trị theo thời gian là một dạng chia để trị. Tư tưởng cơ bản là: để xử lý các thao tác sửa và hỏi trong khoảng thời gian $[l, r]$, trước hết xử lý đóng góp của các thao tác sửa trong nửa trái $[l, \lfloor (l+r)/2 \rfloor]$ lên các truy vấn trong nửa phải $[\lfloor (l+r)/2 \rfloor + 1, r]$, sau đó đệ quy trên hai nửa. Cách làm này yêu cầu đóng góp của từng sửa đối với truy vấn không ảnh hưởng lẫn nhau, các thao tác sửa độc lập với nhau, và bài toán cho phép xử lý offline. Nếu xử lý k thao tác sửa/hỏi tốn $O(f(k))$, thì theo định lý chủ, tổng thời gian là $O(f(n) \log n)$.

11.1.3 Bài chia để trị theo thời gian có thao tác thu hồi

Vì chia để trị theo thời gian yêu cầu các thao tác độc lập, nếu có thao tác dạng “thu hồi một thao tác trước đó” thì không thể áp dụng trực tiếp. Trong tình huống này, cần chuyển bài toán về một bài mới không có thao tác xóa. Một cách hay là sau một bước chia để trị thì đảo ngược thời gian, khi đó thao tác xóa biến thành thao tác thêm của chiều thời gian ngược. Phương pháp “dòng thời gian đảo ngược” này có thể xem thêm trong luận văn đội tuyển quốc gia năm 2013 của Xu Haoran.

Bài viết này đưa ra một phương pháp tổng quát khác, áp dụng cho các bài có thao tác thu hồi, đóng góp của sửa lên truy vấn độc lập với nhau, và cho phép offline. Đây là chủ đề chính của bài. Ta bắt đầu bằng một ví dụ.

Ví dụ một: giao nửa mặt phẳng động. Cho một mặt phẳng, cần hỗ trợ các thao tác:

- thêm một nửa mặt phẳng;
- xóa một nửa mặt phẳng hiện đang tồn tại;
- hỏi một điểm có nằm trong giao của các nửa mặt phẳng hiện tại hay không.

Nếu không có thao tác thứ hai, đây là một bài luyện tập rất phù hợp cho chia để trị theo thời gian.

Quan sát định nghĩa cây đoạn và phương pháp chia để trị theo thời gian ở trên, ta thấy chúng có liên hệ chặt chẽ: cây đoạn cũng liên tục chia $[l, r]$ thành $[l, \text{mid}]$ và $[\text{mid} + 1, r]$.

Trước hết xét bài con không có xóa. Trong chia để trị theo thời gian, mỗi bước xử lý đóng góp của sửa ở đoạn trái lên truy vấn ở đoạn phải; trên cây đoạn, điều này tương đương với xử lý đóng góp của các sửa trong con trái lên các truy vấn trong con phải. Khi chia đến cuối, đoạn còn lại có độ dài 1, tức là thao tác tương ứng được đặt tại một lá của cây đoạn.

Nói cách khác, chia để trị theo thời gian tương đương với một cây đoạn lấy thời gian làm chỉ số. Khác biệt chỉ là thứ tự xử lý: chia để trị thường xử lý đoạn lớn trước rồi đến đoạn nhỏ, còn cách nhìn cây đoạn có thể cho phép xử lý từ các đoạn nhỏ lên hoặc gom theo các nút cụ thể. Từ đây nảy sinh câu hỏi: nếu dùng thứ tự xử lý của cây đoạn để giải bài chia để trị, liệu có thu được một cách làm mạnh hơn hoặc tiện hơn không?

Cụ thể, dựng sẵn một cây đoạn có kích thước bằng số thao tác. Lần lượt chen mỗi thao tác vào lá ứng với thời điểm của nó. Với một nút không lá x , lấy mọi nửa mặt phẳng trong con trái của x , dựng bao trên hoặc bao dưới, rồi xét các điểm truy vấn ở con phải. Với một điểm (X, Y) , dùng tìm kiếm nhị phân để tìm nửa mặt phẳng cao nhất hoặc thấp nhất tại hoành độ X , rồi so sánh với Y . Nếu một điểm không nằm trong một bao nào đó thì đánh dấu vĩnh viễn là không nằm trong giao. Cách này có thời gian $O(n \log^2 n)$, với n là số thao tác. Nếu dùng trộn để sắp thứ tự hệ số góc của nửa mặt phẳng và hoành độ của điểm hỏi, có thể đạt $O(n \log n)$.

Tuy nhiên cách trên nhìn qua không khác nhiều so với chia để trị trực tiếp. Đổi góc nhìn: mỗi nửa mặt phẳng đóng góp cho những truy vấn nào? Chính là những truy vấn xảy ra sau thời điểm nó được thêm. Biết vậy, ta có thể đánh dấu trực tiếp trên cây đoạn theo thời gian mà nửa mặt phẳng ấy có hiệu lực, biểu thị rằng mọi điểm truy vấn trong đoạn đó cần nằm trong nửa mặt phẳng này.

Thuật toán trở thành: nếu một nửa mặt phẳng xuất hiện từ thời điểm T , thì đánh dấu thao tác ấy lên các nút cây đoạn phủ đoạn $[T, n]$. Sau đó, với mỗi nút của cây đoạn, lấy các nửa mặt phẳng đánh dấu tại nút ấy và các điểm truy vấn thuộc đoạn thời gian của nút, rồi làm giao nửa mặt phẳng và truy vấn.

Điều đáng mừng là thuật toán này mở rộng được sang trường hợp có thao tác xóa: mỗi nửa mặt phẳng vẫn tương ứng với một đoạn thời gian liên tục mà nó tồn tại. Phần còn lại giống cách xử lý vừa nêu.

Xét độ phức tạp. Bỏ qua sắp xếp, dựng một giao nửa mặt phẳng và kiểm tra một số điểm có nằm trong giao đó hay không đều là tuyến tính. Thứ tự điểm truy vấn có thể duy trì bằng trộn. Các nửa mặt phẳng có thể được sắp trước khi chen vào cây đoạn; nửa mặt phẳng được chen trước luôn đứng trước nửa mặt phẳng được chen sau, nên khi xử lý mỗi nút, dãy nửa mặt phẳng đã có thứ tự. Do đó thời gian và bộ nhớ đều là $O(n \log n)$.

Bộ nhớ của cách này kém hơn cách cổ điển. Có thể tối ưu bằng cách nhận ra rằng mỗi nửa mặt phẳng khi chen đoạn vào cây đoạn chỉ phân nhánh một lần, nên ở mỗi tầng của cây đoạn có không quá hằng số bản sao của cùng một nửa mặt phẳng. Vì vậy chỉ cần xử lý từng tầng một là có thể giữ bộ nhớ tuyến tính. Trực giác là ta thay toàn bộ cây bằng một tầng đang xét. Như vậy thu được cách làm dùng $O(n)$ bộ nhớ và $O(n \log n)$ thời gian.

Các đoạn trên cho thấy cây đoạn và chia để trị ở đây có cùng bản chất, chỉ khác hình thức biểu diễn. Chia để trị mỗi lần chỉ xét một đường đi trong cây đoạn, còn cây đoạn biểu diễn toàn bộ quá trình chia để trị. Nhờ mô hình cụ thể hơn, trực quan hơn, cây đoạn giúp ta xử lý bài toán rõ ràng và dễ nghĩ ra tối ưu hơn; còn chia để trị vẫn có ưu điểm là viết nhanh và gọn.

Ví dụ hai: Homework. Cho một dãy n số. Mỗi truy vấn hỏi trong các vị trí từ l tới r , số phân tử có giá trị thuộc $[a, b]$, và số giá trị phân biệt trong các phân tử ấy. Có Q truy vấn, với $n \leq 100000$, $Q \leq 1000000$. Nguồn bài: AHOI 2013.

Vì số giá trị phân biệt liên quan mạnh tới giá trị, ta dựng cây đoạn theo miền giá trị. Ở các nút giá trị khác nhau, các số chắc chắn khác nhau. Mỗi nút của cây đoạn giá trị lưu vị trí của những phân tử có giá trị nằm trong miền mà nút biểu diễn. Một truy vấn $[a, b]$ ứng với $O(\log n)$ nút của cây đoạn giá trị; trong

từng nút, cần hỏi số vị trí thuộc $[l, r]$ và số giá trị phân biệt tương ứng. Hai bài con này đều xử lý được bằng cây Fenwick. Tổng bộ nhớ là $O(n + Q)$, thời gian là $O((n + Q) \log^2 n)$.

11.1.4 Một phương pháp chuyển đổi tổng quát

Trong các cuộc thi hiện nay, bài có thao tác kiểu “thu hồi một lần thêm trước đó” không nhiều, nên phương pháp trên không phải lúc nào cũng áp dụng trực tiếp. Ta đưa ra một cách chuyển thao tác dạng “biến mọi phần tử trong $[l, r]$ thành cùng một giá trị” thành các thao tác thêm và thu hồi.

Cách làm như sau. Lần lượt chèn các thao tác sửa vào một cây đoạn theo vị trí, và đánh dấu lên các nút phủ $[l, r]$. Nếu trong quá trình chèn gặp một nút đã có đánh dấu, đẩy đánh dấu của nút ấy xuống hai con và xóa đánh dấu ở nút hiện tại. Khi một thao tác sửa đã tìm được các nút tương ứng trên cây đoạn, lấy ra và xóa mọi đánh dấu trong các cây con của những nút ấy. Nếu thời điểm chèn của thao tác sửa hiện tại là i , thời điểm chèn của một đánh dấu được lấy ra là j , và vị trí của đánh dấu ấy trên cây đoạn là đoạn $[L, R]$, thì trong bài đã chuyển đổi thêm hai thao tác:

- tại thời điểm j , biến mọi phần tử ở các vị trí $[L, R]$ thành một giá trị;
- tại thời điểm i , thu hồi thao tác thứ nhất.

Khi hiện thực, chỉ cần ghi nhận trong cây con của mỗi nút có đánh dấu nào không. Nếu không có thì khỏi tìm tiếp; nếu có thì tiếp tục đi xuống để lấy các đánh dấu.

Tính đúng đắn có thể thấy như sau. Trong bài phủ đoạn, tại bất kỳ thời điểm nào, phần tử thứ p của dãy chính là giá trị do đánh dấu cuối cùng trên đường từ lá p lên gốc biểu diễn. Trong cây con của một nút đã có đánh dấu, các đánh dấu khác đều không còn tác dụng đối với phủ đoạn, nên xóa chúng không làm thay đổi kết quả. Mỗi lần chèn một thao tác sửa, ta đang lấy ra mọi phần tử trong đoạn tương ứng và đổi chúng thành giá trị mới, chỉ khác là các phần tử liên tiếp và cùng giá trị được gộp lại.

Xét độ phức tạp. Sau mỗi lần chèn, trong cây con của một nút có đánh dấu sẽ không còn đánh dấu khác. Vì vậy khi chèn vào một nút, đánh dấu chỉ bị đẩy xuống một lần, không bị dây chuyền do vị trí đẩy xuống lại có đánh dấu. Một lần đẩy xuống làm tăng số đánh dấu thêm $O(1)$, và làm tổng độ sâu các đánh dấu tăng $O(\log n)$. Nếu m là số thao tác sửa và n là độ dài dãy, độ phức tạp chèn tổng cộng là $O(m \log^2 n)$, bao gồm cả thời gian chèn và đẩy xuống; chi phí xóa không vượt quá chi phí chèn. Số thao tác trong bài mới không vượt quá tổng số đánh dấu, tức $O(m \log n)$.

Mức này chưa đủ tốt. Quan sát từ dãy ban đầu, mỗi thao tác sửa đoạn mới chỉ phủ hoàn toàn một số đoạn giá trị bằng nhau, đồng thời cắt hai đoạn giá trị bằng nhau ở hai đầu. Mỗi lần nhiều nhất sinh thêm hai đoạn giá trị mới, nên tổng số đoạn giá trị khác nhau cuối cùng là $O(m)$. Ví dụ, phủ giá trị 6 lên dãy 1112334445555 từ vị trí thứ 3 tới vị trí áp chót sẽ phủ trọn các đoạn 2, 33, 444, còn 111 bị tách thành 1 và 11, 5555 bị tách thành 55 và 55.

Quay lại cây đoạn, nhiều đánh dấu bị xóa thực ra tương ứng với một đoạn liên tục trên dãy; chỉ cần gộp các đánh dấu ấy lại thì tổng số thao tác trong bài mới là $O(m)$. Tìm mọi đánh dấu gộp được tốn $O(\log n)$, tương đương tìm các nút cây đoạn phủ một đoạn. Vì vậy độ phức tạp xây dựng là $O(m \log n)$.

Ví dụ ba: một bài đoạn đơn giản. Cho một dãy dài m , ban đầu mọi phần tử có giá trị cho trước. Cần hỗ trợ:

- đổi mọi phần tử từ vị trí l tới vị trí r thành x ;
- hỏi trong các vị trí từ l tới r có bao nhiêu phần tử có giá trị không vượt quá x .

Có tổng cộng n thao tác. Để đơn giản ký hiệu, giả sử $m = O(n)$.

Dùng phương pháp chuyển đổi ở trên. Trong bài mới, thao tác thứ nhất biến thành thêm một giá trị x

vào mọi vị trí thuộc $[l, r]$, cùng với các thao tác thu hồi một số lần thêm cũ. Theo phương pháp phân trước, ta tiếp tục đưa bài toán về các bài con chỉ có thao tác thêm một giá trị vào đoạn và mọi thao tác thêm đều đứng trước truy vấn. Với mỗi bài con, sắp theo giá trị rồi dùng cây đoạn duy trì cộng đoạn và hỏi tổng đoạn.

Ban đầu có $O(n)$ thao tác sửa; sau bước chuyển thứ nhất vẫn có $O(n)$ thao tác, và sau khi đưa lên cây đoạn thời gian có $O(n \log n)$ thao tác. Mỗi thao tác còn phải cập nhật trên cây đoạn vị trí, nên phần này tốn $O(n \log^2 n)$. Mỗi truy vấn xuất hiện ở $O(\log n)$ nút của cây đoạn thời gian, và mỗi lần cần hỏi tổng đoạn, nên phần truy vấn cũng tốn $O(n \log^2 n)$. Tổng thời gian là $O(n \log^2 n)$.

Do đặc thù của bài này, sau bước chuyển đầu tiên, thao tác thu hồi có thể xem như một thao tác thêm với trọng số -1 . Khi đó có thể viết theo chia để trị thời gian thông thường để giảm hằng số, nhưng bậc độ phức tạp không đổi.

Ví dụ bốn: Robot. Có n robot và m ổ cắm xếp thành một hàng; ban đầu mọi ổ cắm đều trống.

Tại mỗi thời điểm, có một robot lên sân khấu. Robot i chọn một robot j đang cắm phích vào ổ k , rồi hấp thụ năng lượng của nó; bảo đảm lúc ấy robot j có phích ở ổ k . Nếu năng lượng của robot j là f_j , thì năng lượng của robot i là

$$f_j + (i - j)^2 + (i - k)^2 + (j - k)^2.$$

Sau khi bị hấp thụ, năng lượng của robot j không mất đi. Nếu không có robot nào có thể hấp thụ, năng lượng của robot i là 0.

Sau khi nhận năng lượng, robot i cắm phích của mình vào mọi ổ thuộc một đoạn $[l, r]$; mọi phích đang cắm ở các ổ ấy bị rút ra. Tại thời điểm $i + M$, robot i rút mọi phích của mình và rời đi. Giá trị M là như nhau với mọi robot. Hỏi năng lượng lớn nhất mỗi robot có thể nhận được.

Trước hết bỏ qua thao tác rút phích ở thời điểm $i + M$. Dù không biết trước các giá trị f_i , đoạn hiệu lực của mỗi robot là đã biết. Sau khi xử lý bằng phương pháp ở trên, mỗi nút của cây đoạn chỉ có các thao tác kiểu “thêm robot j đang cắm một đoạn phích”. Xử lý mọi thao tác theo thứ tự trung tự của cây đoạn là có thể thu được mọi giá trị f_i .

Vì

$$f_i = \max_j (f_j + (i - j)^2 + (i - k)^2 + (j - k)^2),$$

nếu i, j đã cố định thì biểu thức $(i - k)^2 + (j - k)^2$ đạt lớn nhất tại một trong hai đầu của đoạn ổ cắm. Do đó chỉ cần lấy hai đầu của mỗi đoạn, còn phần giữa là vô dụng. Ta có

$$f_j + (i - j)^2 + (i - k)^2 + (j - k)^2 = f_j + (j - k)^2 + j^2 + k^2 - 2(j + k)i + 2i^2.$$

Chuyển trạng thái này giải được bằng tối ưu hóa độ dốc trong $O(n)$ cho mỗi tập cần xử lý. Thời gian là $O(n \log n + n \log m)$.

Thao tác rút phích chỉ cần tìm tại thời điểm $i + M$, các đánh dấu của robot i hiện nằm ở những nút nào của cây đoạn, rồi lấy chúng ra; thao tác này giống thu hồi thông thường, chỉ cần đánh dấu trên cây đoạn thời gian. Độ phức tạp vẫn là $O(n \log n + n \log m)$.

EXT. Giữ nguyên mọi điều kiện của bài, chỉ đổi yêu cầu từ năng lượng lớn nhất thành năng lượng nhỏ nhất.

Khi đổi sang nhỏ nhất, chỉ lấy hai đầu của mỗi đoạn là sai. Giá trị nhỏ nhất của

$$f_j + (i - j)^2 + (i - k)^2 + (j - k)^2$$

còn có thể đạt tại $k = (i + j)/2$. Khi đó cần

$$l \leq \frac{i+j}{2} \leq r, \quad \text{tức} \quad 2l - j \leq i \leq 2r - j.$$

Có thể trên nền cách làm cũ thực hiện thêm một lần chia để trị tương tự. Do miền của k là số nguyên, cần xử lý cẩn thận biên, nhưng điều này không làm đổi độ phức tạp. Thời gian là $O(n \log^2 n)$.

11.2 Chia để trị liên quan tới cấu trúc cây

11.2.1 Bền vững hóa bài toán dãy

Bền vững hóa là ứng dụng kinh điển của nhiều cấu trúc dữ liệu. Ở đây chỉ xét loại bền vững đơn giản: “đưa trạng thái toàn cục trở về sau một thao tác nào đó”.

Tại mỗi thời điểm T , có thể có nhiều thao tác sửa khác nhau biến trạng thái hiện tại thành các nhánh mới T' . Những nhánh mới này có hậu duệ riêng và không liên quan tới trạng thái ở nhánh khác. Quay về một thao tác tức là quay về một nút trên cây phiên bản. Vì vậy quan hệ giữa các thao tác sửa cuối cùng tạo thành một cấu trúc cây.

Mỗi thao tác sửa ảnh hưởng tới mọi nút trong cây con phiên bản của nó. Khi DFS một cây, các nút trong một cây con được thăm trên một đoạn thời gian liên tục. Vì vậy, khi vừa đi vào cây con x , ta thêm thao tác do x biểu diễn; khi rời cây con x , ta thu hồi thao tác ấy. Thu hồi của một thao tác thu hồi lại là một thao tác thêm. Sắp các nút của cây phiên bản theo thứ tự DFS, ta chuyển bài toán về bài toán trên dãy mà số thao tác không đổi bậc.

11.2.2 Cây ảo

Trong chia để trị trên dãy, độ phức tạp của mỗi bài con không được phụ thuộc vào độ dài toàn dãy, mà chỉ được phụ thuộc vào số phần tử trong bài con. Với dãy, chỉ cần sắp các phần tử theo vị trí là tránh được phụ thuộc vào độ dài toàn cục.

Nhưng cấu trúc cây phức tạp hơn; không thể chỉ sắp thứ tự là nắm được quan hệ giữa các nút trong bài con. Vì vậy cần dùng cây ảo để tối ưu độ phức tạp.

Định nghĩa. Giả sử cây gốc T có hình dạng cố định và có n nút. Cây ảo của nó là một cây con liên thông của T , hoặc là cây thu được từ một cây con liên thông của T sau khi co các chuỗi cạnh liên tiếp không phân nhánh thành một cạnh. Vì các cạnh mới này không nhất thiết tồn tại trong T , ta gọi nó một cách hình tượng là cây ảo.

Với cây có hình dạng thay đổi cũng có thể dựng cây ảo, nhưng điều đó không liên quan nhiều tới bài này. Khái niệm “cây con” ở đây gần với đồ thị con, không phải cây con của một cây có gốc.

Cách dựng. Trong bài toán thực tế, điều kiện dựng cây ảo thường là: tìm một cây ảo nhỏ, bao hàm mọi điểm khóa $x_1, x_2, \dots, x_m$, và giữ đúng quan hệ giữa chúng.

Trước hết, nếu một điểm là điểm khóa, nó chắc chắn phải là một nút trong cây ảo cuối cùng. Nếu lấy mọi điểm nằm trên các đường đi giữa các điểm khóa, ta được một cây con hợp lệ, dù có thể rất không tối ưu. Xét một điểm không khóa trong đó. Xóa điểm này khỏi cây thì toàn cây bị chia thành nhiều cây con; ít nhất hai cây con chứa điểm khóa, nếu không điểm ấy đã không được chọn. Nếu chỉ cần giữ quan hệ phân nhánh giữa các điểm khóa, thì những điểm không tạo phân nhánh thật có thể bị co đi; ngược lại, điểm tạo phân nhánh cần được giữ làm nút cây ảo.

Cách dựng trực tiếp như vậy hơi rườm rà, có thể nói điều kiện một chút. Xem cây ban đầu là cây có gốc. Nếu một điểm thỏa điều kiện trở thành nút cây ảo, thì trong các cây con bị nó chia ra, ít nhất hai cây con là cây con của các con trực tiếp của nó; điểm ấy chính là LCA của hai điểm khóa nằm trong hai cây con đó. Do đó chỉ cần thêm vào tập nút cây ảo các LCA của từng cặp điểm khóa liên nhau theo thứ tự DFS.

Giải thích vì sao chỉ cần các cặp liên nhau. Với hai điểm x, y , trong kỹ thuật Euler tour, LCA của chúng là điểm có độ sâu nhỏ nhất trên đoạn giữa hai lần xuất hiện tương ứng. Nếu ba điểm x, y, z có thứ tự trong Euler tour là x, y, z , nhờ tính kết hợp của truy vấn min trên đoạn, ta có

$$h[\text{LCA}(x, z)] = \min(h[\text{LCA}(x, y)], h[\text{LCA}(y, z)]).$$

Trên đoạn này điểm có độ sâu nhỏ nhất là duy nhất, nên $\text{LCA}(x, z)$ bằng một trong hai LCA ở vế phải. Vì vậy, chỉ cần sắp $x_1, x_2, \dots, x_m$ theo thứ tự DFS, rồi lấy thêm $\text{LCA}(x_i, x_{i+1})$ với $1 \leq i < m$; cùng với các x_i , đó là tập nút cây ảo.

Sau khi có các nút cây ảo, mô phỏng DFS của cây ảo theo thứ tự DFS của cây gốc là dựng được các cạnh giữa chúng. Cụ thể, sắp các nút cây ảo theo thứ tự DFS trong cây gốc rồi chèn lần lượt. Duy trì đường đi từ gốc cây ảo tới nút hiện tại. Khi chèn nút x , kiểm tra nút cuối y trên đường đi hiện tại có phải tổ tiên của x không. Nếu không, loại y khỏi đường đi và tiếp tục kiểm tra; nếu có, nối cạnh giữa x và y , rồi đưa x vào đường đi.

Phương pháp dựng cây ảo này tốn $O(m \log n)$, và kích thước cây ảo dựng ra là $O(m)$. Có thể kiểm tra quan hệ tổ tiên bằng đoạn DFS của cây con trong cây gốc, tốn $O(1)$ sau tiền xử lý.

Thao tác trên cây ảo. Các thao tác trong bài thường không nhắm trực tiếp vào nút cây ảo, mà nhắm vào nút của toàn cây. Trong cây ảo, ta có thể xem trọng số của các nút không thuộc cây ảo là vô dụng. Vì vậy, thao tác trên một cây ảo nghĩa là thao tác lên những nút vừa thuộc cây ảo vừa liên quan tới thao tác đang xét.

Thao tác trên cây thường có hai loại: đường đi và cây con. Thao tác cây con có thể xử lý trực tiếp bằng thứ tự DFS, nên ở đây chỉ xét thao tác trên một đường đi (X, Y) trong cây gốc. Đường đi này tách được thành hai đường đi (X, Z) và (Y, Z) , trong đó $Z = \text{LCA}(X, Y)$. Tìm trong các tổ tiên của X , kể cả X , nút a sâu nhất thuộc cây ảo; tương tự tìm nút b cho Y . Nếu Z là nút cây ảo, đường đi tương ứng trong cây ảo là (a, b) . Nếu không, nó tương ứng với hai đường đi từ a tới ngay dưới Z , và từ b tới ngay dưới Z . Có thể chứng minh nhiều nhất một trong hai đường đi này là hữu ích; nếu độ sâu của cả a và b đều lớn hơn Z , thì Z bắt buộc phải là nút cây ảo.

Sau khi biết a , dùng nhảy bậc hai để tìm tổ tiên c của a có độ sâu lớn hơn Z nhưng nhỏ nhất có thể; đường đi (a, c) là phần cần thao tác.

Để tìm các nút a, b , có thể sắp các nút cây ảo theo độ sâu tăng dần. Với mỗi nút cây ảo c , gán trọng số c cho mọi nút trong cây con của c . Khi đó trọng số của X chính là a , trọng số của Y chính là b . Các phép gán và hỏi này xử lý bằng cây đoạn trên thứ tự DFS. Tiền xử lý tốn $O(n \log n)$, còn mỗi lần tìm nút cây ảo tương ứng tốn $O(\log n)$.

Ví dụ năm: Ký túc xá của Jc. Cho một cây n nút, mỗi nút có một trọng số. Có Q truy vấn; mỗi truy vấn lấy các trọng số trên đường đi (x, y) đưa vào mảng w , sắp tăng dần, rồi hỏi

$$\sum_{1 \leq k \leq m} k w_k.$$

Ràng buộc $1 \leq n, Q \leq 50000$, bộ nhớ 3 GB. Nguồn ghi trong PDF là của tác giả.

Chia các trọng số thành các khối theo thứ tự tăng dần. Với mỗi khối, dựng cây ảo gồm các nút trong khối, rồi tiền xử lý đáp án giữa mọi cặp nút của cây ảo. Việc này có thể làm bằng cách xuất phát từ từng nút, DFS trên cây ảo và duy trì một cây cân bằng trên đường đi. Đáp án giữa hai nút ở đây nghĩa là đáp án của đường đi nếu chỉ xét các nút thuộc cây ảo của khối này.

Với mỗi truy vấn, lần lượt hỏi trên cây ảo của từng khối: đáp án trên đường đi, số nút trên đường đi, và tổng trọng số trên đường đi. Đáp án đã có từ tiền xử lý; số nút và tổng trọng số có thể tiền xử lý bằng tổng trên đường từ nút tới gốc rồi trừ đi.

Giả sử có X khối. Với khối k , gọi đáp án nội bộ là ans_k , tổng trọng số là w_k , và tổng số nút trong các khối trước đó là S_{k-1} . Đáp án cuối cùng là

$$\sum_{1 \leq k \leq X} (\text{ans}_k + w_k S_{k-1}).$$

Tính đúng đắn hiển nhiên: mọi nút thuộc khối k đứng sau đúng S_{k-1} nút ở các khối trước nó.

Nếu kích thước mỗi khối là T , tiền xử lý tốn $O(n^2 \log n / T)$, còn mỗi truy vấn tốn $O(T \log n)$. Chọn $T = \sqrt{n}$, ta được $O((n + Q) \sqrt{n} \log n)$. Thực ra, số điểm giữa mọi cặp nút của cây ảo, tổng trọng số giữa chúng, và nút cây ảo tương ứng với mỗi nút của cây gốc đều có thể tiền xử lý. Khi đó mỗi truy vấn tốn $O(n/T)$. Chọn $T = \sqrt{n / \log n}$, độ phức tạp thành $O((n + Q) \sqrt{n \log n})$.

Ví dụ sáu: CHANGE. Cho một cây có gốc gồm n nút. Mỗi nút có hai thuộc tính a, b ; ban đầu mọi thuộc tính giống nhau. Có Q thao tác:

- đổi giá trị a của một cây con thành một giá trị cho trước;
- đổi giá trị b của một cây con thành một giá trị cho trước;
- hỏi trên một đường đi (x, y) , trong các nút có $a \in [X, Y]$, giá trị lớn nhất của b .

Nguồn bài: tác giả.

Vì sửa cây con tương đương sửa một đoạn trong thứ tự DFS, có thể dùng phương pháp ở mục 1.4 để chuyển thành thêm và thu hồi thao tác. Tuy nhiên nếu chỉ chuyển như vậy sẽ làm thuật toán rối. Do mỗi lần đều sửa cây con, sau mỗi thao tác, các đoạn được chuyển đổi và có cùng đánh dấu sẽ gộp được thành một đồ thị con liên thông. Ta chèn các đồ thị con liên thông này vào cây đoạn thời gian.

Với mỗi nút của cây đoạn thời gian, dựng một cây đoạn giá trị theo a . Mỗi nút của cây đoạn giá trị chứa một số thao tác sửa giá trị b của một đồ thị con liên thông. Xét cách xử lý tại một nút của cây đoạn giá trị.

Với mỗi thao tác sửa đồ thị con liên thông, lấy điểm có độ sâu nhỏ nhất trong đồ thị con ấy làm điểm khóa; điểm này là duy nhất. Dựng cây ảo chứa các điểm khóa đó. Đồng thời dùng cây đoạn để sửa giá trị b trong đồ thị con liên thông thành giá trị cho trước; chỉ cần sửa những đoạn thu được từ cách dựng ở mục 1.4.

Với mỗi truy vấn, đường đi luôn tách được thành hai đường đi hướng lên tổ tiên. Một đường đi có hướng như vậy hoặc nằm trong một đồ thị con liên thông, hoặc đi qua điểm có độ sâu nhỏ nhất của đồ thị con ấy rồi sang đồ thị con kế tiếp. Vì vậy chỉ cần tìm giá trị b lớn nhất trong các điểm khóa mà đường đi đi qua, và giá trị b tại điểm cuối cùng mà đường đi đạt tới; giá trị lớn hơn trong hai giá trị này chính là đóng góp của nút cây đoạn giá trị hiện tại cho truy vấn.

Tóm lại: chuyển sửa cây con thành thao tác thêm một cặp (a, b) trên một đồ thị con liên thông, cùng với thao tác thu hồi một lần thêm nào đó. Chèn mỗi thao tác thêm vào cây đoạn thời gian tương ứng. Với mỗi nút của cây đoạn thời gian, lấy các thao tác thêm bao phủ nó, dựng cây đoạn giá trị theo a , rồi dựng cây ảo ở từng nút cây đoạn giá trị. Với các truy vấn nằm trong nút cây đoạn thời gian này, tìm các nút cây đoạn giá trị phủ $[X, Y]$, rồi hỏi trên các cây ảo tương ứng.

Cây đoạn thời gian nhân số nút lên $\log Q$, cây đoạn giá trị nhân tiếp $\log n$, và các thao tác trong cây đoạn giá trị đều tốn $O(\log n)$. Do đó độ phức tạp là $O(Q \log^2 n \log Q)$, bỏ qua một số chi phí tiền xử lý trên cây gốc.

11.3 Tổng kết

Phương pháp kết hợp cây đoạn với chia để trị này có hiệu quả tốt với một lớp bài sửa đoạn. Ưu điểm của nó là mã nguồn tương đối đơn giản, tư tưởng rõ ràng, và chạy hiệu quả; trong bài toán thực tế có phạm vi ứng dụng khá rộng. Khi kết hợp với cây ảo, nó còn có thể giải một số bài khó hơn.

Lời cảm ơn

- Cảm ơn cha mẹ và nhà trường đã nuôi dưỡng, đào tạo tôi.
- Cảm ơn các bạn Du Yubei, Huang Jiatai, Zhou Zikai và nhiều bạn học khác đã giúp đỡ tôi.
- Cảm ơn CCF đã cho tôi cơ hội thể hiện bản thân.

Tài liệu tham khảo

1. Chen Danqi, *Từ Cash bàn về ứng dụng của một lớp thuật toán chia để trị*.
2. Xu Haoran, *Bàn sơ về một vài lời giải phi kinh điển cho bài cấu trúc dữ liệu*.
3. Chen Lijie, *Báo cáo lời giải JustForFun EXT*.

12 Thuật toán căn bậc hai: không chỉ là chia khối

Wang Yuetong, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Thuật toán căn bậc hai ngày càng phổ biến, và độ phức tạp của các bài OI cũng ngày càng “đa dạng”, không còn chỉ là những dạng đơn giản như $O(n)$, $O(n^2)$, hay $O(n \log n)$. Nhiều bài có độ phức tạp chứa căn bậc hai; trong đó các thuật toán căn bậc hai ở bài cấu trúc dữ liệu là thường gặp nhất, chẳng hạn duy trì theo khối hoặc thuật toán Mo, vốn đã được trình bày kỹ trong năm trước.

Nhưng thuật toán căn bậc hai tuyệt đối không chỉ giới hạn ở những chỗ ấy. Chúng còn xuất hiện trong nhiều thuật toán khác, đi kèm những tư tưởng rất khác nhau. Nghiên cứu sâu các tư tưởng ấy đôi khi còn đào được những ứng dụng có độ phức tạp tốt hơn cả mức căn. Bài viết này giới thiệu vài ứng dụng của thuật toán căn bậc hai ngoài các bài cấu trúc dữ liệu, rồi thử di chuyển tư tưởng ấy để thu được thuật toán tốt hơn.

12.1 Các bài toán cơ bản

12.1.1 Bắt đầu từ một bài dễ

Ví dụ một: Codeforces #80 Div1 D. Cho một dãy $a[i]$ độ dài $N \leq 300000$, và $M \leq 300000$ truy vấn. Mỗi truy vấn có dạng (x, y) ; cần xuất

$$a[x] + a[x + y] + a[x + 2y] + \dots + a[x + ky],$$

trong đó $x + (k + 1)y > N$. Giới hạn thời gian là 4 giây.

Truy vấn của bài này khác những truy vấn thông thường. Phần lớn cấu trúc dữ liệu ta học đều giải xử lý các đoạn liên tiếp, nhưng không giỏi xử lý những vị trí cách đều nhau. Với bài này, cây đoạn và các

cấu trúc truyền thống tương tự khó phát huy tác dụng. Tuy nhiên, nếu mọi truy vấn có cùng y , ta có thể chia dãy gốc thành y nhóm theo giá trị chỉ số modulo y . Trong nhóm tương ứng, truy vấn trở thành truy vấn đoạn liên tiếp và chỉ cần tổng tiền tố.

Vấn đề là với $y = 1$ ta cần duy trì vài mảng có tổng kích thước N ; với $y = 2, y = 3, 4, \dots, N$ cũng đều cần, nên không thể chấp nhận. Nhưng nếu y rất lớn thì vì sao phải lưu trước? Quét trực tiếp cũng nhanh, vì mỗi lần chỉ tốn $O(N/y)$. Do đó, lấy $K = \sqrt{N}$: với $1 \leq y \leq K$, tiền xử lý tổng tiền tố theo lớp modulo y ; với $y > K$, trả lời bằng quét trực tiếp. Độ phức tạp được bảo đảm ở mức $O(N\sqrt{N})$.

Nhìn lại bài này, không thể nói nó hoàn toàn không phải bài cấu trúc dữ liệu, nhưng nó thể hiện một tư tưởng rất khác với chia khối thông thường. Hai đại lượng ràng buộc nhau theo quan hệ tích: bước nhảy và số hạng, luôn có ít nhất một đại lượng không quá lớn. Ta thiết kế thuật toán riêng cho hai tình huống, rồi ghép hai thuật toán ấy thành một lời giải hoàn chỉnh.

Ví dụ hai: Topcoder SRM589 Level-3 FlippingBits. Cho một xâu nhị phân độ dài $N \leq 300$, và một số nguyên dương M . Mỗi thao tác có thể đảo một vị trí, hoặc đảo một tiền tố có độ dài là bội của M . Hỏi cần ít nhất bao nhiêu thao tác để biến xâu thành một xâu tuần hoàn có chu kỳ M .

Chu kỳ M nghĩa là với mọi i , nếu vị trí $i + M$ tồn tại thì ký tự ở i và $i + M$ phải bằng nhau.

Bài này cũng không khó. Xét độ dài chu kỳ và số chu kỳ; tích của hai đại lượng này là độ dài xâu gốc, nên luôn có một đại lượng không vượt quá $\sqrt{N}$. Điều đó gợi ý thiết kế hai thuật toán chuyên biệt.

Trước hết xét khi chu kỳ không dài, cụ thể không quá 17, xấp xỉ $\sqrt{300}$. Khi đó vì đáp án yêu cầu mọi chu kỳ đều giống nhau, ta có thể liệt kê trực tiếp chu kỳ đích. Sau khi liệt kê, mỗi đoạn độ dài M trong xâu gốc phải bằng chu kỳ ấy hoặc bằng chu kỳ ấy sau khi đảo bit. Phần còn lại là một quy hoạch động đơn giản: nếu ghi lại mỗi đoạn có bị đảo toàn bộ hay không, thì số lần dùng thao tác đảo tiền tố bội của M chính là số lần dãy trạng thái đoạn đổi giữa 0 và 1.

Trường hợp còn lại là số chu kỳ không nhiều. Khi đó thao tác đảo tiền tố bội của M chỉ có thể xảy ra ở không quá $\sqrt{N}$ vị trí, nên vẫn có thể liệt kê. Lúc này, ta đã biết từng đoạn có bị đảo hay không và cần xây một chu kỳ đích “phù hợp” với chúng. Việc này cũng đơn giản: với từng vị trí trong chu kỳ đích, nếu sau khi xét mọi đoạn số ký tự 0 nhiều hơn thì đặt là 0, nếu số 1 nhiều hơn thì đặt là 1.

Như vậy toàn bộ bài được giải quyết với độ phức tạp $O(N2^{\sqrt{N}})$. Bản chất vẫn giống ví dụ trước: kết hợp hai thuật toán thô sơ để thu được một cách làm đủ hiệu quả. Quan trọng hơn, kiểu “thuật toán tổ hợp” này đôi khi gần như không thể thay thế: có những bài chỉ có thể giải bằng cách phối hợp nhiều thuật toán, hoặc chỉ như vậy mới giảm được độ phức tạp. Tư tưởng ấy có thể mở rộng sang các bài phức tạp hơn.

12.1.2 Ứng dụng xa hơn

Ví dụ ba: IOI 2009 Regions. Cho một cây N nút, có R loại thuộc tính, mỗi nút thuộc đúng một thuộc tính. Có Q truy vấn; mỗi truy vấn cho r_1, r_2 , hỏi có bao nhiêu cặp (e_1, e_2) sao cho e_1 có thuộc tính r_1 , e_2 có thuộc tính r_2 , và e_1 là tổ tiên của e_2 . Ràng buộc: $N, Q \leq 200000$, $R \leq 25000$, giới hạn thời gian 5 giây.

Thoạt nhìn đây là một truy vấn tính khá chính thống trên cây. Bài bắt buộc trực tuyến, nếu không ta rất dễ đi lệch hướng sang các cách offline. Tác giả ban đầu thử nghĩ các thuật toán cỡ $O(\sqrt{n} \log^2 n)$ nhưng không tiến triển. Mấu chốt của lời giải là thiết kế thuật toán khác nhau theo số lượng nút của hai thuộc tính trong truy vấn.

Giả sử trong truy vấn, thuộc tính r_1 có A nút, thuộc tính r_2 có B nút. Nếu A nhỏ còn B lớn, ta có thể quét A nhưng không thể quét B . Dùng thứ tự DFS: cây con của mỗi nút trong A ứng với một đoạn $[l, r]$, còn mỗi nút trong B ứng với một vị trí p . Nếu $l \leq p \leq r$, nút thuộc A là tổ tiên của nút thuộc B . Vì vậy, với mỗi thuộc tính, lưu các thứ tự DFS của các nút thuộc tính ấy trong một vector đã sắp. Khi quét từng

nút thuộc A , chỉ cần tìm nhị phân trong vector của B . Một truy vấn được xử lý trong $O(A \log B)$.

Nếu A lớn còn B nhỏ, trực giác cho thấy cần một thuật toán $O(B \log A)$. Có thể làm như sau: quét các nút thuộc B , và hỏi vị trí của nút đó bị bao phủ bởi bao nhiêu đoạn cây con của A . Tiền xử lý toàn bộ A đoạn bằng cách cộng 1 trên từng đoạn, chẳng hạn đoạn $[5, 10]$ cộng trên các vị trí từ 5 tới 10. Sau đó, giá trị tại vị trí của một nút thuộc B chính là số tổ tiên cần đếm. Các đầu mút đoạn của A có thể rời rạc hóa, còn phép cộng đoạn có thể xử lý bằng hiệu hoặc tổng tiền tố, nên cũng đạt $O(B \log A)$.

Vẫn còn hai vấn đề. Thứ nhất, nếu cả A và B đều lớn thì dù thế nào cũng khó tốt hơn $O(A + B)$; nếu truy vấn nào cũng như vậy thì có vẻ sẽ quá chậm. Nhưng thực ra một truy vấn liên quan tới rất nhiều nút thì số truy vấn khác nhau kiểu ấy không thể quá nhiều. Nếu có hai truy vấn giống hệt nhau, ta lưu lại đáp án trước đó và trả lời trực tiếp; còn với truy vấn khác nhau thì cứ xử lý. Phân tích độ phức tạp nằm ở dưới.

Thứ hai, ngay cả khi đã có cách $O(A \log B)$ và $O(B \log A)$, vẫn có tình huống xấu: giả sử N và Q cùng bậc, có $\sqrt{N}$ thuộc tính, mỗi thuộc tính có $\sqrt{N}$ nút, rồi hỏi mọi cặp thuộc tính. Thuật toán có thể suy biến thành $O(N\sqrt{N} \log N)$, không chấp nhận được với $N = 200000$.

Qua phân tích trên, trường hợp một bên lớn một bên nhỏ không khó; vấn đề là khi hai bên xấp xỉ nhau. Nếu chấp nhận quét cả A và B , ta nên tìm cách loại bỏ thừa số log. Xem A là các đoạn và B là các điểm, bài toán trở thành đếm mỗi điểm nằm trong bao nhiêu đoạn. Có thể trộn các đầu đoạn của A với các điểm của B , đồng thời duy trì một ngăn xếp hoặc bộ đếm đang mở. Vì các đoạn cây con của cùng một thuộc tính không chứa nhau, có nhiều tính đơn điệu rõ ràng, nên phần này làm được trong $O(A + B)$.

Phân tích cuối cùng như sau. Đặt một ngưỡng C . Nếu $A > C$, dùng cách $O(B \log A)$; nếu $B > C$, dùng cách $O(A \log B)$; các trường hợp còn lại dùng cách $O(A + B)$. Nhờ không xử lý lặp lại cùng một cặp thuộc tính, hai nhóm lớn có tổng chi phí cỡ $O(N^2 \log N / C)$, còn nhóm nhỏ có tổng chi phí $O(NC)$. Chọn $C = \sqrt{N \log N}$, độ phức tạp toàn bộ là $O(N\sqrt{N \log N})$, đủ dùng.

Bài này phức tạp hơn các ví dụ trước nhiều; ngay cả đặt trong kỳ thi IOI khi đó cũng không dễ. Tư tưởng lấy ưu điểm bù nhược điểm giữa nhiều thuật toán để bảo đảm độ phức tạp đã được thể hiện rất rõ.

12.2 Những cách cân bằng khác

12.2.1 Duy trì theo phân loại

“Duy trì theo phân loại” là một tư tưởng khác trong bài cấu trúc dữ liệu. Nó cũng là thuật toán căn bậc hai, nhưng khác khá xa với duy trì theo khối. Ta dùng một ví dụ để nói ngắn gọn về ý nghĩa và ứng dụng của cách làm này.

Ví dụ bốn. Cho một đồ thị vô hướng có N đỉnh và M cạnh, với $N, M \leq 100000$. Ban đầu mọi đỉnh đều màu đen. Mỗi thao tác thuộc một trong hai loại: đổi màu một đỉnh, hoặc hỏi trong các đỉnh kề trực tiếp với một đỉnh cho trước có bao nhiêu đỉnh màu đen. Giới hạn thời gian là 1 giây.

Bài này không khó, nhưng thể hiện một tư tưởng rất tốt. Trong đa số tình huống, nhất là dữ liệu ngẫu nhiên, quét trực tiếp các cạnh kề của một đỉnh rất nhanh, vì bậc thường nhỏ. Dù trường hợp xấu theo lý thuyết là $O(N)$, nó không dễ xảy ra. Vì vậy, nếu một đỉnh có bậc nhỏ, khi truy vấn ta liệt kê trực tiếp.

Nếu bậc rất lớn thì sao? Phản xạ nên là: một bên nhiều thì bên kia ít. Đỉnh có bậc lớn thì số đỉnh như vậy ít. Lấy $\sqrt{N}$ làm ranh giới để bàn. Nếu một đỉnh có bậc không quá $\sqrt{N}$, truy vấn trực tiếp tốn $O(\sqrt{N})$. Nếu bậc lớn hơn, số đỉnh loại này nhiều nhất là $\sqrt{N}$, nên mỗi khi đổi màu một đỉnh, ta cập nhật câu trả lời cho toàn bộ các đỉnh bậc lớn bằng cách kiểm tra đỉnh vừa đổi màu có kề với chúng hay không. Việc này cũng tốn $O(\sqrt{N})$. Kết hợp hai cách duy trì, mỗi thao tác đạt $O(\sqrt{N})$ sau tiền xử lý.

12.2.2 Một khoảng nghỉ nhẹ

Ta xét một bài đơn giản và thú vị hơn.

Ví dụ năm: Kế hoạch phá cầu (JSOI 2013). Cho một dãy $A_1, A_2, \dots, A_N$, với $N \leq 100000$ và $1000 \leq A_i \leq 2000$. Cần chọn một số phần tử, có thể không chọn phần tử nào. Giả sử chọn K phần tử tại các vị trí tăng dần $p_1, p_2, \dots, p_K$. Chi phí chọn là

$$A_{p_1} \cdot 1 + A_{p_2} \cdot 2 + \dots + A_{p_K} \cdot K.$$

Các phần tử được chọn chia những phần tử còn lại thành $K + 1$ đoạn, có thể có đoạn rỗng. Gọi tổng các A_i trong mỗi đoạn là T_i . Với mỗi đoạn có $T_i > M$, phát sinh thêm chi phí T_i ; nếu không thì không phát sinh. Giá trị M giống nhau cho mọi đoạn và không vượt quá $2 \cdot 10^8$. Hãy tìm cách chọn để tổng chi phí nhỏ nhất. Giới hạn thời gian là 7 giây.

Không khó thiết kế trạng thái hai chiều $F[i][j]$: vị trí được chọn cuối cùng hiện là i , và đây là vị trí chọn thứ j từ trái sang phải; khi đó tổng chi phí nhỏ nhất của j vị trí chọn và j đoạn đã sinh ra là bao nhiêu. Đặt $\text{sum}[i]$ là tổng $A_1 + \dots + A_i$, ta có chuyển:

$$F[i][j] = \min(F[k][j-1] + A_i \cdot j), \quad \text{sum}[i-1] - \text{sum}[k] \leq M,$$

$$F[i][j] = \min(F[k][j-1] + A_i \cdot j + \text{sum}[i-1] - \text{sum}[k]), \quad \text{sum}[i-1] - \text{sum}[k] > M,$$

trong đó $0 \leq k < i$. Phân tích thêm một chút sẽ thấy có thể dùng hàng đợi đơn điệu để hạ phần chuyển trạng thái xuống cỡ bậc hai; chi tiết khá hiển nhiên nên không nhắc lại. Nhưng với $N = 100000$, như vậy vẫn quá chậm.

Đây là bài tác giả ra trong kỳ thi thử đội tuyển tỉnh JSOI 2013. Tại chỗ không ai làm được; thật ra nó không khó, mà là một mẹo cần nhận ra. Điều kiện $1000 \leq A_i \leq 2000$ có ích gì? Vì chi phí chọn một phần tử A_i tăng dần theo thứ tự chọn, ta không thể chọn quá nhiều phần tử.

Nếu không chọn phần tử nào, tổng chi phí không quá $2000N$. Nếu chọn i phần tử, riêng chi phí chọn đã ít nhất

$$1000 \cdot \frac{i(i+1)}{2} > 500i^2.$$

Để phương án chọn i phần tử có khả năng tối ưu, cần

$$500i^2 \leq 2000N,$$

tức là $i \leq 2\sqrt{N}$. Vì vậy số trạng thái hữu ích chỉ còn $O(N\sqrt{N})$, đủ để qua bài.

Bài này được đặt ở đây không phải vì nó hoàn toàn tách rời các bài khác. Nó cũng là một thuật toán $O(N\sqrt{N})$, và nguồn gốc của căn bậc hai cũng rất đặc biệt: xuất phát từ việc phân tích cận dưới trong đề bài.

Một bài Topcoder khác có cùng tinh thần: cho một đồ thị gồm K đỉnh chia thành N tầng, giữa hai tầng kề nhau có một số cạnh, và giữa tầng đầu với tầng cuối cũng có cạnh. Hỏi tập độc lập lớn nhất. Ràng buộc $K \leq 100$, $10 \leq N \leq 100$. Cần dùng tính chất đồ thị hai phía để ghép cặp, nhưng nếu N lẻ thì đồ thị không còn hai phía. Vì $N \geq 10$ và tổng số đỉnh không quá 100, chắc chắn có một tầng không quá 10 đỉnh. Liệt kê mọi trạng thái của tầng ấy là có thể “cắt vòng thành dây”. Bài này không liên quan nhiều tới chủ đề chính, nhưng nó nhắc rằng khi cận trên và cận dưới cùng bậc, chẳng hạn chỉ hơn kém nhau một hằng số nhỏ, phân tích hai cận thường cho ta một tính chất căn bậc hai làm điểm đột phá.

12.2.3 Tối ưu bộ nhớ

Tình huống bị siết bộ nhớ có lẽ không phổ biến trong OI. Nhưng nếu thời gian có thể được tối ưu bằng kiểu cân bằng này, vì sao bộ nhớ lại không? Bằng cách mở rộng tương tự, bộ nhớ cũng có thể được tối ưu ở mức đáng kể.

Ví dụ sáu: Codeforces #79 Div1 E. Có $N \leq 20000$ viên đá và $M \leq 20000$ viên kẹo cần ăn hết. Cho các tham số $A[i]$ và $B[j]$. Mỗi lần có thể chọn ăn một viên kẹo hoặc một viên đá. Sau khi ăn xong, nếu còn x viên đá và y viên kẹo, ta nhận phần thưởng $(A[x] + B[y]) \bmod P$, trong đó P cũng cho trước; sau đó tiếp tục ăn. Cần tối đa hóa tổng phần thưởng qua $M + N$ lần ăn, và phải xuất phương án: mỗi lần ăn kẹo hay ăn đá. Giới hạn thời gian 13 giây, giới hạn bộ nhớ 43 MB.

Nếu bỏ qua bộ nhớ, bài này rất dễ. Dùng quy hoạch động $F[i][j]$ biểu diễn đáp án tốt nhất sau khi đã ăn i viên kẹo và j viên đá. Giá trị chuyển trạng thái là phần thưởng của trạng thái còn lại cộng với giá trị tốt hơn giữa hai trạng thái trước đó. Để ghi phương án, dùng thêm một mảng G lưu mỗi ô đến từ đâu.

Quay lại giới hạn bộ nhớ. Mảng F có thể cuộn, còn G có thể nén bit, nhưng tính ra vẫn cần hơn 50 MB. Vì vậy cần một đường khác.

Ta không đủ bộ nhớ để lưu cả bảng. Nhưng tại sao phải lưu tất cả? Chẳng hạn, nếu đã lưu đáp án của dòng cách cuối K dòng, thì không gian từ dòng $K + 1$ tới dòng cuối có thể dùng chung với phần trước: sau khi khôi phục đáp án của đoạn ấy và xuất ra, bài toán còn lại được quy về đoạn trước dòng K . Từ gợi ý này, ta lưu một số dòng thích hợp để tận dụng lại không gian.

Rất tự nhiên nghĩ tới căn bậc hai: cứ mỗi $\sqrt{N}$ dòng lưu lại một dòng đáp án F đầy đủ. Khi đó chỉ cần không gian cỡ $O(N\sqrt{N})$ trong trường hợp N và M cùng bậc để lưu các thông tin này, rồi có thể khôi phục phương án theo từng đoạn. Muốn lấy đáp án ở ô cuối, ta dùng dòng mốc trước đó để chạy lại quy hoạch động trong đoạn $\sqrt{N}$ dòng và tìm đường đi tối ưu; sau đó cùng vùng nhớ ấy lại dùng để khôi phục đoạn phía trên, cứ thế tiếp tục. Đây là một dạng cân bằng khác. Ý tưởng không khó hiểu nhưng khá mới, và là một ứng dụng mở rộng của tư tưởng căn bậc hai. Thời gian vẫn là $O(NM)$, còn bộ nhớ giảm xuống mức căn theo một chiều của bảng.

12.3 Căn xuất hiện tự nhiên

Cũng có lúc ta không cố ý đưa căn bậc hai vào thuật toán, nhưng nó tự xuất hiện. Ví dụ, khi i chạy từ 1 tới N , giá trị $\lfloor N/i \rfloor$ chỉ có $O(\sqrt{N})$ khả năng khác nhau. Chứng minh rất dễ: với $i \leq \sqrt{N}$, có nhiều nhất $\sqrt{N}$ giá trị; với $i > \sqrt{N}$, kết quả nằm trong đoạn $[1, \sqrt{N}]$, cũng chỉ có nhiều nhất $\sqrt{N}$ giá trị. Tính chất này có một số ứng dụng.

Ví dụ bảy: POI 2007 Queries. Có nhiều truy vấn. Với các số nguyên a, b, d , hỏi có bao nhiêu cặp số nguyên dương (x, y) thỏa $x \leq a$, $y \leq b$, và $\gcd(x, y) = d$. Ràng buộc số lượng truy vấn và miền giá trị ở mức 50000.

Đây là một bài toán số học. Trước hết, $\gcd(x, y) = d$ có thể chuyển thành $\gcd(x/d, y/d) = 1$, đồng thời thu nhỏ giới hạn thành $\lfloor a/d \rfloor$ và $\lfloor b/d \rfloor$. Vì vậy ta cần đếm số cặp nguyên tố cùng nhau, một số không vượt quá a , một số không vượt quá b . Bài này rất giống bài *Energy Collection* của NOI 2010 và có thể giải bằng nguyên lý bao hàm–loại trừ.

Đặt $F(x)$ là số cặp mà hai số đều lần lượt không vượt quá a, b , và $\gcd$ của chúng là bội của x . Rõ ràng

$$F(x) = \left\lfloor \frac{a}{x} \right\rfloor \left\lfloor \frac{b}{x} \right\rfloor.$$

Sau đó dùng bao hàm–loại trừ:

$$\text{ans} = \sum_{x=1}^{\min(a,b)} \mu(x)F(x),$$

trong đó $\mu(x)$ bằng 0 nếu x có một thừa số nguyên tố xuất hiện hơn một lần, còn nếu không thì $\mu(x) = (-1)^k$, với k là số thừa số nguyên tố khác nhau của x . Làm trực tiếp mất $O(n)$ cho mỗi truy vấn; tốt hơn vét cạn $O(n^2)$ rất nhiều nhưng vẫn chưa đủ.

Quan sát công thức, ta cần tính nhanh tổng $\sum F(x)\mu(x)$. Hàm $\mu(x)$ không có quy luật đơn giản, nhưng $F(x)$ thì có: như đã nói, các giá trị $\lfloor a/x \rfloor$ và $\lfloor b/x \rfloor$ chỉ đổi $O(\sqrt{n})$ lần, nên $F(x)$ cũng chỉ có cùng bậc số giá trị khác nhau. Với một đoạn liên tiếp mà $F(x)$ không đổi, ta tính gộp cả đoạn; chỉ cần tiền xử lý tổng tiền tố của $\mu(x)$. Nhờ vậy mỗi truy vấn được trả lời trong $O(\sqrt{n})$.

Hàm $\mu(x)$ còn có tên là hàm Mobius; luận văn đội tuyển năm trước của Wang Di đã trình bày về nó. Ở đây bài viết không đào sâu thêm các tính chất số học và ứng dụng rộng hơn, mà chỉ dùng nó làm ví dụ cho kiểu tối ưu dựa trên một tính chất căn bậc hai xuất hiện tự nhiên.

Tài liệu tham khảo

1. *21st International Olympiad in Informatics Tasks and Solutions*, từ www.ioi2009.org.
2. Zhou Mengyu, Trường Trung học số 7 Thành Đô, *POI XIV solutions*, bài tập đội tuyển quốc gia năm 2008.
3. Wang Di, *Bàn sơ về nguyên lý bao hàm–loại trừ*, luận văn đội tuyển quốc gia năm 2013.

13 Bàn về các vấn đề liên quan tới cây động và một số mở rộng đơn giản

Huang Zhi'ao, Trường Yali, Trường Sa

Tóm tắt

Cây động là một lớp bài toán rất thường gặp trong OI. Các thuật toán xử lý cây động như phân rã nặng-nhẹ, chia để trị trên cây và link/cut tree đã được biết và học rộng rãi. Đồng thời, các cách dùng thứ tự DFS của cây hoặc Euler tour tree để duy trì thông tin cây con cũng đã trở nên quen thuộc. Bài viết này tổng kết một số vấn đề liên quan và đưa ra vài cách giải mang tính sáng tạo. Cuối cùng, bài viết giới thiệu hai mở rộng của LCT: top tree và link/cut cactus, dùng để giải các bài toán liên quan.

13.1 Bài toán cây động

Bài toán cây động là bài toán duy trì động thông tin trên cây. Tuy nhiên, khi nhắc tới cây động, đôi khi người ta dùng riêng cụm này để chỉ LCT.

Trong một bài toán cây động điển hình, các thao tác sau có thể xuất hiện.

1. Thêm hoặc xóa một cạnh để duy trì hình dạng của cây.
2. Thực hiện một phép biến đổi trên trọng số của một đường đi trong cây, đồng thời duy trì trọng số của từng đỉnh.
3. Truy vấn thông tin trên một đường đi trong cây.
4. Thực hiện một phép biến đổi trên trọng số của một cây con.
5. Truy vấn thông tin cây con.

13.2 Những bài toán cây động cổ điển

Trong thi thuật toán đã xuất hiện rất nhiều bài toán liên quan tới cây. Cách giải thường cũng dùng nhiều cấu trúc dạng cây, chẳng hạn cây cân bằng.

Trong các bài cũ hơn, hình dạng của cây thường tương đối cố định. Khi đó người giải không phải dành quá nhiều chú ý cho việc duy trì hình dạng cây, mà tập trung hơn vào cách tổ chức thông tin trên cây để hỗ trợ cập nhật và truy vấn.

13.2.1 Phân rã nặng-nhẹ

Cốt lõi của phân rã nặng-nhẹ là chia các cạnh của cây thành cạnh nặng và cạnh nhẹ theo một quy tắc nhất định, rồi dùng cấu trúc dữ liệu thích hợp để duy trì các chuỗi do cạnh nặng tạo thành. Có thể chứng minh rằng đường đi từ một đỉnh tới gốc được chia thành nhiều nhất $O(\log n)$ chuỗi nặng và $O(\log n)$ cạnh nhẹ. Do đó ta có thể hoàn thành các thao tác trong

$$O(\log n \cdot \text{độ phức tạp của một truy vấn trên chuỗi}).$$

13.2.2 Chia để trị theo đỉnh

Chia để trị theo đỉnh tách cây bằng trọng tâm, nhờ đó sau mỗi tầng chia để trị kích thước cây giảm ít nhất một nửa.

Kỹ thuật này thường xử lý các bài toán liên quan tới đường đi trên cây hoặc cặp đỉnh. Trong cách làm trực tiếp, ta hay thống kê thông tin một đường đi bằng cách liệt kê LCA của hai đầu mút. Trong cách chia để trị theo đỉnh, chỉ cần liệt kê trọng tâm được chọn ở mỗi tầng chia để trị để đạt hiệu quả tương tự.

13.3 Link/cut tree

13.3.1 LCT là gì

Có thể tham khảo tài liệu kinh điển về LCT. Nói cụ thể hơn, có thể xem cây động như một phép phân rã nặng-nhẹ mà mọi chuỗi cạnh thực đều được duy trì bằng Splay. Nhờ thao tác `access`, ta lấy được chuỗi từ một đỉnh tới gốc. Nhờ phép đảo trong Splay, ta thực hiện được đổi gốc. Nhờ hai lần `access`, ta lấy được LCA của hai đỉnh.

Theo phân tích khấu hao, mỗi thao tác có độ phức tạp $O(\log n)$, vì vậy LCT có thể giải quyết rất nhiều bài toán.

13.3.2 Thông tin cạnh và thông tin đỉnh

Một số bài toán cần duy trì cả thông tin cạnh lẫn thông tin đỉnh. Thường có hai cách xử lý.

Cách thứ nhất là chẻ mỗi cạnh (u, v) : thêm một đỉnh mới u' , thay cạnh (u, v) bằng hai cạnh (u, u') và (u', v) , rồi lưu thông tin cạnh ban đầu trên đỉnh mới này.

Cách thứ hai là đồng thời ghi nhận, đối với mỗi chuỗi và mỗi đỉnh, hai cạnh liên quan với đỉnh đó. Khi đổi gốc, ta duy trì lại các thông tin tương ứng.

13.3.3 Cách dùng LCT

Xét một bài toán đơn giản:

1. mỗi lần thêm hoặc xóa một cạnh;

2. cộng một số vào mọi đỉnh trên một chuỗi;
3. hỏi tổng trọng số trên một chuỗi.

Có thể hiện thực bằng LCT như sau.

- Thêm cạnh: đặt một đỉnh làm gốc của cây hiện tại của nó, rồi thiết lập một cạnh ảo nối nó tới cha.
- Xóa cạnh: thực hiện access trên một đầu mút để biến cạnh ấy thành cạnh ảo, sau đó xóa trực tiếp.
- Cộng trên chuỗi: đặt một đầu mút của chuỗi làm gốc, access đầu mút còn lại để toàn bộ chuỗi nằm trong một Splay, rồi dùng lazy tag của Splay.
- Hỏi tổng chuỗi: cũng lấy Splay tương ứng với chuỗi, rồi hỏi tổng của cả cây cân bằng.

Vì vậy, chỉ cần một bài toán duy trì bằng cây cân bằng, ta thường có thể đẩy nó lên cây bằng LCT. Ví dụ, phép cộng/trừ đoạn có thể đổi thành gán đoạn; thông tin truy vấn cũng không nhất thiết là tổng mà có thể là giá trị lớn nhất hoặc nhiều đại lượng khác trên chuỗi.

13.3.4 Một số ứng dụng đơn giản của LCT

Bài toán cây động rõ ràng. Nhiều bài chỉ cần dùng trực tiếp LCT. Ví dụ, BZOJ 3091 *City Travel* yêu cầu duy trì một cây, hỗ trợ thêm/xóa cạnh, cộng/trừ trọng số trên chuỗi và hỏi tổng đoạn con lớn nhất trên chuỗi.

Rõ ràng LCT duy trì được hình dạng cây; phần còn lại chỉ là bài toán gán/cộng đoạn và hỏi tổng đoạn con lớn nhất trên dây. Tổng đoạn con lớn nhất có thể duy trì bằng cách lưu tổng tiền tố lớn nhất và tổng hậu tố lớn nhất rồi gộp thông tin, đây là kỹ thuật rất kinh điển.

Cấu trúc cây ẩn. Có nhiều bài không cho cây một cách hiển nhiên, nhưng sau khi phân tích lại thấy một cấu trúc cây rõ ràng, nên có thể dùng LCT. Chẳng hạn bài *Elephant* của IOI 2011: chỉ cần nhận ra mỗi điểm có một cha duy nhất, hoặc những tính chất tương tự của cây, thì bài toán trở nên tự nhiên.

13.3.5 Ứng dụng trong một số bài đồ thị đơn giản

Cây là công cụ quan trọng để mô tả đồ thị. Tính linh hoạt của cây động cho phép dùng nó để xử lý một số bài đồ thị.

Cây khung nhỏ nhất khi chỉ thêm cạnh. Cây khung nhỏ nhất là bài toán kinh điển, còn duy trì động cây khung nhỏ nhất là một bài toán khá khó.

Nếu dùng chia để trị offline, có thể đạt $O(n \log^2 n)$; có thể tham khảo lời giải HNOI 2010. Nhưng cách này là offline. Nếu bắt buộc online, các thuật toán tổng quát thường phức tạp và hằng số không đẹp.

Nếu chỉ có thao tác thêm cạnh và vẫn phải duy trì online, ta có thể dùng LCT. Tính chất vòng-cắt của cây khung nhỏ nhất đảm bảo rằng sau khi thêm một cạnh, chỉ cần xóa cạnh có trọng số lớn nhất trên vòng vừa tạo thì kết quả vẫn là cây khung nhỏ nhất. Truy vấn cạnh lớn nhất trên đường đi là thao tác cơ bản của LCT.

Đường đi có khoảng chênh nhỏ nhất. Cho một đồ thị vô hướng có trọng số, cần tìm một đường đi từ S tới T sao cho hiệu giữa trọng số lớn nhất và nhỏ nhất trên đường đi là nhỏ nhất.

Có thể giải bằng LCT. Sắp xếp các cạnh, rồi liệt kê giá trị lớn nhất của cạnh trên đường đi theo thứ tự tăng dần. Với các cạnh có trọng số không vượt quá giá trị hiện tại, ta cần một đường đi mà cạnh nhỏ nhất

trên đường đi là lớn nhất. Đây là một phần của bài toán cây khung cổ chai nhỏ nhất, và cây khung nhỏ nhất cũng là cây khung cổ chai nhỏ nhất. Vì vậy ta có thể dùng lại cách duy trì cây khung ở trên.

Duy trì động tính hai phía của đồ thị. Cho một đồ thị, cho phép thêm hoặc xóa cạnh; sau mỗi thời điểm cần biết đồ thị có phải đồ thị hai phía hay không. Các thao tác có thể xử lý offline.

Bài này giống duy trì offline tính liên thông động của đồ thị, có thể dùng chia để trị theo thời gian. Một cách khác là xử lý trước thời điểm xóa của mỗi cạnh, rồi duy trì cây khung lớn nhất theo thời điểm xóa: khi thêm cạnh mà tạo thành vòng, ta xóa khỏi vòng cạnh bị xóa sớm nhất. Khi xóa một cạnh, cần hỏi cạnh ấy cùng các cạnh cây có tạo chu trình lẻ hay không; nếu có thì đưa nó vào một tập đánh dấu. Khi xóa cạnh, nếu nó là cạnh cây thì xóa trực tiếp, còn nếu nó nằm trong tập đánh dấu thì loại khỏi tập ấy. Nếu tại một thời điểm tập đánh dấu rỗng thì đồ thị là hai phía, ngược lại thì không. Tính đúng đắn không quá hiển nhiên nhưng có thể chứng minh.

Các thao tác cần thiết đều có thể duy trì trực tiếp bằng LCT.

13.4 Thứ tự DFS và ETT

Có thể thấy các bài cây động ở trên hầu như đều liên quan tới thông tin trên chuỗi, vì bản thân LCT dùng chuỗi để duy trì cấu trúc cây. Tuy nhiên, nhiều bài liên quan tới thông tin cây con; nếu chỉ dùng LCT thì sẽ gặp khó khăn.

Khi đó có một công cụ mạnh khác: thứ tự DFS của cây. Trong DFS, chỉ khi duyệt xong toàn bộ cây con của một đỉnh ta mới đi sang đỉnh khác. Vì vậy, nếu ghi các đỉnh theo thứ tự DFS, cây con của một đỉnh luôn là một đoạn liên tiếp. Ta có thể dùng nhiều cấu trúc dữ liệu khác nhau để duy trì đoạn này.

13.4.1 Bài toán cây con đơn giản

Nhiều bài có thao tác truy vấn cây con rất rõ ràng và có thể giải trực tiếp bằng thứ tự DFS.

Tree. Bài lấy từ đợt luyện tập đội tuyển quốc gia Trung Quốc cho IOI 2012. Cho một cây, cần hỗ trợ:

1. đổi gốc;
2. sửa trọng số đỉnh;
3. hỏi trọng số nhỏ nhất trong cây con.

Sau khi phân tích đơn giản, có thể thấy xuất phát từ LCT không phải hướng tốt. Dù thao tác đổi gốc rất linh hoạt, hình dạng của cây không hề thay đổi.

Ta dùng thứ tự DFS và cây đoạn để duy trì giá trị nhỏ nhất trên đoạn. Vậy đổi gốc và truy vấn xử lý thế nào? Dù gốc hiện tại là đâu, cây con cần hỏi hoặc chính là cây con của đỉnh đó khi gốc là 1, hoặc là phần bù của cây con ấy. Trường hợp đầu hỏi trực tiếp đoạn DFS; trường hợp sau hỏi giá trị nhỏ nhất trên phần bù của đoạn đó.

13.4.2 Dùng cây cân bằng duy trì thứ tự DFS

Trước khi có thứ tự DFS của một cây, ta phải DFS toàn cây. Nếu hình dạng cây thay đổi thì sao?

Nếu cố định gốc, sau khi thêm hoặc xóa một đỉnh, thay đổi trong thứ tự DFS tương đương với chèn hoặc xóa một phần tử. Chỉ cần tìm được vị trí của phần tử này là cây cân bằng có thể duy trì được, mà việc này không khó.

Tương tự, nếu thêm hoặc xóa cả một cây con, thay đổi trong thứ tự DFS chỉ là chèn hoặc xóa một đoạn. Có thể dùng Splay hoặc Treap không xoay. Nhờ cây cân bằng, ta có thể giải một số bài không còn quá đơn giản.

Mashmokh's Designed Problem. Bài từ Codeforces Round 240 Div1 E. Cho một cây có gốc; các cạnh đi ra từ mỗi đỉnh có thứ tự. Các thao tác gồm:

1. hỏi khoảng cách giữa hai đỉnh;
2. tách cây con gốc z khỏi cây, rồi thêm cạnh nối cây con ấy tới một tổ tiên nào đó, đặt nó làm con cuối cùng của tổ tiên này;
3. từ một đỉnh, duyệt DFS theo thứ tự cạnh và hỏi đỉnh được duyệt cuối cùng ở độ sâu k .

Bài này không dễ giải đẹp bằng LCT, nhưng nhìn từ thứ tự DFS thì rất rõ ràng. Thêm/xóa cạnh tương ứng với thao tác trên đoạn. Còn lại cần xử lý vài việc:

1. Tìm phần tử cuối cùng trong thứ tự DFS có độ sâu k . Do độ sâu trong thứ tự DFS thay đổi liên tục, nếu duy trì trong mỗi đoạn đỉnh sâu nhất và nông nhất, ta biết đoạn ấy có chứa độ sâu k hay không, rồi nhị phân vị trí xuất hiện cuối cùng.
2. Cùng phương pháp đó cũng truy vấn được tổ tiên ở độ sâu bất kỳ của một đỉnh.
3. Trong loại truy vấn thứ nhất cần LCA của hai đỉnh x, y . Cách nhân đôi cho $O(\log^2 n)$, nhưng có thể xét đỉnh có độ sâu nhỏ nhất giữa x và y trong thứ tự DFS; cha của đỉnh ấy chính là LCA. Vì vậy bài toán chuyển thành truy vấn nhỏ nhất trên đoạn và xử lý trong $O(\log n)$.

Thứ tự DFS và khả năng lưu phiên bản. Cây cân bằng duy trì thứ tự DFS có thể làm persistent, dẫn tới bài sau. Cho một cây gốc 0, cần hỗ trợ:

1. thêm một đỉnh trọng số 0 vào cây;
2. cộng một giá trị vào toàn bộ cây con;
3. hỏi trọng số của một đỉnh;
4. hỏi tổng trọng số của một cây con;
5. đưa trạng thái cây hiện tại về trạng thái sau một thao tác trước đó.

Cộng cây con, thêm đỉnh và truy vấn cây con đều là các thao tác đoạn cơ bản trên cây cân bằng. Nhưng yêu cầu persistent gây khó khăn: muốn truy vấn đoạn tương ứng với một đỉnh, ta thường phải đi theo cha của đỉnh đó; nếu cây cân bằng cũng duy trì con trỏ cha thì gần như không thể persistent được. Persistent hóa cả bộ nhớ cấp phát chỉ có ý nghĩa lý thuyết, còn trong thực tế hằng số và độ phức tạp cài đặt đều khó chấp nhận.

Vì vậy phải dùng cây cân bằng không duy trì cha, nhưng vẫn cần truy vấn tới đúng một đỉnh. Cách giải đưa vào khái niệm nhân động: mỗi nút trong cây cân bằng có một nhân, đồng thời persistent hóa cả nhân lẫn cây cân bằng. Để duy trì nhân, cần dùng cây cân bằng theo trọng lượng. Nếu dùng Treap persistent, sau khi chèn một đỉnh ta sửa nhân của toàn bộ cây con tương ứng. Nhờ tính chất của cây cân bằng theo trọng lượng, kích thước kỳ vọng của cây con bị sửa là $O(\log n)$. Để persistent hóa nhân, ta dùng thêm một mảng persistent; nếu hiện thực bằng cây đoạn thì độ phức tạp là $O(\log^2 n)$.

13.4.3 Euler tour tree

Tiếp theo là Euler tour tree, viết tắt là ETT.

Euler tour là gì. Gọi T là một cây vô hướng. Tách mỗi cạnh của T thành hai cạnh có hướng, rồi chọn tùy ý một đỉnh s làm điểm bắt đầu để DFS. Quá trình đó gọi là một Euler tour của T . Chú ý rằng chọn đỉnh bắt đầu nào cũng có thể thu được kết quả tương đương, vì Euler tour tương ứng với một vòng.

Dãy Euler tour. Đó là dãy các cạnh có hướng đi qua trong Euler tour.

Euler tour tree. Đây là cây tìm kiếm nhị phân duy trì dãy Euler tour, thường hiện thực bằng Splay hoặc Treap.

ETT chú trọng duy trì thông tin tổng thể của một cây. Khác với thứ tự DFS, nó có thể duy trì thông tin cây con khi chọn bất kỳ đỉnh nào làm gốc, nên xử lý được đổi gốc, thao tác vốn hơi khó với thứ tự DFS tĩnh.

Một số chi tiết hiện thực. Thực chất chỉ cần hỗ trợ hai thao tác: thêm cạnh và xóa cạnh.

- Thêm cạnh, tức nối hai cây: có thể xem là ghép hai vòng. Ta cắt hai vòng ra ở vị trí thích hợp rồi nối lại.
- Xóa cạnh, tức tách một cây thành hai cây: có thể xem là cắt một vòng thành hai vòng. Thao tác này cũng không khó.

13.4.4 Ứng dụng đơn giản của ETT

Cho một đồ thị, mỗi lần có thể thêm cạnh, xóa cạnh, hoặc hỏi tính liên thông giữa hai đỉnh bất kỳ.

Cách giải bài này khá phức tạp, ở đây chỉ mô tả ngắn gọn. Vì cần biết hai đỉnh có liên thông hay không, ta tự nhiên nghĩ tới rừng khung; nhưng rừng khung đơn thuần không xử lý được xóa cạnh.

Dùng thuật toán chia tầng: gán cho mỗi cạnh một nhãn e_i , sao cho với các cạnh có nhãn không vượt quá e , kích thước thành phần liên thông được khống chế theo tầng; nhãn lớn nhất chỉ cỡ $\log n$. Với mỗi tầng e , dùng ETT để duy trì cây khung nhỏ nhất theo nhãn trong đồ thị gồm các cạnh có nhãn không vượt quá e .

Khi thêm cạnh, đặt nhãn của nó là $\log n$ rồi thêm vào cây khung nhỏ nhất của tầng ấy, mất $O(\log n)$. Khi hỏi liên thông, kiểm tra ở tầng cao nhất, cũng mất $O(\log n)$.

Điểm mấu chốt là xóa cạnh. Trước hết tìm nhãn của cạnh này. Nếu nó không nằm trong cây khung nhỏ nhất của tầng hiện tại thì xóa trực tiếp. Nếu có, giả sử hai đầu mút là x, y ; trong mọi đồ thị ở tầng không nhỏ hơn nhãn của cạnh, cần tìm một cạnh thay thế nối hai cây con bị tách. Lấy cây con nhỏ hơn, quét các cạnh của nhãn hiện tại đi ra ngoài nó; nếu cạnh nối sang cây con còn lại thì ta tìm được cạnh thay thế. Nếu không, hạ nhãn các cạnh đã quét xuống 1 và đưa chúng vào đồ thị tầng thấp hơn. Vì cây con được quét có kích thước không quá một nửa thành phần cũ, tính chất phân tầng không bị phá vỡ.

Số lần hạ nhãn quyết định tổng độ phức tạp; tổng các nhãn là $O(m \log n)$, nên tổng thời gian vào khoảng $O(n \log^2 n)$. Các việc như duy trì kích thước cây con, thêm/xóa cạnh và tìm cạnh của nhãn hiện tại nối ra ngoài một cây đều có thể hiện thực bằng ETT.

13.5 Một số mở rộng của LCT

LCT không chỉ làm được các thao tác chuỗi đơn giản mà còn có thể giải một số vấn đề thú vị hơn. Dưới đây là hai ví dụ.

13.5.1 LCT và duy trì thông tin cây con

Ở trên, thứ tự DFS và ETT đã được dùng cho truy vấn cây con. Vậy LCT có hoàn toàn không xử lý được cây con hay không?

Chẳng hạn bài *Tree* ở mục trước, liệu có thể dùng LCT không? Thực ra là có.

Xét cách duy trì trong cây động: với mỗi đỉnh, lưu giá trị lớn nhất trong các cây con đi ra từ mọi cạnh ảo của đỉnh đó, thường tính cả chính đỉnh để tiện cài đặt. Nếu biết giá trị lớn nhất của từng cây con tương ứng với cạnh ảo, ta có thể duy trì bằng heap; gọi đại lượng này là giá trị A .

Tương tự, định nghĩa giá trị lớn nhất của một chuỗi là giá trị lớn nhất trong các giá trị A của mọi đỉnh trên chuỗi, tức là lớn nhất trong các cây con đi ra từ cạnh ảo của các đỉnh trên chuỗi. Đại lượng này rất dễ duy trì trong Splay. Vì vậy, muốn hỏi giá trị lớn nhất trong cây con lấy gốc tại một đỉnh nào đó trên chuỗi, chỉ cần hỏi giá trị A lớn nhất trên đoạn tương ứng trong Splay.

Còn giá trị A thay đổi thế nào? Thao tác then chốt của LCT là *access*. Trong một bước của *access*, một cạnh thực biến thành cạnh ảo, và một cạnh ảo biến thành cạnh thực. Khi cạnh thực chuyển thành cạnh ảo, ta đưa giá trị lớn nhất của cây con ở phía con vào cấu trúc A của cha; khi cạnh ảo chuyển thành cạnh thực thì xóa nó khỏi đó. Khi *access* chạy, các thông tin trên đường từ đỉnh tới gốc được cập nhật tương ứng. Vì LCT đảm bảo độ phức tạp khâu hao $O(\log n)$, nhân thêm chi phí heap để duy trì A , bài toán được giải trong $O(\log^2 n)$ khâu hao.

Về đối gốc, cần nói thêm rằng cách duy trì A hoàn toàn không phụ thuộc vào gốc, nên thao tác đổi gốc không làm phương pháp mất hiệu lực.

13.5.2 Một biến thể LCT

Xét bài *Lord* từ một đợt chọn đội IOI 2012. Cần hỏi tổng các số trong một cây con và hỗ trợ:

1. hỏi, khi lấy x làm gốc, tổng các số trong cây con của y ;
2. đảo các số trên đường đi x, y ;
3. đổi giá trị của x thành y ;
4. nối x với y ;
5. khi lấy x làm gốc, xóa cạnh nối y với cha của nó.

Bài này chỉ hỏi tổng cây con, nhưng vì có thao tác đảo trên chuỗi nên không thể dùng ETT. Ta buộc phải nghĩ tới cây động.

Dùng cách vừa nêu có thể duy trì thông tin cây con, nhưng làm sao hiện thực thao tác đảo trọng số trên đường đi? Một cách là không duy trì trực tiếp trọng số trên cây động. Thay vào đó, với mỗi chuỗi trên cây động, dùng một Splay mới để duy trì các trọng số trên chuỗi ấy; khi đó phép đảo chuỗi có thể thực hiện được. Trên cây động, với mỗi đỉnh chỉ cần duy trì tổng trọng số của các cây con đi ra từ cạnh ảo. Nhìn ban đầu độ phức tạp có vẻ là $O(n \log^2 n)$, nhưng hai loại thao tác Splay là tương đương trong phân tích khâu hao, nên tổng thời gian vẫn nên ở mức $O(n \log n)$.

13.6 Bài toán tổng hợp

13.6.1 Phát biểu

Cho một cây, cần hỗ trợ:

1. cộng hoặc gán một giá trị cho mọi đỉnh trong một cây con;

2. cộng hoặc gán một giá trị cho mọi đỉnh trên một chuỗi;
3. hỏi giá trị lớn nhất, nhỏ nhất hoặc tổng trọng số trong một cây con;
4. hỏi giá trị lớn nhất, nhỏ nhất hoặc tổng trọng số trên một chuỗi;
5. thêm cạnh và xóa cạnh.

13.6.2 Cách làm bằng ETT

Bài này có thao tác trên chuỗi, còn ETT vốn không thể thao tác trực tiếp trên chuỗi. Nếu dùng LCT mà lại có tag cây con, cài đặt cũng rất phức tạp.

Tuy nhiên vẫn có cách giải. Xét Euler tour của một cây. Trong ETT, sau lần xuất hiện đầu tiên của một đỉnh, một đoạn liên tiếp biểu diễn đường đi bắt đầu từ đỉnh đó và liên tục đi xuống người con đầu tiên. Điều này cho thấy việc biểu diễn đường đi trong ETT là khả thi.

Ta kết hợp với LCT: đặt người con đầu tiên của mỗi đỉnh thành đứa con duy nhất nối bằng cạnh thực trong LCT. Thay đổi thứ tự cây con trong ETT chỉ tương ứng với thay đổi thứ tự các đoạn.

Nếu LCT thực hiện một lần access, một số cạnh lần lượt trở thành cạnh thực; ta thực hiện các sửa đổi tương ứng trong ETT. Đối gốc cũng được cả LCT và ETT hỗ trợ tốt nên xử lý được. Độ phức tạp của LCT là $O(n \log n)$, vì thế số thao tác tương ứng trong ETT cũng là $O(n \log n)$, tổng thời gian $O(n \log^2 n)$.

Với truy vấn và cập nhật, vì cả chuỗi lẫn cây con đều đã tương ứng với các đoạn trong ETT, ta chỉ cần dùng cây cân bằng trong ETT để duy trì thông tin.

13.6.3 Một cách làm bằng LCT

Chỉ dùng cây động thông thường thì không thể làm thao tác cây con, trong khi trọng tâm của bài này lại là sửa và hỏi cây con. Vì thế độ khó tăng lên nhiều bậc. Dù vậy, vẫn có thể dùng LCT để đạt $O(n \log n)$.

Một mẹo đơn giản. Các thao tác sửa hoặc cộng thêm một số, hoặc gán thành một số. Ta có thể thống nhất tag dưới dạng $ax + b$, chỉ cần lưu a, b ; khi viết chương trình sẽ rất tiện.

Một hình minh họa. Nếu dùng LCT truyền thống, vấn đề nằm ở việc đẩy tag xuống và kéo thông tin lên. Hình dưới vẽ các cạnh thực bằng nét liền và cạnh ảo bằng nét đứt; các đỉnh được sắp theo độ sâu từ trên xuống dưới.



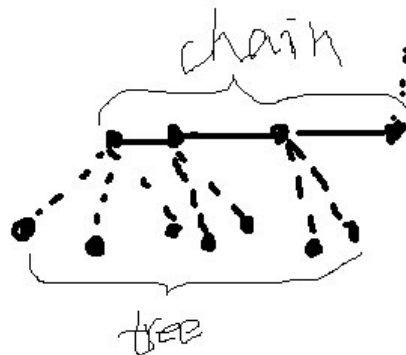
Cạnh thực và cạnh ảo trong LCT khi xét thông tin cây con.

Gộp thông tin. Một Splay duy trì chuỗi nối bởi cạnh thực, vì thế thông tin lớn nhất, nhỏ nhất trên chuỗi rất dễ thống kê.

Tiếp theo xét cạnh ảo. Vì cần gộp thông tin cây con, ta dùng thông tin của cây con đi ra từ mỗi cạnh ảo để cập nhật thông tin trên chuỗi thực. Với một chuỗi cần duy trì ba loại thông tin:

1. *chain*: thông tin của các đỉnh nằm bên trong chuỗi;
2. *tree*: tổng thông tin của mọi cây con đi ra từ cạnh ảo của các đỉnh trên chuỗi, không tính bản thân chuỗi;
3. *all*: tổng của hai loại trên, tức thông tin cây con của đỉnh cao nhất của chuỗi.

Vì chuỗi được duy trì bằng Splay, mỗi nút Splay đều lưu các giá trị này, không khác nhiều so với cây động thông thường. Có thể hình dung bằng cách xoay các chuỗi nằm ngang: *chain* được Splay duy trì trực tiếp; nếu tính được *tree* thì *all* chỉ là phép gộp đơn giản.

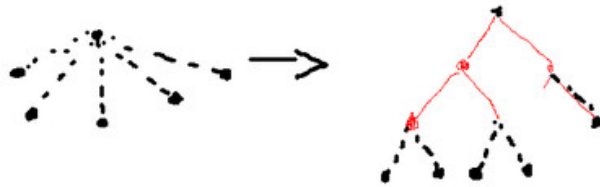


Ba lớp thông tin *chain*, *tree* và *all*.

AAA tree. Vấn đề còn lại là tính *tree*. Nếu với mỗi đỉnh ta lưu thô mọi cạnh ảo đi ra, có thể liệt kê các chuỗi tương ứng và dùng *all* của chúng để cập nhật *tree*; nhưng cách này quá chậm. Vì vậy cần sửa

đổi LCT ban đầu.

AAA tree được đưa vào cho mục đích này: với các cạnh ảo đi ra từ mỗi đỉnh, cũng dùng Splay để duy trì các điểm tương ứng. Để cài đặt tiện hơn, ta thêm một số nút nội bộ, gọi là *inner*. Trong hình dưới, các nút đỏ là nút nội bộ mới thêm; mọi lá màu đen chính là các đỉnh tương ứng với cạnh ảo trong cây gốc. Số nút nội bộ không vượt quá số nút lá, nên không làm đổi độ phức tạp.



AAA tree dùng thêm nút nội bộ để duy trì tập cạnh ảo bằng Splay.

Rõ ràng mỗi chuỗi chỉ thuộc một AAA tree, và AAA tree của một đỉnh chứa mọi cạnh ảo đi ra từ đỉnh ấy. Nhờ AAA tree, ta thực hiện được hai thao tác:

1. **add**: nối gốc của một chuỗi vào một đỉnh của chuỗi khác. Chỉ cần chèn gốc chuỗi vào AAA tree của đỉnh đó như chèn Splay thông thường; để bảo đảm nó là lá, cần thêm nút nội bộ nếu cần.
2. **del**: cắt một chuỗi đi ra từ cạnh ảo. Dùng xóa Splay thông thường; nếu sau khi xóa có nút nội bộ có quá ít con đen, cũng xóa nút nội bộ ấy, hoặc nếu nó chỉ còn một con thì để con ấy thay nó nối lên cha.

Khi cần cập nhật thông tin của một đỉnh trên chuỗi thực, ta chỉ dùng thông tin ở gốc AAA tree tương ứng.

AAA tree và access. Xét access dưới cấu trúc này. Đây là thao tác quan trọng nhất của cây động; có nó thì về cơ bản các bài cây động đều xử lý được.

Sau khi có hai thao tác **add** và **del**, **access** không bị ảnh hưởng. Trong mỗi vòng lặp của **access**, ta chỉ làm các việc:

1. tìm cha của chuỗi hiện tại, có thể tìm thông qua cha của gốc AAA tree của chuỗi ấy;
2. biến một cạnh thực thành cạnh ảo, tức là thao tác **add**;
3. biến một cạnh ảo thành cạnh thực, tức là thao tác **del**.

Nhờ vậy việc gộp thông tin trong LCT trở nên khả thi.

Tag. Cần hiện thực các thao tác sửa, tức là tag và đẩy tag xuống. Có hai loại tag rõ ràng.

Loại thứ nhất là **treeflag**, biểu thị sửa đổi trong cây con tương ứng với chuỗi này, chú ý không bao gồm sửa đổi trên chính chuỗi. Loại thứ hai là **chainflag**, tức sửa đổi trên chuỗi.

Đẩy **chainflag** xuống là việc đơn giản và quen thuộc. Còn **treeflag** có hai tình huống:

1. Nếu ở trong Splay của chuỗi thực hoặc đẩy xuống nút nội bộ, cứ đẩy trực tiếp, không cần sửa đổi.
2. Nếu đẩy từ AAA tree xuống nút ngoài, cần tách **treeflag** thành **chainflag** cộng **treeflag**, vì **treeflag** không bao gồm phần sửa đổi trên chuỗi.

Chi tiết hiện thực. AAA tree nhìn có vẻ rắc rối, nhưng thực ra chỉ cần đổi mảng hai con ch[2] trong nút LCT thành bốn con ch[4]. Truyền thêm một biến type để quyết định thao tác hiện tại đang xét trong LCT hay trong AAA tree. Đặt thêm một cờ inner cho nút nội bộ để tiện xử lý tag.

Độ phức tạp. Theo phân tích không quá nghiêm ngặt, độ phức tạp của cách làm này không vượt quá $O(\log n)$ khấu hao mỗi thao tác. Trong bài báo về self-adjusting top tree, độ phức tạp khấu hao cũng được chứng minh là $O(\log n)$, tuy có một hằng số 96. Trong thực tế, cách làm này vẫn chạy khá nhanh.

Tên gọi. Trong tài liệu của Tarjan và Werneck, cấu trúc này được gọi là *self-adjusting top tree*. Nhưng nếu xem nó như một mở rộng đơn giản của LCT thì sẽ dễ hiểu hơn.

13.6.4 Phương pháp rake và compress

Trong một tài liệu khác có nhắc tới một cấu trúc dữ liệu thú vị: RC tree.

Cụ thể, với một cây, có thể dùng hai kiểu nút để chia nó thành các cây con nhỏ hơn. Nút *rake* đại diện cho cây con gốc tại một đỉnh trong cây ban đầu. Nút *compress* đại diện cho toàn bộ cây con của các đỉnh trên một chuỗi trong cây ban đầu.

Dùng hai loại nút này để phân rã cây có thể đạt hiệu quả tương tự cây động. Kết hợp chiến lược ngẫu nhiên, độ sâu kỳ vọng là $O(\log n)$. Tuy nhiên, trong bài báo về top tree có nêu hạn chế của cách này: nó yêu cầu số đỉnh trong mỗi cụm là hằng số. Nếu hy sinh một phần độ phức tạp thời gian, có lẽ vẫn giải được bài cây động ban đầu; tác giả chưa hiện thực nên không dám khẳng định.

13.7 Cây xương rồng động

Phần này tri ân Lu Kaifeng. Ở đây bài viết giới thiệu ngắn về bài toán cây xương rồng động và lời giải liên quan.

13.7.1 Tóm tắt bài toán

Đồ thị xương rồng là đồ thị trong đó mỗi cạnh thuộc nhiều nhất một chu trình đơn. Bài toán xương rồng động yêu cầu duy trì động một đồ thị xương rồng. Do kỹ thuật còn hạn chế, hiện tại chủ yếu xét cập nhật và truy vấn trên đường đi.

Trong xương rồng, giữa hai đỉnh có thể có nhiều đường đi đơn; khi nói tới đường đi (u, v) , ta thường chọn đường đi ngắn hơn hoặc đường đi theo quy ước của bài toán.

Bài toán xương rồng động được chia thành ba biến thể có độ khó tăng dần:

1. thêm/xóa cạnh và hỏi độ dài đường đi ngắn nhất;
2. trên cơ sở biến thể trước, mỗi cạnh có thêm một trọng số cố định, cần hỏi giá trị nhỏ nhất trên một đường đi;
3. trên cơ sở đó, hỗ trợ thêm các phép sửa đổi trên một đoạn đường đi.

13.7.2 Một thuật toán xuất sắc

Cách rõ ràng nhất là dùng LCT để duy trì cây khung của xương rồng. Có thể đạt độ phức tạp $O(n \log n)$, tức mức tối ưu về lý thuyết.

Nhưng hiện thực cụ thể của cách này quá phức tạp, nên bài viết không bàn tiếp. Bạn đọc quan tâm có thể xem lời giải liên quan của Lu Kaifeng.

13.7.3 Link/cut cactus

Vì sao duy trì cây khung lại phiền như vậy? Vì áp thẳng một thuật toán trên cây vào đồ thị xương rồng sẽ không tránh khỏi nhiều phân tích trường hợp phức tạp. Để có một cách dễ viết hơn, cần thoát khỏi khuôn khổ cây. Hãy nghĩ cách sửa cấu trúc link/cut tree để nó thích ứng với đồ thị xương rồng; từ đó thu được một cấu trúc mới: link/cut cactus.

13.7.4 Phân rã đồ thị xương rồng

Bắt chước LCT, để có LCC trước hết phải xét cách chia đồ thị xương rồng thành nhiều chuỗi không giao nhau. Chọn tùy ý một đỉnh làm gốc.

Nếu không có chu trình, mọi khái niệm định nghĩa giống LCT. Nếu có chu trình thì sao?

Trước hết, cần bảo đảm mỗi chuỗi thu được là một đường đi hợp lệ, tức đường đi ngắn nhất giữa hai đầu mút của nó. Nếu xem một chu trình như một đỉnh trong LCT, còn phải bảo đảm với mỗi chu trình, theo hướng đi xuống xa gốc chỉ có nhiều nhất một cạnh đi ra.

Thứ hai, mặc dù một đỉnh có thể thuộc rất nhiều chu trình, nó chỉ được thuộc một chuỗi.

Gọi các cạnh trong chuỗi là cạnh thực, còn các cạnh khác là cạnh ảo. Với một chu trình, gọi đỉnh gần gốc nhất là gốc của chu trình. Nếu từ một đỉnh trên chu trình có cạnh đi xuống, chu trình có thể được phân rã theo hai kiểu trong hình gốc. Mô tả bằng lời: bỏ gốc ra, một chu trình nhiều nhất tách thành hai chuỗi; chỉ một trong hai phần có thể có một đầu mút nối tiếp xuống dưới bằng cạnh thực. Phần đó gọi là chuỗi b , phần còn lại gọi là chuỗi other.

Điểm khác nhau giữa hai kiểu phân rã nằm ở việc gốc có nối với các đỉnh khác trên chu trình hay không. Có thể thấy một đỉnh có thể làm gốc của nhiều chu trình, nhưng nhiều nhất chỉ thuộc chuỗi b hoặc other của một chu trình. Vì vậy có thể duy trì trực tiếp tất cả các chuỗi.

13.7.5 access

Tiếp theo xét access. Với cây động, chỉ cần thực hiện được access thì bài toán về cơ bản được giải quyết.

Ý nghĩa của nó là nối đường đi ngắn nhất từ một đỉnh tới gốc thành một chuỗi. Giả sử trong quá trình access, đỉnh hiện tại là u . Ta tìm cạnh đi trước u trong LCC, có thể dùng định nghĩa chuỗi và đường đi ngắn nhất. Nếu cạnh này không nằm trong chu trình, cách làm giống LCT.

Nếu cạnh ấy nằm trong một chu trình, tìm chu trình chứa nó. Chắc chắn u không phải gốc của chu trình, gọi gốc là a . Ta thực hiện thao tác sau: nối chuỗi b và chuỗi other, rồi cắt tại u , sao cho u nằm trên chuỗi ngắn hơn; thu được các chuỗi b và other mới, đồng thời u đúng là một đầu mút của b . Gọi thao tác này là *Expand*. Sau đó nối chuỗi đi xuống từ u với chuỗi b mới là hoàn thành xử lý đỉnh u .

Cần chú ý đỉnh a : nếu a nối với chuỗi b ban đầu, cuối cùng phải nối a với chuỗi b mới; nếu không, trước hết thực hiện *Expand* trên a , rồi mới nối với chuỗi b mới.

13.7.6 Đổi gốc

May mắn là đổi gốc rất đơn giản. Thực hiện access trên gốc mới để lấy đường đi, rồi đảo đường đi đó.

LCC yêu cầu với mỗi chu trình phải duy trì các chuỗi b và other tương ứng. Sau khi đảo, các giá trị này có thể thay đổi. Cách xử lý đơn giản là: mỗi khi đi qua một chu trình, kiểm tra thứ tự của b trên đường đi; nếu thứ tự khác trước, đảo chuỗi tương ứng trong chu trình và hoán đổi các giá trị liên quan.

13.7.7 Các thao tác khác

Sau khi đã có access và đổi gốc, các thao tác còn lại trở nên rất đơn giản.

13.7.8 Một vài chi tiết nhỏ

Vì trọng số trong bài nằm trên cạnh, với mỗi đỉnh cần duy trì hai cạnh của chuỗi kề với nó, để sau khi đảo vẫn lấy được đúng tập cạnh.

Bài toán còn yêu cầu phán đoán đường đi ngắn nhất có duy nhất hay không. Cách xử lý là với mỗi chu trình ghi nhận độ dài chuỗi b và chuỗi $other$ có bằng nhau hay không, rồi dùng thông tin đó để đánh dấu các đường đi tương ứng.

13.7.9 Độ phức tạp

Vì việc chuyển đổi cạnh thực và cạnh ảo trong LCC tương tự LCT, tổng số lần sẽ không vượt quá bậc $n \log n$. Tính thêm Splay, độ phức tạp là $O(n \log^2 n)$. Theo thử nghiệm, hiệu suất cũng khá tốt.

13.7.10 Một số hướng mở rộng

LCC cũng là một cấu trúc dữ liệu linh hoạt như LCT. Có thể suy nghĩ tiếp các hướng sau:

1. liệu có thể persistent hóa hay không;
2. liệu có thể kết hợp self-adjusting top tree với LCC để xử lý thao tác và truy vấn trên “xương rồng con” hay không;
3. liệu có thể mở rộng thứ tự DFS sang đồ thị xương rồng hay không;
4. dùng LCC thế nào để tối ưu các bài thống kê và tối ưu hóa trên đồ thị xương rồng.

Bạn đọc quan tâm được khuyến khích tiếp tục suy nghĩ về các vấn đề này.

Hậu ký

Bài viết đã giới thiệu sơ lược các bài toán cây động trong thi tin học.

Tác giả chọn chủ đề này có lẽ vì bản thân khá hứng thú với các vấn đề liên quan. Hai năm gần đây, cây động trở thành đề tài nóng trong nhiều kỳ thi mô phỏng; độ khó tăng dần và dạng bài cũng ngày càng nhiều. Vì vậy tác giả muốn tổng kết lại lớp vấn đề này.

Do thời gian gấp và năng lực tác giả có hạn, nhiều vấn đề chưa được đề cập. Dù vậy, hy vọng bài viết có thể giúp những OIer sắp học hoặc đang học cây động; nếu được như vậy thì bài viết cũng đã có ý nghĩa.

Lời cảm ơn

Cảm ơn thầy Liao Xiaogang đã quan tâm và chăm sóc trong học tập cũng như đời sống. Cảm ơn các thầy Zhu Quanmin và Qu Yunhua đã chỉ dạy. Cảm ơn các huấn luyện viên đội tuyển quốc gia vì sự cống hiến vất vả. Cảm ơn CCF đã cung cấp nền tảng và cơ hội.

Cảm ơn các bạn Gan Zhengfei, Liu Yanyi, Mao Xuan, Peng Yuxiang, Yang Dingcheng, Liu Jiancheng, Tan Bowen, Yang Zhanglin, Su Yufeng và nhiều bạn khác đã góp ý và giúp đỡ. Cảm ơn Chen Lijie, từ bạn ấy tác giả học được top tree và nhiều thuật toán thú vị khác. Cảm ơn Lu Kaifeng vì nghiên cứu tiên

phong về bài toán xương rồng động. Cảm ơn các bạn trong đội tuyển đã cùng tác giả trải qua một khoảng thời gian vui vẻ.

Tài liệu tham khảo

1. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*, Nhà xuất bản Đại học Thanh Hoa.
2. Liu Rujia, *Nhập môn kinh điển thi thuật toán*, Nhà xuất bản Đại học Thanh Hoa.
3. Cen Ruoxu, *Bàn về các bài toán xây dựng đơn giản trong thi thuật toán*.
4. Yang Zhe, *Một số nghiên cứu về lời giải SPOJ375 QTREE*, bài tập đội tuyển quốc gia năm 2007.
5. the, *Báo cáo lời giải HNOI 2010*, bài tập đội tuyển quốc gia năm 2011.
6. Chen Lijie, *Ứng dụng của cây cân bằng theo trọng lượng và cây cân bằng hậu tố trong Olympic Tin học*, luận văn đội tuyển quốc gia năm 2013.
7. 6.851: *Advanced Data Structures*, bài giảng 20.
8. Một tài liệu trại đông Olympic Tin học 2014, *Hai thuật toán đồ thị ít gặp*.
9. Robert E. Tarjan, Renato F. Werneck, *Self-Adjusting Top Trees*.
10. Chen Shouyuan, *Duy trì tính liên thông của rừng: cây động*.
11. Một tài liệu về *Dynamic Trees Problem and its Applications*.
12. Lu Kaifeng, *Loạt lời giải về xương rồng động*.

14 Ứng dụng thuật toán ngẫu nhiên trong thi tin học

Hu Zecong, Trường THPT trực thuộc Đại học Sư phạm Hồ Nam

Tóm tắt

Thuật toán ngẫu nhiên là một lớp thuật toán rất hữu ích. Bằng cách đưa hàm ngẫu nhiên vào quá trình tính toán, nhiều thuật toán có thể đạt độ phức tạp thời gian kỳ vọng tốt hơn thuật toán tất định. Bài viết này giới thiệu ngắn gọn định nghĩa và phân loại thuật toán ngẫu nhiên, phân tích cách áp dụng chúng qua một số ví dụ trong thi tin học, rồi tổng kết vài hướng suy nghĩ thường gặp khi thiết kế thuật toán ngẫu nhiên.

14.1 Dẫn nhập

Trong thi tin học, những bài buộc thí sinh thiết kế thuật toán ngẫu nhiên có vẻ không nhiều. Nhưng một khi gặp dạng này, không ít thí sinh sẽ lúng túng. Nhiều người chỉ biết thuật toán ngẫu nhiên qua các cách “ăn điểm” không chắc chắn; vì vậy ba phần đầu của bài viết sẽ giới thiệu định nghĩa và phân loại cơ bản.

Phạm vi của thuật toán ngẫu nhiên khá rộng, nên không có một phương pháp chung cho mọi bài. Cần phân tích theo từng bài cụ thể. Ở phần ví dụ, ta sẽ xem thuật toán ngẫu nhiên được dùng như thế nào, hiệu quả ra sao, rồi rút ra một số cách thiết kế và phân tích thường dùng.

14.2 Định nghĩa và phân loại

Thuật toán ngẫu nhiên, còn gọi là thuật toán xác suất, là thuật toán có sử dụng hàm ngẫu nhiên, và giá trị trả về của hàm này trực tiếp hoặc gián tiếp ảnh hưởng tới luồng thực thi hay kết quả. Một số quyết định của thuật toán phụ thuộc vào lựa chọn ngẫu nhiên; nhờ đó ta có thể cải thiện độ phức tạp kỳ vọng hoặc đạt một xác suất đúng chấp nhận được.

Có thể chia thuật toán ngẫu nhiên thành ba loại sau.

- Thuật toán xác suất số trị.
- Thuật toán Monte Carlo.
- Thuật toán Las Vegas.

Ba loại này có đặc điểm khác nhau.

Thuật toán xác suất số trị. Loại này chọn ngẫu nhiên một số phần tử để tìm nghiệm xấp xỉ về mặt số học. So với phương pháp truyền thống, tốc độ thường nhanh hơn; thời gian chạy càng dài thì nghiệm xấp xỉ càng chính xác. Khi không thể hoặc không cần tìm nghiệm chính xác, thuật toán xác suất số trị có thể cho kết quả rất thỏa đáng. Ví dụ quen thuộc là phương pháp rải điểm ngẫu nhiên để ước lượng diện tích những hình khó tính trực tiếp.

Thuật toán Monte Carlo. Loại này luôn cho ra một kết quả trong thời gian xác định, nhưng kết quả có xác suất sai. Thông thường xác suất sai nhỏ, nên có thể lặp lại thuật toán nhiều lần để đạt độ tin cậy mong muốn. Một ví dụ kinh điển là kiểm tra nguyên tố Miller–Rabin.

Thuật toán Las Vegas. Loại này luôn trả về kết quả đúng, nhưng thời gian chạy không xác định trước. Với một số thuật toán tất định có thời gian trung bình tốt nhưng trường hợp xấu rất tệ, ta có thể thêm ngẫu nhiên để giảm xác suất rơi vào trường hợp xấu và biến chúng thành thuật toán Las Vegas với độ phức tạp kỳ vọng tốt. Sắp xếp nhanh ngẫu nhiên là ví dụ điển hình, trong đó tính ngẫu nhiên đến từ việc chọn *pivot*.

Bài viết này tập trung vào Monte Carlo và Las Vegas; thuật toán xác suất số trị không được bàn sâu.

14.3 Thuật toán Monte Carlo

Trong thi tin học, Monte Carlo có lẽ là loại thuật toán ngẫu nhiên được dùng tương đối nhiều. So với Las Vegas, đôi khi ta có thể dựa vào tính chất của đề để sửa trực tiếp một thuật toán tất định thành thuật toán Monte Carlo.

14.3.1 Phân loại thuật toán Monte Carlo

Theo tính chất bài toán, có thể chia thuật toán Monte Carlo thành hai nhóm:

- thuật toán Monte Carlo cho bài toán tối ưu;
- thuật toán Monte Carlo cho bài toán quyết định.

Với bài toán quyết định, còn có thể phân tiếp thành ba dạng:

- *thiên lệch về sai (false-biased)*: nếu thuật toán trả về false thì kết quả chắc chắn đúng, còn nếu trả về true thì có xác suất sai;

- *thiên lệch về đúng (true-biased)*: nếu thuật toán trả về true thì kết quả chắc chắn đúng, còn nếu trả về false thì có xác suất sai;
- *sai hai phía (two-sided error)*: cả hai kết quả true và false đều có xác suất sai.

Hai dạng đầu được gọi chung là sai một phía (*one-sided error*). Trong thi tin học, phần lớn thuật toán Monte Carlo gặp được là sai một phía, nên phần sau bỏ qua trường hợp sai hai phía.

14.3.2 Xác suất đúng và độ phức tạp

Giả sử ta có một thuật toán Monte Carlo với xác suất đúng p và độ phức tạp thời gian $O(f(n))$. Nếu chạy độc lập thuật toán đó k lần, ta có:

$$\Pr[\text{ít nhất một lần đúng}] = 1 - (1 - p)^k, \quad T = O(kf(n)).$$

Trong nhiều bài, chỉ cần p là một hằng số không quá nhỏ thì vài chục hoặc vài nghìn lần lặp đã đủ đưa xác suất sai xuống rất thấp.

14.4 Thuật toán Las Vegas

Las Vegas xuất hiện ít hơn Monte Carlo trong thi tin học. Những thuật toán loại này ta thường dùng là thuật toán đã biết sẵn, chẳng hạn sắp xếp nhanh, thuật toán gia tăng ngẫu nhiên trong hình học tính toán, hoặc Treap; bài yêu cầu tự thiết kế Las Vegas tương đối hiếm. Phần ví dụ cuối sẽ có một bài như vậy.

Las Vegas không có nhiều phân loại như Monte Carlo, nên điều quan trọng là cách phân tích độ phức tạp kỳ vọng.

14.4.1 Ví dụ: sắp xếp nhanh

Với dãy độ dài n , sắp xếp nhanh có trường hợp tốt $O(n \log n)$, trường hợp xấu $O(n^2)$, và độ phức tạp kỳ vọng $O(n \log n)$. Gọi $T(n)$ là thời gian kỳ vọng khi sắp xếp dãy độ dài n . Ta có

$$T(0) = 0,$$

và

$$T(n) = n + \frac{1}{n} \sum_{i=0}^{n-1} (T(i) + T(n-i-1)).$$

Do tính đối xứng,

$$T(n) = n + \frac{2}{n} \sum_{i=n/2}^{n-1} (T(i) + T(n-i-1)).$$

Vì $T(n) \geq n$, với $n/2 \leq i \leq j$ ta có

$$T(i) + T(n-i) \leq T(j) + T(n-j).$$

Suy ra

$$\begin{aligned} T(n) &\leq n + \frac{2}{n} \sum_{i=n/2}^{3n/4} (T(3n/4) + T(n/4)) + \frac{2}{n} \sum_{i=3n/4}^{n-1} (T(n-1) + T(0)) \\ &\leq n + \frac{1}{2} (T(3n/4) + T(n/4)) + \frac{1}{2} T(n-1). \end{aligned}$$

Ta chứng minh $T(n) = O(n \log n)$ bằng quy nạp. Giả sử với mọi $m < n$, $T(m) \leq cm \log m$. Khi đó

$$\begin{aligned} T(n) &\leq n + \frac{1}{2}(c(3n/4) \log(3n/4) + c(n/4) \log(n/4)) + \frac{1}{2}c(n-1) \log(n-1) \\ &\leq cn \log n + n \left(1 - \frac{3c}{8} \log(4/3) - \frac{c}{4}\right). \end{aligned}$$

Chỉ cần chọn hằng số c đủ lớn để biểu thức trong ngoặc không dương, ta có $T(n) \leq cn \log n$, do đó $T(n) = O(n \log n)$.

14.4.2 Biến Monte Carlo thành Las Vegas

Với một lớp bài xây dựng luôn có nghiệm, giả sử ta có thuật toán Monte Carlo sai một phía, xác suất đúng p , độ phức tạp $O(f(n))$. Nếu lặp thuật toán cho đến khi tìm được nghiệm, ta thu được một thuật toán Las Vegas. Gọi K là số lần chạy kỳ vọng. Ta có

$$K = \sum_{i=1}^{\infty} p(1-p)^{i-1}i.$$

Nhân hai vế với $1-p$:

$$(1-p)K = \sum_{i=2}^{\infty} p(1-p)^{i-1}(i-1).$$

Lấy hiệu hai công thức:

$$pK = p + \sum_{i=2}^{\infty} p(1-p)^{i-1} = p \sum_{i=0}^{\infty} (1-p)^i.$$

Vì đây là cấp số nhân,

$$K = \sum_{i=0}^{\infty} (1-p)^i = \frac{1}{p}.$$

Vậy thời gian kỳ vọng là $O(p^{-1}f(n))$.

14.5 Ví dụ và phân tích

14.5.1 Ví dụ 1: *MSTONE*

Trên mặt phẳng có n điểm đôi một khác nhau. Biết rằng tồn tại không quá 7 đường thẳng phủ được toàn bộ các điểm. Hỏi nếu vẽ một đường thẳng thì nhiều nhất có thể phủ bao nhiêu điểm. Ràng buộc $n \leq 10000$.

Phân tích. Cách ngây thơ là chọn hai điểm để xác định một đường thẳng, rồi đếm số điểm nằm trên đường đó. Độ phức tạp $O(n^3)$ nếu cài đặt trực tiếp hoặc ít nhất cũng quá lớn cho giới hạn thời gian.

Điều kiện “không quá 7 đường thẳng phủ được tất cả các điểm” rất quan trọng. Đường thẳng phủ nhiều điểm nhất phải phủ ít nhất $\lceil n/7 \rceil$ điểm. Do đó nếu chọn ngẫu nhiên hai điểm, xác suất hai điểm đó cùng nằm trên đường thẳng tối ưu không nhỏ hơn xấp xỉ $1/49$, và có thể lấy cận an toàn là $1/50$ khi n đủ lớn.

Vì vậy ta thay bước “liệt kê hai điểm” bằng “chọn ngẫu nhiên hai điểm”. Mỗi lần chạy chỉ cần $O(n)$ để đếm điểm trên đường thẳng được chọn. Nếu chạy $k = 1000$ lần, xác suất tìm được đường tối ưu ít nhất là

$$1 - \left(1 - \frac{1}{50}\right)^k > 1 - 10^{-9},$$

một mức rất tốt trong thực tế.

14.5.2 Ví dụ 2: GCD

Với một tập S , định nghĩa giá trị $\text{GCD}(S)$ là ước chung lớn nhất lớn nhất có thể đạt được bởi một tập con đủ lớn, cụ thể là một tập con có kích thước không nhỏ hơn một nửa kích thước của S . Cho dãy $a_1, a_2, \dots, a_n$, hãy tính $\text{GCD}(\{a_1, a_2, \dots, a_n\})$. Trong bài gốc, n rất lớn và a_i có thể tới 10^{12} .

Phân tích. Một cách tất định tự nhiên là liệt kê ứng viên cho đáp án rồi đếm bao nhiêu số chia hết cho ứng viên đó. Cũng có thể liệt kê các ước của từng số, nhưng nếu làm cho mọi số thì vẫn quá chậm.

Ta đổi góc nhìn: nếu biết một số a_x thuộc tập con tối ưu, thì đáp án phải là ước của $\text{gcd}(a_x, a_i)$ với các số a_i khác. Với a_x đã chọn, ta tính $g_i = \text{gcd}(a_x, a_i)$, rồi với mỗi ước d của a_x đếm số lượng g_i chia hết cho d . Ước lớn nhất có số đếm đủ một nửa dãy là đáp án ứng với lựa chọn a_x .

Nếu a_x được liệt kê cho mọi x thì vẫn chậm; nhưng nếu chọn ngẫu nhiên một số trong dãy, xác suất nó thuộc tập con tối ưu không nhỏ hơn $1/2$. Vì vậy ta chỉ cần lặp vài chục lần. Đây là một thuật toán Monte Carlo có xác suất đúng rất cao.

14.5.3 Hướng thiết kế 1

Từ hai ví dụ trên có thể rút ra một cách thiết kế thường gặp:

1. Thiết kế trước một thuật toán tất định giải được bài toán. Thuật toán này có một phần liệt kê tốn kém; giả sử độ phức tạp là $O(f(n)g(n))$, trong đó $f(n)$ là phần liệt kê và $g(n)$ là chi phí xử lý một lựa chọn. Cần bảo đảm chạy $O(g(n))$ nhiều lần vẫn không quá thời gian.
2. Đưa ngẫu nhiên vào bằng cách thay phần liệt kê bằng chọn ngẫu nhiên. Phải chứng minh xác suất chọn trúng phần tử cần thiết không quá nhỏ, rồi tính số lần lặp cần thiết để đạt xác suất đúng chấp nhận được.

Các bài cần dùng cách này thường có ràng buộc mạnh, bảo đảm xác suất chọn trúng không quá thấp. Ngay cả khi xác suất đúng chưa lý tưởng, thuật toán vẫn có thể đem lại một phần điểm, nhất là ở dạng bài nộp đáp án.

14.5.4 Ví dụ 3: Couriers

Cho dãy độ dài n và m truy vấn. Mỗi truy vấn cho l, r , hỏi trong đoạn $a_l, \dots, a_r$ có phần tử nào xuất hiện quá $(r - l + 1)/2$ lần hay không; nếu có thì in phần tử đó, ngược lại in -1 . Ràng buộc $n, m \leq 500000$.

Phân tích. Ta có thể tiền xử lý, với mỗi giá trị, danh sách các vị trí nó xuất hiện. Khi cần đếm số lần xuất hiện của một giá trị trong đoạn, chỉ cần hai lần tìm kiếm nhị phân. Nếu vẫn liệt kê mọi phần tử trong đoạn thì độ phức tạp còn quá cao.

Theo hướng thiết kế 1, thay việc liệt kê phần tử trong đoạn bằng chọn ngẫu nhiên một vị trí trong đoạn. Nếu đoạn có đáp án, đáp án xuất hiện hơn một nửa số vị trí, nên một lần chọn trúng đáp án với xác suất lớn hơn $1/2$. Nếu đoạn không có đáp án, kiểm tra sau khi chọn ngẫu nhiên sẽ không bao giờ trả về một đáp án sai, vì ta luôn đếm lại số lần xuất hiện. Do đó đây là Monte Carlo sai một phía.

Thực nghiệm cho thấy mỗi truy vấn chạy khoảng 20 lần là đủ qua bài. Độ phức tạp là $O(km \log n)$, với k là số lần thử.

14.5.5 Ví dụ 4: *TKCONVEX*

Cho độ dài của n đoạn thẳng. Hãy chọn $2k$ đoạn sao cho chúng có thể chia thành hai tập, mỗi tập k đoạn và mỗi tập lập được một đa giác lồi; hoặc kết luận vô nghiệm. Ràng buộc $n \leq 1000, 3 \leq k \leq 6$.

Phân tích. Trước hết xét bài con: với k đoạn đã cho, khi nào chúng lập được một đa giác lồi? Nếu độ dài là $a_1, \dots, a_k$, điều kiện tương đương là

$$\max(a_1, \dots, a_k) < \sum_i a_i - \max(a_1, \dots, a_k),$$

tức cạnh dài nhất nhỏ hơn tổng các cạnh còn lại. Mọi đa giác đơn không lõm có thể được biến đổi bằng tịnh tiến và quay cạnh để thu được một đa giác lồi, nên điều kiện này đủ cho bài đang xét.

Một tính chất hữu ích là: nếu tồn tại một tập k đoạn lập được đa giác lồi, thì cũng tồn tại một tập như vậy mà sau khi sắp các đoạn theo độ dài, các đoạn được chọn tương ứng với một đoạn liên tiếp trong dãy đã sắp. Lý do là nếu trong tập chọn có một đoạn dài hơn nhưng có đoạn ngắn hơn nằm ngoài tập ở giữa thứ tự, ta thay đoạn dài bằng đoạn ngắn; điều kiện cạnh lớn nhất nhỏ hơn tổng các cạnh còn lại không bị phá vỡ. Lắp lại quá trình thay thế sẽ đưa tập chọn về một đoạn liên tiếp.

Với bài gốc, một ý tưởng trực tiếp là chia các đoạn thành hai nhóm rồi trong mỗi nhóm tìm một đoạn liên tiếp độ dài k thỏa điều kiện trên. Ta ngẫu nhiên gán mỗi đoạn vào một trong hai nhóm với xác suất $1/2$. Sau đó chạy kiểm tra tuyến tính hoặc $O(n \log n)$ cho từng nhóm.

Phân tích trường hợp xấu: giả sử chỉ có đúng hai tập rời nhau X, Y , mỗi tập kích thước k , có thể lập đa giác lồi. Một phép chia là tốt khi toàn bộ X nằm một bên và toàn bộ Y nằm bên kia, theo một trong hai thứ tự. Xác suất này là

$$\frac{2 \cdot 2^{n-2k}}{2^n} = 2^{1-2k}.$$

Với $k \leq 6$, xác suất một lần thử nhỏ nhất là $1/2048$. Chạy khoảng 20000 lần vẫn chấp nhận được, và xác suất thất bại chỉ còn xấp xỉ

$$\left(1 - \frac{1}{2048}\right)^{20000} \approx 5.7 \cdot 10^{-5}.$$

Các ví dụ trên cũng có thuật toán tất định, nhưng thường cần phân tích sâu hơn nhiều. Điều này cho thấy ngẫu nhiên hóa có thể giảm độ khó tư duy, đơn giản hóa cài đặt, và trong một số bài còn đạt độ phức tạp tốt hơn.

14.5.6 Ví dụ 5: Tích trong của vector

Định nghĩa tích trong của hai vector d chiều

$$A = [a_1, a_2, \dots, a_d], \quad B = [b_1, b_2, \dots, b_d]$$

là

$$(A, B) = \sum_{i=1}^d a_i b_i.$$

Cho n vector d chiều. Với trường hợp $k = 2$, cần tìm hai vector có tích trong chia hết cho 2, hoặc kết luận không tồn tại. Dữ liệu có $n \leq 10000, d \leq 100$. Bài gốc còn có trường hợp $k = 3$, nhưng phần khó của trường hợp đó không liên quan trực tiếp tới chủ đề bài viết, nên ở đây chỉ xét $k = 2$.

Phân tích. Xem n vector là một ma trận A kích thước $n \times d$, và đặt

$$B = AA^T.$$

Khi đó B_{ij} chính là tích trong của vector thứ i và vector thứ j . Ta chỉ cần biết ngoài đường chéo chính có vị trí nào bằng 0 (mod 2) hay không. Mọi phép tính có thể thực hiện modulo 2.

Xây dựng ma trận C kích thước $n \times n$, trong đó đường chéo chính giống đường chéo của B , còn mọi phần tử ngoài đường chéo đều bằng 1. Khi đó bài toán trở thành kiểm tra $AA^T = C$. Tính trực tiếp phép nhân ma trận là không khả thi.

Ý tưởng ngẫu nhiên mới là nhân hai vế với một vector hàng ngẫu nhiên X độ dài n . Nếu $AA^T = C$, chắc chắn

$$XAA^T = XC$$

với mọi X . Vế trái có thể tính trong $O(nd)$ bằng cách nhân lần lượt XA rồi $(XA)A^T$. Vế phải cũng tính nhanh được vì C có cấu trúc đặc biệt; nếu $Y = XC$, thì

$$Y_i = \sum_j X_j C_{ji}.$$

Ta chứng minh xác suất sai. Nếu $AA^T \neq C$, đặt

$$D = AA^T - C.$$

Tồn tại một phần tử $D_{ij} \neq 0$. Thuật toán trả về “bằng nhau” khi XD là vector không. Xét thành phần thứ j :

$$(XD)_j = \sum_k X_k D_{kj} = 0.$$

Khi mọi X_k với $k \neq i$ đã cố định, vì $D_{ij} \neq 0$ trong modulo 2, chỉ có đúng một giá trị của $X_i \in \{0, 1\}$ làm phương trình trên đúng. Do đó xác suất sai không vượt quá $1/2$. Lặp lại vài chục lần là đủ.

Nếu phát hiện hai vế khác nhau, ta còn cần tìm vị trí tích trong bằng 0. Từ một thành phần khác không trong XD , có thể suy ra tồn tại nghiệm liên quan tới vector tương ứng; sau đó liệt kê vector còn lại và kiểm tra trực tiếp trong $O(nd)$.

14.5.7 Hướng thiết kế 2

Từ ví dụ trên, có thể rút ra một kiểu thiết kế khác:

1. Thiết kế một thuật toán tất định nhận thêm một vector làm tham số. Giá trị trong vector này không được phụ thuộc vào dữ liệu vào; nếu vector được rút ra từ dữ liệu vào thì thực chất đã quay về hướng thiết kế 1.
2. Đưa ngẫu nhiên vào bằng cách sinh vector đó ngẫu nhiên rồi truyền cho thuật toán kiểm tra.

Kiểu này khó nghĩ hơn, và không nhiều bài có lời giải chuẩn thuộc dạng này. Nhưng nó vẫn rất hữu ích. Với một số bài tối ưu có chi phí nhiều chiều, có thể ngẫu nhiên một vector trọng số, định nghĩa chi phí mới là tích trong giữa vector chi phí gốc và vector trọng số, giải bài tối ưu một chiều, rồi dựng lại phương án để tính chi phí thật. Một số bài hình học tính toán cũng có thể dùng cách tương tự.

14.5.8 Ví dụ 6: Graph Reconstruction

Cho đồ thị vô hướng n đỉnh, m cạnh, mỗi đỉnh bậc không quá 2, không có khuyên và không có cạnh trùng. Hãy dựng một đồ thị mới thỏa:

- có n đỉnh và m cạnh;
- mỗi đỉnh bậc không quá 2, không có khuyên và không có cạnh trùng;
- nếu cạnh (u, v) tồn tại trong đồ thị cũ thì cạnh đó không được tồn tại trong đồ thị mới.

Nếu không thể, in vô nghiệm. Ràng buộc $1 < m < n \leq 100000$.

Phân tích. Vì mọi đỉnh trong đồ thị cũ có bậc không quá 2, đồ thị cũ là hợp của các đường đi và chu trình. Trong đồ thị bù, mỗi đỉnh có bậc ít nhất $n - 3$. Bài toán trở thành chọn m cạnh trong đồ thị bù sao cho bậc vẫn không quá 2.

Ta xét một yêu cầu mạnh hơn: tìm một chu trình Hamilton trong đồ thị bù. Khi đó chỉ cần lấy tùy ý m cạnh trên chu trình này làm đồ thị mới.

Dùng định lý Ore: với đồ thị vô hướng n đỉnh, nếu với mọi cặp đỉnh không kề nhau u, v đều có

$$\deg u + \deg v \geq n,$$

thì đồ thị có chu trình Hamilton. Trong đồ thị bù của ta, mọi đỉnh có bậc ít nhất $n - 3$, nên điều kiện đủ là

$$2(n - 3) \geq n,$$

tức $n \geq 6$. Vì vậy với n đủ lớn, nghiệm luôn tồn tại; các trường hợp nhỏ có thể xử lý vét cạn.

Thuật toán Las Vegas rất đơn giản:

1. sinh ngẫu nhiên một hoán vị $p_1, p_2, \dots, p_n$;
2. kiểm tra các cạnh (p_i, p_{i+1}) và (p_n, p_1) có xuất hiện trong đồ thị cũ hay không;
3. nếu không cạnh nào thuộc đồ thị cũ, ta đã tìm được một chu trình Hamilton trong đồ thị bù; nếu thất bại thì sinh lại hoán vị.

Mỗi lần thử có thể cài trong $O(n \log n)$ hoặc $O(n)$, tùy cách lưu cạnh.

Điểm chính là số lần thử kỳ vọng. Xét xấp xỉ trường hợp xấu khi đồ thị cũ là một chu trình dài n . Ta lần lượt cố định từng vị trí của hoán vị và chỉ ước lượng xác suất không sinh ra cạnh xấu, bỏ qua việc chính xác các đỉnh nào đã được chọn. Khi n lớn, xác suất một hoán vị hợp lệ xấp xỉ

$$\left(\frac{n-3}{n-1} \right)^n,$$

và giới hạn của biểu thức này là $e^{-2} \approx 0.135335$. Với n không quá nhỏ, có thể lấy cận thực dụng lớn hơn 0.1, nên số lần thử kỳ vọng dưới 10. Với $n < 9$, dùng vét cạn là đủ.

14.5.9 Hướng thiết kế 3

Từ ví dụ trên, ta có một hướng thiết kế cho Las Vegas:

1. Bắt đầu từ một thuật toán tất định cần liệt kê hoán vị của các phần tử. Dạng bài phù hợp thường chỉ cần tìm một phương án khả thi, hoặc có rất nhiều phương án tối ưu, sao cho số hoán vị “có ích” không quá ít.
2. Thay việc liệt kê hoán vị bằng sinh ngẫu nhiên một hoán vị. Phân tích xác suất thành công thường là phần khó nhất, và nhiều khi chỉ có thể ước lượng gần đúng.

Trong thi đấu, nếu nghĩ ra thuật toán kiểu này, đôi khi kiểm chứng bằng thực nghiệm còn hữu ích hơn cố tìm một chứng minh quá chặt trong thời gian ngắn. Cũng có thể giới hạn số lần chạy, tức “cắt theo thời gian”, để biến nó thành Monte Carlo; với bài cho điểm theo chất lượng lời giải, cách này đôi khi vẫn đem lại điểm tốt.

14.6 Tổng kết

Qua phần giới thiệu và các ví dụ, có thể thấy thuật toán ngẫu nhiên áp dụng được vào nhiều kiểu bài khác nhau. Tuy vậy, vì bản thân khái niệm này bao trùm rất rộng, không có một khuôn mẫu chung cho mọi bài; phải phân tích theo từng tình huống.

So với thuật toán tất định, thuật toán ngẫu nhiên có vài ưu điểm:

- không cần phân tích quá sâu một số tính chất của bài toán;
- độ phức tạp cài đặt thường thấp hơn;
- trong một số trường hợp, đạt độ phức tạp thời gian tốt hơn;
- với một vài bài, ngẫu nhiên hóa là lời giải đúng tự nhiên nhất.

Nhược điểm cũng rõ ràng:

- cần chứng minh độ phức tạp và xác suất đúng tương đối nghiêm ngặt, đôi khi phần chứng minh khá phức tạp;
- không phải bài nào cũng có cách ngẫu nhiên hóa; phần lớn thời gian, ngẫu nhiên hóa chỉ là công cụ để lấy một phần điểm.

Bài viết đã phân loại một số thuật toán ngẫu nhiên theo thao tác và đưa ra ba hướng thiết kế tương đối chung. Dĩ nhiên đó không phải toàn bộ cách nghĩ có thể có. Khi đối mặt với bài cần thuật toán ngẫu nhiên, ta vẫn cần phát huy tư duy sáng tạo, đồng thời cố gắng giữ phân tích đủ chặt chẽ.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Cảm ơn cô Li Shuping, huấn luyện viên của tác giả, vì sự hướng dẫn và quan tâm. Cảm ơn cha mẹ vì đã ủng hộ và động viên tác giả học thi tin học. Cảm ơn rất nhiều bạn học đã đồng hành và giúp đỡ tác giả trong suốt chặng đường.

Tài liệu tham khảo

1. Wikipedia, “Randomized algorithm”.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, *Introduction to Algorithms*, Second Edition, MIT Press.
3. Rajeev Motwani, *Randomized Algorithms*, Cambridge University Press, 1995.

15 Hiện thực chương trình tính tế: bàn về tối ưu hằng số trong OI

He Qi, Trường Trung học Nam Khai, Thiên Tân

Tóm tắt

Thời gian chạy của chương trình chủ yếu do độ phức tạp thuật toán quyết định: độ phức tạp mô tả cách thời gian chạy thay đổi khi quy mô dữ liệu tăng. Hằng số của chương trình là hệ số nhân trước phần độ phức tạp nút thắt; nó không tăng theo quy mô dữ liệu nên thường bị xem là yếu tố phụ. Tuy nhiên, với cùng một quy mô dữ liệu, tối ưu hằng số có thể làm thời gian chạy giảm vài lần và khiến chương trình hiệu quả hơn nhiều.

Bài viết này bàn về những xử lý không làm thay đổi bậc độ phức tạp trong OI, nhưng ảnh hưởng rõ rệt tới thời gian chạy: chọn giữa các thuật toán cùng bậc, chọn thuật toán gần bậc tối ưu nhưng hằng số nhỏ hơn, và tinh chỉnh chi tiết hiện thực.

15.1 So sánh và lựa chọn hằng số giữa các thuật toán cùng loại

Với cùng một bài, thường có nhiều thuật toán đều tối ưu hoặc gần tối ưu về độ phức tạp. Cách hiện thực khác nhau làm hằng số khác nhau. Vì hằng số phụ thuộc mạnh vào chi tiết cài đặt, các so sánh dưới đây dùng phần khung thuật toán ít thay đổi để ước lượng; kết quả thực tế có thể lệch đi.

15.1.1 Thuật toán có cùng độ phức tạp

Ta lấy Fenwick tree, cây đoạn và cây cân bằng làm ví dụ. Chúng đều thường xuất hiện với độ phức tạp dạng $O(n \log n)$.

Vì sao thường dùng chia đôi. Khi mô tả độ phức tạp, ta viết $O(n \log n)$ chứ không cố định đáy logarit, vì với mọi hằng số x, y ,

$$\frac{n \log_x n}{n \log_y n} = \log_x y = C.$$

Đáy logarit không ảnh hưởng tới bậc độ phức tạp, nhưng ảnh hưởng tới hằng số.

Nhìn thuần túy theo số tầng, chia ba có vẻ tốt hơn chia đôi: hệ số gộp là 3, còn số tầng giảm theo $\log_3 n$. Nhưng trong hiện thực, chia ba cần tính hai vị trí chia, thường kéo theo phép chia nguyên. Trên máy tính nhị phân, chia đôi có thể dùng dịch bit, nên hằng số thực tế nhỏ hơn nhiều. Các cấu trúc cây cân bằng dựa trên xoay cũng lấy chia đôi làm nền tảng; nếu cố thay bằng chia ba, ngay cả phân tích đã trở nên nặng nề.

Cây đoạn. Cây đoạn thường được viết từ trên xuống, có thể hỗ trợ tạo nút động, lưu phiên bản và nhiều biến thể khác. Để tìm con bằng bit nhanh, nút x thường có hai con $2x$ và $2x + 1$.

Với cách viết này, thao tác trên một điểm mỗi tầng chỉ đi qua một nút, nhưng còn chi phí quay lui đệ quy. Với thao tác trên đoạn, mỗi tầng có thể gặp tối đa bốn nút và lấy ra tối đa hai nút; nếu lấy số nút được thăm làm hằng số, ta có thể xem hằng số là khoảng 4.

Một dạng cây đoạn từ dưới lên, thường được gọi theo kiểu cài đặt của Zhang Kunwei, đệm dây tới một độ dài lũy thừa của hai rồi đặt dữ liệu ở tầng lá; khi hỏi đoạn, ta đổi đoạn hỏi thành khoảng mở và kéo hai đầu nút từ đáy lên. Nếu không có lazy tag dạng cập nhật, mỗi tầng chỉ cần xét hai nút được lấy ra, nên hằng số gần 2. Nếu có lazy tag, phải xét thêm tag của hai đầu khoảng mở, hằng số tăng lên gần 4.

Ngay cả khi có lazy tag, cách từ dưới lên vẫn thường nhanh hơn cách từ trên xuống, vì phân tích bằng số nút chưa tính các phép kiểm tra bao hàm đoạn trong quá trình đi từ gốc xuống. Điểm yếu của nó là không xử lý thuận tiện các bài không thể trực tiếp xác định lá cần đến, chẳng hạn nhị phân trên cây đoạn. Khi phù hợp, cây đoạn từ dưới lên thường có hiệu quả tốt hơn.

Fenwick tree. Fenwick tree có cấu trúc gần với cây nhị thức. Cách cập nhật và truy vấn của nó không giống một thuật toán chia để trị rõ ràng, nhưng hiệu quả hiện thực thường tốt hơn cây đoạn.

Một Fenwick tree kích thước 2^{k+1} có thể nhìn như hai Fenwick tree kích thước 2^k , trong đó gốc của một cây được nối vào gốc của cây kia. Cây đoạn kích thước tương ứng lại tạo một nút gốc mới nối hai nửa. Vì vậy, mỗi lần gộp, Fenwick tree tiết kiệm được một nút, mà nút bị lược bỏ thực chất là gốc của nửa phải.

Fenwick tree hỗ trợ truy vấn tiền tố $[1, x]$. Khi truy vấn cần một đoạn phải trọn vẹn, chỉ cần dùng luôn nút cha, nên nút phải riêng lẻ đó không bao giờ thực sự cần. Bit operation trong cập nhật nhảy tới tổ tiên tồn tại gần nhất; bit operation trong truy vấn nhảy tới anh em trái của cha để tiếp tục cộng.

Cấu trúc này tiết kiệm khoảng một nửa bộ nhớ so với cây đoạn. Do các nút bị bỏ không phân bố đều, hằng số tệ nhất theo số nút vẫn có thể là 1, chẳng hạn cập nhật vị trí 1 hoặc truy vấn $[1, 2^k - 1]$. Nhưng với dữ liệu ngẫu nhiên, hằng số kỳ vọng gần 0.5, tốt hơn cây đoạn.

Cây cân bằng. Cây cân bằng có nhiều cách hiện thực, đại thể gồm loại dựa trên xoay và loại dựa trên tái cấu trúc. Ở đây, ta lấy số nút được thăm cộng với số con trở bị sửa làm thước đo hằng số.

Với cây dựa trên xoay, nếu lưu con trở cha, mỗi lần xoay phải sửa khoảng sáu con trở. AVL có độ sâu tốt, nhưng mỗi tầng mất tới hai phép xoay, nên hằng số có thể lên tới cỡ một chục. Phân tích thể năng của splay tự mang hằng số 3; số lần truy cập và số lần xoay cùng bậc, nên hằng số thực dụng còn lớn hơn, bài gốc ước lượng khoảng 21.

Treap dựa trên xoay có số lần xoay kỳ vọng bị chặn bởi một hằng số nhỏ, nên nút thất nằm ở độ sâu; độ sâu thực tế thường chỉ có hệ số khoảng 1.5 tới 2. Nhìn trên khung thuật toán, treap rất tốt. Nhưng phần ngoài nút thất không nhỏ và còn phải xử lý bộ sinh số ngẫu nhiên, nên hiệu quả thực tế không luôn đẹp như ước lượng, dù đây vẫn là một thuật toán hằng số khá nhỏ.

Với cây dựa trên tái cấu trúc, không cần lưu con trở cha, nhưng chi phí tái cấu trúc tỷ lệ với kích thước đoạn bị xây lại. Chẳng hạn scapegoat tree với hệ số cân bằng 0.7 có hằng số độ sâu và hằng số tái cấu trúc nhân lại vào khoảng mười mấy. Treap kiểu tái cấu trúc mỗi lần xây lại có kích thước kỳ vọng theo độ sâu, nên thường chỉ dùng khi bài toán không cho phép xoay.

Các con số trên vẫn cách khá xa tỷ lệ thời gian chạy thực tế, vì truy cập và sửa con trở không phải toàn bộ thao tác của cây cân bằng. Dù vậy, chúng cho thấy họ cây cân bằng có hằng số không nhỏ, đặc biệt splay. Khi bài toán cho phép, ta nên ưu tiên cây đoạn tĩnh đã cấp sẵn vị trí thay cho cây cân bằng.

15.1.2 Thuật toán có độ phức tạp gần nhau

Đôi khi thuật toán không có bậc tối ưu lại có hằng số nhỏ hơn nhiều, đủ để cạnh tranh với thuật toán tối ưu trên quy mô dữ liệu hiện có.

Heap. Heap có nhiều cách hiện thực. Dùng rộng rãi nhất là binary heap bằng mảng. Khi cần hỗ trợ gộp heap, ta hay gặp leftist heap, binomial heap và Fibonacci heap.

Binary heap bằng mảng có hầu hết thao tác ngoài lấy min ở $O(\log n)$. Nó dùng quan hệ cha-con như cây đoạn, nút x có con $2x$ và $2x + 1$, nên có thể tăng tốc bằng bit operation; hằng số rất nhỏ, gần 1.

Leftist heap là binary heap có hỗ trợ gộp. Độ phức tạp gần binary heap, nhưng có thao tác đổi con nên hằng số tăng khoảng gấp đôi. Dù cây không cân bằng, thao tác chỉ đi theo phía nông hơn, trừ DecreaseKey, nên hệ số độ sâu vẫn nhỏ.

Binomial heap, nếu lưu riêng nút min, có độ phức tạp cùng bậc với các cách trên. Nó dùng con trở theo dạng con trái và anh em phải. Khi gộp hai cây nhị thức cùng kích thước, hằng số khoảng 2; trường hợp xấu phải gộp qua nhiều bậc nên hằng số lớn hơn. Trung bình số cây bị gộp gần với số bit bật trong một số, tương tự Fenwick tree, nên hằng số kỳ vọng tốt hơn trường hợp xấu.

Fibonacci heap đạt $O(1)$ khấu hao cho nhiều thao tác ngoài ExtractMin/DeleteMin. Mỗi lần nối hoặc tách danh sách kép lại có hằng số sửa liên kết không nhỏ; hệ số bậc của cây cũng xuất hiện trong phân tích. Ưu thế lý thuyết của Fibonacci heap là rõ ràng, nhưng trong nhiều bài, số lần chèn và số lần lấy không chênh nhau đủ lớn, khiến các heap đơn giản hơn vẫn nhanh hơn. Chỉ khi số thao tác chèn hoặc giảm khóa áp đảo đáng kể so với nhóm thao tác lấy, Fibonacci heap mới đáng cân nhắc.

Mảng hậu tố. Hai cách dựng mảng hậu tố thường gặp là thuật toán nhân đôi và DC3. Phần chính của các thuật toán này dựa trên radix sort, nên có thể lấy số vòng quét làm hằng số.

Với nhân đôi, mỗi vòng có hai lần radix sort, tạo lại rank, và có thể cần đủ $\log n$ vòng trong trường hợp xấu. Độ phức tạp là $O(n \log n)$, hằng số không nhỏ. Phần RMQ sau đó có thể dùng tiền xử lý $O(n \log n)$, hỏi $O(1)$.

DC3 là thuật toán tuyến tính $O(n)$, nhưng mỗi mức đệ quy có nhiều lần sắp xếp theo khóa, gộp và xử lý rank. Hằng số vì vậy khá lớn; phần RMQ tuyến tính thật sự cũng rườm rà. Trong thực tế, DC3 thường hơi nhanh hơn nhân đôi, nhưng khoảng cách không lớn như bậc độ phức tạp gợi ý.

Thao tác trên đường đi của cây. Với thao tác trên một đường đi trong cây, hai lựa chọn quen thuộc là phân rã nặng-nhẹ và dynamic tree. Hai khung này khác nhau nên đơn vị hằng số cũng khác, chỉ có thể ước lượng thô.

Dynamic tree có phân tích gần splay, hằng số lớn. Phân rã nặng-nhẹ có thêm một logarit: tìm LCA hoặc tách đường đi rồi dùng cây đoạn trên từng đoạn. Tuy vậy, hai lớp logarit khó cùng đạt trường hợp xấu; tổng độ dài các đoạn theo chuỗi nặng thường nhỏ hơn ước lượng bi quan. Vì vậy, dù có thêm $\log n$, phân rã nặng-nhẹ trong thực tế vẫn thường không kém dynamic tree.

15.2 Tối ưu hằng số trong hiện thực

Ngay trong cùng một thuật toán, nhiều chi tiết nhỏ vẫn quyết định hằng số của toàn bộ chương trình.

15.2.1 Phần vô dụng trong một khối

Nhiều thuật toán dùng mảng và vòng lặp, nhưng không phải mọi vị trí mảng đều có ý nghĩa. Tránh tính các vị trí vô dụng có thể giảm hằng số đáng kể.

Quy hoạch động và tìm kiếm có nhớ. Trong quy hoạch động nhiều chiều, thường có nhiều trạng thái không hợp lệ hoặc không cần xét. Ta có thể dùng tìm kiếm có nhớ để chỉ chạm tới trạng thái hợp lệ.

Ví dụ, bài NOIP 2008 *Truyền giấy* yêu cầu trên lưới 200×200 tìm hai đường đi từ góc trên trái tới góc dưới phải, không giao nhau, sao cho tổng trọng số đạt tối ưu. Cách dễ nghĩ là đặt $dp[x_1][y_1][x_2][y_2]$, biểu thị hai đường đi đang ở (x_1, y_1) và (x_2, y_2) . Bậc $O(n^4)$ có vẻ không qua hết dữ liệu, nhưng rất nhiều trạng thái không thể xảy ra. Chỉ hai ô có cùng khoảng cách tới điểm xuất phát mới có thể thuộc cùng một trạng thái, nên lời giải chuẩn dùng $dp[khoảng\ cách][x_1][x_2]$ với độ phức tạp $O(n^3)$. Tìm kiếm có nhớ cũng chỉ ghé các trạng thái hợp lệ này, giúp ta đạt cùng hiệu quả mà ít phải suy nghĩ lại mô hình hơn.

Lược bớt phần vô dụng trong ma trận. Trong một số bài, ta dùng nhân ma trận hoặc nhân đôi để tăng tốc, nhưng không phải mọi ô trong ma trận đều cần tính.

Ví dụ, bài NOI 2013 ngày 2 bài 1 cần lập các biến đổi tuyến tính dạng $y = ax + b$ và $y = cx + d$, rồi lấy đáp án modulo p . Có thể mô tả biến đổi bằng ma trận affine 2×2 và dùng nhân đôi. Nếu nhân ma trận 2×2 đầy đủ, mỗi lần nhân cần tám phép nhân có modulo và sẽ quá chậm. Nhưng trong ma trận affine, cột cố định chứa 0 và 1 không đổi sau mọi phép nhân. Chỉ cần lưu phần thật sự biến đổi, hằng số giảm còn hai phép nhân modulo và hai phép cộng modulo. Bài gốc cho biết chính kiểu hiện thực thật tình này là nguồn cảm hứng cho nhan đề bài viết.

15.2.2 Chọn hằng số tự định nghĩa

Một số thuật toán có tham số do người viết tự chọn. Tham số không đổi bậc độ phức tạp nhưng ảnh hưởng lớn tới thời gian chạy.

Chia khối. Trong thuật toán chia khối, kích thước khối thường cần được chỉnh theo chi phí thật.

Ví dụ, bài *Tất của Tiểu Z* cho dãy độ dài n và m truy vấn; mỗi truy vấn hỏi xác suất chọn ngẫu nhiên hai số bằng nhau trong đoạn $[l_i, r_i]$. Với thuật toán Mo, dịch một đầu mút một đơn vị có chi phí hằng số. Nếu chia theo đầu trái thành x khối và trong mỗi khối sắp truy vấn theo đầu phải, tổng độ dài di chuyển của đầu trái xấp xỉ mn/x , còn đầu phải xấp xỉ xn . Vì vậy nên chọn số khối $x = \sqrt{m}$, không phải mặc định $\sqrt{n}$. Thử nghiệm trong bài gốc với $n = 65536$, $m = 262144$ cho thấy chọn 512 khối nhanh hơn chọn 256 khối khoảng 1.5 lần.

Chia để trị trên cây. Trong chia để trị trên cây, cách thông dụng là chia theo trọng tâm. Có hai tham số hằng số đáng chú ý: chia các cây con thành hai nửa như thế nào, và khi kích thước còn bao nhiêu thì chuyển sang vét cạn.

Cách chia đơn giản là xét cây con theo thứ tự bất kỳ, khi tổng vượt quá một nửa thì đưa cây con đó sang nửa nhỏ hơn. Do tính chất trọng tâm, cây con lớn nhất không quá một nửa tổng kích thước, nhưng tỷ lệ xấu nhất của hai phần có thể tới $1 : 3$. Cải tiến là tạm chia thành ba phần đều không quá một nửa, rồi gộp hai phần nhỏ hơn lại. Khi đó một phía không quá $2/3$, phía còn lại không quá $1/2$, tỷ lệ xấu nhất thành $1 : 2$. Trường hợp có ba cây con bằng nhau cho thấy khó làm tốt hơn tỷ lệ này.

Với ngưỡng chuyển sang vét cạn, mỗi bước chia để trị phải duyệt cây để tính kích thước và trọng tâm, duyệt các phần để lập danh sách, rồi gộp. Tổng cộng có thể tương đương sáu tới tám lần duyệt. So với chi phí vét cạn trên cây con nhỏ, phân tích và thử nghiệm của bài gốc đều gợi ý chuyển sang vét cạn khi kích thước khoảng 20 là hợp lý.

15.2.3 Một số chi tiết hiện thực

Bit operation. Khi cần thao tác đồng loạt trên biến boolean, bit operation có thể tăng tốc rất mạnh. Ngay cả trên số nguyên 64 bit, phép bit vẫn rất nhanh; trong một số bài, nó nâng được quy mô xử lý lên cả một bậc.

Khi dùng quy hoạch động nén trạng thái, nên ưu tiên biểu diễn trạng thái bằng cơ số 2, 4 hoặc 8, vì có thể lấy phần cần dùng bằng dịch và mask. Nếu có trạng thái thừa, có thể kết hợp với ý tưởng bỏ qua phần vô dụng trong khối. Phép nhân và chia thường chậm hơn cộng trừ, nên các thao tác thường gặp như nhân đôi hoặc chia đôi cũng nên đổi thành dịch bit khi phù hợp. Cần chú ý độ ưu tiên toán tử.

Đọc dữ liệu. Khi đọc số thập phân, chương trình phải chuyển sang nhị phân, kéo theo nhiều phép nhân theo số chữ số. Điều này rất rõ trong các thuật toán $O(n)$ hoặc $O(n \log n)$ có hằng số nhỏ. Trên môi trường bài gốc, đọc một tệp chứa 10^5 số nguyên bằng scanf đã mất khoảng một giây, còn luồng vào chuẩn của C++ chậm hơn nữa.

Hàm đọc của hệ thống phải xử lý mọi tình huống hợp lệ của ngôn ngữ, nên ổn định nhưng hằng số lớn. Trong thi đấu, ta thường có thể giả định dữ liệu đúng định dạng đề bài, bỏ qua nhiều nhánh kiểm tra và tự viết hàm đọc dựa trên đọc từng ký tự. Hiệu quả thường nhanh hơn hàm hệ thống khoảng 60% tới 80%.

Sinh số ngẫu nhiên. Hàm sinh số ngẫu nhiên của hệ thống khá chậm; trong môi trường bài gốc, sinh khoảng $5 \cdot 10^5$ số đã mất gần một giây. Khi tự viết bộ sinh, có thể dùng lặp tuyến tính, lặp bậc hai, hoặc luân phiên nhiều hàm để giữ tính ngẫu nhiên đủ dùng trong bài, đồng thời giảm hằng số.

Tình hóa cấp phát động. Hàm cấp phát bộ nhớ của hệ thống rất chậm nếu gọi nhiều lần. Với các cấu trúc dữ liệu tiêu tốn nhiều nút động như cây đoạn lưu phiên bản, dùng cấp phát động có thể làm chương trình chậm hơn 10 tới 20 lần. Cách xử lý là ước lượng số nút cần dùng, cấp trước một memory pool và tự quản lý con trỏ cấp phát. Nếu cần thu hồi, có thể dùng thêm stack để tái sử dụng nút.

Tính liên tục khi truy cập bộ nhớ. Vấn đề này rất rõ trong nhân ma trận lớn. Nếu vòng lặp trong cùng thay đổi chỉ số không liên tục trong bộ nhớ, bộ nhớ đệm hoạt động kém và chương trình chậm đi đáng kể. Chẳng hạn, trong nhân ma trận theo thứ tự chỉ số quen thuộc, vòng trong cùng có thể quét theo cột của ma trận thứ hai, tức thay đổi chiều không liên tục của mảng hai chiều; với ma trận 200×200 đã có thể mất khoảng một giây trên môi trường bài gốc. Đổi thứ tự vòng lặp để vòng trong cùng quét theo chiều liên tục của mảng thứ hai có thể xử lý tới cỡ 400×400 .

Kết luận thực dụng là: cố tránh việc chỉ số không phải chiều cuối thay đổi liên tục trong vòng lặp nóng. Nếu cần, hãy đổi hai chiều của mảng hoặc đổi thứ tự vòng lặp.

15.3 So sánh thuật toán trên một bài kinh điển

Bài toán: cho n số nguyên có tổng không vượt quá C , chia chúng thành hai phần sao cho hai tổng càng gần nhau càng tốt; cần tìm chênh lệch nhỏ nhất. Quy mô của n và C cùng cấp.

15.3.1 Thuật toán 1: ba lô 0/1

Đặt $dp[i][j]$ cho biết dùng i số đầu có thể đạt tổng j hay không. Ban đầu $dp[0][0] = 1$, sau đó chuyển theo việc không chọn hoặc chọn số thứ i . Độ phức tạp $O(nC)$, xử lý được dữ liệu cỡ 10^4 .

15.3.2 Thuật toán 2: tối ưu bằng bit operation

Giá trị dp thực chất là boolean, nên có thể dùng bitset để nén mỗi trạng thái thành một bit. Với số c_i , chuyển trạng thái bằng cách lấy bitset cũ OR với bản dịch trái c_i vị trí. Độ phức tạp vẫn nhìn theo $O(nC)$, nhưng hằng số giảm theo số bit trong một word máy. Cách này có thể xử lý quy mô $5 \cdot 10^4$, thậm chí 10^5 .

15.3.3 Thuật toán 3: chia khối và FFT

Thuật toán này do Guo Xiaoxu của Đại học Giao thông Thượng Hải đề xuất.

Trong n số, số phần tử lớn hơn $\sqrt{C}$ nhiều nhất chỉ là $\sqrt{C}$. Các số không vượt quá $\sqrt{C}$ có thể chia thành nhiều khối sao cho tổng mỗi khối không quá $\sqrt{C}$; mỗi khối cũng có không quá $\sqrt{C}$ phần tử. Dùng thuật toán ba lô cơ bản, ta tính được tập tổng có thể đạt của một khối trong $O(C)$, nên tính toàn bộ khối mất $O(C\sqrt{C})$.

Khi gộp hai khối, nếu a_i và b_i biểu thị hai phía có đạt tổng i hay không, thì kết quả c_i là OR của mọi $a_j \wedge b_{i-j}$. Thay OR boolean bằng phép cộng rồi dùng FFT, có thể gộp trong $O(C \log C)$; tổng thời gian gộp là $O(C\sqrt{C} \log C)$. Sau đó xử lý các số lớn hơn $\sqrt{C}$ bằng chuyển ba lô thường, tổng thể vẫn bị FFT chi phối.

Vì không dùng được bit operation kiểu thuật toán 2, hằng số của FFT rất lớn. Hiệu quả thực tế chỉ đạt khoảng quy mô 2^{15} , thậm chí kém thuật toán 2.

15.3.4 Thuật toán 4: chia để trị và FFT

Ta có thể dùng chia để trị để hỗ trợ thao tác gộp. Với một tập số có tổng C , có thể tách thành ba phần: hai phần có tổng không quá $C/2$, và phần thứ ba chỉ gồm một số. Giải đệ quy hai phần đầu, gộp bằng một FFT kích thước C , rồi chuyển thô số còn lại trong $O(C)$.

Theo định lý chủ,

$$T(C) = 2T(C/2) + O(C \log C) + O(C) = O(C \log^2 C).$$

Ta có một thuật toán có bậc rất tốt. Nhưng nút thắt vẫn là FFT, nên hằng số vẫn lớn. Ngay cả ở quy mô $5 \cdot 10^5$, khi $n = C$, bài gốc ghi nhận thuật toán này mất khoảng 6 giây, còn thuật toán 2 mất khoảng 15 giây; chênh lệch chưa đủ lớn so với khác biệt về bậc độ phức tạp.

15.3.5 Thuật toán 5: nhiều ba lô

Khi n lớn, nhiều số sẽ trùng nhau; có thể dùng cách làm nhiều ba lô. Trong trường hợp xấu, t số khác nhau nhỏ nhất có tổng $1 + 2 + \dots + t = O(t^2)$, nên số giá trị khác nhau nhiều nhất là $O(\sqrt{C})$.

Dùng chuyển trạng thái cổ điển của nhiều ba lô: với mỗi giá trị, lưu số lần ít nhất cần dùng giá trị đó để đạt tổng j ; -1 biểu thị không đạt được. Độ phức tạp là $O(C\sqrt{C})$. Do không nén bit được, hằng số lớn hơn thuật toán 2, nhưng vẫn có thể đạt quy mô 10^5 .

15.3.6 Thuật toán 6: cải tiến ba lô 0/1

Một cách xử lý nhiều ba lô khác là tách các phần tử bằng nhau theo nhị phân: từ x phần tử giống nhau, tạo ra $O(\log x)$ phần tử có trọng lượng gộp. Như vậy có thể giảm số phần tử hiệu dụng xuống $O(\sqrt{C} \log C)$.

Quan sát thêm: chỉ cần một giá trị xuất hiện quá hai lần là ta còn có thể tiếp tục gộp. Nếu xét từ nhỏ tới lớn và gộp dần, cuối cùng mỗi giá trị xuất hiện không quá hai lần, nên số phần tử hiệu dụng chỉ còn cỡ $O(\sqrt{C})$. Sau đó chạy ba lô 0/1 và dùng tối ưu bitset của thuật toán 2. Bài gốc xem độ phức tạp là $O(C\sqrt{C})$, nhưng hằng số sau nén bit rất nhỏ, đủ xử lý quy mô 10^5 .

15.3.7 Tiểu kết

Trong các thuật toán trên, thuật toán có bậc tốt nhất là chia để trị FFT, chỉ $O(C \log^2 C)$. Nhưng hiệu quả thực tế tốt nhất lại là thuật toán 6, với bậc $O(C\sqrt{C})$. Lý do là hằng số của hai thuật toán có thể chênh nhau gần hai trăm lần. Đây là một ví dụ rõ ràng cho thấy chú ý tới hằng số có thể cải thiện hiệu suất chương trình rất mạnh.

15.4 Tổng kết

Hãy trở thành một Oler biết chú ý tới hằng số.

Tài liệu tham khảo

1. Zhang Kunwei, *Sức mạnh của thống kê: tiếp xúc toàn diện với cây đoạn*.
2. Thomas H. Cormen, Charles E. Leiserson và các tác giả khác, *Introduction to Algorithms*.
3. Yang Zhe, *Một số nghiên cứu về lời giải QTREE*.
4. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*.

16 Trở về gốc rễ: phép toán bit và ứng dụng

Shen Yang, Trường Trung học Trần Hải

16.1 Dẫn nhập

Trong máy tính điện tử, phép toán bit còn cơ bản hơn cả bốn phép toán số học quen thuộc. Tuy vậy, vì chúng không phải phép toán cơ bản trong toán học phổ thông và ít xuất hiện trong đời sống hằng ngày, nhiều thí sinh, đặc biệt là người mới học tin, dễ bỏ qua chúng. Bài viết này hệ thống hóa các phép toán bit và một số ứng dụng thường gặp trong OI.

Quy ước của bài viết là: bit cao nhất nằm bên trái, bit thấp nhất nằm bên phải, và bit thấp nhất được gọi là bit 0. Nếu không nói khác, các biểu thức dùng cú pháp C++ và xét trên trình biên dịch GNU C++; phần nhắc tới hợp ngữ hiểu là hợp ngữ x86.

16.2 Các phép toán cơ bản

16.2.1 Và, hoặc, phủ định và xor

Bốn phép logic cơ bản là AND, OR, NOT và XOR. Với hai bit p, q , bảng chân trị của ba phép hai ngôi là:

p	q	$p \wedge q$	$p \vee q$	$p \oplus q$
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

và phép phủ định đổi 0 thành 1, đổi 1 thành 0.

Phép toán bit có thể xem là mở rộng của các phép logic trên toàn bộ các bit của một số nguyên. Kết quả của $x \& y$, $x | y$, $\sim x$, $x \oplus y$ được tạo bằng cách áp dụng phép logic tương ứng trên từng vị trí bit. Trên x86, các phép này tương ứng với các lệnh `and`, `or`, `not`, `xor`; số có dấu và không dấu không có lệnh riêng, nên bit dấu cũng được xử lý như mọi bit khác.

16.2.2 Dịch bit

Khi nói dịch x sang trái hoặc sang phải y bit, mặc định $0 \leq y < w$, trong đó w là độ rộng bit của kiểu đang xét. Nếu vượt quá giới hạn này, lệnh hợp ngữ x86 chỉ lấy vài bit thấp của y , còn trong C/C++ hành vi có thể là không xác định.

Dịch logic. Dịch trái logic `shl` đưa mọi bit sang trái, các ô trống bên phải được điền 0, các bit vượt ra bên trái bị tràn. Dịch phải logic `shr` làm ngược lại: đưa bit sang phải, điền 0 vào bên trái.

Dịch số học. Dịch trái số học `sar` giống dịch trái logic. Dịch phải số học `sar` khác dịch phải logic ở chỗ các ô trống bên trái được điền bằng bit dấu, tức bit cao nhất ban đầu.

Dịch vòng. Dịch vòng trái `rol` và dịch vòng phải `ror` đưa phần bị tràn ở một đầu quay lại đầu kia. C++ không có toán tử riêng cho dịch vòng, nhưng với số không dấu 32 bit có thể viết dịch vòng trái là

$$(x \ll y) | (x \gg (32 - y)),$$

và dịch vòng phải là

$$(x \gg y) | (x \ll (32 - y)).$$

Dịch logic được thiết kế cho số không dấu: dịch trái tương ứng nhân với 2^y theo modulo 2^w , còn dịch phải tương ứng chia lấy phần nguyên cho 2^y . Dịch số học được thiết kế cho số có dấu theo biểu diễn bù hai: dịch trái vẫn tương ứng nhân 2^y khi không xét tràn, còn dịch phải tương ứng chia cho 2^y với làm tròn xuống. Trong C++, dịch trên số không dấu dùng dịch logic, còn dịch trên số có dấu thường dùng dịch số học; nếu cần dịch phải logic trên số có dấu, có thể ép sang kiểu không dấu trước.

16.3 Sửa đổi và truy vấn bit

Phần này chỉ xét số nguyên không dấu 32 bit, ký hiệu là `u32`.

16.3.1 Đọc một số bit

Để đọc bit thứ *pos* của *x*, dịch *x* sang phải *pos* vị trí rồi lấy bit thấp nhất:

$$\text{bit}(x, pos) = (x \gg pos) \& 1.$$

Khi nhiều giá trị nhỏ được đóng gói trong một số nguyên, ta thường cần đọc *cnt* bit bắt đầu từ vị trí *pos*. Khi đó dùng mặt nạ có *cnt* bit thấp bằng 1:

$$((x \gg pos) \& ((1u \ll cnt) - 1)).$$

Ý tưởng chung là dùng AND với một mặt nạ: bit mặt nạ bằng 1 thì giữ lại, bit mặt nạ bằng 0 thì xóa.

16.3.2 Sửa một số bit

Để đặt một bit thành 1, OR với mặt nạ có bit đó bằng 1:

$$x | (1u \ll pos).$$

Để đặt một bit thành 0, AND với mặt nạ đảo:

$$x \& \sim (1u \ll pos).$$

Để đảo một bit, XOR với mặt nạ:

$$x \wedge (1u \ll pos).$$

Các công thức này mở rộng trực tiếp cho nhiều bit bằng cách thay mặt nạ một bit bằng mặt nạ nhiều bit.

16.3.3 Đếm số bit 1

Đếm số bit 1 tương đương cộng các bit của số. Cách phân trị song song là: trước hết cộng từng cặp bit kề nhau, rồi cộng từng nhóm 2 bit thành nhóm 4 bit, tiếp tục tới khi còn một tổng. Các mặt nạ quen thuộc là `0xAAAAAAAA`, `0x55555555`, `0xCCCCCCCC`, `0x33333333`, `0xF0F0F0F0`, `0x0F0F0F0F`, rồi các mặt nạ theo byte và theo nửa word.

Một tối ưu nhỏ ở bước đầu là dùng

$$x - ((x \& 0xAAAAAAAAu) \gg 1)$$

để thu được tổng trong từng cặp bit, vì biểu thức này tương đương lấy bit chẵn cộng với bit lẻ đã dịch về đúng vị trí. Ở các bước sau, do tổng trong mỗi nhóm vẫn nhỏ hơn độ rộng nhóm, có thể cộng x với bản dịch của chính nó rồi AND lấy phần cần giữ.

Một tối ưu kinh điển khác thay hai bước cộng cuối bằng phép nhân:

$$x = (x \cdot 0x01010101u) \gg 24.$$

Sau khi mỗi byte đã chứa số bit 1 trong byte tương ứng, phép nhân với $0x01010101$ gom bốn byte vào byte cao nhất; ba phần thấp không thể tràn lên byte cao vì mỗi tổng byte không vượt quá 8.

Cách khác là tra bảng. Ta tiền xử lý $f(0) = 0$ và

$$f(i) = f(i \gg 1) + (i \& 1)$$

cho mọi số 16 bit; khi cần đếm số bit 1 của số 32 bit, tách nó thành hai nửa 16 bit rồi cộng hai giá trị tra bảng. GCC cung cấp `__builtin_popcount`; nếu máy và tùy chọn biên dịch hỗ trợ, hàm này có thể được dịch thành lệnh `popcnt`, nếu không thường dùng phương pháp gần với tra bảng.

16.3.4 Đảo thứ tự bit

Với số 32 bit, đảo thứ tự bit nghĩa là đổi bit 0 với bit 31, bit 1 với bit 30, v.v. Ví dụ 5 sau khi đảo bit thành 2684354560.

Cách phân trị tương tự như đếm bit: đầu tiên đổi chỗ từng cặp bit kề nhau, sau đó đổi chỗ từng cặp nhóm 2 bit, tiếp tục với nhóm 4, 8, 16 bit. Mỗi bước dùng hai mặt nạ đối xứng rồi dịch hai phần theo hai hướng ngược nhau. Cũng có thể tiền xử lý bảng đảo bit cho mọi số 16 bit bằng truy hồi

$$f(0) = 0, \quad f(i) = (f(i \gg 1) \gg 1) | ((i \& 1) \ll 15),$$

rồi đảo số 32 bit bằng cách đảo riêng hai nửa 16 bit và đổi vị trí hai nửa.

16.3.5 Đếm số bit 0 ở đầu hoặc ở cuối

Để đếm số bit 0 ở đầu, có thể nhị phân trên các nửa: xét 16 bit cao có rỗng không; nếu rỗng thì cộng 16 vào đáp án và tiếp tục ở nửa thấp, nếu không thì dịch nửa cao xuống và tiếp tục. Sau đó lặp lại với 8, 4, 2, 1 bit. Đếm số bit 0 ở cuối làm tương tự nhưng kiểm tra các bit thấp.

Cách tra bảng cũng tách theo 16 bit. Với số 16 bit, có thể tính bảng đếm số 0 ở đầu bằng

$$f(0) = 16, \quad f(i) = f(i \gg 1) - 1.$$

Nếu 16 bit cao của x khác 0, kết quả là bảng của nửa cao; nếu không, kết quả là 16 cộng bảng của nửa thấp. GCC có hai hàm tương ứng là `__builtin_clz` và `__builtin_ctz`, thường dựa trên `bsr` và `bsf`; cần chú ý rằng với tham số 0, hành vi của hai hàm này là không xác định.

16.3.6 Tìm bit 1 thứ k

Bài toán này có thể chuyển thành tìm w lớn nhất sao cho trong các bit từ 0 tới $w - 1$, số bit 1 nhỏ hơn k . Nếu đã có bảng đếm bit cho số 16 bit, ta nhị phân theo các khối 16, 8, 4, 2, 1 bit: nếu số bit 1 ở nửa đang xét nhỏ hơn k , bỏ qua nửa ấy và trừ số lượng tương ứng khỏi k ; nếu không, đi vào nửa ấy.

Ngay cả khi không dùng bảng, ta vẫn có thể tận dụng các tổng trung gian trong thuật toán đếm bit phân trị: sau khi đã có tổng cho từng khối 2, 4, 8, 16 bit, các tổng này chính là thông tin cần để đi xuống cây nhị phân tìm vị trí của bit 1 thứ k .

16.3.7 Tách dãy bit 1 ở cuối

Khi cộng 1 vào x , dãy 1 liên tiếp ở cuối chuyển thành 0, bit 0 ngay bên trái dãy đó chuyển thành 1, các bit khác giữ nguyên. Vì vậy dãy 1 ở cuối có thể lấy bằng:

$$x \& (x \wedge (x + 1)).$$

16.3.8 lowbit

Với số dương x , $\text{lowbit}(x)$ là phần từ bit 1 thấp nhất của x tới bit thấp nhất. Chẳng hạn $\text{lowbit}(28) = 4$, vì $28 = (11100)_2$ và $4 = (100)_2$. Ba công thức thường dùng là:

$$\text{lowbit}(x) = x \& (x \wedge (x - 1)),$$

$$\text{lowbit}(x) = x \wedge (x \& (x - 1)),$$

$$\text{lowbit}(x) = x \& (-x).$$

Hai công thức đầu dựa trên việc trừ 1 làm bit 1 thấp nhất thành 0 và biến các bit thấp hơn thành 1; công thức cuối dùng biểu diễn bù hai. `lowbit` là công cụ quen thuộc trong Fenwick tree và khi duyệt tập con.

16.3.9 Duyệt mọi bit 1

Muốn duyệt mọi vị trí có bit 1, ta lặp lại: lấy $\text{lowbit}(x)$, xử lý bit đó, rồi xóa nó khỏi x , chẳng hạn bằng XOR. Nếu cần biết vị trí cụ thể, dùng đếm số bit 0 ở cuối của $\text{lowbit}(x)$.

16.4 Dùng số nguyên làm tập hợp

Mọi tập hữu hạn sau khi đánh số phần tử đều có thể xem là tập con của $[0, n)$. Một bit bằng 1 biểu thị phần tử tương ứng có mặt, bit bằng 0 biểu thị không có mặt. Nếu một word máy xử lý được w bit, ta dùng $\lceil n/w \rceil$ word, gọi mỗi word là một khối. Cấu trúc này chính là `bitset`; về bản chất, nó là một số nguyên lớn được nén theo từng word, nên hỗ trợ AND, OR, NOT, XOR, dịch trái, dịch phải, và đôi khi cả cộng trừ.

16.4.1 Các thao tác tập cơ bản

Thêm một phần tử: tìm khối chứa phần tử đó, rồi đặt bit tương ứng thành 1. Xóa một phần tử: tìm khối rồi đặt bit tương ứng thành 0. Kiểm tra một phần tử: tìm khối rồi đọc bit. Ba thao tác này tốn $O(1)$.

Giao của hai tập là AND từng khối; hợp là OR từng khối; hiệu $A \setminus B$ có thể viết là $A \& (\sim B)$, hoặc $A \wedge (A \& B)$; phần bù là NOT từng khối. Các thao tác toàn tập này tốn $O(n/w)$.

Đếm số phần tử bằng cách cộng `popcount` của mọi khối, tốn $O(n/w)$. Duyệt mọi phần tử bằng cách duyệt từng khối rồi dùng `lowbit` để lấy các bit 1, tốn $O(n/w + c)$, với c là số phần tử thật sự được duyệt.

16.4.2 Phần tử nhỏ thứ k

Cách đơn giản là quét các khối từ nhỏ tới lớn, trừ dần số bit 1 trong từng khối cho tới khi tìm được khối chứa phần tử nhỏ thứ k , rồi dùng kỹ thuật tìm bit 1 thứ k bên trong khối. Độ phức tạp là $O(n/w + \log w)$.

Nếu truy vấn phần tử nhỏ thứ k xuất hiện rất nhiều, có thể dựng thêm cây đoạn lưu số phần tử trong mỗi khối. Khi đó ta nhị phân trên cây đoạn để tìm khối chứa phần tử cần tìm, rồi xử lý trong khối bằng kỹ thuật bit. Truy vấn giảm xuống $O(\log n)$. Khi thêm hoặc xóa phần tử, cần cập nhật cây đoạn nên tốn $O(\log n)$; khi lấy giao, hợp, hiệu hoặc phần bù, có thể tính lại cây đoạn sau khi đã xử lý các khối, tổng thời gian vẫn cùng bậc $O(n/w)$.

16.4.3 Phần tử nhỏ nhất lớn hơn x

Cách thô là kiểm tra phần còn lại của khối chứa x . Nếu còn bit 1 lớn hơn x , dùng `ctz` để lấy vị trí bit đầu tiên; nếu không, quét các khối sau đó cho tới khi gặp một khối khác 0. Độ phức tạp xấu nhất là $O(n/w)$.

Để nhanh hơn, dùng thêm một bitset phụ đánh dấu khối nào không rỗng. Khi cần tìm phần tử nhỏ nhất lớn hơn x , trước hết xét phần còn lại trong khối hiện tại; nếu không có, dùng bitset phụ để tìm khối không rỗng đầu tiên phía sau, rồi lấy bit đầu tiên trong khối đó. Vì bitset phụ cũng có thể dùng chính kỹ thuật này một cách đệ quy, truy vấn có thể đạt $O(\log_w n)$. Khi thêm hoặc xóa phần tử, ta cập nhật cả khối gốc lẫn bitset phụ; khi tính lại toàn tập sau giao, hợp, hiệu hoặc phần bù, có thể dựng lại bitset phụ trong $O(n/w)$.

16.4.4 Dịch một tập

Muốn cộng x cho mọi phần tử của tập A , dịch bitset của A sang trái x bit. Muốn trừ x , dịch sang phải x bit. Độ phức tạp là $O(n/w)$.

Nếu chỉ cần dịch một phần $B \subseteq A$, tách A thành hai phần: phần không đối $P = A \setminus B$ và phần cần dịch $Q = A \cap B$. Dịch Q , rồi lấy hợp với P . Cách này vẫn tốn $O(n/w)$.

16.4.5 Duyệt tập con

Với tập x và một tập con y của nó, tập con đứng ngay trước y theo thứ tự từ lớn xuống có thể tính bằng:

$$(y - 1) \& x.$$

Vì vậy có thể duyệt mọi tập con của x bằng cách bắt đầu từ x , liên tục thay $y \leftarrow (y - 1) \& x$, cho tới khi gặp tập rỗng.

16.4.6 Duyệt các tập có đúng K phần tử

Cần một phép sinh tập kế tiếp có cùng số bit 1. Giả sử các bit 1 liên tiếp thấp nhất của x nằm từ vị trí i tới j . Tập kế tiếp theo thứ tự tăng có thể thu được bằng cách đưa bit j sang trái một vị trí, rồi gom các bit 1 từ i tới $j - 1$ về sát bên phải. Công thức kinh điển là:

$$\begin{aligned} l &= x \& (-x), \\ y &= x + l, \\ \text{next}(x) &= y \mid (((x \hat{y}) / l) \gg 2). \end{aligned}$$

Trong đó l là `lowbit(x)`, $y = x + l$ thực hiện bước đẩy bit cao của dãy 1 sang trái, còn phần cuối lấy các bit 1 còn lại và đưa chúng về thấp nhất.

16.5 Một số ứng dụng khác

Kiểm tra chẵn lẻ: $x \& 1$ bằng 1 thì x lẻ, bằng 0 thì x chẵn.

Nhân hoặc chia cho lũy thừa của 2: dùng dịch trái hoặc dịch phải như đã nêu ở phần dịch bit.

Lấy $x \bmod 2^y$: chỉ cần giữ y bit thấp,

$$x \& ((1 \ll y) - 1).$$

Tính phần nguyên của $\log_2 x$: nếu x có độ rộng w bit và số bit 0 ở đầu là l , thì

$$\lfloor \log_2 x \rfloor = w - 1 - l.$$

Đổi chỗ hai biến có thể làm bằng ba phép XOR:

$$a \hat{=} b, \quad b \hat{=} a, \quad a \hat{=} b.$$

Cách này chủ yếu có giá trị minh họa; trong C++ hiện đại, dùng biến tạm hoặc `std::swap` thường rõ ràng và an toàn hơn.

So sánh hai số bằng nhau: $x \hat{=} y = 0$ khi và chỉ khi $x = y$.

Lấy trị tuyệt đối không rẽ nhánh cho số có dấu w bit có thể dựa trên bit dấu. Nếu $sign = x \gg (w - 1)$ cho giá trị 0 hoặc -1 , công thức thường dùng là:

$$(x \hat{=} sign) - sign.$$

Chọn không rẽ nhánh giữa x và y có thể dùng:

$$y \hat{=} ((x \hat{=} y) \& - cond),$$

trong đó $cond = 1$ trả về x , còn $cond = 0$ trả về y .

16.6 Ví dụ

16.6.1 RF

Bài toán kinh điển: có $2m + 1$ số nguyên, một số xuất hiện số lần lẻ, mọi số khác xuất hiện số lần chẵn. Vì XOR có tính giao hoán và $x \hat{=} x = 0$, chỉ cần XOR toàn bộ dãy, kết quả còn lại chính là số xuất hiện lẻ lần.

16.6.2 Robot in Basement

Bài Codeforces 97D cho một lưới $n \times m$ với $n, m \leq 150$, một số ô là vật cản, biên lưới cũng là vật cản, và một ô không cản là lối ra. Một robot thực hiện chương trình gồm các lệnh U, D, L, R; nếu lệnh đưa robot vào vật cản thì robot đứng yên. Cần tìm tiền tố ngắn nhất của chương trình sao cho robot, dù bắt đầu ở ô trống nào, đều dừng ở lối ra.

Cách thô mô phỏng tập các ô có thể có robot sau từng lệnh, tốn $O(nml)$. Tối ưu là dùng một bitset biểu diễn toàn bộ lưới: ô (i, j) ứng với bit $im + j$. Lệnh trái/phải tương ứng dịch bitset 1 vị trí, lệnh lên/xuống tương ứng dịch m vị trí. Để xử lý vật cản, tiền xử lý bốn bitset c_L, c_R, c_U, c_D , mỗi bitset đánh dấu các ô mà nếu đi theo hướng đó sẽ đâm vào vật cản. Ví dụ với lệnh trái và trạng thái x , phần $x \& c_L$ phải đứng yên, còn phần $x \setminus (x \& c_L)$ được dịch trái một bước; lấy hợp hai phần này cho trạng thái mới. Độ phức tạp giảm theo hệ số word máy, xấp xỉ $O(nml/w)$.

16.6.3 Quick Tortoise

Bài Codeforces 232E cho lưới $n \times m$ với $n, m \leq 500$, có vật cản, và q truy vấn rất lớn. Mỗi truy vấn hỏi có thể đi từ (x_1, y_1) tới (x_2, y_2) chỉ bằng bước xuống và bước sang phải hay không.

Trước hết xét các truy vấn đi qua một hàng trung gian p , tức $x_1 \leq p \leq x_2$. Với mọi ô phía trên hàng p , ta tính bitset các cột trên hàng p mà ô đó đi tới được; công thức chuyển chỉ lấy hợp từ ô bên dưới và ô bên phải/trái phù hợp với hướng đi ngược. Tương tự, với mọi ô phía dưới hàng p , ta tính bitset các cột trên hàng p có thể đi tới ô đó. Một truy vấn qua hàng p có lời đáp “có” khi giao của hai bitset tương ứng khác rỗng.

Với truy vấn bất kỳ, dùng chia để trị theo hàng: ở đoạn hàng $[l, r]$, lấy mid , xử lý mọi truy vấn có $x_1 \leq mid \leq x_2$, rồi đệ quy hai phía cho các truy vấn còn lại. Nếu chỉ chia theo một chiều, độ phức tạp có

dạng $O(nm^2 \log n/w + q \log n)$; chia luân phiên theo hai chiều giúp cân bằng hơn và giảm hằng số cũng như bậc phụ thuộc vào cạnh dài của lưới.

16.6.4 Bags and Coins

Bài Codeforces 356D có $n \leq 70000$ túi; mỗi túi hoặc đặt trên đất, hoặc đặt trong một túi khác. Có $s \leq 70000$ đồng xu được phân bố trong các túi. Nếu muốn lấy một đồng xu phải mở túi i , ta nói đồng xu đó nằm trong túi i . Biết số xu a_i trong từng túi, cần dựng một cách lồng túi và phân bố xu, hoặc kết luận vô nghiệm.

Khi tổng cần đạt đúng bằng một trường hợp biên của dãy a_i , có thể dựng trực tiếp bằng cách xếp túi thành một chuỗi lồng nhau và phân phối phần chênh giữa hai giá trị liên tiếp. Trường hợp tổng quát chuyển thành bài toán chọn một số túi đặt trên đất sao cho tổng số xu của chúng bằng mục tiêu; túi đầu tiên là bắt buộc. Đây là ba lô 0/1: dùng bitset S biểu thị các tổng hiện đạt được, khi thêm vật có trọng lượng x thì cập nhật

$$S \leftarrow S \mid (S \ll x).$$

Sau khi thêm hết các vật, kiểm tra bit mục tiêu để biết có nghiệm hay không.

Để khôi phục phương án, dùng mảng $from$, trong đó $from[j]$ ghi vật cuối cùng được thêm trong một cách đạt tổng j . Khi thêm vật thứ i , so sánh bitset trước và sau; các bit mới xuất hiện chính là những tổng lần đầu đạt được nhờ vật i , nên gán $from$ cho chúng. Mỗi vị trí $from$ chỉ bị gán một lần, vì vậy việc lưu vết không phá bậc độ phức tạp. Thuật toán có độ phức tạp xấp xỉ $O(ns/w)$.

16.7 Tài liệu tham khảo

1. Sean Eron Anderson, *Bit Twiddling Hacks*.
2. Christer Ericson, *Advanced bit manipulation-fu*.
3. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*.

17 Một số phương pháp tìm nghiệm tốt thứ k

Yu Dingli, Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Tìm nghiệm tốt thứ k là một kiểu bài rất thường gặp trong thi tin học. Bài viết này phân loại và hệ thống hóa các phương pháp giải, để nhiều bài có thể được chuyển thành những bài toán quen thuộc và đơn giản hơn. Những thuật toán được nhấn mạnh gồm thuật toán tìm k đường đi ngắn nhất với độ phức tạp $O(n \log n + m + k \log k)$, thuật toán tìm k cây khung nhỏ nhất với độ phức tạp $O(m \log n + k^2)$, và thuật toán tìm k đường đi đơn ngắn nhất với độ phức tạp $O(kn(m + n \log n))$. Ngoài các mục thuần giới thiệu thuật toán, mỗi phần đều có ví dụ để người đọc dễ đối chiếu và tổng quát hóa.

17.1 Dẫn nhập

Sau khi giải xong một bài toán, ta thường tự hỏi các biến thể của nó: nếu đổi một điều kiện, đổi hàm mục tiêu, hoặc không chỉ cần nghiệm tốt nhất mà cần nghiệm tốt thứ k , bài toán sẽ ra sao. Những bài “lớn thứ k ”, “nhỏ thứ k ”, hay tổng quát hơn là “nghiệm tốt thứ k ”, vì vậy xuất hiện rất nhiều trong OI.

Các công cụ có thể dùng rất đa dạng: cấu trúc dữ liệu, quy hoạch động, đồ thị, tìm kiếm theo thứ tự từ điển, hàng đợi ưu tiên. Có những cách quen thuộc như nhị phân đáp án, cây đoạn theo giá trị, thống kê trên trie; cũng có những thuật toán ít phổ biến hơn như k đường đi ngắn nhất, k cây khung nhỏ nhất và k đường đi đơn ngắn nhất. Bài viết đi từ các phép chuyển hóa đơn giản tới những thuật toán chuyên dụng hơn.

17.2 Nhị phân để tìm nghiệm tốt thứ k

Trước hết cần một mô tả toán học. Với bài toán cực tiểu, ta có tập nghiệm khả dĩ U và hàm mục tiêu J . Bài toán tối ưu thông thường là

$$\min_{u \in U} J(u).$$

Ở đây $u \in U$ biểu thị một nghiệm thỏa các ràng buộc, còn $J(u)$ đo độ tốt của nghiệm.

Với bài toán cực tiểu, nghiệm tốt thứ k có thể hiểu là một tập $K \subseteq U$ sao cho

$$|K| = k, \quad \max_{u \in K} J(u) \leq \min_{v \in U - K} J(v).$$

Giá trị cần tìm là $\max_{u \in K} J(u)$. Trường hợp cực đại hoàn toàn tương tự nếu đảo chiều bất đẳng thức.

Nếu chỉ cần giá trị của nghiệm tốt thứ k , không cần khôi phục phương án cụ thể, và $J(U) \subseteq \mathbb{Z}$, bài toán có thể chuyển thành bài toán đếm. Đặt

$$S(x) = \{u \in U : J(u) \leq x\}.$$

Khi đó giá trị của nghiệm nhỏ thứ k là

$$\min_{|S(x)| \geq k} x.$$

Vì $|S(x)|$ tăng đơn điệu theo x , ta nhị phân trên miền giá trị của J . Chi phí chuyển hóa là

$$O(\log(\sup J(U) - \inf J(U)))$$

lần gọi thủ tục đếm

$$\sum_{u \in U, J(u) \leq x} 1.$$

Nếu J nhận giá trị thực nhưng đề bài chỉ yêu cầu làm tròn tới d chữ số thập phân, có thể thay $J(u)$ bằng $\text{round}(J(u)10^d)$ để đưa về miền nguyên. Điểm quan trọng là điều kiện $J(u) \leq x$ phải đủ đơn giản; nếu việc kiểm tra hoặc đếm dưới điều kiện này còn khó hơn bài gốc, nhị phân đáp án không đem lại lợi ích.

17.2.1 Ví dụ: COCI BROJ

Cho một số nguyên tố p . Hãy tìm số nguyên dương nhỏ thứ k có ước nguyên tố nhỏ nhất bằng p ; nếu đáp án lớn hơn 10^9 , in 0.

Ta có $J(u) = u$ và

$$U = \{u \in \mathbb{Z}^+ : u \leq 10^9, \text{ ước nguyên tố nhỏ nhất của } u \text{ là } p\}.$$

Mọi phần tử của U có dạng vp , trong đó $v \leq 10^9/p$ và v không có ước nguyên tố nhỏ hơn p . Nếu p đủ lớn, chẳng hạn $p \geq 67$, tập ứng viên nhỏ và có thể xét trực tiếp. Với $p \leq 61$, nhị phân $x \in [0, 10^9]$ và đếm

$$\sum_{v=1}^{\lfloor x/p \rfloor} (1 - [v \text{ có ước nguyên tố nhỏ hơn } p]).$$

Bằng bao hàm–loại trừ, số này có thể viết thành

$$\sum_{d=1}^{\lfloor x/p \rfloor} [d \text{ chỉ có ước nguyên tố nhỏ hơn } p] \mu(d) \left\lfloor \frac{x}{pd} \right\rfloor.$$

Vì chỉ cần xét các tích không lặp của những nguyên tố nhỏ hơn p , độ phức tạp có dạng

$$O(\min\{L/p, 2^{\pi(p)} \log L\}), \quad L = 10^9.$$

17.2.2 Ví dụ: TopCoder PulleyTautLine

Trên mặt bàn có hai đỉnh tại $(0, 0)$ và $((n+1)d, 0)$, cùng n vòng rọc bán kính r có tâm tại

$$(d, 0), (2d, 0), \dots, (nd, 0).$$

Ròng rọc không chồng nhau. Một sợi dây căng nối hai đỉnh có thể quấn quanh các vòng rọc tùy ý. Với n, d, r, k cho trước, cần tìm độ dài nhỏ thứ k của dây, sai số không quá 10^{-9} . Ràng buộc chính là

$$1 \leq r \leq 499999999, \quad 2r+1 \leq d \leq 10^9, \quad 1 \leq n \leq 50, \quad 1 \leq k \leq 10^{18}.$$

Tiếp tục nhị phân độ dài *bound* và đếm số cách có độ dài không vượt quá ngưỡng này. Mỗi đoạn của dây được chia thành bốn loại độ dài e, A, B, R , đều tính được bằng hình học phẳng từ d, r . Khi đó cần đếm các bộ (x, y, z) thỏa

$$2e + xA + yB + zR \leq \text{bound}.$$

Một quy hoạch động trực tiếp có thể ghi trạng thái gồm số đoạn loại A, B, R , vòng rọc hiện tại, hướng đi và vị trí trên vòng rọc. Sau khi quan sát đối xứng, hướng đi chỉ phụ thuộc vào $z \bmod 2$, còn A và B có chuyển trạng thái giống nhau; vì vậy chỉ cần lưu $w = x + y$, rồi nhân thêm hệ số tổ hợp $\binom{x+y}{y}$ khi cộng đáp án.

Không cần ghi mọi lần quấn tròn quanh vòng rọc: hai vòng tròn liên tiếp triệt tiêu về mặt trạng thái, phần R dư có thể được phân phối lại bằng công thức tổ hợp. Khi đã cố định w, x, y, z , số vòng R còn có thể thêm là

$$\text{rest} = \left\lfloor \frac{\text{bound} - (2e + xA + yB + zR)}{R} \right\rfloor.$$

Nhân số trạng thái cơ bản với hệ số tổ hợp tương ứng là đủ. Vì w chỉ cần xét tới khoảng $O(\log k)$, bài toán đếm đủ nhanh để đặt trong vòng nhị phân.

17.3 Chia nhị phân toàn cục và cây đoạn theo giá trị

Trong nhiều bài cấu trúc dữ liệu, cần trả lời nhiều truy vấn kiểu “phần tử nhỏ thứ k_i trong đoạn $[l_i, r_i]$ ”. Nhị phân riêng từng truy vấn thường lãng phí; chia nhị phân toàn cục xử lý đồng thời các truy vấn trên miền đáp án. Cách này cần các tính chất quen thuộc:

- đáp án của truy vấn có tính nhị phân;
- đóng góp của các thao tác sửa đổi vào phép kiểm tra độc lập với nhau;
- nếu một sửa đổi có đóng góp, giá trị đóng góp không phụ thuộc vào ngưỡng đang xét;
- phép cộng đóng góp có tính giao hoán và kết hợp;
- đề bài cho phép xử lý ngoại tuyến.

Trong bài nghiệm tốt thứ k , tính chất đầu tiên chính là hệ quả của phần nhị phân đáp án ở trên.

Khi bài toán cần xử lý trực tuyến hoặc có sửa giá trị điểm, điều kiện của chia nhị phân toàn cục thường quá mạnh. Một hướng khác là dựng cây đoạn theo giá trị. Mỗi nút $[l, r]$ của cây đoạn lưu cấu trúc dữ liệu mô tả các nghiệm u có $J(u) \in [l, r]$. Để trả lời truy vấn trên một tập con U_i , đếm số nghiệm thuộc nửa phải; nếu số đó ít nhất k thì đi sang phải, nếu không thì trừ nó khỏi k rồi đi sang trái. Thêm một nghiệm mới v nghĩa là chèn v vào $O(\log W)$ nút chứa $J(v)$, với

$$W = \sup J(U) - \inf J(U).$$

Sửa giá trị một điểm thì xóa giá trị cũ rồi chèn giá trị mới. Đổi lại, phương pháp này không phù hợp với thao tác làm thay đổi đồng thời $J(u)$ của cả một tập nghiệm lớn.

17.3.1 Ví dụ: BZOJ 3065

Ban đầu có n số $a_1, \dots, a_n$. Có ba loại thao tác: hỏi giá trị nhỏ thứ k trong đoạn từ số thứ x tới số thứ y theo thứ tự hiện tại; sửa giá trị của số thứ x thành val ; và chèn một số giá trị val trước vị trí thứ x . Bài yêu cầu trực tuyến bắt buộc, với cỡ dữ liệu khoảng 35000 phần tử ban đầu và tối đa 70000 thao tác sửa, 70000 truy vấn.

Dùng cây đoạn theo giá trị; ở mỗi nút đặt một cây cân bằng lưu các phần tử có giá trị trong khoảng của nút, theo thứ tự hiện tại. Khi đi trong cây giá trị, ta cần biết trong đoạn vị trí đang hỏi có bao nhiêu phần tử rơi vào nửa trái hoặc nửa phải. Vị trí tương ứng ở tầng con có thể tìm bằng truy vấn “phần tử đặc trưng đầu tiên trước/sau vị trí hiện tại” trong cây cân bằng. Vì mỗi phần tử cần biết vị trí của nó ở từng tầng cây giá trị, hiện thực khá phức tạp dù độ phức tạp chỉ khoảng

$$O((n + q) \log W \log n).$$

17.3.2 Một cách dùng khác của cây đoạn theo giá trị

Nếu xem cây đoạn theo giá trị như thông tin gắn trên nút của một cấu trúc dữ liệu khác, ta gặp khó khăn mới: nhiều cấu trúc cần gộp thông tin con lên cha, trong khi hai cây đoạn theo giá trị không gộp nhanh được. Có ba mẹo thường dùng. Thứ nhất, khi chỉ cần nghiệm tốt thứ k , không nhất thiết dựng sẵn cây đã gộp; ta vừa đi trên cây giá trị vừa tính lượng cần đếm ở nút hiện tại. Thứ hai, nếu nhãn lười cũng là cây đoạn theo giá trị, hãy dùng nhãn vĩnh cửu thay vì đẩy xuống. Thứ ba, nếu cấu trúc nền cần xoay hoặc cập nhật cha từ con, nên chọn cấu trúc cân bằng theo trọng lượng, chẳng hạn treap hoặc scapegoat tree, để chấp nhận tái dựng bạo lực với độ phức tạp kỳ vọng hoặc khấu hao tốt.

17.3.3 Ví dụ: ZJOI 2013

Có n vị trí và m thao tác. Một thao tác thêm một số c vào từng vị trí trong đoạn $[a, b]$; thao tác còn lại hỏi số lớn thứ k trong toàn bộ các số đã được thêm vào các vị trí thuộc $[a, b]$. Với $n, m \leq 50000$, có thể dùng cây đoạn theo vị trí, mỗi nút lưu hai cây đoạn theo giá trị. Cây C_v là nhãn lười vĩnh cửu: mọi vị trí trong đoạn của nút v đều nhận các giá trị này. Cây T_v lưu toàn bộ giá trị xuất hiện trong đoạn của nút.

Khi hỏi đúng đoạn của một nút, trả về T_v . Khi hỏi cắt qua hai con, lấy kết quả hai phía rồi cộng thêm $(b - a + 1)C_v$, vì nhãn của nút hiện tại áp dụng cho mọi vị trí trong đoạn hỏi. Khi sửa đúng đoạn nút, thêm c vào C_v và thêm $(b - a + 1)$ bản sao của c vào T_v ; nếu không thì đệ quy xuống con. Độ phức tạp là $O(m \log^2 n)$. Nếu cần hỗ trợ thêm thao tác cộng một lượng vào cả đoạn, có thể duy trì một nhãn dịch d_v ; lúc đi qua nút thì điều chỉnh giá trị truy vấn theo nhãn này, còn khi đi trên cây giá trị phải trả thêm một hệ số $\log n$.

17.4 Thống kê trên trie

Một lớp bài khác yêu cầu xâu nhỏ thứ k theo thứ tự từ điển thỏa một điều kiện nào đó. Dùng nhị phân trên xâu ít tự nhiên hơn cách xác định từng ký tự. Bản chất của phương pháp xác định từng ký tự là đi trên trie: tại một nút, thử các con theo thứ tự từ điển. Nếu số phương án trong cây con là $s \geq k$, ta đi vào con đó; nếu không, bỏ qua cây con và thay $k \leftarrow k - s$. Công việc cốt lõi là đếm số xâu thỏa điều kiện khi đã cố định tiền tố hiện tại.

Không chỉ xâu mới dùng được cách này. Tìm hoán vị nhỏ thứ k , dãy quyết định nhỏ thứ k , hoặc nhiều đối tượng tổ hợp khác cũng có thể nhìn như đi trên một cây chữ cái trừu tượng.

17.4.1 Ví dụ: xâu con nhỏ thứ k

Cho một xâu str , hãy tìm xâu con nhỏ thứ k theo thứ tự từ điển; hai xâu con có vị trí khác nhau được xem là khác nhau. Với $|str| \leq 100000$, dựng cây hậu tố của str . Cây hậu tố cũng là một trie nén của mọi hậu tố, nên đi trên cây này theo thứ tự từ điển sẽ cho đáp án. Số xâu con trong một cây con có thể tính bằng quy hoạch động trên cây. Nếu chỉ có mảng hậu tố, có thể dựng cấu trúc tương đương dạng “con trái, anh phải” bằng ngăn xếp đơn điệu trên LCP; bản chất vẫn là đi trên cây hậu tố.

17.4.2 Ví dụ: câu có lượng thông tin cho trước

Một câu gồm V từ, hai từ kề nhau cách nhau bởi một dấu cách, từ chỉ gồm chữ cái thường. Lượng thông tin của câu là tổng lượng thông tin của mọi ký tự: dấu cách có lượng 1, chữ a có lượng 2, ..., chữ z có lượng 27. Hỏi câu có tổng lượng thông tin W và nhỏ thứ k theo thứ tự từ điển. Ràng buộc:

$$1 \leq V \leq 1000, \quad 1 \leq W \leq 300, \quad 1 \leq k \leq 10^{18}.$$

Vẫn đi trên trie của mọi câu hợp lệ. Nếu mỗi lần thử một ký tự lại làm quy hoạch động từ đầu thì quá chậm; cần tiền xử lý số cách hoàn thành phần hậu tố với lượng thông tin còn lại. Khi đó tổng chi phí có thể giữ ở cỡ $O(VW)$.

17.5 Hàng đợi ưu tiên trong bài nghiệm tốt thứ k

Một ví dụ cơ bản là trộn m danh sách đã sắp xếp. Đưa phần tử đầu của mỗi danh sách vào một min-heap; mỗi lần lấy phần tử nhỏ nhất ra, rồi đưa phần tử kế tiếp trong cùng danh sách vào heap. Với tổng số phần tử n , độ phức tạp là $O(n \log m)$.

Ví dụ khác: cho $a_1, \dots, a_n$ và $b_1, \dots, b_m$, cần tìm k giá trị nhỏ nhất trong mọi tổng $a_i + b_j$. Sắp xếp dãy b . Với mỗi i , dãy

$$a_i + b_1, a_i + b_2, \dots, a_i + b_m$$

đã tăng dần; bài toán trở thành trộn n danh sách có thứ tự. Sau khi sắp xếp, chạy heap tới k bước là đủ. Cách đếm rồi nhị phân cũng cho giá trị nhỏ thứ k , nhưng không liệt kê được toàn bộ k giá trị đầu.

Tổng quát hơn, ta dựng một đồ thị phụ P trên các nghiệm sao cho mỗi nghiệm là một hoặc vài đỉnh, mỗi đỉnh là nghiệm hợp lệ, và đồ thị có tính chất heap: nếu có cạnh $u \rightarrow v$ thì $J(u) \leq J(v)$. Thêm một nguồn $source$ có trọng số $-\infty$, nối tới mọi đỉnh không có bậc vào. Dùng hàng đợi ưu tiên chứa các đỉnh đã mở rộng: ban đầu chỉ có $source$, mỗi lần lấy nghiệm tốt nhất chưa lấy ra và đưa các đỉnh kề chưa thăm vào heap. Nếu số trạng thái được thăm là $State$, độ phức tạp là $O(State \log k)$. Đồ thị P thường không cần dựng tường minh; ta chỉ sinh cạnh khi mở rộng trạng thái.

Có thể mở rộng khái niệm trên sang đồ thị có trọng số cạnh không âm: mỗi nghiệm tương ứng với một đường đi xuất phát từ $source$, và giá trị nghiệm là tổng trọng số trên đường đi. Cách nhìn này là nền

tăng cho các thuật toán ở những phần sau. Với bài tổng $a_i + b_j$, có thể dùng heap trên dãy b để dựng P với bậc ra nhỏ, đạt dạng độ phức tạp

$$O(k \log k + n + m).$$

17.6 k đường đi ngắn nhất

Trong phần này, đường đi được phép đi qua lại một đỉnh nhiều lần. Bài toán là tìm độ dài đường đi ngắn thứ k , không giảm nghiêm ngặt, từ s tới t trong đồ thị có hướng có trọng số.

17.6.1 Chuyển về các cạnh lệch khỏi cây ngắn nhất

Lấy một cây đường đi ngắn nhất T hướng về t . Với cạnh e , gọi $\ell(e)$ là độ dài, $\text{head}(e)$ là đỉnh đầu, $\text{tail}(e)$ là đỉnh cuối, và $d(u, t)$ là khoảng cách ngắn nhất từ u tới t . Định nghĩa

$$\delta(e) = \ell(e) + d(\text{tail}(e), t) - d(\text{head}(e), t).$$

Khi đó $\delta(e) \geq 0$ với mọi cạnh, và $\delta(e) = 0$ với cạnh thuộc cây T .

Với một đường đi p từ s tới t , bỏ các cạnh thuộc T ; dãy còn lại gọi là $\text{sidetracks}(p)$. Dãy cạnh lệch này có ba tính chất:

- mọi cạnh trong dãy đều thuộc $G - T$;
- nếu hai cạnh lệch liên tiếp là e, f , thì $\text{head}(f)$ nằm trên đường cây từ $\text{tail}(e)$ tới t , hoặc $\text{head}(f) = \text{tail}(e)$;
- độ dài của p bằng

$$d(s, t) + \sum_{e \in \text{sidetracks}(p)} \delta(e).$$

Ngược lại, mỗi dãy cạnh lệch thỏa điều kiện trên xác định duy nhất một đường đi từ s tới t . Do đó bài toán trở thành tìm dãy cạnh lệch có tổng δ nhỏ thứ k .

17.6.2 Dựng đồ thị phụ

Cách thô: một trạng thái là một dãy cạnh lệch q . Nếu cạnh cuối có đỉnh cuối là v (dãy rỗng thì $v = s$), có thể nối tới mọi cạnh $e \in G - T$ có đỉnh đầu nằm trên đường cây từ v tới t , với trọng số $\delta(e)$. Bậc ra quá lớn.

Ta cải thiện bằng cách với mỗi đỉnh v , lập danh sách $g(v)$ gồm mọi cạnh ngoài cây có đỉnh đầu nằm trên đường từ v tới t , sắp theo δ . Trạng thái có thể thay cạnh cuối bằng phần tử kế tiếp trong $g(v)$, hoặc thêm phần tử đầu của $g(\text{tail}(e))$. Dùng biểu diễn “con trái, anh phải” giảm bậc ra xuống 2, nhưng việc dựng mọi danh sách vẫn đắt.

Nhận xét $g(v)$ và $g(\text{next}_T(v))$ gần như giống nhau: từ đỉnh v chỉ thêm các cạnh ngoài cây đi ra khỏi v . Vì vậy dùng heap khá dĩ lâu bền. Gọi $H(v)$ là heap chứa các cạnh trong $g(v)$; ta có được $H(v)$ bằng cách thêm lâu bền các cạnh ngoài cây của v vào $H(\text{next}_T(v))$. Khi cần lấy “phần tử kế tiếp”, chỉ thay đỉnh heap hiện tại bằng hai con của nó. Thêm một tầng heap nữa để tận dụng phép dựng heap tuyến tính, thuật toán đạt

$$O(n \log n + m + k \log k).$$

Heap trái có thể làm lâu bền bằng cách sao chép các nút trên đường trộn; mỗi phép trộn vẫn có độ phức tạp $O(\log m)$.

17.7 Một lớp bài quy hoạch động

Nếu một quy hoạch động có thể xem như đường đi ngắn nhất trên DAG, thì nghiệm tốt thứ k của quy hoạch động có thể tìm bằng thuật toán k đường đi ngắn nhất. Cụ thể, nếu cạnh (a, b) biểu thị chuyển trạng thái

$$b = \min\{b, a + \ell(a, b)\},$$

và mọi phương án quyết định của quy hoạch động tương ứng một-một với một đường đi từ trạng thái đầu tới trạng thái cuối, thì nghiệm nhỏ thứ k chính là đường đi ngắn thứ k . Nếu có nhiều trạng thái đầu/cuối, thêm nguồn và đích ảo với cạnh trọng số 0. Với n trạng thái và m chuyển trạng thái, độ phức tạp là

$$O(n \log n + m + k \log k).$$

Trường hợp tối đa hóa đổi dấu mọi trọng số cạnh rồi đổi dấu đáp án.

17.7.1 Ví dụ: chia điểm vào hai tập

Cho n điểm một chiều x_i . Cần chia các điểm vào hai tập S_A, S_B . Với một tập

$$S = \{p_1, p_2, \dots, p_{|S|}\}, \quad p_1 < p_2 < \dots < p_{|S|},$$

định nghĩa

$$L(S) = \sum_{i=0}^{|S|-1} |p_{i+1} - p_i|,$$

trong đó mặc định $p_0 = 0$. Hãy tìm giá trị nhỏ thứ k của

$$L(S_A) + L(S_B).$$

Ràng buộc trong bài gốc là $n \leq 10000, k \leq 500000$.

Với $k = 1$, đặt $f(i, j)$ là chi phí nhỏ nhất khi hai tập có phần tử cuối lần lượt là i và j , giả sử $i \geq j$. Điểm $i + 1$ hoặc được đưa vào tập đang kết thúc ở i , hoặc đưa vào tập đang kết thúc ở j :

$$f(i, j) + |x_{i+1} - x_i| \rightarrow f(i + 1, j),$$

$$f(i, j) + |x_{i+1} - x_j| \rightarrow f(i + 1, i).$$

Khởi tạo $f(0, 0) = 0$, các trạng thái khác là $+\infty$; đáp án tối ưu là $\min_{j < n} f(n, j)$. Xem các chuyển trạng thái này là cạnh DAG rồi áp dụng phần trước sẽ cho nghiệm nhỏ thứ k , nhưng số cạnh ban đầu là $O(n^2)$.

Có thể tối ưu chuyển thứ hai bằng cách duy trì

$$M_i = \min_j \{f(i, j) + |x_i - x_j|\}.$$

Đặt

$$g(i, x_j) = f(i, j) - x_j, \quad h(i, x_j) = f(i, j) + x_j,$$

ta có thể tính M_i bằng hai truy vấn min trên cây đoạn:

$$M_i = \min \left\{ x_i + \min_{j: x_j < x_i} g(i, x_j), -x_i + \min_{j: x_j > x_i} h(i, x_j) \right\}.$$

Cây đoạn nhị phân đưa số cạnh xuống $O(n \log n)$, cho độ phức tạp khoảng $O(n \log^2 n + k \log k)$. Nếu thay cây nhị phân bằng cây d -phân, chi phí sửa và hỏi có thể cân bằng tốt hơn; lựa chọn d tối ưu liên quan tới hàm Lambert W , nhưng ý chính là điều chỉnh bậc cây để giảm tổng số cạnh phụ trợ.

17.8 k cây khung nhỏ nhất

17.8.1 Từ cây khung nhỏ nhất tới cây khung nhỏ thứ hai

Prim và Kruskal là hai thuật toán quen thuộc để tìm cây khung nhỏ nhất, với độ phức tạp điển hình $O(n \log n + m)$ và $O(m \log m + m\alpha(n))$. Với cây khung nhỏ thứ hai, trước hết tìm cây khung nhỏ nhất T . Với mỗi cạnh ngoài cây (u, v) , nếu thêm nó vào T thì tạo một chu trình; bỏ cạnh lớn nhất trên đường $u \rightarrow v$ trong T sẽ thu được một cây khung mới. Cây khung nhỏ thứ hai là phương án tốt nhất trong các thay thế này. Cạnh lớn nhất trên đường cây có thể tìm bằng nhân đôi LCA, nên tổng độ phức tạp là $O(m \log n)$ sau khi đã có T .

17.8.2 Co các cạnh bắt buộc

Với đồ thị vô hướng G và cạnh $e = (x, y)$, ký hiệu $G.e$ là đồ thị thu được bằng cách co e . Nếu T là cây khung của G chứa e , thì $c_e(T)$ là cây khung nhỏ nhất của $G.e$ khi và chỉ khi T là cây khung nhỏ nhất của G trong số các cây bắt buộc chứa e .

Với một cạnh cây e , bỏ e sẽ chia T thành hai phần. Gọi $\text{ref}(e)$ là cạnh nhẹ nhất, khác e , bắc qua hai phần đó. Nếu $G - e$ còn liên thông, thì

$$T - e + \text{ref}(e)$$

là cây khung nhỏ nhất của $G - e$. Tất cả $\text{ref}(e)$ có thể tính trong $O(m \log m)$: duyệt các cạnh ngoài cây theo trọng số tăng dần, phủ đường đi giữa hai đầu mút của nó trên T , và dùng DSU để nhảy qua các đoạn đã được phủ.

Đặt

$$\text{lb}(e) = \ell(\text{ref}(e)) - \ell(e).$$

Nếu $\text{lb}(e)$ không nằm trong $k - 1$ giá trị nhỏ nhất, thì có ít nhất k cây khung không tệ hơn mọi cây không chứa e . Vì vậy e chắc chắn xuất hiện trong cây khung nhỏ thứ k , và có thể co lại. Bằng cách này ta tìm được một tập cạnh chắc chắn thuộc đáp án và giảm số đỉnh cần xét xuống cỡ $O(k)$.

17.8.3 Bỏ các cạnh ngoài cây vô dụng

Với cạnh ngoài cây $e = (u, v)$, gọi $\text{Re}(e)$ là cạnh nặng nhất trên đường $u \rightarrow v$ trong cây khung nhỏ nhất T . Đại lượng

$$\zeta(e) = \ell(e) - \ell(\text{Re}(e))$$

chính là chi phí tăng thêm khi ép dùng e theo thay thế tốt nhất. Nếu $\zeta(e)$ quá lớn, cụ thể lớn hơn giá trị nhỏ thứ $k - 1$ trong các cạnh ngoài cây, thì đã có ít nhất k cây khung không tệ hơn mọi cây chứa e ; do đó e chắc chắn không nằm trong cây khung nhỏ thứ k . Sau bước này có thể giả sử $n \leq k$ và $m \leq 2k - 1$.

17.8.4 Dựng đồ thị phụ cho các cây khung

Một đỉnh của đồ thị phụ P là bộ

$$R = (T, I, X),$$

trong đó T là một cây khung, I là tập cạnh bị ép phải có, còn X là tập cạnh bị ép không được có. Nút gốc là

$$R_0 = (T_0, \emptyset, \emptyset),$$

với T_0 là cây khung nhỏ nhất của G .

Từ $R_i = (T_i, I_i, X_i)$, xét các thay thế hợp lệ

$$T_i - e + \text{rc}_{X_i}(e), \quad e \in T_i - I_i,$$

trong đó $\text{rc}_{X_i}(e)$ là cạnh nhẹ nhất, không thuộc X_i , có thể thay e . Sắp các thay thế theo độ tăng trọng số, với các cạnh cây bị thay lần lượt là $e_1, e_2, \dots$. Con thứ j của R_i là

$$(T_i - e_j + \text{rc}_{X_i}(e_j), I_i + \{e_1, \dots, e_{j-1}\}, X_i + \{e_j\}).$$

Tập I ghi rằng các thay thế trước đó đã bị bỏ qua nên các cạnh ấy phải giữ lại; tập X ghi cạnh vừa bị cấm để tách các nhánh. Có thể chứng minh các nút của P tương ứng một-một với mọi cây khung, và trọng số không giảm dọc cạnh của P . Vì vậy dùng hàng đợi ưu tiên trên P sẽ sinh lần lượt các cây khung nhỏ nhất.

Vấn đề còn lại là tìm thay thế nhỏ nhất đủ nhanh. Có hai hướng: duy trì rc cho mọi cạnh cây, hoặc duy trì Re cho mọi cạnh ngoài cây. Hướng thứ hai có thể cập nhật $O(m)$ mỗi lần thay một cạnh cây. Khi thay $e = (u_0, v_0)$ bằng $f = (u_p, v_q)$, chỉ những cạnh ngoài cây bắc qua hai thành phần của $T - e$ mới đổi đường đi trong cây. Duyệt cây để biết mỗi đỉnh gần điểm nào nhất trên chu trình mới, rồi tính lại Re từ vài đoạn tiền tố trên chu trình. Tổng cộng thu được định lý:

$$k \text{ cây khung nhỏ nhất có thể tìm trong } O(m \log n + k^2).$$

Nếu có một cấu trúc dữ liệu lâu bền hỗ trợ thay cạnh cây, đổi trọng số cạnh và duy trì thay thế nhỏ nhất, chi phí phần sau có thể giảm theo số thao tác của cấu trúc đó; tài liệu gốc nhắc tới topology tree lâu bền cho hướng này.

17.9 k đường đi đơn ngắn nhất

Ở đây đường đi từ s tới t không được đi qua một đỉnh quá một lần. Chỉ khác k đường đi ngắn nhất ở chữ “đơn”, nhưng thuật toán lại gần tinh thần của k cây khung nhỏ nhất hơn: chia tập nghiệm thành các tập con, mỗi tập con lấy nghiệm tốt nhất rồi đưa vào hàng đợi ưu tiên.

Một đỉnh của đồ thị phụ là

$$R = (p, I, X),$$

trong đó $p = (v_1 = s, v_2, \dots, v_{|p|} = t)$ là đường đi đơn tốt nhất hiện tại dưới các ràng buộc, I là tập cạnh bị ép phải dùng theo một tiền tố của đường đi, còn X là các cạnh bị cấm xuất phát từ đỉnh phân nhánh hiện tại. Giả sử

$$I = \{(v_1, v_2), (v_2, v_3), \dots, (v_{q-1}, v_q)\},$$

và X chỉ gồm các cạnh đi ra từ v_q . Để tách tập nghiệm chứa p nhưng không lấy lại đúng p , ta tạo các nhánh: nhánh đầu cấm cạnh (v_q, v_{q+1}) ; các nhánh sau lần lượt ép thêm cạnh kế tiếp của tiền tố rồi cấm cạnh rẽ tiếp theo. Cách chia này giống phương pháp Lawler: các tập con không giao nhau và phủ hết nghiệm còn lại.

Trong mỗi tập con, cần tìm đường đi đơn ngắn nhất thỏa I, X . Nếu mọi cạnh có trọng số không âm, dùng Dijkstra. Nếu có cạnh âm nhưng không có chu trình âm, dùng Bellman–Ford hoặc SPFA để lấy thể năng $\text{dist}(s, u)$, rồi đổi trọng số cạnh thành

$$\ell'(u, v) = \ell(u, v) + \text{dist}(s, u) - \text{dist}(s, v),$$

tương tự phần δ của k đường đi ngắn nhất; cuối cùng cộng lại độ dài đường đi ngắn nhất ban đầu từ s tới t .

Mỗi lần lấy một nút khỏi hàng đợi ưu tiên, ta sinh các nhánh con và chạy lại thuật toán đường đi ngắn nhất dưới ràng buộc. Tối đa k đường đi, mỗi đường có độ dài không quá n , nên số lần tính đường đi ngắn nhất là $O(kn)$. Do đó:

$$\text{đường đi đơn ngắn thứ } k \text{ có thể tìm trong } O(kn(m + n \log n)).$$

Heap trực tiếp cho phần hàng đợi ưu tiên đủ dùng; bộ nhớ có thể giữ ở $O(kn)$.

Lời cảm ơn

Tác giả cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi; cảm ơn các thầy Chen Heli, Shao Hongxiang, You Guanghui và Dong Yehua của Trường Trung học số 1 Thiệu Hưng vì sự quan tâm và hướng dẫn nhiều năm; cảm ơn huấn luyện viên Hu Weidong và Yu Linyun của đội tuyển quốc gia; cảm ơn các anh chị, bạn học ở Thiệu Hưng và Trần Hải đã góp ý, giúp đỡ và hiệu đính; cuối cùng cảm ơn cha mẹ vì sự chăm sóc bền bỉ.

Tài liệu tham khảo

1. Wikipedia, *Optimization problem*.
2. Wheeler, *PulleyTautLine*, TopCoder Algorithm Problem Set Analysis.
3. Xu Haoran, bài viết về chia nhị phân toàn cục, tập luận văn đội tuyển quốc gia năm 2013.
4. Wang Kaifeng, lời giải loạt bài giá trị nhỏ thứ k trên đoạn có chèn.
5. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Introduction to Algorithms*.
6. D. Eppstein, *Finding the k shortest paths*, SIAM Journal on Computing, 28(2):652–673, 1998.
7. Yu Dingli, *Heap lâu bền và k đường đi ngắn nhất*, thảo luận trại đông năm 2014.
8. Yu Dingli, lời giải *Người đưa thư nhàm chán*, bài tập đội tuyển quốc gia lần hai năm 2014.
9. Wikipedia, *Prim's algorithm*.
10. Wikipedia, *Kruskal's algorithm*.
11. Wikipedia, *Minimum spanning tree*.
12. Tang Wenbin, chuyên đề đồ thị về cây khung, bài giảng trại đông năm 2012.
13. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*.
14. D. Eppstein, *Finding the smallest spanning trees*, BIT 32(2): 237–248.
15. Greg N. Frederickson, *Ambivalent data structures for dynamic 2-edge-connectivity and k smallest spanning trees*, SIAM Journal on Computing, 26(2):484–538, 1997.
16. E. L. Lawler, *Combinatorial Optimization: Networks and Matroids*.
17. J. Y. Yen, *Finding the k shortest loopless paths in a network*, Management Science, 17:712–716, 1971.

Phạm vi bản dịch

Tập PDF có 228 trang. Bản dịch bao gồm toàn bộ mười lăm bài trong tập, từ *MSS* ở trang in 1–8 đến *Một số phương pháp tìm nghiệm tốt thứ k* ở trang in 195–224. Các trang trắng ngăn cách giữa bài đã được bỏ qua; các hình minh họa và đoạn mã mẫu được chuyển thành mô tả thuật toán hoặc sơ đồ LaTeX khi cần để giữ phần nội dung chính.